CLOUD DEVELOPERS

Cloud Developers — Course Book

Foundations to Operations

Lars Appel

v2026.04.29-4b26d48

Contents

Preface 27

Understanding Cloud Computing 31

- What Is Cloud Computing? 31
- As A Service Models (XaaS) 34
- The Shared Responsibility Model 37
- Deployment Models 39
- Cloud Providers' Global Presence 43
- Common Services 45
- The Role of Linux in Cloud Computing 49

Technical Opportunities 55

- Automation 55
- High Availability 57
- Virtualization 59
- Architectural Patterns 62

Organizational Aspects 69
Organizational Opportunities 69
Best Practices 73
GitOps: Version-Controlled Infrastructure 78
Financing Cloud Computing 80
Capacity Management 82

Cloud Native Development 85
Key Aspects of Cloud Native Development 85
Architectural Patterns For Cloud Native Apps 88
The 12-Factor Model 90

Compute 95
Azure VM Sizing and Cost (Azure) 95

HTTP Fundamentals 111
Why HTTP exists 112
The request-response cycle 113
HTTP methods 114
Status codes 116
URIs 118
Headers 119
Worked example: curl hitting an MVC action 120
Summary 122

The .NET Platform 123

Why .NET exists 124

The compilation pipeline 125

Assemblies and NuGet packages 126

ASP.NET Core as the web layer 128

A worked example: scaffolding an MVC project 129

.NET compared with other web stacks 131

Summary 132

The MVC Pattern 135

The three roles of MVC 136

Routing matches a URI to an action 138

Model binding turns request data into typed parameters 139

Action results describe the response 140

Razor view syntax 141

The request flow 142

Worked example: HomeController and Index.cshtml 143

Why this pattern matters 145

Summary 145

Three-Tier Architecture 147
Why monolithic controllers break down 148
The three layers and their responsibilities 149
Dependency direction and the role of interfaces 150
How a request flows through the layers 152
The trade-off: more files, more reasoning power 156
Summary 157

Configuration and Environments 159
Why configuration is separate from code 160
The configuration provider chain 160
Environment selection 163
User-secrets for development 164
Environment variables and the override precedence 165
Strongly-typed configuration with IOptions 167
The boundary between development and production secret stores 169
Summary 170

Dependency Injection 173
The problem with constructing collaborators directly 174
Inverting the dependency 175
The ASP.NET Core DI container 176
Service lifetimes 178
A worked example: the newsletter service 181
Dependency injection enables unit testing 184
Summary 186

Relational vs NoSQL Data Models. 189
How the two models structure records 190
Schema rigidity and the cost of migration 191
Denormalization as the answer to the join problem 192
ACID, CAP, and eventual consistency 193
Worked example: a Cosmos DB collection for the MongoDB API 194
A decision framework 196
Summary 197

ORM and the Repository Pattern 199

Why raw data access becomes a liability 200

What an ORM does 201

The repository pattern 202

How ORM and repository compose 204

When the abstraction is worth its cost 209

Summary 210

Connections, Pooling, and Transactions 213

The cost of opening a database connection 214

Connection pooling 215

Reusing the client across the application 217

Transactions 219

Isolation levels 221

Optimistic and pessimistic locking 222

Practical advice 224

Summary 224

Object Storage and File Uploads 227

Why databases struggle with large binary payloads 228

Blobs and containers 229

Access tiers as the cost-vs-latency knob 230

SAS tokens for delegated access 231

Streaming uploads from the SDK 233

Content type and metadata 235

Choosing between blob storage and a database BLOB column 236

Summary 238

Authentication vs Authorization 241

The two questions every secured request must answer 242

The principal — the identity attached to a request 243

Where each decision happens in the pipeline 244

Credentials authentication accepts 245

Decisions authorization makes 247

A worked example 248

How the rest of Part V fits together 249

Summary 250

Cookie-Based Authentication and Sessions 251

How browsers handle cookies 252

The authentication ticket inside the cookie 253

The sign-in handshake 254

The sign-out handshake 258

Cookie attributes that matter for security 258

Why CSRF is a cookie-specific problem 261

Putting the pieces together 262

Summary 263

ASP.NET Core Identity 265

What ASP.NET Core Identity provides 266

User stores: where the records live 267

UserManager and SignInManager 270

A worked example: registering and signing in a user. 273

When not to use Identity 275

Summary 277

Roles, Claims, and Policies 279

Claims as the unit of fact 280

Roles as a special claim 281

When roles stop being enough 283

Authorization policies 283

A decision framework 286

Where claims come from 288

Summary 289

Bearer Tokens and JWT 291

Why cookies do not fit API calls 292

Bearer tokens 293

The JWT format 294

Validating a JWT 297

Lifetimes and refresh tokens 299

When JWT is the wrong choice 300

Summary 301

API Keys and Machine-to-Machine 303

The machine-to-machine problem 304

How an API key travels with a request 305

ASP.NET Core middleware as the enforcement point 306

Why API keys are weaker than JWT 308

Generating, storing, and rotating keys 310

When an API key is enough — and when it is not 314

Summary 315

OAuth 2.0 and OpenID Connect 317
The delegated-access problem 318
The four actors 319
Grant types at a high level 320
Authorization code with PKCE 321
Scopes — what the token may do 323
What OIDC adds — the ID token 324
Authentication versus authorization, restated 324
Worked example — registering OIDC with Microsoft Entra ID 325
Summary 327

Secret Management 329
Why production needs more than environment variables 330
Azure Key Vault as the production secret store 331
Managed identity removes the bootstrap problem 332
RBAC role assignments grant access at the right scope 333
Wiring Key Vault into the configuration chain 335
Application secrets versus pipeline secrets 337
Audit logging and incident response 338
A checklist for handling secrets 339
Summary 340

REST Principles 343

What REST is and why it exists 344

The architectural constraints 344

A worked example: GET /quotes/42 348

The Richardson Maturity Model 349

When REST is not the right choice 351

Summary 351

Resource Modeling and URIs 353

The resource as the unit of API design 354

Naming conventions 354

Collection and item URIs 356

Hierarchical relationships 357

Filtering, pagination, and sub-resources at a glance 360

URI stability — the contract 361

Worked example — the `CloudCiApi` quote endpoints 362

Summary 363

DTOs vs Entities 365

Why entities make poor wire shapes 366

What a DTO is and how to design it 367

Mapping between entities and DTOs 369

The cost of the split 372

Summary 373

Status Codes, Versioning, and Error Responses 375
Status codes as the primary signal 376
Problem details: a standard error shape 379
A worked example: create and validation failure 381
API versioning 383
Summary 385

OpenAPI and Swagger 387
The contract problem 388
The OpenAPI specification 388
Swagger UI as the browser explorer 392
Swashbuckle generates OpenAPI from the controller code. 393
The toolchain payoff 397
Worked example tied to the exercise 398
Summary 399

Pagination, Idempotency, and Rate Limiting. 401
Why returning everything is not an option 402
Idempotency as a contract 405
Rate limiting as a backstop 407
A worked example 409
Summary 412

Containers vs Virtual Machines 415
The reproducibility problem 416
How a virtual machine virtualizes hardware 416
How a container isolates processes on a shared kernel 417
What each model gives up and gains 419
Density and startup time in practice 420
When each model fits 421
Summary 423

Images and Layers 425
What an image is 426
The layer model 427
Why layer ordering matters 428
The base image 429
The image manifest 430
Storage and pull efficiency 431
Worked example: pulling and inspecting an image 432
Summary 433

Dockerfiles and Multi-Stage Builds 435
The Dockerfile as a build recipe 436
The build context and .dockerignore 438
The layer cache and instruction order 439
ENTRYPOINT versus CMD 440
The multi-stage pattern 441
Summary 444

Multi-Platform Builds 445
What a platform means for an image 446
Buildx and the multi-architecture pipeline 447
The manifest list as the indirection layer 449
A worked example 450
Trade-offs and when multi-platform matters 451
Summary 452

Docker Compose 455
The problem Compose solves 456
The compose file 457
Services 458
Named networks 459
Volumes 460
Environment variables and .env files 462
Dependency ordering 463
Worked example: web and database 464
Compose and production orchestration 466
Summary 467

Container Registries 469
What a registry is 470
Tags versus digests 471
Push and pull 472
Public and private registries 473
Authenticating to a registry 476
Summary 477

The DevOps Philosophy 481
The legacy split and its handoff cost 482
What DevOps changed 483
The four DORA metrics 484
The value stream and where waste hides 486
Worked example: a team's two-year arc 487
How the rest of Part VIII operationalizes the philosophy 489
Summary 489

Continuous Integration vs Continuous Deployment 491

The pain that motivated continuous integration. 492

What CI actually requires 493

The spectrum: integration, delivery, deployment 494

Trunk-based development 495

The pull-request gate 496

A worked example 497

Choosing where on the spectrum to land. 499

Where this connects to the exercises 500

Summary 501

Pipelines as Code 503

Why the pipeline belongs in the repository 504

The GitHub Actions vocabulary 505

Triggers: when a workflow runs 507

Code review for the pipeline itself 507

Artifacts: passing files between jobs 508

Worked example: a minimal CI workflow 509

The trade-off: YAML readability degrades 511

Summary 512

Build, Test, and Smoke Gates 515
From build script to continuous integration 516
The build stage 516
Test layers 517
Why each layer earns its keep 520
Gate ordering and fail fast 521
Test-result publishing 522
The smoke test as deployment gate 523
Summary 525

Deployment Strategies 527
Why a strategy is needed 528
The manual gate 529
Rolling deployment 530
Blue-green deployment 531
Canary deployment 532
Feature flags 532
Choosing a strategy 533
A worked example with Container Apps 536
Summary 537

Pipeline Secrets and OIDC Federation. 539
Why pipeline credentials are dangerous 540
The GitHub secret store 541
The service principal model 542
The federated-credential model 543
Comparison: long-lived vs federated 546
Worked example: azure/login with OIDC. 547
A note on managed identity 548
Summary 549

Azure Container Apps as a Deployment Target 551
Why a managed runtime for stateless containers 552
The Container Apps model 553
Single and multiple revision modes 554
Ingress 555
Scale rules 556
The deployment update flow 557
Pulling images from ACR with managed identity 559
Composing with the rest of the Part. 559
Summary 560

The Three Pillars: Logs, Metrics, Traces 565

From monitoring to observability 566

Logs: the narrative of what happened 567

Metrics: numbers over time 568

Traces: end-to-end flow across services 569

How the three pillars combine 570

Cost trade-offs in practice 573

Summary 574

Structured Logging with ILogger 575

From free-text strings to queryable data 576

How `ILogger<T>` works 577

Log levels 579

Logging scopes 580

Enrichment 582

A worked example 582

Operational practices 585

Summary 587

Application Insights and Telemetry 589
Why an APM destination 590
How the SDK captures telemetry without explicit calls 591
Configuration via connection string 592
The dashboards developers actually use 594
Sampling as a cost lever 596
Custom events and metrics 597
What Application Insights does not solve 599
Summary 600

Log Analytics and KQL Basics 601
Where the data lives 602
Tables as the unit of organization 603
KQL as a pipe of transforms 604
The workhorse operators 605
Time range first, always 606
A worked example: failed responses by operation 607
Retention as a cost knob 608
Saving useful queries as workbooks 609
Summary 610

Health Checks 613

Why a TCP probe is not enough 614

What a health check is 615

The three canonical probe types 615

The /healthz endpoint convention 618

ASP.NET Core health checks 619

Health checks and the smoke test gate 621

How probes integrate with autoscaling 621

Cross-link to the exercise 622

Summary 622

Alerts and SLOs 625

What an alert rule is 626

Action groups as the contact-list abstraction 627

The alert fatigue problem 628

SLI, SLO, and error budget 629

A worked example: alerting on 5xx error rate 631

How SLOs counter alert fatigue in practice 632

Summary 633

Version Control with Git 637

Why version control matters 638

What a Git repository actually is 639

The three states of a file 640

Commits as immutable snapshots 641

Branches as movable pointers 642

Remotes and synchronization 642

The daily workflow 644

Worked example: from empty folder to remote repository 645

Where Git fits in the larger picture 646

Summary 646

Branching, Pull Requests, and Code Review 649

Why pushing to main is risky 650

The topic-branch flow 651

What a pull request actually is 652

Branch protection: turning convention into enforcement 653

Merge strategies 654

Conflict resolution 656

Draft pull requests for work in progress 657

Code review as a craft 658

Where this fits with the exercises 659

Summary 659

Inner Loop vs Outer Loop 661

The two loops of software development 662

Why optimizing the inner loop matters 664

Tools that shorten the inner loop 665

The boundary conversation 669

Summary 671

Agile, Sprints, and User Stories..... 673

Why iterative delivery replaces big-bang planning 674

The four values of the Agile Manifesto 674

Scrum as the dominant framework 675

The user story format 677

Acceptance criteria as the “how do we know it’s done” answer..... 678

Definition of done as the team’s quality contract..... 678

Worked example: a sign-up form 679

Scrum compared with Kanban 680

Connecting to practice 681

Summary 682

Working with Jira 683
What Jira is 684
The issue type hierarchy 684
The workflow as a state machine 686
The sprint board 687
Git integration through branch naming 688
Comment hygiene and estimating discipline 690
Summary 691

Preface

The CLO25 Course Book is the textbook companion to the hands-on exercises. Read it end-to-end, or jump to the Part that matches the week you're in.

The book has ten Parts. Parts I and II cover the foundations every cloud developer needs. Parts III and IV develop the application-development half of the BCD course. Parts V through X cover the advanced topics — identity, services, containers, delivery, observability, and collaboration — that make up the ACD course.

0.0.1 How to read this book

Each Part is independent enough that you can pick it up cold, but reading in order pays off — terminology and worked examples build on each other. Every chapter pairs with at least one practical exercise; the chapter introduces the concept and the exercise gives you the muscle memory.

Each chapter ends with a **Summary** section that recaps its load-bearing claims in a few sentences. If you're scanning, read the summary first.

0.0.1 Parts

PART I

Cloud Foundations

The conceptual ground every cloud developer stands on. This Part covers what cloud computing actually is, what it makes technically possible, what it changes for organizations, and what "cloud native" means in practice.

PART I · CLOUD FOUNDATIONS

CHAPTER 1

Understanding Cloud Computing

1.1 What Is Cloud Computing?

1.1.1 Introduction

Cloud computing has become a ubiquitous technology that powers everything from our smartphones and streaming services to businesses of all sizes. But what exactly is cloud computing? In this chapter, we'll explore the fundamental concepts and key aspects of cloud computing, shedding light on its significance and the ways it has transformed the way we store, process, and access data and applications.

1.1.2 Defining Cloud Computing

At its core, cloud computing is a technology that enables the delivery of computing services, such as storage, processing power and software services, over the internet. These services are hosted on remote servers in data centers, and users can access them through web browsers or dedicated applications without the need for extensive local hardware or infrastructure.

1.1.3 Key Characteristics of Cloud Computing

- On-Demand and Self-Service: Cloud services can be provisioned and managed by users on an as-needed basis, allowing for flexibility and scalability. Users can easily adjust resources to match their specific requirements.
- Broad Network Access: Cloud services are accessible over the internet from a wide range of devices, including laptops, smartphones, tablets, and more. This accessibility fosters remote work and collaboration.
- Resource Pooling: Cloud providers use multi-tenant models, where resources are shared among multiple users or organizations while ensuring data isolation and security. This approach optimizes resource utilization.
- Rapid Elasticity: Cloud services can quickly scale up or down to accommodate changing workloads. This elasticity is essential for handling fluctuations in demand without incurring unnecessary costs. The preferred scaling technique in the cloud is what is called horizontal scaling as opposed to the more traditional approach of vertical scaling.
- Measured Service: Cloud computing typically follows a pay-as-you-go model, where users are billed based on

their actual usage. This approach minimizes upfront expenses and provides cost transparency.

1.1.4 Benefits of Cloud Computing

The adoption of cloud computing offers several compelling advantages, including:

- Cost-Efficiency: Cloud services reduce the need for upfront capital investments in hardware and infrastructure. Users pay only for the resources they consume, which can result in significant cost savings.
- Scalability: Cloud services can easily scale up or down to accommodate changing workloads, ensuring that businesses can adapt to varying demands.
- Accessibility: Cloud computing enables remote access to data and applications, facilitating remote work, collaboration, and global connectivity.
- Reliability and Availability: Cloud providers invest heavily in redundant infrastructure and data centers to ensure high availability and reliability, reducing downtime and data loss.
- Security and Compliance: Cloud providers implement robust security measures, and many offer compliance certifications to meet industry-specific regulatory requirements.

1.1.5 Conclusion

Cloud computing has become the backbone of countless businesses and industries. Its flexibility, scalability, and cost-efficiency make it a transformative technology that continues to shape the way we work, innovate, and interact with the digital world. As cloud computing evolves, it will likely bring even more innovation, empowering individuals and organizations

to achieve new heights in efficiency, productivity and innovation.

1.2 As A Service Models (XaaS)

1.2.1 Introduction

Cloud services are typically categorized into three main models: - Infrastructure as a Service (IaaS) - Platform as a Service (PaaS) - Software as a Service (SaaS)

Let's examine how they work and their respective advantages.

1.2.2 Infrastructure as a Service (IaaS)

IaaS is the foundational layer of cloud computing, providing virtualized computing resources over the internet. With IaaS, businesses can rent virtual machines, storage, and networking infrastructure from a cloud service provider. This model offers a high degree of flexibility, as users can scale their resources up or down based on their needs.

Key characteristics of IaaS include:

- Virtualization: IaaS leverages virtualization technology to create a pool of virtual resources, allowing users to run multiple virtual machines on a single physical server.
- Self-Service Provisioning: Users can provision and manage their virtual infrastructure through a web-based interface, enabling quick deployment and scalability. Many times you can also access the resources through the use of Command Line Interfaces (CLI) or Software Development Kits (SDK) for different languages.
- Pay-as-You-Go Pricing: IaaS follows a pay-as-you-go pricing model, meaning users only pay for the resources they consume, reducing upfront infrastructure costs. This

model has huge implications on how businesses finance their infrastructure moving from CAPEX to OPEX.

- Scalability: IaaS allows businesses to easily scale their infrastructure to accommodate increased workloads or seasonal demands.

Examples of popular IaaS providers include Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP).

1.2.3 Platform as a Service (PaaS)

PaaS builds upon the IaaS layer by providing a platform that includes not only infrastructure but also development tools and services. PaaS is designed to simplify the application development process and speed up the deployment of software applications.

Key characteristics of PaaS include:

- Application Development Tools: PaaS offers a range of development tools, including databases, frameworks, and runtime environments, making it easier for developers to create, test, and deploy applications.
- Automatic Scaling: PaaS platforms typically provide automatic scaling capabilities, ensuring that applications can handle varying levels of traffic without manual intervention.
- DevOps Integration: PaaS encourages collaboration between development and operations teams through built-in DevOps tools and workflows.
- Reduced Management Overhead: With PaaS, users can focus more on coding and less on managing infrastructure, as many operational tasks are handled by the platform.

Examples of PaaS providers include Heroku, IBM Cloud Foundry, and Microsoft Azure App Service.

1.2.4 Software as a Service (SaaS)

SaaS is the top layer of the cloud computing stack, offering fully functional software applications accessible through a web browser. SaaS providers host and maintain the software, allowing users to access it over the internet without the need for installation or updates on their local devices.

Key characteristics of SaaS include:

- Accessibility: SaaS applications are accessible from anywhere with an internet connection, making them ideal for remote work and collaboration.
- Automatic Updates: SaaS providers manage software updates, ensuring that users always have access to the latest features and security patches.
- Subscription-Based Pricing: SaaS follows a subscription-based pricing model, where users pay a recurring fee for access to the software.
- Multi-Tenancy: SaaS applications are typically multi-tenant, meaning multiple users or organizations share the same software instance while their data remains isolated and secure.

Popular SaaS examples include Microsoft Office 365, Google Workspace, Salesforce, and Dropbox.

1.2.5 Conclusion

Understanding the fundamentals of cloud computing service models - IaaS, PaaS, and SaaS - is essential for businesses and individuals looking to harness the power of the cloud. These models offer various advantages, from infrastructure flexibility and development efficiency to software accessibil-

ity and maintenance simplicity. Depending on your specific needs and objectives, you can choose the cloud service model that best suits your requirements, enabling you to take full advantage of the cloud's capabilities.

1.3 The Shared Responsibility Model

1.3.1 Introduction

The Shared Responsibility Model is a framework that delineates who is responsible for securing what within the cloud environment. It creates a clear demarcation of security responsibilities, ensuring that both Cloud Service Providers (CSP) and customers understand their roles in maintaining a secure cloud ecosystem. This model is designed to mitigate security risks and provide accountability in the cloud computing landscape.

1.3.2 Connecting the Shared Responsibility Model with Cloud Service Models

1. **Infrastructure as a Service (IaaS):**
   - **CSP Responsibilities**: In an IaaS model, the CSP is responsible for securing the foundational infrastructure, including physical data centers, network infrastructure, and the hypervisor layer.
   - **Customer Responsibilities**: Customers take on more security responsibilities at the IaaS level. They are responsible for securing their virtual machines, operating systems, applications, data, and access control. This includes tasks such as configuring firewalls, patching, and setting up encryption.

   Relationship with Shared Responsibility Model: IaaS requires customers to manage and secure their virtual

environments extensively, aligning with the model's principle of shared responsibility. CSPs provide the infrastructure and foundational services, while customers are accountable for the security of their applications and data.

2. **Platform as a Service (PaaS):**
   - **CSP Responsibilities**: PaaS providers extend their security responsibilities beyond infrastructure to include the management of the platform itself. This encompasses aspects like database security, runtime environments, and application frameworks.
   - **Customer Responsibilities**: Customers using PaaS focus primarily on securing their applications, custom code, and user access management. They have less control over the underlying platform, as it is managed by the CSP.

   Relationship with Shared Responsibility Model: PaaS offers a more streamlined security model, with CSPs assuming a broader role in securing the platform. Customers still bear responsibility for application security, access control, and data protection.

3. **Software as a Service (SaaS):**
   - **CSP Responsibilities**: In a SaaS model, the CSP takes on the majority of security responsibilities. This includes securing the application, data, and user access. CSPs often handle application maintenance, updates, and security patches.
   - **Customer Responsibilities**: Customers using SaaS solutions have limited control over the application's security aspects. They are primarily responsible for managing user access and ensuring that they adhere to security best practices.

Relationship with Shared Responsibility Model: SaaS exemplifies the concept of shifting security responsibilities towards the CSP. Customers are more focused on user access and adherence to security policies, while CSPs manage the bulk of security tasks.

Conclusion

The Shared Responsibility Model defines the distribution of security responsibilities between CSPs and customers. Understanding how this model relates to the various cloud service models (IaaS, PaaS, and SaaS) is crucial for organizations to ensure the security of their cloud deployments.

Different cloud service models offer varying levels of control and responsibility, and the Shared Responsibility Model adapts to these differences. By aligning security practices with this model, organizations can create a robust and well-defined security posture in the cloud, reduce risks, and promote accountability. Ultimately, leveraging the Shared Responsibility Model is essential to achieving a secure and successful cloud computing strategy regardless of the chosen service model.

1.4 Deployment Models

1.4.1 Introduction

Cloud computing deployment models are frameworks that define the management and allocation of computational resources over the cloud. These models are essential in determining how cloud services are provided and accessed by users. The primary deployment models in cloud computing are: Public Cloud, Private Cloud and Hybrid Cloud. Let's explore each model in detail.

1.4.2 Public Cloud

The public cloud is a model where cloud services are delivered over the internet and shared across different organizations. Notable examples include Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform.

This model is ideal for businesses that prioritize flexibility and want to avoid the capital expenditure of hardware. By offloading infrastructure management to the cloud provider, organizations can focus on their core business activities. However, this model comes with concerns regarding security and control, as the shared resources are managed by an external party. Despite these concerns, the public cloud is a compelling choice for businesses seeking a quick, scalable, and cost-effective solution.

Pros: - **Cost-Effective**: Users pay only for the resources they use, which eliminates the need for large capital investments. - **Scalability**: Public clouds offer vast resources and scalability to handle varying workload demands. - **Maintenance-Free**: The cloud provider is responsible for all maintenance, updates, and infrastructure management.

Cons: - **Security Concerns**: Shared resources can lead to potential vulnerabilities and security concerns. - **Less Control**: Users have limited control over the infrastructure and its configuration. - **Variable Performance**: Due to shared resources, performance can be impacted by other users' activities.

1.4.3 Private Cloud

A private cloud is a cloud environment dedicated solely to one organization. It can be hosted internally or by a third-party service provider.

For organizations where security and control are paramount, the private cloud offers a secluded environment. This

model is often chosen by organizations with stringent regulatory requirements or those needing high-level data protection. The private cloud allows for greater customization to meet specific organizational needs and ensures consistent performance. However, the flip side is the higher cost and complexity in management, often requiring dedicated IT teams. Despite these challenges, the private cloud is a powerful choice for organizations needing a secure, controlled environment with the flexibility of cloud computing.

Pros: - **Enhanced Security and Privacy**: As the resources are not shared, there is a higher level of control and security. - **Customization**: Organizations can tailor the environment to meet their specific needs. - **Performance Consistency**: Since resources are not shared, performance is more predictable and consistent.

Cons: - **Higher Costs**: Requires significant investment in hardware and software, as well as ongoing maintenance costs. - **Complexity in Management**: Requires skilled professionals to manage and maintain the infrastructure. - **Limited Scalability**: Scaling might be limited by the organization's physical capacities.

1.4.4 Hybrid Cloud

A hybrid cloud combines private and public clouds, allowing data and applications to be shared between them.

The hybrid cloud model is increasingly popular for its ability to offer flexibility, scalability, and security. By combining public and private clouds, organizations can keep their sensitive data protected in a private cloud while leveraging the cost-effective and scalable resources of the public cloud. This model is complex in terms of management and security but offers unparalleled flexibility. Organizations that want the benefits of both public and private clouds, such as dynamic

scalability along with high-level security for sensitive operations, will find the hybrid model most beneficial.

Pros: - **Flexibility and Scalability**: Offers a mix of scalable resources from the public cloud and secure resources from the private cloud. - **Cost-Effectiveness**: Businesses can optimize their costs by using public resources for less-sensitive operations. - **Business Continuity**: Enhances disaster recovery and business continuity by providing multiple options.

Cons: - **Complexity**: Managing and integrating different platforms can be complex. - **Security Challenges**: Maintaining data security across multiple environments can be challenging. - **Compatibility Issues**: Ensuring compatibility between different cloud services and infrastructures can be difficult.

1.4.5 Making the Right Choice

Choosing the right cloud computing deployment model depends on several factors: the organization's size, budget, security needs, and the specific requirements of their operations. Public clouds are best for flexibility and scalability, private clouds for security and control, community clouds for collaborative environments with similar needs, and hybrid clouds for a balanced approach. Each model presents a different pathway to achieving efficiency, innovation, and agility in the cloud era.

1.4.6 Conclusion

Each cloud computing deployment model offers unique benefits and challenges, and the choice largely depends on the specific needs, budget, and objectives of an organization. While public and private clouds offer simplicity and security, respectively, hybrid clouds offer a balance of shared resources and tailored services.

1.5 Cloud Providers' Global Presence

1.5.1 Introduction

Central to the success of cloud providers is their global infrastructure, which consists of Regions, Availability Zones and Edge Locations. In this chapter, we will explain the significance of these elements and how they contribute to the global presence of cloud providers.

1.5.2 Regions: The Backbone of Global Cloud Providers

A Region, in the context of cloud computing, refers to a geographical area where a cloud provider has established a presence. Each Region is equipped with one or multiple data centers and serves as an independent entity within the cloud ecosystem. These Regions are strategically located around the world to cater to the needs of businesses and users across diverse locations.

Key points about Regions:

1. **Geographic Diversity**: Cloud providers strategically position their Regions across continents to ensure geographic diversity. This redundancy minimizes the risk of data loss or service interruptions due to natural disasters or regional issues.
2. **Data Sovereignty**: Regions allow organizations to store data within specific geographic boundaries, addressing data sovereignty and regulatory compliance requirements.
3. **Latency Optimization**: Proximity to data centers reduces latency, ensuring that users experience minimal delays when accessing cloud services.
4. **High Availability**: Cloud providers replicate services across multiple data centers within a Region to ensure high availability and fault tolerance.

1.5.3 Availability Zones: Enhancing Reliability and Redundancy

Within each Region, cloud providers establish Availability Zones (AZs) as isolated data center facilities. These Availability Zones are designed to operate independently, with redundant power, cooling, and networking infrastructure. They are crucial for enhancing the reliability and fault tolerance of cloud services.

Key points about Availability Zones:

1. **Fault Tolerance**: Services hosted in different Availability Zones are resilient to failures in one zone. In the event of hardware or network issues, services can automatically failover to another Availability Zone.
2. **Scalability**: Availability Zones enable businesses to scale their applications and workloads horizontally, distributing them across multiple zones to handle increased demand.
3. **Disaster Recovery**: Organizations can set up disaster recovery solutions that replicate data and applications across Availability Zones for rapid recovery in case of disruptions.

1.5.4 Edge Locations: Delivering Content and Reducing Latency

Edge Locations are a vital part of cloud providers' global infrastructure, but they differ from Regions and Availability Zones. Instead of serving as independent data centers, Edge Locations are smaller, distributed points of presence located closer to end-users. These locations are strategically placed in cities and regions to optimize content delivery and reduce latency for services like content delivery networks (CDNs).

Key points about Edge Locations:

1. **Content Delivery**: Edge Locations help cache and distribute content, including web pages, videos, and large files, closer to end-users. This accelerates content delivery and reduces load times.
2. **Low Latency**: By bringing content closer to users, Edge Locations minimize the latency experienced when accessing web applications or streaming media.
3. **Improved User Experience**: Faster load times and reduced latency lead to an enhanced user experience, making websites and applications more responsive and enjoyable to use.

1.5.5 Conclusion

The global presence of cloud providers, as manifested through Regions, Availability Zones, and Edge Locations, is essential for delivering reliable, scalable, and low-latency cloud services to users worldwide. This infrastructure enables businesses to expand globally, maintain high availability, optimize content delivery, and meet regulatory compliance requirements. As cloud providers continue to expand their global footprints, users and organizations can expect even greater accessibility and efficiency.

1.6 Common Services

1.6.1 Introduction

Cloud service platforms offer a wide range of services to cater to diverse needs, from infrastructure provisioning to advanced data analytics and machine learning. In this chapter, we will explore the typical services provided by a cloud service platform.

1.6.2 Infrastructure Services

Compute Services: IaaS provides the foundational building blocks of cloud computing. It offers virtualized computing resources, including virtual machines, storage, and networking. Users can provision and manage these resources to build their IT infrastructure in a scalable and cost-effective manner.

Storage Services: Cloud platforms offer various storage options, including object storage, block storage, and file storage. These services allow users to store, retrieve, and manage data efficiently, with features like data replication, versioning, and backup.

Networking Services: Cloud providers offer a suite of networking services, including virtual private clouds (VPCs), load balancers, content delivery networks (CDNs), and domain name system (DNS) management. These services help users configure and manage their network infrastructure for improved performance and security.

1.6.3 Platform Services

Platform as a Service (PaaS): PaaS goes beyond infrastructure to provide a complete development and deployment platform. It offers tools and services for application development, such as databases, application frameworks, and runtime environments. PaaS simplifies the development process, allowing developers to focus on coding instead of managing infrastructure.

Database Services: Cloud platforms offer a variety of managed database services, including relational databases, NoSQL databases, and in-memory databases. These services provide scalable, highly available, and fully managed database solutions for applications of all sizes.

Development Tools: PaaS platforms often include a range of development tools and services, such as integrated development environments (IDEs), code repositories, and continuous integration/continuous deployment (CI/CD) pipelines. These tools streamline the software development lifecycle.

1.6.4 Application Services

Software as a Service (SaaS): SaaS delivers fully functional software applications over the internet. Users can access these applications without the need for installation or maintenance. Popular SaaS examples include office productivity suites, customer relationship management (CRM) software, and collaboration tools.

Content Delivery and Streaming: Cloud providers offer content delivery networks (CDNs) and media services for delivering web content, videos, and live streaming efficiently to global audiences. CDNs cache content at edge locations to reduce latency and improve user experience.

1.6.5 Security and Compliance Services

Identity and Access Management (IAM): Cloud platforms offer robust IAM services to manage user access, permissions, and authentication. IAM helps organizations enforce security policies and ensure data protection.

Security and Compliance Tools: Cloud providers invest heavily in security and compliance, offering tools and services for threat detection, monitoring, and auditing. They also provide compliance certifications for various industry standards and regulations.

1.6.6 Logging and Monitoring Services

Logging Services: Cloud platforms provide logging services that allow organizations to capture and store logs from vari-

ous resources, including applications, virtual machines, and databases. These logs provide a detailed record of events and activities, facilitating troubleshooting, auditing, and compliance.

Monitoring Services: Cloud monitoring services enable organizations to track the performance and health of their cloud resources in real-time. They offer features such as custom alerts, dashboards, and automated scaling based on predefined thresholds. Monitoring helps organizations proactively identify and address issues before they impact users.

1.6.7 Advanced Services

Big Data and Analytics: Cloud platforms provide services for big data storage, processing, and analytics. Users can leverage tools like Hadoop, Apache Spark, and machine learning frameworks to extract insights from vast datasets.

Machine Learning and Artificial Intelligence: Cloud providers offer machine learning services, allowing users to build, train, and deploy machine learning models for various applications, including image recognition, natural language processing, and predictive analytics.

IoT (Internet of Things) Services: Cloud platforms support IoT by providing services for device management, data ingestion, analytics, and real-time processing. These services enable businesses to harness the potential of IoT in various industries.

1.6.8 Conclusion

The services provided by a cloud service platform are extensive and diverse, catering to a wide range of IT needs and business requirements from foundational infrastructure and development tools to advanced analytics, machine learning, and security services.

1.7 The Role of Linux in Cloud Computing

1.7.1 Introduction

Cloud computing has been evolving rapidly, driven by an increasing demand for reliable, scalable, and cost-effective solutions. Linux, an open-source operating system, has become a corner stone in this transformation.

Based on general trends and industry observations up to early 2023, it's reasonable to suggest that Linux might account for around 70-90% of the cloud market, with Windows covering the remaining 10-30%.

This chapter explores the significance of Linux in the cloud domain, examining its dominance and the reasons behind its widespread adoption.

1.7.2 The Rise of Linux in Cloud Computing

Linux's journey to becoming the backbone of cloud computing is a testament to its robust architecture, security, and flexibility. While cloud computing represents a broad spectrum of services, including Software as a Service (SaaS), Platform as a Service (PaaS), and Infrastructure as a Service (IaaS), Linux has established itself as a fundamental component across these models. The versatility of Linux makes it an ideal choice for various cloud applications, from hosting websites and applications to managing large data centers and supporting big data analytics.

1.7.3 Why Linux Dominates the Cloud

Several factors contribute to the dominance of Linux in cloud computing:

1. **Open Source Nature**: Being open-source, Linux offers a cost-effective solution for cloud providers and users. The ability to modify and distribute the code without licensing

fees aligns perfectly with the dynamic nature of cloud environments.

2. **Security and Stability**: Linux is renowned for its robust security features, making it a preferred choice in cloud computing, where data security and system stability are paramount.
3. **Flexibility and Scalability**: The flexibility to work with a wide range of software and hardware and the ability to scale easily makes Linux ideal for the cloud.
4. **Strong Community Support**: The extensive Linux community contributes to its ongoing development and provides a rich resource for troubleshooting and innovation.
5. **Compatibility with Modern Technologies**: Linux's compatibility with contemporary technologies like containerization and microservices architecture (e.g., Docker and Kubernetes) further cements its position in the cloud ecosystem.

1.7.4 Popular Linux Distributions in Cloud Computing

Several Linux distributions have gained popularity in cloud environments, each with unique features catering to different needs. Ubuntu's user-friendliness and strong community support make it a favorite for both cloud and enterprise environments. Red Hat Enterprise Linux (RHEL) offers enterprise-grade reliability and support, making it suitable for critical business applications. Similarly, CentOS, Debian, Fedora, SUSE Linux Enterprise Server, and Amazon Linux each play significant roles, chosen for their stability, security, and compatibility with various applications.

1.7.5 Linux's Impact on Cloud Economics

The economics of cloud computing are significantly influenced by Linux. Its open-source model reduces costs for cloud service providers, translating into more competitive pricing for end-users. Additionally, the ability to customize and optimize Linux for specific cloud workloads leads to efficient resource utilization, further driving down operational costs.

1.7.6 Conclusion

The role of Linux in shaping the cloud computing landscape cannot be overstated. Its open-source nature, security, flexibility, and compatibility with modern cloud technologies make it an indispensable tool in the cloud computing arsenal. As the cloud continues to evolve, Linux's adaptability suggests it will remain at the forefront, driving innovation and efficiency in this ever-expanding field. Whether for large-scale enterprises or small-scale applications, Linux's influence in cloud computing is a key factor in the continued growth and success of cloud technologies.

Several Linux distributions (distros) are particularly popular, each serving different needs and preferences in various environments, including desktops, servers, and cloud computing. Here are some of the most widely-used Linux distros:

1. ***Ubuntu**: Known for its user-friendliness and widespread use, Ubuntu is a popular choice for both desktop and server environments. It's backed by Canonical Ltd. and offers regular updates and a strong community support base. Ubuntu is also a prevalent choice in cloud computing due to its stability and ease of use.*
2. ***Red Hat Enterprise Linux (RHEL)**: RHEL is a major player in enterprise environments, known for its robustness, security, and support. It's a preferred choice for critical business applications and is widely used in the corporate sector. RHEL is a commercial product, offering extensive support and certified compatibility with numerous software applications.*
3. ***CentOS**: Before its shift in strategy in late 2020, CentOS was widely used as a free, community-supported computing platform, functionally compatible with its upstream source, RHEL. Its shift to CentOS Stream, which is a rolling-release version, has led to mixed reactions in the community.*
4. ***Debian**: Praised for its stability and security, Debian is a volunteer-driven project and forms the basis for many other distros, includ-*

ing Ubuntu. It's a popular choice for servers and experienced Linux users.

5. ***Fedora**: As a community-supported counterpart to RHEL, Fedora is known for its innovation and features the latest Linux technology. It's a good choice for those who want the newest developments in the Linux world.*
6. ***SUSE Linux Enterprise Server (SLES)**: SUSE is known for its enterprise-grade reliability and scalability, making it a solid choice for business applications, particularly in Europe.*
7. ***Arch Linux**: Preferred by enthusiasts and advanced users, Arch Linux is known for its simplicity and customization. While not as beginner-friendly, it offers extensive documentation and a rolling release model.*
8. ***Amazon Linux**: Specifically designed for the AWS cloud, Amazon Linux is optimized for performance and security in Amazon's cloud environment.*

The popularity of these distros can vary based on the specific requirements, such as ease of use, stability, support, and community backing. In a cloud or server context, the choices are often dictated by the need for reliability, support, and compatibility with various enterprise applications.

CHAPTER 2

Technical Opportunities

2.1 Automation

2.1.1 Introduction

The advent of cloud computing has introduced a transformative approach to managing IT resources, fostering a new era of efficiency, agility, and innovation. This chapter explores three pillars that underpin automation in cloud infrastructure:

- Infrastructure as Code (IaC)
- Immutable Infrastructure
- Provisioning Automation.

2.1.2 Infrastructure as Code (IaC)

Infrastructure as Code (IaC) is a key practice in cloud computing that involves managing and provisioning computing infrastructure through machine-readable definition files, rather than physical hardware configuration or interactive configuration tools. This paradigm shift means that the same rigor and discipline applied to application code can now be applied to the infrastructure.

The benefits of IaC are manifold. It enables developers and operations teams to automatically manage, monitor, and provision resources through code, which is version controlled, allowing for consistency, repeatability, and rapid deployment. IaC reduces the potential for human error during setup and configuration, ensures that the deployment process is repeatable and standardized, and simplifies the management of infrastructure across various environments.

2.1.3 Immutable Infrastructure

Immutable Infrastructure is a model where components are replaced rather than changed. Once a piece of infrastructure is deployed, it is never modified; if changes are needed, a new piece of infrastructure is created and replaced as a whole. This concept complements IaC and aligns with the microservices architecture approach.

The immutable approach has several advantages. It eradicates configuration drift which is the divergence of environments due to manual changes and updates. By treating infrastructure components as disposable, systems become more predictable, scalable, and secure. Rollbacks are easier as well since one can simply revert to a previous version of the infrastructure definition.

2.1.4 Provisioning Automation

Provisioning automation is about streamlining the process of managing the resources. By automating the provisioning processes, organizations can deploy infrastructure and applications quickly and consistently. This facet of cloud computing is critical for managing large-scale systems and for implementing continuous integration and continuous delivery (CI/CD) pipelines.

Automated provisioning tools work in conjunction with IaC and immutable infrastructure to create a highly efficient workflow. This enables quick setup of development, testing, and production environments that are identical, thereby reducing the “works on my machine” syndrome and improving the quality and reliability of deployments.

2.1.5 Conclusion

The combination of IaC, Immutable Infrastructure, and Provisioning Automation represents a modern approach to IT infrastructure management. These practices enable organizations to build robust, secure, and highly available systems while significantly reducing the operational overhead and risk of human error.

2.2 High Availability

Cloud computing makes it possible to develop IT infrastructures that are not only robust but also adaptive to the changing demands and resilient to failures. High availability, scalability and elasticity, and self-healing mechanisms are fundamental features of such systems. This chapter outlines these three critical topics that shape the backbone of cloud computing architectures.

2.2.1 High Availability

High availability refers to a system's ability to remain accessible and functional for a maximum period, minimizing downtime and ensuring users can always access the services they need. This is crucial for businesses where downtime directly translates to lost revenue and diminished customer trust. High availability is achieved through redundant components, failover mechanisms, and the elimination of single points of failure. Systems are designed to detect failures and seamlessly switch to backup systems without interrupting service. In cloud computing, high availability is facilitated by distributed resources, enabling services to be replicated across multiple geographic locations.

2.2.2 Scalability and Elasticity

Scalability is the capability of a system to handle a growing amount of work, or its potential to be enlarged to accommodate that growth. Elasticity takes scalability further by allowing the system to scale out or in automatically in response to the demand. This means resources are added or removed dynamically, ensuring optimal performance and cost-effectiveness.

In cloud environments, scalability and elasticity are paramount. As demand fluctuates, the infrastructure can automatically scale without manual intervention. For example, during a traffic surge on an e-commerce website, additional virtual servers can be brought online to handle the load, and similarly, they can be decommissioned during low traffic periods.

2.2.3 Self-Healing

Self-healing systems are designed to detect when they are not operating correctly and, without human intervention, make

the necessary adjustments to return to the desired state. This is an advanced feature of modern systems that relies on automation and monitoring tools to continually assess the health of the environment.

In cloud computing, self-healing mechanisms might involve restarting failed instances, replacing unhealthy components with new ones, or automatically redistributing workloads to healthy resources in the event of a failure. This capability not only reduces downtime but also lessens the burden on IT staff, allowing them to focus on more strategic tasks.

2.2.4 Conclusion

High availability, scalability and elasticity, and self-healing are not just beneficial but essential characteristics of cloud computing environments. They enable businesses to deliver a seamless, uninterrupted service experience to their customers. By leveraging these features, companies can ensure their digital assets are robust against failures, flexible to changing demands, and resilient in the face of inevitable system anomalies. As technology continues to advance, these features will become even more sophisticated, further enhancing the reliability and efficiency of cloud-based systems and services.

2.3 Virtualization

2.3.1 Introduction

The concepts of virtualization, containerization, and serverless architecture represent a spectrum of approaches that cater to diverse computing needs while promoting efficiency, flexibility, and scalability. These technologies have become staples in the toolkit of developers and system architects, enabling more sophisticated and agile application deploy-

ment. This chapter explores the nuances of each approach and their respective roles in modern cloud infrastructure.

2.3.2 Virtualization: The Foundation of Modern Computing

Virtualization technology acts as the bedrock of cloud computing by allowing multiple virtual machines (VMs) to run on a single physical server's hardware. Each VM operates independently with its own operating system and applications, thinking it has the host's resources to itself. This abstraction provides significant improvements in resource utilization, as underutilized physical servers can be consolidated into fewer machines, saving costs on hardware and energy.

Beyond mere cost savings, virtualization offers enhanced disaster recovery plans, simplified management through virtual desktop infrastructure (VDI), and better security through isolation between VMs. The technology lays the groundwork for building scalable and resilient IT infrastructures that can easily adapt to changing business needs.

2.3.3 Containerization: Efficiency and Portability at Scale

While virtualization abstracts the hardware, containerization goes a step further by abstracting the operating system. Containers package an application's code together with its libraries, dependencies, and other binaries into a single object. Unlike VMs, which each have their own OS, containers share the host system's kernel but run isolated processes in user space.

Containerization, powered by technologies like Docker and orchestrated by systems like Kubernetes, offers a lightweight alternative to VMs, with faster start-up times and lower overhead. This makes it an ideal choice for microservices architectures, where each service can be deployed in a separate

container. The portability of containers - they can run on any system that supports the containerization platform - simplifies deployment and testing across different environments, enhancing developer productivity and operational efficiency.

2.3.4 Serverless: Ultimate Abstraction and Operational Simplification

Serverless computing represents the pinnacle of abstraction in cloud services, where the cloud provider dynamically manages the allocation of machine resources. This model abstracts server management and infrastructure decisions away from the developers, who can then focus solely on writing code that serves their business logic. The serverless model, which includes Backend as a Service (BaaS) and Function as a Service (FaaS), is inherently event-driven and scales automatically with the application's needs.

The serverless paradigm significantly reduces operational responsibilities and costs, as users are charged based on the execution of functions and the actual consumption of resources, rather than on pre-purchased capacity. It's particularly beneficial for applications with variable workloads, where managing and provisioning servers can be complex and inefficient.

2.3.5 Conclusion

Virtualization, containerization, and serverless computing are not mutually exclusive and often complement each other within an enterprise's cloud strategy.

Virtualization continues to be a reliable solution for full hardware abstraction and is ideal for applications that require full operating systems or need to be isolated at the hardware level.

Containerization strikes a balance by providing OS-level abstraction, optimizing for microservices and DevOps practices, and offering a middle ground between traditional virtualization and serverless computing.

Finally, serverless computing is the epitome of cloud-native development, enabling developers to focus on code rather than infrastructure management.

Together, these technologies empower businesses to create complex, scalable, and efficient cloud architectures that can respond quickly to the market's needs. Virtualization and containerization provide the environments and flexibility necessary for running a wide array of applications, while serverless computing offers a cost-effective, highly scalable platform for event-driven applications.

2.4 Architectural Patterns

2.4.1 Introduction

The evolution of cloud computing has been accompanied by the emergence and convergence of various architectural patterns. These patterns are designed to tackle specific problems in cloud computing, such as scalability, availability, and continuous delivery.

Below is an exploration of some of the most influential architectural patterns:

- Microservices Architecture
- Serverless Architecture
- Multi-Tenancy Architecture
- Load Balanced Architecture
- API Gateway Pattern
- Domain-Driven Design (DDD) and Clean Architecture.

2.4.2 Microservices Architecture

Microservices Architecture is a design approach that structures an application as a collection of loosely coupled services. Each service is fine-grained and implements a single business capability. This pattern allows for:

- **Independent Deployment**: Services can be deployed independently of each other, which speeds up the deployment process and reduces risk.
- **Technology Diversity**: Different services can be written in different programming languages, use different data storage technologies, and have different data management models.
- **Resilience**: If one service fails, it does not bring down the entire application.
- **Scalability**: Individual components can be scaled as needed, without scaling the entire application.

2.4.3 Serverless Architecture

Serverless Architecture refers to applications that significantly depend on third-party services (Backend as a Service - BaaS) or on custom code that runs in ephemeral containers (Function as a Service - FaaS). Key characteristics include:

- **No Server Management**: Developers do not need to manage server infrastructure.
- **Cost-Effectiveness**: You pay only for the runtime of your functions, which can be more cost-effective than paying for idle servers.
- **Scalability**: The cloud provider automatically scales the functions up or down based on the demand.

2.4.4 Multi-Tenancy Architecture

Multi-Tenancy Architecture is a principle where a single instance of the software application serves multiple customers or "tenants." Multi-tenancy allows for:

- **Resource Sharing**: Tenants share the same infrastructure and platforms, which drives down costs.
- **Scalability**: It is easier to scale a single application than to scale multiple instances of the application.
- **Maintenance Efficiency**: Updating and maintaining a single, centralized application is simpler and more efficient.

2.4.5 Load Balanced Architecture

Load Balanced Architecture involves distributing workloads across multiple computing resources. This can maximize performance, reduce response time, and ensure the availability of applications and websites. Load balancing is critical for:

- **High Availability**: If one server fails, the load balancer redirects traffic to the remaining online servers.
- **Scalability**: As demand increases, more servers can be added to the pool without any disruption to services.

2.4.6 API Gateway Pattern

The API Gateway Pattern is an interface that sits between clients and services. It acts as a reverse proxy, routing requests from clients to services. It can handle cross-cutting concerns such as:

- **Authentication and Security**: Verifying user credentials and ensuring that only authorized users can access services.
- **Rate Limiting**: Preventing any single user from overloading the services with too many requests.

- **Analytics**: Gathering data on how services are used.

2.4.7 Domain-Driven Design (DDD)

DDD is a software design philosophy focused on modeling software to match a domain's complexity with which the software is intended to solve problems. DDD is useful for:

- **Complexity Management**: Breaking down a complex domain into manageable parts.
- **Ubiquitous Language**: Creating a common language that all team members can understand.
- **Focused on Core Logic**: Keeping the team focused on the core logic of the application rather than technical details.

2.4.8 Clean Architecture

Clean Architecture is a set of principles aimed at creating software that is easy to manage and test. It emphasizes the separation of concerns, where the software is divided into layers with distinct responsibilities. Benefits include:

- **Independence of Frameworks**: The system is not dependent on the database, UI, or any external agency.
- **Testability**: Business logic can be tested without UI, database, or any external element.

2.4.9 Combining Architectural Patterns

These architectural patterns can be combined to leverage their individual strengths. For instance:

- A **Microservices Architecture** can be implemented with a **Load Balanced Architecture** to ensure each service can handle high traffic and remain available.
- **Serverless Architecture** can be used to run the business logic of microservices, eliminating the need to manage server infrastructure.

- An **API Gateway** serves as the entry point for a microservices-based application, handling authentication and directing requests to the appropriate services.
- **DDD** can guide the design of microservices by aligning them with the domain model.
- **Clean Architecture** can be used within each microservice to ensure its independence from frameworks and databases.

2.4.10 Popularity in Cloud Computing

These patterns are popular in cloud computing due to their ability to provide scalable, resilient, and maintainable solutions in a cost-effective manner. The cloud's inherent characteristics — on-demand resource availability, scalability, and managed services — complement these architectural patterns well.

- **Microservices** thrive in the cloud where services can be easily containerized and orchestrated.
- **Serverless** computing takes full advantage of the cloud's elasticity and consumption-based pricing models.
- **Multi-Tenancy** maximizes resource utilization, which aligns perfectly with the cloud's pay-as-you-go pricing structure.
- **Load Balancing** is simplified through cloud services that can dynamically adjust resources in response to changing loads.
- **API Gateways** offered by cloud providers can manage the complexity of handling requests to various microservices while providing additional layers of security.
- **DDD** and **Clean Architecture** provide a blueprint for building cloud applications that are scalable, robust, and maintainable over time.

The popularity of these patterns in cloud computing is also due to their alignment with the principles of agile development and continuous delivery. They support an iterative approach, enable quick and reliable deployment of new features, and help manage complex systems at scale.

2.4.11 Conclusion

In summary, these architectural patterns are not just technical choices; they embody a philosophy of building systems that are robust, flexible, and capable of adapting to the changing needs of the business. Their adoption in cloud computing has been a key factor in enabling businesses to accelerate their digital transformation and innovate at a rapid pace.

CHAPTER 3

Organizational Aspects

3.1 Organizational Opportunities

3.1.1 Introduction

Cloud computing has fundamentally transformed business operations, creating a plethora of new organizational opportunities while also presenting a unique set of challenges. This shift has been particularly notable as enterprises increasingly rely on cloud services for their day-to-day operations, data management, and strategic initiatives. In this article, we explore the organizational opportunities and challenges brought about by the proliferation of cloud computing.

3.1.2 Organizational Opportunities in Cloud Computing

1. **Cost Efficiency and Capital Expenditure Reduction** Cloud computing has ushered in significant cost savings by transitioning from capital expenditure (CapEx) to operational expenditure (OpEx). Organizations can avoid the upfront costs of purchasing and maintaining IT infrastructure by utilizing pay-as-you-go models offered by cloud providers.
2. **Agility and Innovation** The cloud enables businesses to rapidly deploy and scale applications, which can accelerate development cycles and time-to-market for new products. This agility fosters an environment conducive to innovation, where new ideas can be tested and deployed quickly.
3. **Remote Workforce Enablement** With cloud services, teams can access necessary applications and data from anywhere, facilitating remote work and collaboration. This has been especially valuable in adapting to the changes necessitated by global events such as the COVID-19 pandemic.
4. **Focus on Core Business Functions** By outsourcing IT infrastructure management to cloud providers, organizations can concentrate on their core business functions. This shift allows them to allocate more resources toward strategic initiatives rather than managing IT operations.
5. **Data-Driven Decision Making** The cloud provides advanced analytics tools and vast data storage capabilities, enabling organizations to become more data-driven. They can leverage these tools for improved decision-making and to gain insights into customer behavior and market trends.

6. **Global Expansion** The global reach of cloud providers allows businesses to deploy services in multiple regions quickly, reducing the barriers to global expansion and enabling them to enter new markets more easily.

3.1.3 Organizational Challenges in Cloud Computing

1. **Security and Compliance** As data moves to the cloud, organizations face the challenge of ensuring that their information is secure and compliant with various regulations. The shared responsibility model of cloud security requires a clear understanding of what aspects of security are managed by the provider and what remains the organization's responsibility.
2. **Managing Multi-Cloud Environments** With the adoption of services from multiple cloud providers, managing these environments becomes complex. Organizations must deal with different standards, technologies, and interfaces, which can lead to increased operational complexity.
3. **Cost Management and Optimization** While cloud computing can be cost-efficient, it also presents the challenge of managing and optimizing cloud spending. Without proper governance, organizations can experience budget overruns due to unused or underutilized resources.
4. **Talent Acquisition and Training** There is a growing need for skilled professionals who understand cloud technologies. However, the market is currently facing a skills gap. Organizations must invest in training existing staff or attract new talent to fill these roles.
5. **Data Sovereignty and Localization** Different countries have regulations regarding data sovereignty, which can complicate cloud strategies for multinational companies. Organizations must ensure that data is stored and pro-

cessed in compliance with local laws, which may require the use of data centers in specific locations.

6. **Technical Debt** As organizations migrate legacy systems to the cloud, they may accumulate technical debt if they don't update or re-architect their applications to fully utilize cloud-native features. This can negate some of the cloud's benefits and lead to increased costs and complexity.
7. **Vendor Lock-in** Dependence on a single cloud provider's infrastructure and services can lead to vendor lock-in, making it difficult and potentially expensive for organizations to change providers or bring services back in-house.
8. **Cultural Shift** Moving to the cloud often requires a cultural shift within an organization. Traditional IT departments may resist the change due to concerns about job security or a lack of familiarity with the new technology.

3.1.4 Navigating the Opportunities and Challenges

To fully capitalize on the opportunities presented by cloud computing, organizations must adopt a strategic approach. This includes embracing best practices for cloud security, investing in cloud management tools, and continually training staff. Additionally, businesses should conduct regular reviews of their cloud spending and usage to ensure they are optimizing their investment.

On the challenge front, companies can mitigate risks by adopting multi-cloud strategies, ensuring proper compliance controls are in place, and focusing on upskilling their workforce. By understanding the shared responsibility model of cloud security, organizations can better protect their data and applications. Lastly, fostering a culture that is open to change

and innovation is crucial for a successful transition to cloud-based operations.

3.1.5 Conclusion

Cloud computing offers a paradigm shift that enables organizations to be more flexible, scalable, and focused on their core competencies. However, with these benefits come new challenges that must be managed with careful planning and strategic execution. Organizations that successfully navigate these waters will not only mitigate the risks associated with cloud computing but will also position themselves to take full advantage of the transformational opportunities it provides. As cloud technologies continue to mature, we can expect the landscape of opportunities and challenges to evolve, compelling organizations to adapt and innovate continually.

3.2 Best Practices

3.2.1 Introduction

In the rapidly evolving landscape of software development, methodologies such as Agile development, DevOps, Continuous Improvement, and Continuous Integration/Continuous Deployment (CI/CD) have become foundational. These practices are interlinked, each reinforcing the others to create a synergistic effect that enhances the efficiency and quality of software development. Cloud computing has acted as a catalyst, enabling these practices by providing the necessary tools and infrastructure. Let's explore how these methodologies, their interrelationships, and how cloud computing underpins them.

3.2.2 Agile Development (Agile utveckling)

Agile development is a set of principles for software development under which requirements and solutions evolve through the collaborative effort of cross-functional teams. It promotes adaptive planning, evolutionary development, early delivery, and continual improvement, and it encourages flexible responses to change. The core values and principles of Agile are encapsulated in the Agile Manifesto.

In practice, Agile involves iterative development, where projects are divided into small parts that are completed in work sessions that run from the design phase to testing and quality assurance (QA). These parts are known as iterations. Agile helps teams deliver value faster and with fewer headaches because of its emphasis on real-time communication and feedback.

3.2.3 DevOps

DevOps is a set of practices that combines software development (Dev) and IT operations (Ops), aiming to shorten the system development life cycle and provide continuous delivery with high software quality. DevOps is complementary to Agile; while Agile focuses on the software development life cycle, DevOps brings in the operational perspective.

With DevOps, teams aim for faster deployment frequencies, lower failure rates of new releases, shortened lead time between fixes, and faster mean time to recovery in the event of a new release crashing or otherwise disabling the current system. Automation is a key tenet of DevOps, with the goal of creating a CI/CD pipeline that minimizes manual work and error.

3.2.4 Continuous Improvement

Continuous Improvement is a method to identify opportunities for streamlining work and reducing waste. The practice involves regularly examining and improving your processes. In the context of Agile and DevOps, Continuous Improvement is woven into the fabric of daily operations; teams are always looking for ways to improve performance, enhance quality, and increase efficiency.

This methodology relies on a culture of feedback where every stakeholder is involved in the process of suggesting and implementing improvements. It is a cyclical process of planning, doing, checking, and acting (PDCA), which ensures that improvements are based on systematic analysis and feedback data.

3.2.5 Continuous Integration/Continuous Deployment (CI/CD)

CI/CD is a method to frequently deliver apps to customers by introducing automation into the stages of app development. The main concepts attributed to CI/CD are continuous integration, continuous deployment, and continuous delivery.

- **Continuous Integration**: Developers merge their changes back to the main branch as often as possible. The developer's changes are validated by creating a build and running automated tests against the build. This helps to identify errors quickly.
- **Continuous Deployment**: Each change that passes the automated tests is deployed to the production environment automatically.
- **Continuous Delivery**: An extension of continuous deployment to ensure that you can release new changes to your customers quickly in a sustainable way. This means that on top of automated testing, you have an automated

release process and can deploy your application at any point in time.

3.2.6 Leveraging Each Other

Agile, DevOps, Continuous Improvement, and CI/CD form a close-knit matrix of practices that bolster each other:

- Agile provides the iterative framework that emphasizes customer satisfaction through continuous delivery of valuable software.
- DevOps bridges the gap between development and operations, ensuring that software can be reliably released at any time.
- Continuous Improvement ensures that the Agile and DevOps processes are always evolving and optimizing.
- CI/CD automates the software delivery process, ensuring that software can be rapidly and reliably released at any time.

3.2.7 Enabled by Cloud Computing

Cloud computing provides the on-demand infrastructure and services that are critical for implementing Agile, DevOps, Continuous Improvement, and CI/CD practices. Here's how cloud computing enables these methodologies:

- **Elasticity**: Cloud services can be scaled up or down as needed, which is essential for Agile development that may require rapid changes in resource allocation.
- **Automation**: Cloud environments support the automation tools necessary for CI/CD pipelines, making it easier to deploy and manage applications.
- **Collaboration**: Cloud platforms offer tools that enhance collaboration among distributed teams, a key aspect of Agile development. These include source code repositor-

ies, project tracking tools, and real-time communication applications.

- **Service Availability**: With a wide array of services like computing power, storage, and databases readily available, teams can focus on their core product without worrying about underlying infrastructure, which is a fundamental aspect of DevOps practices.
- **Continuous Monitoring and Analytics**: The cloud offers advanced monitoring and analytics tools, allowing teams to gather insights and feedback necessary for Continuous Improvement.
- **Infrastructure as Code (IaC)**: Cloud computing leverages IaC, which is essential for replicating and fixing environments quickly - a critical component of CI/CD pipelines.

The cloud effectively democratizes the ability to deploy sophisticated development environments, making these best practices accessible to organizations of all sizes.

3.2.8 Conclusion

The convergence of Agile, DevOps, Continuous Improvement, and CI/CD reflects a maturation in software development processes, focusing on speed, efficiency, and quality. Cloud computing not only enables these practices but also accelerates their adoption by providing a dynamic, flexible, and scalable environment. As organizations continue to embrace these intertwined methodologies, they can expect to see improved deployment frequency, more dependable releases, and faster recovery times when issues arise. This leads to enhanced customer satisfaction, which is the ultimate goal of any software development endeavor.

3.3 GitOps: Version-Controlled Infrastructure

3.3.1 Introduction

GitOps is a paradigm that extends DevOps by using Git as a single source of truth for declarative infrastructure and applications. With Git at the center of the delivery pipelines, every change is observable, verifiable, and auditable.

3.3.2 Fundamentals of GitOps

- **Version Control for Everything**: All environment specifications and application configurations are stored in a Git repository. This approach treats infrastructure as code, which can be versioned and audited.
- **Automated Delivery**: Changes to the repository automatically trigger deployments to the environment. This ensures alignment between the code repository and the state of the deployed infrastructure.
- **Merge Requests for Change Management**: Changes to infrastructure and applications are made through merge requests, facilitating peer reviews, discussions, and a record of what changed and why.

3.3.3 The Role of GitOps in Cloud Computing Practices

GitOps is particularly well-suited to cloud computing due to its reliance on dynamic and scalable infrastructure. By managing infrastructure and application deployments using pull requests, organizations can enhance the practices of Agile, DevOps, and CI/CD:

- **Enhancing Agile Practices**: GitOps complements Agile's iterative approach by allowing for fast feedback loops and incremental changes to the infrastructure that hosts the applications developed using Agile methodologies.

- **Complementing DevOps Culture**: GitOps fits naturally into the DevOps ethos of collaboration and communication, bridging gaps between development and operations. The use of Git as the single source of truth helps maintain consistency across the entire pipeline, from development to production.
- **Driving Continuous Improvement**: With the Git repository at the core, the Continuous Improvement process benefits from a clear audit trail of what changes were made, who made them, and why. This historical data supports better decision-making and process optimization.
- **Integrating with CI/CD**: GitOps can be integrated into CI/CD workflows, where the CI system pushes the built artifacts into a Git repository, and the CD system watches for changes to deploy. This ensures that only the changes that pass the CI pipeline are deployed to production, maintaining the integrity of the deployment process.

3.3.4 Benefits of GitOps

The benefits of GitOps in cloud environments are multifold:

- **Improved Productivity**: Developers can use familiar tools like Git without needing to learn different interfaces for deployment and operations, which streamlines workflows.
- **Enhanced Security**: The use of pull requests and code reviews adds layers of security checks, while version control provides an audit trail for compliance and governance.
- **Faster Recovery**: Should a failure occur, GitOps enables quick and reliable recovery by reverting to a previous state in the Git history, making disaster recovery as simple as a `git revert`.

- **Automated Synchronization**: The declarative nature of GitOps means the desired state of the system is in Git, and any drift in the actual state can be automatically corrected.

3.3.5 Conclusion

Incorporating GitOps into Agile, DevOps, Continuous Improvement, and CI/CD practices provides a robust framework for managing the complexities of modern cloud-native environments. It ensures that the benefits of these practices are maximized, leading to more secure, stable, and efficient development cycles. As organizations strive to streamline their cloud operations, GitOps stands out as a key strategy that can help maintain order within the fast-paced and ever-evolving cloud landscape.

3.4 Financing Cloud Computing

3.4.1 Introduction

In the domain of cloud computing, two crucial financial and operational concepts are frequently at the forefront of strategic planning: Operational Expenditure (OPEX) vs. Capital Expenditure (CAPEX). Understanding and optimizing these two can significantly impact an organization's agility, efficiency, and competitive edge. This article will explore both concepts in detail, particularly how they pertain to cloud computing.

3.4.2 OPEX and CAPEX

The distinction between OPEX and CAPEX is foundational in financial accounting and plays a significant role in cloud computing strategies.

Capital Expenditures (CAPEX) refer to the funds used by a company to acquire, upgrade, and maintain physical assets such as property, buildings, technology, or equipment. In traditional IT infrastructure, CAPEX is a significant component, as companies need to invest heavily in data centers, servers, networking hardware, and software licenses before they can start or expand operations.

Operational Expenditures (OPEX), on the other hand, are the costs for a company to run its day-to-day operations. These expenses include rent, utilities, and in the context of IT, the ongoing costs for services and resources that are consumed.

3.4.3 Cloud Computing's Impact on OPEX/CAPEX

Cloud computing has shifted the paradigm from a CAPEX-heavy investment model to one that favors OPEX. This shift is driven by the cloud's service-oriented model, where computing resources are rented, not owned. Here's how cloud computing impacts both:

- **Reduced Upfront Investments**: With cloud services, the need for upfront investment in physical infrastructure is significantly reduced or eliminated. Organizations can access state-of-the-art infrastructure on a pay-as-you-go basis.
- **Predictable Operating Costs**: Cloud services often come with predictable payment plans, which means businesses can plan their budgets with more accuracy. OPEX becomes a more consistent line item in financial planning.
- **Tax Efficiency**: CAPEX often involves depreciation of assets over several years, while OPEX can typically be written off in the same fiscal year they are incurred, which can be more tax-efficient for companies.

- **Cash Flow Management**: Shifting to an OPEX model helps in better cash flow management since it avoids large one-time expenses and spreads costs over time.

However, it's worth noting that while the cloud can decrease CAPEX, it doesn't eliminate it entirely—there may still be some initial costs associated with migration and setup. Additionally, without proper management, OPEX can grow unchecked as more services are consumed.

3.4.4 Conclusion

The strategic management of OPEX vs. CAPEX in cloud computing is essential for organizations looking to harness the cloud's full potential. By shifting to an OPEX model, companies can enjoy lower upfront costs and greater scalability. As cloud technologies continue to advance, organizations that master these financial and operational domains will be well-positioned to thrive.

3.5 Capacity Management

3.5.1 Introduction

Capacity management is a critical component in the IT strategy of any organization, especially in the context of cloud computing. It refers to the process of planning, managing, and optimizing the use of IT resources to ensure that the right level of capacity is provided to meet service demands at the right time and cost.

3.5.2 Key Elements of Capacity Management

- **Resource Allocation**: The process involves determining the precise amount of computing resources needed—such

as CPU, memory, and storage—to support application workloads effectively.

- **Performance Monitoring**: Continuous monitoring of system performance is vital to ensure that allocated resources meet current service level agreements and to adjust allocations as workload demands change.
- **Scalability**: A core benefit of cloud computing is scalability. Capacity management in the cloud ensures that resources can be scaled up to handle high demand or scaled down to save costs when demand is low.
- **Demand Forecasting**: This involves using predictive analytics to forecast future demand and resource requirements, helping to prevent both overprovisioning, which can lead to unnecessary costs, and underprovisioning, which can lead to poor system performance and user dissatisfaction.

3.5.3 Capacity Management in the Cloud

Cloud computing has transformed capacity management by offering dynamic scalability and flexible resource provisioning.

- **Elasticity**: The cloud's elastic nature means that resources can be automatically adjusted based on real-time demand, ensuring efficient utilization.
- **Automation**: Many cloud services feature automated scaling, which simplifies capacity management by automatically adjusting resources without manual intervention.
- **Cost Optimization**: By closely aligning resource consumption with demand, organizations can optimize their spending, paying only for what they use and avoiding the costs associated with idle resources.

- **Disaster Recovery**: Cloud computing offers robust options for backup and recovery, enhancing an organization's capacity to manage resources effectively in crisis situations.

3.5.4 Challenges in Capacity Management

Despite the advantages, there are challenges that organizations must navigate:

- **Complex Cost Structures**: The pricing models of cloud services can be intricate, making it challenging to forecast and manage costs effectively.
- **Resource Sprawl**: Without proper governance, the ease of spinning up new resources can lead to uncontrolled growth and spiraling costs.
- **Compliance and Data Management**: Navigating the compliance landscape, especially when scaling across regions, can be complex and requires diligent capacity planning.

3.5.5 Conclusion

Effective capacity management is essential for leveraging the full benefits of cloud computing. By ensuring that IT resources are optimized to meet service demands, organizations can improve their operational efficiency, reduce costs, and maintain high levels of service quality. As cloud technologies continue to mature, capacity management will remain a critical discipline for organizations aiming to maximize their cloud investments.

CHAPTER 4

Cloud Native Development

4.1 Key Aspects of Cloud Native Development

"Cloud native" is a term used in the context of cloud computing and application development to describe a style of building and running applications that fully exploits the advantages of the cloud computing model. Here are the key aspects of cloud-native applications:

1. **Designed for the Cloud:** Cloud-native apps are specifically built for cloud environments. This means they're designed to run in the scalable, flexible, and automated computing environments provided by cloud service platforms, like AWS, Azure, or Google Cloud. They are designed to handle

frequent changes in load and to be deployed across multiple servers or even multiple data centers.

2. **Elastic Scalability:** Unlike traditional applications, which might require significant manual effort to scale, cloud-native apps can automatically scale up or down in response to demand. This is facilitated by the cloud infrastructure, which allows for rapid provisioning and de-provisioning of resources.
3. **Resilience and Self-healing:** Cloud-native apps are designed to be resilient to failures. They can tolerate and recover from infrastructure failures, network issues, and other common disruptions. Techniques like circuit breakers, load balancing, and automatic restarts help in maintaining high availability.
4. **Utilization of Cloud-Specific Services:** Cloud-native apps often take advantage of specific services offered by cloud providers, like managed databases, AI and machine learning services, and serverless computing capabilities. This reliance on cloud-specific services is a clear differentiator from traditional applications. These services are often referred to as *Managed Services*
5. **Containers and Orchestration:** Cloud-native apps typically use containers, such as Docker, to package and run services. Containers encapsulate the microservice and its dependencies, making it easy to move the service between environments. Orchestration tools like Kubernetes are used to manage these containers – deploying, scaling, and networking them across the cloud.
6. **DevOps and Agile Methodologies:** While DevOps and Agile can be applied in traditional settings, they are almost a necessity in cloud-native development. This integration

results in a culture and practice that enhances collaboration, flexibility, and speed in the development process.

7. **Continuous Delivery and Deployment:** Cloud-native development typically involves a CI/CD pipeline, enabling frequent and reliable software updates. This approach is more integral and streamlined in cloud-native applications compared to traditional ones, where updates might be more infrequent and involve significant downtime.
8. **API-Based Communication:** In a cloud-native environment, services communicate through APIs. This ensures loose coupling between services, making it easier to modify and scale individual components without affecting the whole system.
9. **Infrastructure as Code (IaC):** Cloud-native apps often use IaC, which means the infrastructure is provisioned and managed using code, enabling automation and consistency. While IaC can be used in traditional environments, it is a fundamental part of cloud-native deployment.
10. **Service Oriented:** Even if not strictly following a microservices architecture, cloud-native apps tend to be more service-oriented and modular compared to traditional monolithic applications. This modularity facilitates easier updates, maintenance, and scalability.
11. **Pay-as-you-go Pricing Model:** Cloud-native applications benefit from the cloud's pay-as-you-go pricing model. This allows for cost savings as you pay only for the resources you use, which is generally not the case with traditional applications where resources are statically allocated.

Cloud-native apps are designed for virtualized cloud environments, using modern architectural patterns and operational strategies to maximize the benefits of the cloud. This approach enhances flexibility, scalability, and speed, thereby

allowing businesses to respond more rapidly to market changes and customer needs.

4.2 Architectural Patterns For Cloud Native Apps

Modern architectural patterns play a crucial role in the development of cloud-native applications. These patterns help in achieving scalability, resilience, and agility, which are essential for applications operating in cloud environments. Here are some key examples:

1. **Microservices Architecture:** Perhaps the most fundamental pattern in cloud-native applications, microservices involve decomposing applications into smaller, independent services. Each service performs a specific function and communicates with others via APIs. This pattern promotes scalability, easier maintenance, and faster development cycles.
2. **Serverless Architecture:** In a serverless model, developers write application code without concerning themselves with the underlying infrastructure. Cloud providers manage the servers and dynamically allocate resources. This model is highly scalable and cost-effective, as you pay only for the resources you use.
3. **Event-Driven Architecture:** This pattern is designed around the production, detection, consumption, and reaction to events. It enables applications to be highly responsive and scalable, as services can independently react to events (like a user action or a data update) without waiting for other services to complete their tasks.
4. **Containerization:** Using containers, applications and their runtime environment (including libraries and dependen-

cies) are packaged together. This ensures consistency across different environments and simplifies deployment and scaling. Docker is a popular platform for containerization.

5. **Orchestration with Kubernetes:** Kubernetes is an orchestration system for containers. It automates the deployment, scaling, and management of containerized applications. This pattern is essential for managing complex applications with many moving parts.
6. **API Gateway:** An API gateway is a management tool that sits between a client and a collection of microservices. It acts as a reverse proxy to route requests, simplify API management, and enforce security policies. This pattern is key in handling different API requests efficiently.
7. **Circuit Breaker Pattern:** This pattern prevents a failure in one service from cascading to other services. When a microservice fails to respond, the circuit breaker trips, and the call to the failing service is redirected or an alternative workflow is executed.
8. **Sidecar Pattern:** In this pattern, each microservice is paired with a sidecar container which runs alongside the main container, providing additional features like logging, monitoring, and configuration management. This allows developers to offload some of the application infrastructure concerns.
9. **Service Mesh:** In a service mesh, communication between service instances is managed by a dedicated layer of infrastructure. This provides capabilities like service discovery, load balancing, encryption, and traceability without changing the application code.
10. **Immutable Infrastructure:** This pattern treats infrastructure components as replaceable and disposable. Instead of

updating existing servers, new, pre-configured servers replace them. This approach increases consistency and reliability in the deployment process.

These patterns, individually and in combination, contribute significantly to the agility, resilience, and efficiency of cloud-native applications, making them well-suited for the dynamic and distributed nature of cloud computing environments.

4.3 The 12-Factor Model

The 12-factor model is a set of principles for developing cloud native applications that are scalable, maintainable, and deployable on modern cloud platforms. Developed by Heroku, it's a widely respected methodology for developing web applications. Let's explore these 12 factors and then see how they correlate with modern architectural patterns like microservices, serverless architecture, etc., and contrast with other models.

The 12-Factor Model:

1. **Codebase:** One codebase tracked in revision control, many deploys.
2. **Dependencies:** Explicitly declare and isolate dependencies.
3. **Config:** Store config in the environment.
4. **Backing Services:** Treat backing services as attached resources.
5. **Build, Release, Run:** Strictly separate build and run stages.
6. **Processes:** Execute the app as one or more stateless processes.
7. **Port Binding:** Export services via port binding.

8. **Concurrency:** Scale out via the process model.
9. **Disposability:** Maximize robustness with fast startup and graceful shutdown.
10. **Dev/prod Parity:** Keep development, staging, and production as similar as possible.
11. **Logs:** Treat logs as event streams.
12. **Admin Processes:** Run admin/management tasks as one-off processes.

4.3.1 Correlation with Modern Architectural Patterns

- **Microservices Architecture:** The 12-factor model emphasizes decentralized, independent processes, similar to microservices. Each factor like codebase, dependencies, and config supports the development of small, loosely coupled services.
- **Serverless Architecture:** The principles of disposability, stateless processes, and environment-specific configurations align well with the serverless model where infrastructure management is abstracted away.
- **Containerization and Orchestration:** The model's emphasis on stateless processes and disposability is similar to how containerized applications are designed and orchestrated, for example, using Docker and Kubernetes.
- **Event-Driven Architecture:** While not directly addressed, the principles of the 12-factor model, like disposability and port binding, support the development of event-driven architectures by facilitating loosely coupled services.

4.3.2 Contrast with Other Models

1. **Traditional Monolithic Applications:** Contrary to the 12-factor model, traditional monolithic applications often

have a single codebase that is not designed for cloud environments, tightly coupled components, shared state, and often lack clear separations between build, release, and run stages.

2. **Non-Cloud-Native Applications:** These might not adhere to factors such as environment-specific configurations, disposability, and strict separation of build and run stages. The 12-factor model is specifically tailored for cloud environments, whereas non-cloud-native apps may be designed with different considerations in mind.
3. **Enterprise Service Bus (ESB) Based Architectures:** ESB architectures, often found in SOA (Service Oriented Architecture), contrast with the 12-factor model in that they tend to have more centralized governance and control, different approaches to handling dependencies and configurations, and less emphasis on disposability and dev/prod parity.

4.3.3 Conclusion

The 12-factor model is a set of best practices for building applications that are optimized for cloud environments, promoting scalability, maintainability, and agility. It aligns well with modern architectural patterns like microservices and serverless, and contrasts with more traditional, monolithic, or non-cloud-native application development models. The model's emphasis on factors such as codebase management, dependencies, configuration, statelessness, and process disposability makes it particularly suitable for the dynamic, scalable nature of cloud computing.

PART II

Infrastructure

The infrastructure layer underneath every cloud application: how compute is virtualized, how networks are organized, how storage and databases work, and how all of it can be expressed as code.

PART II · INFRASTRUCTURE

CHAPTER 5

Compute

5.1 Azure VM Sizing and Cost (Azure)

Selecting a virtual machine size determines both the performance characteristics and the cost of cloud infrastructure. An undersized VM cannot handle the workload; an oversized VM wastes money on unused capacity. Understanding how VM resources map to workload requirements—and how cloud platforms price these resources—enables matching infrastructure to application needs efficiently.

5.1.1 VM Resource Dimensions

Every virtual machine provides three primary resources that determine its capabilities.

Processing Power (vCPUs)

The number of virtual CPUs determines how much parallel computation the VM can perform. More vCPUs enable handling more concurrent operations—more web requests, more database queries, more data processing threads.

CPU-intensive workloads—compiling code, processing data, rendering, scientific computation—benefit from additional vCPUs. Web servers handling many concurrent requests also benefit, as each request can execute on a separate core. A single-threaded application gains nothing from additional vCPUs beyond one.

Memory (RAM)

Memory capacity determines how much data the VM can hold in fast-access RAM. Applications access RAM in nanoseconds; accessing disk takes milliseconds—a difference of several orders of magnitude.

Memory-intensive workloads include databases (holding indexes and hot data in memory), caching systems (Redis, Memcached), and applications with large working datasets. A VM that exhausts its memory begins swapping to disk, degrading performance severely.

Storage (Disks)

Storage provides persistent capacity for the operating system, applications, and data. Two characteristics matter: capacity (how much data) and performance (how fast operations complete).

Storage performance is measured in IOPS (input/output operations per second) and throughput (megabytes per second). Database servers and applications with heavy disk access require high IOPS; workloads transferring large files require high throughput.

5.1.2 Azure VM Series

Azure organizes VMs into series based on resource ratios and intended use cases. Each series provides a specific balance of vCPU, memory, and storage optimized for different workload types.

B-Series: Burstable

B-series VMs provide a baseline CPU performance level and accumulate credits during idle periods. When the workload demands more CPU, the VM bursts above baseline using accumulated credits.

This model suits workloads with variable CPU usage—development environments, small websites, applications with occasional spikes. During idle periods, the VM accumulates credits at low cost. During activity spikes, it uses those credits for full CPU performance.

SIZE	VCPUS	MEMORY	BASELINE CPU	APPROXIMATE MONTHLY COST
Standard_B1s	1	1 GB	10%	~$8
Standard_B1ms	1	2 GB	20%	~$15
Standard_B2s	2	4 GB	40%	~$30
Standard_B2ms	2	8 GB	60%	~$60

B-series is not suitable for workloads requiring sustained high CPU usage—the credits deplete and performance drops to baseline. For continuous high-CPU workloads, use D-series or F-series instead.

D-Series: General Purpose

D-series VMs provide a balanced ratio of vCPU to memory (approximately 1:4) with consistent performance. No burstable credits—the VM delivers its full rated performance continuously.

This series suits most production workloads: web servers, application servers, development environments with consistent load, small to medium databases.

SIZE	VCPUS	MEMORY	APPROXIMATE MONTHLY COST
Standard_D2s_v5	2	8 GB	~$70
Standard_D4s_v5	4	16 GB	~$140
Standard_D8s_v5	8	32 GB	~$280

D-series is the default choice for workloads without specific optimization requirements.

E-Series: Memory Optimized

E-series VMs provide a high memory-to-vCPU ratio (approximately 1:8), optimized for workloads that require large amounts of RAM relative to compute power.

This series suits databases holding large datasets in memory, in-memory caching systems, and analytics workloads processing large data volumes.

SIZE	VCPUS	MEMORY	APPROXIMATE MONTHLY COST
Standard_E2s_v5	2	16 GB	~$90
Standard_E4s_v5	4	32 GB	~$180
Standard_E8s_v5	8	64 GB	~$360

F-Series: Compute Optimized

F-series VMs provide a high vCPU-to-memory ratio with the fastest per-core performance, optimized for compute-intensive workloads.

This series suits batch processing, scientific modeling, gaming servers, and other CPU-bound applications that benefit from raw compute power rather than large memory.

Costs vary by region. Check the Azure pricing calculator for current rates in your region.

5.1.3 Selecting VM Size

Matching VM size to workload requirements involves identifying the workload type, estimating resource needs, and right-sizing based on actual usage.

Identify Workload Type

Different workloads have different resource profiles:

WORKLOAD TYPE	PRIMARY RESOURCE NEED	RECOMMENDED SERIES
Development/Test	Variable CPU, cost-sensitive	B-series
Web server	Balanced CPU and memory	D-series
Application server	Balanced CPU and memory	D-series
Database server	Memory, storage I/O	E-series
Batch processing	CPU	F-series
Caching	Memory	E-series

Estimate Resource Requirements

For a simple web application (Flask with nginx): - vCPUs: 1-2 (handles typical traffic patterns) - Memory: 1-2 GB (Flask processes are lightweight) - Storage: 30 GB Standard SSD

A Standard_B1s or Standard_B1ms suits development; Standard_D2s_v5 provides headroom for production.

For a production database server: - vCPUs: 4-8 (handles concurrent queries) - Memory: 32-64 GB (holds indexes and working data) - Storage: Premium SSD (low-latency queries)

A Standard_E4s_v5 or Standard_E8s_v5 provides the memory-to-CPU ratio databases require.

Start Small and Scale

Predicting exact resource requirements before deployment is difficult. A better approach: start with a smaller size, mon-

itor actual resource usage, and scale based on observed metrics.

1. Deploy with a conservative size estimate
2. Monitor CPU, memory, and storage metrics
3. Scale up if utilization consistently exceeds 70-80%
4. Scale down if utilization remains consistently low

Azure allows resizing VMs with a brief restart. This flexibility enables adjusting size based on actual usage rather than predictions.

5.1.4 Understanding VM Costs

VM costs comprise multiple components beyond the compute charge.

Cost Components

COMPONENT	DESCRIPTION	BILLING MODEL
Compute	vCPU and memory	Per hour or per second
OS Disk	Boot disk storage	Per GB per month
Data Disks	Additional storage	Per GB per month
Public IP	Static IP address	Per hour
Network Egress	Outbound data transfer	Per GB

Example Cost Calculation

A Standard_B1s in North Europe:

COMPONENT	SPECIFICATION	MONTHLY COST
Compute	1 vCPU, 1 GB RAM	~$8
OS Disk	30 GB Standard SSD	~$5
Public IP	Static	~$3
Total		**~$16/month**

Network egress adds to this cost when users download data from the VM. Ingress (data uploaded to the VM) is free.

Storage Tiers

Storage type significantly affects both performance and cost:

TIER	USE CASE	IOPS	RELATIVE COST
Standard HDD	Backups, archives	500	$
Standard SSD	General workloads	500-6000	$
			||*PremiumSSD*| *Productiondatabases*|120 – 20000|
			$

Premium SSD costs approximately 2-3x more than Standard SSD for the same capacity, but delivers substantially higher IOPS for I/O-intensive workloads.

OS Disks and Data Disks

Azure VMs use two categories of disks with different purposes.

The **OS disk** contains the operating system and is created automatically when provisioning a VM. For Linux VMs, this

disk is typically 30 GB. The OS disk persists with the VM—deleting the VM can optionally delete the OS disk.

Data disks are additional storage attached to a VM for application data, databases, or other content that should be stored separately from the operating system. Data disks have their own lifecycle—they can be detached from one VM and attached to another, or preserved when a VM is deleted.

Separating data from the OS disk provides several benefits: - Data persists independently of the VM lifecycle - Data disks can be resized without affecting the OS - Different performance tiers can be applied (Premium SSD for database, Standard SSD for logs) - Backups and snapshots can target specific disks

For production workloads, store application data on data disks rather than the OS disk.

Managed Disks

Azure offers two disk management models: Managed Disks and unmanaged disks (storage account blobs).

Always use Managed Disks for new deployments. Managed Disks provide: - Automatic storage account management (no storage account limits to monitor) - Higher availability through automatic placement across fault domains - Simpler configuration and management - Support for disk snapshots and Azure Backup integration

Unmanaged disks are a legacy approach that required manually managing storage accounts and their limits. There is no advantage to unmanaged disks for new deployments.

5.1.5 Cost Optimization

Cloud infrastructure costs accumulate quickly across multiple VMs and services. Several strategies reduce costs without sacrificing required performance.

Right-Size VMs

Over-provisioning "just in case" wastes money continuously. Monitor actual resource usage and downsize VMs that consistently show low utilization. A VM running at 10% CPU utilization could likely run on a smaller, cheaper size.

Use B-series for variable workloads instead of paying for consistent capacity that goes unused most of the time.

Use Reserved Instances

Reserved Instances provide significant discounts (up to 72%) in exchange for committing to 1-year or 3-year terms. If you know a VM will run continuously for an extended period, reserved pricing reduces costs substantially.

Reserved instances suit stable production workloads. They are not appropriate for temporary or experimental deployments where the commitment cannot be honored.

Stop VMs When Not Needed

Development and test VMs do not need to run continuously. Stopping (deallocating) a VM stops compute charges while preserving the disk and configuration. Only storage and static IP costs continue.

```
# Stop and deallocate VM (stops compute billing)
az vm deallocate --resource-group MyResourceGroup --name
        MyVM

# Start VM when needed
az vm start --resource-group MyResourceGroup --name MyVM
```

Azure Automation or Azure DevTest Labs can automate start/stop schedules for development environments.

Choose Appropriate Storage

Match storage tier to actual requirements. Premium SSD costs significantly more than Standard SSD—use it only for workloads that require the additional IOPS. Archive infrequently accessed data to Standard HDD or blob storage rather than keeping it on expensive disk tiers.

Monitor Costs

Azure Cost Management provides visibility into spending by resource, resource group, and subscription. Set budgets and alerts to notify when spending approaches limits. Review spending regularly to identify optimization opportunities.

The Azure Advisor service provides specific recommendations for cost optimization, including identifying underutilized VMs and recommending reserved instance purchases.

Estimate Costs Before Deployment

The **Azure Pricing Calculator** (https://azure.microsoft.com/pricing/calculator/) enables estimating costs before deploying resources. Configure VM size, storage, networking, and region to see projected monthly costs. Compare different configurations to find the best balance of performance and cost for your requirements.

5.1.6 Network Cost Considerations

Network costs can be significant for applications that transfer substantial data.

Ingress is free: Data uploaded to Azure (user form submissions, API requests) incurs no network charges.

Egress is charged: Data leaving Azure (web pages served to users, files downloaded, API responses) is billed per GB. Rates vary by volume and region.

To minimize egress costs: - Keep related resources in the same region (intra-region traffic is free or very cheap) - Com-

press responses (gzip reduces transfer size) - Use CDN for static content (CDN egress is often cheaper than VM egress) - Cache content at the edge to reduce origin requests

5.1.7 Practical Recommendations

Development and Learning

- **Size**: Standard_B1s or Standard_B1ms
- **Disk**: 30 GB Standard SSD
- **IP**: Dynamic (free) or static if needed
- **Cost**: ~$8-20/month

Stop VMs when not actively using them. Delete resource groups when projects complete. Use free tier credits if available.

Production Web Application

- **Size**: Standard_D2s_v5 (scale up based on monitoring)
- **Disk**: Premium SSD for database, Standard SSD for application
- **IP**: Static public IP
- **Cost**: ~$80-150/month

Monitor metrics and right-size based on actual usage. Consider reserved instances for stable long-term workloads. Implement auto-scaling for variable traffic patterns.

5.1.8 Summary

Selecting VM size requires matching workload resource requirements to VM capabilities. B-series suits variable workloads; D-series provides general-purpose balance; E-series optimizes for memory; F-series optimizes for compute. Start with conservative sizing, monitor actual usage, and adjust based on metrics.

VM costs include compute, storage, public IP, and network egress. Cost optimization involves right-sizing based on utilization, using reserved instances for stable workloads, stopping idle VMs, and choosing appropriate storage tiers. Monitoring costs and setting budgets prevents unexpected spending.

The Standard_B1s provides adequate resources for development and learning at minimal cost. Production workloads typically start with D-series general-purpose VMs, scaling or changing series as monitoring reveals actual resource requirements.

PART III

Application Development

The application-development half of the BCD course rests on six load-bearing ideas: how HTTP carries requests and responses, what the .NET platform provides, how the MVC pattern organizes a web application, how the three-tier architecture separates responsibilities, how configuration travels with an application across environments, and how dependency injection wires the pieces together at startup.

The chapters in this Part run in reading order and build on each other. The companion exercise is the Webapp

Development chapter, which puts every concept into practice in a single growing ASP.NET Core MVC project.

PART III · APPLICATION DEVELOPMENT

CHAPTER 6

HTTP Fundamentals

Watch the presentation
Se presentationen på svenska

A web application is useless until something on the network can reach it and ask it to do work. Two unrelated programs — a browser written in C++ and a web server written in C# — must agree on the exact bytes that travel between them, the order those bytes arrive in, and the meaning of every field. That agreement is HTTP, the protocol that every chapter in Part III either uses directly or sits on top of. Understanding the request-response cycle, the methods, the status codes, and the URI structure makes the rest of the application stack legible: routing, model binding, controller actions, and

authentication all become refinements of the basic shape introduced here.

6.1 Why HTTP exists

HTTP sits on top of the TCP/IP stack covered in Part II and inherits the client-server model from the same Part. TCP delivers an ordered, reliable stream of bytes between two endpoints, but it has no opinion about what those bytes mean. A browser requesting a page and a database client retrieving a row both speak TCP, yet they cannot understand each other. An application protocol layered on top of TCP fixes the format of messages so that two programs from different vendors can interoperate.

HTTP (Hypertext Transfer Protocol) is the request-response protocol that browsers and APIs use to exchange data over a TCP connection. A client sends an HTTP request specifying a method, URI, and optional body, and the server returns an HTTP response with a status code, headers, and optional body. Every page load, every form submission, every API call from a mobile app, every health probe from an Azure load balancer follows this same shape.

Three properties of HTTP shape the rest of this Part:

- **Text-based message format.** Requests and responses are human-readable strings, which makes debugging with `curl` and browser developer tools tractable.
- **Statelessness.** Each request is processed in isolation; the server keeps no memory of previous requests on the same connection. State that needs to persist (a logged-in user, a shopping cart) is reconstructed from headers, cookies, or tokens carried with each request.
- **Uniform interface.** Every interaction uses the same small vocabulary — methods, URIs, status codes, headers — so a

generic component (a browser, a proxy, a CDN) can handle traffic for any application without knowing its internals.

These properties are why HTTP scales from a single laptop running `dotnet run` to global infrastructure handling millions of concurrent connections.

6.2 The request-response cycle

A single HTTP exchange has four steps. The client opens a TCP connection to the server's host and port and writes a request message into that connection. The server reads the request, dispatches it to the right piece of application code, and writes a response message back. The client reads the response and acts on it — rendering HTML, parsing JSON, following a redirect, or showing an error.

An **HTTP request** is a message sent by a client to a server, consisting of a method (such as `GET` or `POST`), a target URI, headers describing the request, and an optional body carrying data. The first line names the method and the request target. Headers follow as `Name: Value` pairs, one per line. A blank line separates the headers from the body, which may be empty.

An **HTTP response** is the message a server returns after handling a request, consisting of a status code, headers describing the response, and an optional body carrying the requested representation. The first line of the response carries the protocol version, the numeric status code, and a short text reason phrase. Headers describe the body that follows — its content type, its length, its cache rules. The body itself contains the HTML, JSON, image bytes, or whatever representation the URI resolves to.

A minimal request and the response it triggers look like this:

```
GET /Home/Index HTTP/1.1
Host: localhost:7240
Accept: text/html

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 1832

<!DOCTYPE html><html>...
```

The exchange is short, textual, and self-describing. A network capture, a `curl -v` run, or the Network tab in browser developer tools all show the same fields. This transparency is a deliberate design choice: any tool that can read text can inspect HTTP traffic.

6.3 HTTP methods

An **HTTP method** specifies the action a client wants to perform on the resource identified by the URI; the most common are `GET` (retrieve), `POST` (create or submit), `PUT` (replace), `PATCH` (partial update), and `DELETE` (remove). The method appears as the first token of the request and tells the server how to interpret the URI and the body.

Most browser traffic is `GET` and `POST`. Programmatic clients and APIs add `PUT`, `PATCH`, and `DELETE` to model the full lifecycle of a resource. The method is not a free-form label — servers, proxies, and caches all read it and behave differently based on its declared semantics.

6.3.1 GET versus POST

The distinction between `GET` and `POST` is the question students encounter first, and it is the one most often answered

imprecisely. The two methods differ in three ways that matter at every level of the stack.

Where the data goes. A `GET` request encodes its parameters in the URI as a query string — `/products?category=books&page=2`. A `POST` request carries data in the request body, separate from the URI. Body data is not visible in the URL bar, is not stored in browser history, and is not written to most server access logs.

What the request promises. `GET` is *safe* and *idempotent*: the server should not change state in response to a `GET`, and repeating the same `GET` should yield the same outcome. `POST` is neither safe nor idempotent. A `POST` is expected to change server state — create a record, charge a card, send an email — and repeating it produces a new effect each time. Browsers and proxies rely on this contract: a `GET` is freely cacheable, prefetchable, and replayable, while a `POST` triggers a confirmation dialog when the user tries to refresh the page after submitting a form.

How they interact with the URL. A `GET` produces a bookmarkable, shareable link, because the full request (method plus URI) is encoded in the address bar. A `POST` cannot be bookmarked: the body is gone after the request completes. Form submissions that mutate data therefore use `POST`; search forms that filter a listing use `GET`.

The practical guidance follows directly from the contract. Use `GET` to fetch and display data. Use `POST` to submit data that creates, updates, or otherwise mutates state. Sending a credit card number as a query parameter on a `GET` request is wrong twice — the data ends up in browser history and proxy logs, and the action it triggers is not safely repeatable.

6.3.2 Idempotency in practice

A method is *idempotent* when sending the same request many times has the same effect on the server as sending it once. `GET`, `PUT`, and `DELETE` are idempotent; `POST` and `PATCH` are not. The distinction matters when a request fails midway. If a client retries an idempotent request after a network timeout, the server reaches the same state regardless of how many times the request actually arrived. Retrying a `POST` after a timeout, on the other hand, risks creating two records for the same logical action — the canonical reason for the "Confirm form resubmission" dialog and for techniques like idempotency keys in payment APIs.

6.4 Status codes

Every response carries a three-digit **HTTP status code** that signals the outcome, with the first digit grouping outcomes into 1xx informational, 2xx success, 3xx redirection, 4xx client error, and 5xx server error. The status line gives the client enough information to decide what to do next without parsing the response body.

The codes that appear most often in an ASP.NET Core MVC application are listed below.

CODE	MEANING	TYPICAL CAUSE
200 OK	Request succeeded; body holds the representation	A `GET` returning a view or JSON payload
201 Created	A resource was created; `Location` header points to it	A `POST` to a REST endpoint that inserted a row
204 No Content	Success with no body	A `DELETE` or a `POST` that does not return data
301 Moved Permanently	The resource lives at a new URI permanently	URL restructuring; SEO redirects
302 Found	Temporary redirect	Post-Redirect-Get pattern after a successful form submission
400 Bad Request	The request is malformed or fails validation	Invalid model state in a controller action
401 Unauthorized	Authentication is required or has failed	A request missing a valid auth cookie or token
403 Forbidden	The caller is authenticated but not allowed	Authorization policy denied the action
404 Not Found	No resource exists at the URI	Route did not match any controller action
500 Internal		Bug in application code; missing dependency

CODE	MEANING	TYPICAL CAUSE
Server Error	The server raised an unhandled exception	

Two patterns are worth noting. Status codes in the 4xx range place the fault on the client; codes in the 5xx range place it on the server. The difference matters for retry logic — retrying a `404` will keep failing, while retrying a `503` may succeed once the server recovers. And the framework, not the developer, picks the status code in many cases: ASP.NET Core MVC turns an unhandled exception into `500`, an unmatched route into `404`, and a `ValidationProblem` result into `400` automatically.

6.5 URIs

A **URI** (Uniform Resource Identifier) is the string a client uses to address a resource on a server, typically composed of a scheme, host, path, and optional query string and fragment. The URI is the single piece of information that ties an HTTP request to a specific piece of application logic.

The full structure of a URI seen by an ASP.NET Core application looks like this:

```
https://localhost:7240/Newsletter/Subscribe?
source=footer#confirmation
\___/   \____________/\____________________/\____________/
\___________/
scheme  authority      path                  query
fragment
```

The *scheme* (`http` or `https`) selects the transport — plain TCP or TLS-wrapped TCP. The *authority* combines the host name and port. The *path* identifies the resource within the

server and is the part the routing system dispatches on. The *query* is a `?`-prefixed list of `key=value` pairs separated by `&`, used by `GET` requests to pass parameters. The *fragment* after `#` is never sent to the server — it stays in the browser and is used for client-side anchors.

In an ASP.NET Core MVC application, the path conventionally maps to `/{controller}/{action}/{id?}`. A request for `/Home/Index` invokes the `Index` action on `HomeController`. Routing is the subject of the MVC pattern chapter; here it is enough to know that the path of the URI determines which C# method runs.

6.6 Headers

Headers are the metadata channel of an HTTP message. They appear as `Name: Value` pairs, one per line, between the start line and the body. The same header syntax serves both requests and responses, but most headers are used in only one direction.

Common request headers include `Host` (which virtual host on the server), `Accept` (which content types the client can handle), `Authorization` (credentials such as a bearer token), `Cookie` (session and auth cookies the browser previously stored), and `Content-Type` (what is in the body, when there is one). Common response headers include `Content-Type` (what the body contains), `Content-Length` (how many bytes), `Location` (where to redirect to), `Set-Cookie` (cookies the server wants the browser to store), and `Cache-Control` (rules for caches and the browser).

Headers carry the information that does not fit cleanly into the method or URI. Authentication, content negotiation, caching, compression, and CORS all happen through headers. The

body is for the resource representation; headers are for everything about it.

6.7 Worked example: curl hitting an MVC action

A default ASP.NET Core MVC project scaffolded with `dotnet new mvc -n CloudSoft` includes a `HomeController` whose `Index` action returns the welcome view. With the application running locally on port 7240, a `curl` command can drive the full request-response cycle without a browser.

```
curl -v https://localhost:7240/Home/Index
```

The `-v` flag prints the request and response in full. The annotated output shows every concept introduced above:

```
> GET /Home/Index HTTP/1.1
> Host: localhost:7240
> User-Agent: curl/8.4.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Content-Type: text/html; charset=utf-8
< Date: Tue, 28 Apr 2026 09:14:22 GMT
< Server: Kestrel
< Transfer-Encoding: chunked
<
<!DOCTYPE html>
<html lang="en">
...
```

The lines beginning with `>` are the request `curl` sent. The first line names the method (`GET`), the path (`/Home/Index`), and the protocol version. The `Host`, `User-Agent`, and

`Accept` headers describe the client and what it is willing to receive. There is no body, which is expected for a `GET`.

The lines beginning with `<` are the response. The status line reports `200 OK`, signalling success. The `Content-Type` header tells `curl` the body is HTML, encoded as UTF-8. The `Server` header reveals that the request was handled by Kestrel, the cross-platform web server that ships with ASP.NET Core. The HTML body that follows is the rendered output of the MVC view at `Views/Home/Index.cshtml`.

Internally, ASP.NET Core matched the path `/Home/Index` to the `Index` method on `HomeController`, executed the method, rendered the associated view to HTML, and wrote the bytes back through the same TCP connection. Routing, model binding, and view rendering — the subjects of the MVC pattern — all happened inside that single response.

A `POST` example illustrates the contrast. Submitting a newsletter form looks like this:

```
curl -v -X POST https://localhost:7240/Newsletter/Sub-
        scribe \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "email=ada@example.com"
```

The request now has a body; the `Content-Type` header tells the server how to parse it; and the controller action is expected to mutate state — adding the address to the subscriber list. A successful submission typically returns a `302` redirect to a confirmation page rather than a `200` with a body, following the Post-Redirect-Get pattern that prevents accidental duplicate submissions on browser refresh.

The companion exercise Presentation Layer builds these requests step by step against an ASP.NET Core MVC project, starting from the scaffolded `HomeController` and adding forms, validation, and styled views.

6.8 Summary

HTTP is the request-response protocol layered on TCP that lets a client and a server exchange messages whose shape both sides understand. Every request names a method and a URI, carries headers, and may have a body; every response carries a status code, headers, and an optional body. `GET` is safe and idempotent and encodes parameters in the query string; `POST` mutates state and carries data in the body, which is why mutating actions never use `GET`. Status codes tell the client the outcome in one number, with the leading digit narrowing the cause to client or server. URIs combine scheme, authority, path, and query into the address that routing dispatches on. The rest of Part III — ASP.NET Core, the MVC pattern, three-tier architecture, configuration, and dependency injection — refines how a server turns those requests into responses.

PART III · APPLICATION DEVELOPMENT

CHAPTER 7

The .NET Platform

Watch the presentation

Se presentationen på svenska

Building a web application requires a language to express logic, a runtime to execute that logic, libraries to handle common concerns, and a packaging system to share code across projects. .NET provides all four as a single, integrated platform. Understanding how the pieces fit together — from C# source code through compilation to a running web server — gives the foundation for every other chapter in Part III.

7.1 Why .NET exists

.NET is the cross-platform development platform from Microsoft for building applications in C# (and other languages) that compile to a common Intermediate Language and run on the .NET runtime; the unified version replaces the older .NET Framework. The platform addresses a recurring problem in application development: source code written for one operating system, language version, or hardware architecture often fails to run on another. Without a common execution model, every team rewrites the same plumbing — memory management, string handling, networking primitives, package distribution.

The original .NET Framework, released in 2002, solved this only for Windows. Code written against the Framework ran on Windows desktops and Windows Server, but not on Linux or macOS. As cloud workloads moved toward Linux containers and developers adopted macOS for daily work, this Windows-only constraint became a serious limitation. Microsoft responded with .NET Core in 2016 — a complete rewrite of the runtime designed to run identically on Windows, Linux, and macOS. After several release cycles, the Core lineage merged with the original Framework codebase into a unified product called .NET, with major version numbers continuing the Core sequence (5, 6, 7, 8, 9, and now 10).

The unified platform is open source, distributed under the MIT license, and developed in public on GitHub. The same SDK, compiler, and runtime power desktop apps, mobile apps, console tools, microservices, and the ASP.NET Core web stack used throughout this course.

7.2 The compilation pipeline

C# is a statically typed language, which means the compiler verifies types and catches a category of errors before the code ever runs. The compiler does not produce machine code directly. Instead, it produces a portable intermediate representation that the runtime translates to machine code on the target machine.

The pipeline has four stages:

1. **C# source code** in `.cs` files — what the developer writes.
2. **Intermediate Language** (IL) — what the C# compiler emits. IL is a CPU-independent bytecode similar in spirit to Java bytecode. The same IL runs unchanged on x86-64, ARM64, Windows, Linux, and macOS.
3. **Just-in-time compilation** (JIT) — the runtime translates IL to native machine code the first time a method executes. Subsequent calls reuse the compiled code.
4. **Machine code** — the actual CPU instructions that execute on the host hardware.

This two-step model — compile once to IL, JIT-compile at runtime — explains why a single published .NET application can target multiple operating systems and CPU architectures from one codebase. The IL is platform-neutral; the JIT compiler embedded in each platform's runtime handles the platform-specific translation.

7.2.1 The common language runtime

The **CLR** (Common Language Runtime) is the .NET execution environment that loads compiled assemblies, performs just-in-time compilation of Intermediate Language to machine code, and provides services such as garbage collection and exception handling. The CLR is what makes .NET a *managed* runtime: code does not allocate and free memory directly,

and unhandled errors do not corrupt process state in unpredictable ways.

Two CLR services shape how C# code behaves day-to-day. The **garbage collector** tracks every object the program allocates on the managed heap and reclaims memory once no live references remain. Developers do not call `free()` or `delete` — the GC runs periodically, identifies unreachable objects, and frees their memory. This eliminates entire classes of bugs (use-after-free, double-free, memory leaks from forgotten cleanup) at the cost of occasional pauses while collection runs.

The CLR's exception handling model is the second pillar. When code throws an exception, the runtime walks the call stack looking for a matching `catch` clause. If none exists, the runtime terminates the process cleanly rather than allowing corrupted state to spread. Web frameworks like ASP.NET Core layer on top of this, catching exceptions at the request boundary so that one failed request does not crash the whole server.

7.3 Assemblies and NuGet packages

Compiled .NET code is distributed as assemblies. An **assembly** is a compiled .NET unit — typically a `.dll` file — containing Intermediate Language, type metadata, and resources, that the CLR loads at runtime. A small console application produces one assembly. A larger web application produces one assembly for the application itself plus dozens or hundreds of dependency assemblies, each contributed by a different library.

The metadata embedded in every assembly is what makes the runtime self-describing. Tools can inspect a `.dll` and discover the types it defines, the methods on each type, the parameters of each method, and the attributes applied to

each declaration. The DI container, the model binder, and the routing system all rely on this reflection capability.

Sharing assemblies across projects requires a package manager. **NuGet** is the package manager for .NET; published packages contain assemblies and metadata, and projects declare dependencies in their `.csproj` files for the SDK to restore. Public packages live on `nuget.org`; private feeds (Azure Artifacts, GitHub Packages) host internal libraries. The exercises in this course pull packages such as `Microsoft.AspNetCore.Mvc`, `MongoDB.Driver`, and `Azure.Storage.Blobs` from `nuget.org`.

7.3.1 The .NET 10 SDK and the dotnet CLI

The .NET 10 SDK is the toolchain installed on the developer's machine. It bundles the C# compiler, the runtime, the project templates, and the `dotnet` command-line interface. Every project in this course is created, built, and run through `dotnet` commands rather than through an IDE-specific menu, which keeps the workflow identical on Windows, Linux, and macOS.

Five commands cover the bulk of daily work:

COMMAND	PURPOSE
`dotnet new <template>`	Scaffold a new project from a template (`mvc`, `console`, `webapi`, `classlib`).
`dotnet restore`	Download all NuGet dependencies declared in the `.csproj` file.
`dotnet build`	Compile the project to IL, producing assemblies in `bin/Debug/`.
`dotnet run`	Build (if needed) and execute the project locally.
`dotnet add package <name>`	Add a NuGet package reference and trigger restore.

`dotnet run` and `dotnet build` invoke `dotnet restore` automatically when dependencies are missing, so an explicit restore is rarely needed in a fresh clone — running the project triggers the chain.

7.4 ASP.NET Core as the web layer

The CLR and the SDK handle compilation and execution, but a web application also needs an HTTP server, a routing system, a way to render HTML, and an extension point for middleware (logging, authentication, compression). **ASP.NET Core** is the .NET web framework that hosts HTTP applications using a configurable request-handling pipeline, with built-in support for the MVC pattern, dependency injection, configuration, and middleware.

ASP.NET Core ships with Kestrel, a high-performance HTTP server written in C#. When `dotnet run` starts an ASP.NET Core project, Kestrel binds to a port (typically 5000 for HTTP and 7240 for HTTPS in development), accepts incoming connections, parses each HTTP request, and hands it off to the request pipeline configured in `Program.cs`. The pipeline routes the request to a controller action, the action returns a result, and Kestrel writes the response back to the client.

The framework is modular. Features such as authentication, session state, CORS, and response caching are opt-in middleware components, registered in `Program.cs`. A minimal API service may use only routing and JSON serialization; a full MVC application composes a longer pipeline with view rendering, anti-forgery tokens, and authorization checks.

7.5 A worked example: scaffolding an MVC project

The companion exercise Building the Presentation Layer starts by scaffolding a new MVC application. The single command does the work:

```
dotnet new mvc -n CloudSoft
```

The command generates the following folder structure:

```
CloudSoft/
├── CloudSoft.csproj
├── Program.cs
├── appsettings.json
├── appsettings.Development.json
├── Controllers/
│   └── HomeController.cs
├── Models/
│   └── ErrorViewModel.cs
├── Views/
│   ├── Home/
│   │   ├── Index.cshtml
│   │   └── Privacy.cshtml
│   ├── Shared/
│   │   ├── _Layout.cshtml
│   │   ├── _ValidationScriptsPartial.cshtml
│   │   └── Error.cshtml
│   ├── _ViewImports.cshtml
│   └── _ViewStart.cshtml
├── wwwroot/
│   ├── css/
│   ├── js/
│   ├── lib/
│   └── favicon.ico
└── Properties/
    └── launchSettings.json
```

Each top-level folder corresponds to one concept introduced in this Part. `CloudSoft.csproj` declares the project's target framework (`net10.0`), the SDK (`Microsoft.NET.Sdk.Web`), and the NuGet package references — the SDK uses this file to drive `restore`, `build`, and `run`. `Program.cs` is the entry point: it builds the web host, registers services with the DI container, configures the middleware pipeline, and calls `app.Run()` to start Kestrel. `Controllers/` and `Views/` together form the MVC presentation layer; `wwwroot/` holds static assets that Kestrel serves directly without invoking a controller. The

`appsettings.json` files supply configuration values that ASP.NET Core loads at startup.

Running `dotnet run` from the `CloudSoft/` directory triggers the full pipeline: the SDK restores NuGet packages, the C# compiler emits `CloudSoft.dll` (an assembly) into `bin/Debug/net10.0/`, the runtime starts the CLR, the CLR loads the assembly, JIT-compiles `Program.Main` to machine code, and Kestrel begins listening for HTTP requests. Visiting `https://localhost:7240` in a browser produces the welcome page rendered by `HomeController.Index()` and `Views/Home/Index.cshtml`.

7.6 .NET compared with other web stacks

Choosing a platform involves trade-offs across performance, ecosystem, hiring pool, and operational maturity. .NET sits in a similar slot to the JVM (Java, Kotlin) and Go for backend web work, with different strengths.

PLATFORM	COMPILATION	MEMORY MODEL	TYPICAL WEB FRAMEWORK
.NET	C# → IL → JIT to native	Garbage collected	ASP.NET Core
JVM	Java → bytecode → JIT to native	Garbage collected	Spring Boot
Go	Source → native at build time	Garbage collected	Standard library `net/http`
Node.js	JavaScript interpreted by V8 (with JIT)	Garbage collected	Express, Fastify

Cloud-native services on Azure benefit from .NET's first-party tooling (Azure SDK, Application Insights, Azure Functions runtime) and the deepest documentation surface. The exercises in this course standardise on .NET so that the platform itself never becomes the topic — the topic is the architectural pattern being learned.

7.7 Summary

.NET is the cross-platform managed runtime that executes C# code through a compile-to-IL, JIT-to-native pipeline. The CLR provides the execution environment, including garbage collection and exception handling, so developers focus on application logic rather than memory management. Compiled code ships as assemblies — `.dll` files containing IL and metadata — and is shared between projects through NuGet packages declared in `.csproj` files. The .NET 10 SDK

supplies the `dotnet` CLI used to scaffold, build, and run projects (`dotnet new`, `dotnet build`, `dotnet run`, `dotnet add package`). ASP.NET Core layers on top of this foundation as the web framework, providing the Kestrel HTTP server, the MVC request pipeline, and the dependency injection container that subsequent chapters build on. Together, these pieces form the platform on which every exercise in Part III is built.

PART III · APPLICATION DEVELOPMENT

CHAPTER 8

The MVC Pattern

Watch the presentation

Se presentationen på svenska

A web application that handles user input, applies business rules, and renders output entangles three concerns that change for different reasons. Validation rules shift when policies change. Page layouts shift when designers iterate. Routing shifts when URLs are reorganized. Mixing those concerns in a single class produces code that becomes difficult to test, modify, or reason about. The MVC pattern resolves this by assigning each concern to a distinct role with a clearly defined contract between them.

This chapter covers the three roles of MVC, how routing matches a request URI to a controller action, how model bind-

ing turns request data into typed parameters, the action results that controllers return, the basics of Razor view syntax, and the request flow that ties everything together. The treatment focuses on ASP.NET Core MVC, the implementation used throughout the companion exercise Presentation Layer.

8.1 The three roles of MVC

MVC (Model-View-Controller) is an architectural pattern that separates application concerns into three roles: the **model** holds data and business state, the **view** renders that state to the user, and the **controller** receives user input, coordinates the model and view, and decides what to return. Each role has a single reason to change and communicates with the others through narrow interfaces rather than shared internal state.

The split exists because the three concerns evolve at different rates and for different reasons. Input handling depends on URLs, HTTP methods, and form schemas. Business state depends on domain rules. Rendering depends on layout, styling, and presentation logic. Keeping the three apart means a designer can rework a view without touching validation logic, and a developer can change a routing rule without breaking the rendered HTML.

8.1.1 Controller

A **controller** in ASP.NET Core MVC is a class whose public methods (called **actions**) handle incoming HTTP requests; an action returns an `IActionResult` describing what the framework should send back, such as a rendered view, JSON, or a redirect. Controllers in ASP.NET Core inherit from `Controller` and live by convention in a `Controllers/` folder. The class name ends in the suffix `Controller` — `HomeControl-`

`ler`, `NewsletterController`, `AccountController` — and the framework strips that suffix when matching route values.

Each public method on a controller is a candidate action. The framework selects the action to invoke based on the route and the HTTP method. Methods can be decorated with attributes such as `[HttpGet]`, `[HttpPost]`, `[Route]`, or `[Authorize]` to constrain which requests they accept. An action method does the minimum needed to coordinate the response: it reads input from its parameters, calls into the service layer for business logic, and returns an action result. Controllers should not contain validation rules, persistence calls, or rendering logic; those belong to the service layer, the data layer, and views respectively.

8.1.2 View

A **view** is a Razor template (`.cshtml`) that renders HTML by combining static markup with C# expressions; ASP.NET Core resolves view names to template files using conventions over the `Views/` folder. When a controller called `HomeController` returns `View("Index")`, the framework looks for `Views/Home/Index.cshtml`. The folder name matches the controller name (without the suffix), and the file name matches the view name. A shared `Views/Shared/` folder holds layouts and partials used across multiple controllers.

Views are not classes — they are templates compiled into rendering code at build or first-request time. A view receives a model object from the controller, exposes it through the `Model` property, and uses Razor expressions to interpolate values into HTML. Logic in views should stay limited to presentation concerns: looping over a collection of items, conditionally showing a section, formatting a value for dis-

play. Anything heavier — database queries, business rules, branching on user roles — belongs upstream.

In MVC, the **model** for a view is the typed data object passed to it. This usage is narrower than the broader notion of a domain model. Rich domain models — entities with behaviour, business invariants, and persistence concerns — live in the service layer covered in the three-tier architecture chapter. The model passed to a view is often a tailored shape, sometimes called a ViewModel, that contains exactly the fields the view needs to render and nothing more.

8.2 Routing matches a URI to an action

Routing is the ASP.NET Core mechanism that matches an incoming request URI to a controller action, using either conventional templates configured at startup or attribute routes declared on actions. Routing is registered as middleware in `Program.cs` and runs early in the request pipeline. By the time an action executes, routing has already inspected the URI, found a matching template, and extracted route values from the path.

A conventional route template looks like `{controller=Home}/{action=Index}/{id?}`. Three placeholders define the structure: a `controller` segment, an `action` segment, and an optional `id` segment. The defaults `Home` and `Index` mean that a request to the root URL `/` resolves to `HomeController.Index()`. A request to `/Newsletter/Subscribe` resolves to `NewsletterController.Subscribe()`. A request to `/Products/Details/42` resolves to `ProductsController.Details(42)`, with `42` bound to the `id` parameter.

Attribute routing places the route directly on the controller or action with attributes such as `[Route("api/[controller]")]` or `[HttpGet("{id:int}")]`. Attribute routes are

explicit, support route constraints, and are the preferred style for API controllers. Conventional routes are concise and suit traditional MVC applications where most routes follow a predictable controller-action pattern. The two styles can coexist in the same application.

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Figure 1: Default conventional route registered in `Program.cs`. The `=Home` and `=Index` syntax sets defaults for the segments, and the `?` makes `id` optional. With this single line, every controller action becomes reachable through a URL that mirrors its class and method name.

8.3 Model binding turns request data into typed parameters

A controller action declares its inputs as ordinary C# parameters. **Model binding** is the ASP.NET Core process that converts incoming request data — route values, query strings, form fields, and JSON bodies — into the typed parameters of an action method. The framework inspects each parameter, looks for a matching value in the request, converts it to the parameter's declared type, and passes it in. The action body sees only typed data; the parsing of the raw request is handled before the method runs.

Model binding draws values from several sources in a defined order: route values come first, then query string parameters, then form fields, then the request body for content types like JSON. A parameter named `id` on `ProductsController.Details(int id)` matches the `{id}` route segment

when present, falls back to a `?id=42` query string, and so on. For complex types — a class with several properties — binding walks the public settable properties and matches each one against the request by name.

Validation runs alongside binding. Properties decorated with attributes such as `[Required]`, `[StringLength]`, or `[EmailAddress]` are checked after the value is set, and the results accumulate in `ModelState`. The action body inspects `ModelState.IsValid` to decide whether to proceed or return the form to the user with error messages. The combination of typed parameters and declarative validation removes most of the boilerplate that would otherwise appear at the top of every action.

8.4 Action results describe the response

An action method returns an `IActionResult` — an interface that represents what the framework should send back. The framework inspects the returned object and translates it into an HTTP response. Several concrete result types cover the common cases.

RESULT TYPE	RETURNED BY HELPER	WHAT IT PRODUCES
`ViewResult`	`View()` or `View(model)`	Renders a Razor view to HTML
`RedirectToActionResult`	`RedirectToAction("Index")`	Sends a 302 redirect to another action
`JsonResult`	`Json(data)`	Serializes an object to JSON
`NotFoundResult`	`NotFound()`	Returns HTTP 404
`BadRequestResult`	`BadRequest()`	Returns HTTP 400
`ContentResult`	`Content(text)`	Returns plain text or arbitrary content

The choice of action result expresses the controller's intent. Returning `View()` says "render the page that matches this action." Returning `RedirectToAction` after a successful form post follows the post-redirect-get pattern, preventing the browser from re-submitting the form on refresh. Returning `Json` from an API action signals a machine-readable response. The framework handles serialization, status codes, and content-type headers based on the result type chosen.

8.5 Razor view syntax

Razor is the templating syntax used by ASP.NET Core views. A `.cshtml` file is mostly HTML, with C# expressions introduced by the `@` symbol. A single expression like `@Model.Title` interpolates a value. A code block `@{ var count = Model.Items.Count; }` runs C# without producing output. Control flow uses standard C# keywords: `@if`,

`@foreach`, `@switch`. The Razor parser distinguishes markup from code by tracking HTML tag boundaries, so most templates require no explicit signalling beyond the leading `@`.

Views typed against a specific model declare the type at the top with a `@model` directive: `@model Product`. From that point on, `Model` refers to the strongly-typed instance, and IntelliSense in the editor offers property completion. Layouts wrap views in shared markup using `_Layout.cshtml`, and partial views render reusable fragments through `@Html.Partial("_FormFields", Model)`. Tag helpers — attributes prefixed with `asp-` — bind form inputs to model properties and generate validation markup automatically.

8.6 The request flow

A complete request flows through several stages, each isolated by an interface.

1. The browser sends an HTTP request to the server.
2. ASP.NET Core's hosting layer accepts the connection and constructs an `HttpContext`.
3. Middleware components run in order — authentication, static files, routing — each examining the request and either short-circuiting it or passing it on.
4. Routing matches the URI to a controller action and extracts route values.
5. Model binding populates the action's parameters from route values, query string, form, or body.
6. The framework instantiates the controller (using dependency injection for any constructor parameters) and invokes the action.
7. The action does its work and returns an `IActionResult`.

8. The framework executes the result, which may render a view, serialize JSON, or write a redirect header.
9. The response travels back through the middleware pipeline and out to the client.

Each stage has a clear responsibility, and each can be tested or replaced independently. Routing can be exercised without instantiating controllers. Controllers can be tested with mocked services. Views can be rendered against fabricated model objects.

8.7 Worked example: HomeController and Index.cshtml

The default ASP.NET Core MVC project, scaffolded by `dotnet new mvc -n CloudSoft`, ships with a `HomeController` and a matching set of views. The controller is the simplest possible illustration of the pattern.

```
using Microsoft.AspNetCore.Mvc;

namespace CloudSoft.Controllers;

public class HomeController : Controller
{
    public IActionResult Index()
    {
        return View();
    }

    public IActionResult Privacy()
    {
        return View();
    }
}
```

Figure 2: `Controllers/HomeController.cs` from the scaffolded project. Each action is a parameterless method returning `View()` with no model. The framework resolves the view name from the action name — `Index` finds `Views/Home/Index.cshtml`, `Privacy` finds `Views/Home/Privacy.cshtml`.

```
@{
    ViewData["Title"] = "Home Page";
}

<div class="text-center">
    <h1 class="display-4">Welcome</h1>
    <p>Learn about <a href="https://learn.microsoft.com/
        aspnet/core">building Web apps with ASP.NET Core</
        a>.</p>
</div>
```

Figure 3: `Views/Home/Index.cshtml`. The `@{ }` block runs C# to set a value in `ViewData`, a loosely-typed dictionary that the layout reads to populate the `<title>` tag. The rest is plain HTML that Razor passes through unchanged.

When a browser requests the root URL, the default route resolves the empty path to `controller=Home, action=Index`, the framework instantiates `HomeController`, calls `Index()`, receives a `ViewResult`, locates `Views/Home/Index.cshtml`, executes the Razor template against an empty model, and writes the resulting HTML to the response. The companion exercise Presentation Layer extends this scaffold with strongly-typed models, form posts, and validation, exercising every part of the request flow described above.

8.8 Why this pattern matters

The clean separation of controller, view, and model makes ASP.NET Core MVC applications testable and adaptable. A controller can be unit-tested by instantiating it with stub dependencies and asserting on the action result it returns. A view can be examined by rendering it against a hand-built model. A routing rule can be changed without touching either. When a feature evolves — a new field on a form, a different page layout, an alternate URL — the change tends to land in one place rather than rippling across the codebase.

The pattern also defines a clear seam for the next architectural layer. Controllers are thin; they depend on services for anything substantial. Those services form the **service layer** of the three-tier architecture, and they are supplied to controllers through dependency injection. The MVC pattern handles the boundary between HTTP and application code; the service layer handles the boundary between application code and business logic.

8.9 Summary

The MVC pattern separates a web application into three roles: controllers handle input, views render output, and models carry data between them. ASP.NET Core MVC implements the pattern with controllers as classes whose actions return `IActionResult` values, views as Razor templates resolved by convention from a `Views/` folder, and models as the typed data passed from action to view. Routing matches an incoming URI to a specific action, and model binding converts route values, query strings, and form fields into typed parameters before the action runs. Action results — `ViewResult`, `RedirectToActionResult`, `JsonResult`, and the rest — describe the response declaratively, leaving the framework to produce

the HTTP output. The complete request flow runs from middleware through routing, binding, action execution, and result rendering, with each stage isolated behind a clear interface. The result is a structure where presentation, input handling, and business state evolve independently, and where each can be tested without standing up the others.

PART III · APPLICATION DEVELOPMENT

CHAPTER 9

Three-Tier Architecture

Watch the presentation

Se presentationen på svenska

A small ASP.NET Core application can put every concern into the controller: read the form, validate the input, check for duplicates, write to a database, render a confirmation. The code runs and the feature works. The trouble starts when the application grows. A second feature needs the same duplicate check. A third feature needs to call the same database, but with a different validation rule. The controller turns into a 500-line class that mixes HTTP handling, business rules, and data access, and any change risks breaking unrelated beha-

viour. Three-tier architecture addresses this by giving each kind of work its own home, with strict rules about which tier may call which.

9.1 Why monolithic controllers break down

A controller's job is to translate between HTTP and the rest of the application. It receives a request, decides what should happen, and produces a response. When the controller also performs the business logic and reaches into the database directly, three different reasons to change the code collapse onto a single class. A change to the URL routing affects the same file as a change to a validation rule, which affects the same file as a change to the storage backend.

This violates a design principle known as **separation of concerns** — the design principle that each module of an application should have one reason to change; layers, classes, and methods are scoped so that unrelated responsibilities never live together. A class with three reasons to change attracts three streams of edits, three sets of bugs, and three groups of developers stepping on each other.

The practical symptoms appear quickly. Unit tests become hard to write because exercising the validation logic also requires a real database. Swapping an in-memory store for a cloud database forces edits to every controller. Two features that need the same business rule end up with two slightly different copies of it, drifting apart over time.

A **three-tier architecture** organizes an application into a presentation layer (UI), a service layer (business logic), and a data layer (persistence), with each layer depending only on the layer beneath it through abstractions. The structure does not eliminate complexity — a finished feature still needs HTTP handling, validation, and storage — but it places each

piece behind a clear boundary, so changes stay local and substitutions stay possible.

9.2 The three layers and their responsibilities

Each layer has one job. The boundaries between layers are formed by interfaces, not by folder names alone. Folders signal the structure to a developer reading the project; interfaces enforce the structure at compile time.

9.2.1 The presentation layer

The **presentation layer** is the part of an application responsible for accepting user input and rendering output; in an ASP.NET Core MVC app, controllers and views form this layer. Code in this layer handles HTTP-specific concerns: route matching, model binding from form fields, returning `IActionResult`, setting status codes, and rendering Razor views. It does not contain business rules and does not talk to a database.

A controller in this layer reads the incoming request, hands the work to a service through a service interface, and translates the service's result back into an HTTP response. If the service reports success, the controller picks the appropriate view or redirect. If the service reports a validation failure, the controller returns the form with error messages. The controller does not know how the work is performed, only that it was requested and what the outcome was.

9.2.2 The service layer

The **service layer** is the part of an application that contains the business logic — validation, workflow, orchestration — and exposes operations to the presentation layer through service interfaces. Code in this layer enforces rules: the email format

must be valid, a subscriber must not exist twice, a publish action requires a draft to exist first. It coordinates one or more data-layer calls into a single operation that means something to the business.

The service layer does not know it is being called from HTTP. It receives plain objects (or primitive parameters) and returns plain objects (or a result type that signals success or failure). The same service class could be invoked from a background worker, a console tool, or a test, with no change to its code. This independence is what makes business logic reusable.

9.2.3 The data layer

The **data layer** is the part of an application responsible for persistent storage and retrieval, exposed to the service layer through repository abstractions; concrete implementations may use a database, object storage, or an in-memory collection. Code in this layer translates between the application's domain types and the storage technology. A repository class hides whether the store is MongoDB, SQL Server, Azure Blob Storage, or a `ConcurrentDictionary` held in memory for a development environment.

The data layer enforces a different kind of rule: how data is laid out, indexed, and queried. It does not enforce business rules. A repository's `AddAsync(subscriber)` accepts whatever the service hands it; checking whether the subscriber is a duplicate is the service layer's concern.

9.3 Dependency direction and the role of interfaces

The three layers are stacked, and references flow in one direction only. The presentation layer references the service

layer; the service layer references the data layer. The arrows never point upward. A repository must not call a service; a service must not return a `ViewResult`.

This rule exists for two reasons. The first is testability. A service can be tested with a fake repository that stores rows in a list. A controller can be tested with a fake service that returns a canned result. If references pointed upward, every test of a low-level class would drag in the entire stack above it. The second reason is substitution. Replacing the in-memory subscriber repository with a MongoDB-backed implementation changes one class. None of the services or controllers above it need to be touched.

Interfaces enforce the boundary. A controller does not declare a parameter of type `NewsletterService`; it declares a parameter of type `INewsletterService`. A service does not declare a field of type `MongoSubscriberRepository`; it declares a field of type `ISubscriberRepository`. The concrete classes are wired up once, in `Program.cs`, through the dependency injection container. Each layer compiles against an *abstraction* — an interface that names the operations it depends on without committing to any particular implementation.

This is also why dependency injection appears so often in three-tier code. Constructor parameters typed as interfaces are the mechanism by which a class declares its needs without choosing its dependencies. For the purposes of this chapter, the relevant point is that interfaces in constructors are how the layer boundaries are enforced. Concrete substitution is configured at startup; the layers above never name the concrete type. (See the dependency injection chapter for the registration mechanics.)

The MVC pattern lives entirely inside the presentation layer. Controller, view, and the view's model object all sit at

the top tier, and the controller delegates downward to a service interface. The MVC pattern itself is documented in /course-book/3-application-development/3-the-mvc-pattern/; three-tier architecture wraps that pattern in two further layers below it.

9.4 How a request flows through the layers

A single user action travels through all three layers and back. Tracing one request makes the boundaries concrete.

9.4.1 Worked example: subscribing to a newsletter

The companion service layer exercise refactors a monolithic controller into the three-layer structure described here. A user submits a newsletter signup form. The controller receives the POST, hands the email and name to the service, and the service uses a repository to store the subscriber.

The controller belongs to the presentation layer:

```
public class NewsletterController : Controller
{
    private readonly INewsletterService _newsletterService;

    public NewsletterController(INewsletterService newslet-
        terService)
    {
        _newsletterService = newsletterService;
    }

    [HttpPost]
    public async Task<IActionResult> Subscribe(Subscribe-
        FormModel form)
    {
        if (!ModelState.IsValid)
        {
            return View(form);
        }

        var result = await _newsletterService.SubscribeA-
        sync(form.Email, form.Name);

        if (result.Succeeded)
        {
            return RedirectToAction(nameof(Confirmation));
        }

        ModelState.AddModelError(string.Empty,
        result.Error);
        return View(form);
    }
}
```

The controller does three things, all of them HTTP-specific: it checks the model state populated by ASP.NET Core's model binding, it calls a single service method, and it picks a response based on the outcome. There is no email-format check, no duplicate check, no database call.

The service belongs to the service layer:

```
public class NewsletterService : INewsletterService
{
    private readonly ISubscriberRepository _subscribers;

    public NewsletterService(ISubscriberRepository sub-
        scribers)
    {
        _subscribers = subscribers;
    }

    public async Task<OperationResult>
        SubscribeAsync(string email, string name)
    {
        if (string.IsNullOrWhiteSpace(email))
        {
            return OperationResult.Fail("Email is
        required.");
        }

        if (await _subscribers.ExistsAsync(email))
        {
            return OperationResult.Fail("This email is
        already subscribed.");
        }

        var subscriber = new Subscriber(email, name, Date-
        Time.UtcNow);
        await _subscribers.AddAsync(subscriber);
        return OperationResult.Ok();
    }
}
```

The service enforces the business rules: presence of an email, no duplicates, recording when the subscription occurred. It returns an `OperationResult` rather than a view or a status code, because the service has no concept of HTTP. The same method could be called from a unit test or a background job and behave identically.

The repository belongs to the data layer:

```
public class InMemorySubscriberRepository : ISubscriberRe-
        pository
{
    private readonly ConcurrentDictionary<string, Sub-
        scriber> _store =
        new(StringComparer.OrdinalIgnoreCase);

    public Task<bool> ExistsAsync(string email)
        => Task.FromResult(_store.ContainsKey(email));

    public Task AddAsync(Subscriber subscriber)
    {
        _store[subscriber.Email] = subscriber;
        return Task.CompletedTask;
    }
}
```

The repository handles storage and nothing else. It does not validate, it does not check for duplicates as a business rule (the `ContainsKey` check exists to prevent a key collision in the dictionary, not to enforce policy). Replacing this class with a MongoDB-backed `MongoSubscriberRepository` changes the storage technology with no edits to `NewsletterService` or `NewsletterController`, because both depend on `ISubscriberRepository` rather than the concrete class.

The full request flow is: ASP.NET Core routes the POST to `Subscribe`, model binding fills `SubscribeFormModel`, the controller calls `SubscribeAsync`, the service validates and consults the repository, the repository returns a result, the service returns an `OperationResult`, the controller picks a redirect or a re-rendered view. Each layer does its own job and trusts the layers below.

9.5 The trade-off: more files, more reasoning power

Three-tier architecture is not free. A signup feature that lived in one controller method now spans a controller, a service interface, a service class, a repository interface, a repository class, a domain object, and a result type. A reader new to the codebase must follow constructor parameters across several files to trace one operation.

The trade-off only pays off as the application grows. For a single signup form with a single rule, the layered version has more code than the monolithic version. For ten features sharing a duplicate-checking rule, the layered version has dramatically less code, because the rule lives in one place. For a feature that needs to swap from in-memory storage to a cloud database, the layered version requires changes to one file, and the monolithic version requires changes to every controller that touched the store.

CONCERN	MONOLITHIC CONTROLLER	THREE-TIER
Files per feature	1	4–6
Reuse of business rules	Copy-paste	Service method called from anywhere
Substituting storage	Edit every controller	Replace one repository class
Unit testing the rules	Requires real database	Pass a fake repository
New developer's first read	Direct (one file)	Indirect (follow interfaces)

The decision rule is empirical: separate layers when there is more than one consumer of the rules, more than one storage backend in play, or more than one engineer touching the same area. For a throwaway demo, a monolithic controller is fine. For a product expected to live for years and grow new features, the layered structure is the cheaper option once the trajectory is taken into account.

9.6 Summary

Three-tier architecture organizes an application into a presentation layer, a service layer, and a data layer, with dependencies pointing only downward and layer boundaries enforced by interfaces. The presentation layer translates between HTTP and method calls; the service layer enforces business rules and orchestrates work; the data layer hides the

storage technology behind repository abstractions. Each layer has one reason to change, which is the principle of separation of concerns applied at the architectural scale. A request passes through all three layers — controller to service to repository — and the result returns up the same chain. The structure introduces more files, but the cost is repaid by reusable rules, substitutable backends, and tests that can exercise one layer at a time. Subsequent chapters cover how configuration is layered into this structure and how the dependency injection container wires the concrete implementations together at startup.

PART III · APPLICATION DEVELOPMENT

CHAPTER 10

Configuration and Environments

Watch the presentation

Se presentationen på svenska

An application that runs on a developer's laptop, on a build server, and in a production cluster needs different connection strings, different log levels, and different credentials in each location. Hardcoding these values into source code forces a recompile for every environment and pushes secrets into version control. ASP.NET Core solves this by separating settings from code and reading them through a layered configuration system that resolves the right value for the environment the process is running in. This chapter explains how that system

is assembled, how strongly-typed bindings expose it to application code, and where the boundary between development-time and production-time secret storage falls.

10.1 Why configuration is separate from code

The twelve-factor methodology introduced in Part I treats configuration as anything that varies between deploys: hostnames, credentials, feature flags, log levels. Code that contains these values directly cannot move between environments without modification, which means the artifact tested in development is not the artifact deployed to production. Externalizing configuration restores that property — the same compiled binary boots with development settings on one machine and production settings on another, and only the surrounding values differ.

Three forces push the same direction. Secrets must not enter source control because git history is permanent and repository access is broader than production access. Operators need to change settings without rebuilding the application, which rules out compiled constants. Multiple deploys of the same binary — a staging slot and a production slot, or two regions — must coexist with different configuration without divergent code paths. ASP.NET Core's configuration system addresses all three by treating settings as data the host loads at startup.

10.2 The configuration provider chain

ASP.NET Core builds configuration from a chain of **configuration providers**, each contributing key-value pairs into a

single hierarchical tree. The default chain that `WebApplication.CreateBuilder(args)` registers is, in order:

1. `appsettings.json`
2. `appsettings.{Environment}.json`
3. User-secrets (only when the environment is `Development`)
4. Environment variables
5. Command-line arguments

Providers are read in order and later providers override earlier ones for any key they supply. A connection string defined in `appsettings.json` is replaced by the same key in `appsettings.Production.json` when the production environment is active, and replaced again if an environment variable with the matching name is set. Keys that no provider supplies simply do not exist; reading them returns `null`.

10.2.1 IConfiguration as the read interface

`IConfiguration` is the ASP.NET Core abstraction that reads settings from a chain of providers — `appsettings.json`, environment-specific overrides, environment variables, command-line arguments, and user-secrets — and exposes them as a hierarchical key-value tree. Application code receives `IConfiguration` through dependency injection (covered in the dependency injection chapter) and reads values by key path:

```
public class HomeController : Controller
{
    private readonly string _connectionString;

    public HomeController(IConfiguration configuration)
    {
        _connectionString = configuration["Mon-
        goDB:ConnectionString"];
    }
}
```

The colon in `"MongoDB:ConnectionString"` is the section separator. ASP.NET Core treats the configuration tree as nested objects; a JSON file with `{ "MongoDB": { "ConnectionString": "..." } }` produces the key path `MongoDB:ConnectionString` regardless of which provider supplied it. The consumer never asks which file or environment variable a value came from — the abstraction collapses the chain into a single lookup.

10.2.2 Appsettings.json and environment-specific files

`appsettings.json` is the default configuration file shipped with an ASP.NET Core project; environment-specific files such as `appsettings.Development.json` and `appsettings.Production.json` layer on top of it and override individual keys when the matching environment is active. The base file holds settings shared across all environments — log categories, default feature flags, structural defaults — and the environment file holds only the deltas. A developer who opens `appsettings.Development.json` sees a small file containing only what differs from the base, not a full duplicate of every setting.

The file naming follows a strict convention. The host reads the `ASPNETCORE_ENVIRONMENT` variable, looks for `appsettings.<value>.json`, and merges it on top of `appset-`

`tings.json`. Common values are `Development`, `Staging`, and `Production`, but any string is allowed; teams that run integration test suites against a dedicated environment often define `appsettings.IntegrationTest.json` and set `ASPNETCORE_ENVIRONMENT=IntegrationTest` for that pipeline stage.

10.3 Environment selection

The `ASPNETCORE_ENVIRONMENT` environment variable controls which environment-specific configuration file the host loads and which behavioural defaults the framework applies. When the variable is unset, ASP.NET Core defaults to `Production`, which is a deliberate safety choice — a misconfigured server is less dangerous when it assumes production-grade defaults than when it assumes development-grade ones such as detailed exception pages.

Three behaviours hinge on the environment value. The matching `appsettings.<env>.json` file is loaded. The user-secrets provider is registered only when the environment is `Development`. Middleware checks such as `app.Environment.IsDevelopment()` enable developer-only features (the developer exception page, runtime compilation of Razor views) without those features ever activating in production. The same compiled binary therefore behaves differently based on a single environment variable, with no conditional compilation involved.

In Visual Studio and `dotnet run`, the environment defaults to `Development` because the launch profiles in `Properties/launchSettings.json` set the variable explicitly. In a published deployment, the hosting platform sets it — Azure App Service, Container Apps, and Kubernetes all

expose the variable through their configuration UI or manifest.

10.4 User-secrets for development

Storing a database password in `appsettings.Development.json` works until the file enters version control, at which point the password is exposed to anyone with repository access. ASP.NET Core's user-secrets feature solves this by moving sensitive values out of the project directory entirely.

User-secrets is a development-only configuration provider that stores sensitive values outside the project directory (in the user's profile) so credentials are not committed to source control; production environments rely on environment variables or a secret store instead. The secrets file lives at `%APPDATA%\Microsoft\UserSecrets\<id>\secrets.json` on Windows and `~/.microsoft/usersecrets/<id>/secrets.json` on macOS and Linux, where `<id>` is a GUID stored in the `.csproj` file under `UserSecretsId`. Because the file lives outside the project tree, no `git add` can include it.

The workflow uses the `dotnet user-secrets` CLI:

```
dotnet user-secrets init
dotnet user-secrets set "MongoDB:ConnectionString" "mon-
        godb://localhost:27017/cloudsoft"
dotnet user-secrets set "Storage:AccountKey" "dev-key-
        value"
dotnet user-secrets list
```

The `init` command writes a `UserSecretsId` GUID into the `.csproj` file. The `set` command stores a key-value pair in the user-profile JSON file, using the same colon-separated key paths as `appsettings.json`. At runtime, when `ASPNETCORE_ENVIRONMENT=Development`, the user-secrets provider

reads that JSON file and exposes its keys through `IConfiguration`. Application code reads `configuration["MongoDB:ConnectionString"]` without knowing — or caring — whether the value came from `appsettings.json`, an environment file, or user-secrets.

User-secrets are explicitly not a production mechanism. The provider is not registered in non-development environments, and the storage location is the developer's user profile, not a server's filesystem. Production deployments use environment variables or a managed secret store (covered below).

10.5 Environment variables and the override precedence

An **environment variable** is a key-value pair set in the operating-system process environment; ASP.NET Core reads them as a configuration provider, which is the standard mechanism for supplying production secrets and per-deployment settings. The provider sits later in the chain than the JSON files, so any environment variable overrides a value with the same key from any file.

Hierarchical keys translate using a double-underscore separator because most operating systems prohibit colons in environment variable names. The key `MongoDB:ConnectionString` becomes the variable name `MongoDB__ConnectionString`. The double underscore is recognized by the environment-variables provider and converted back into a colon when populating the configuration tree, so the consumer reading `configuration["MongoDB:ConnectionString"]` receives the value transparently.

10.5.1 Worked example

Consider an `appsettings.json` that defines a default MongoDB connection string for local development:

```
{
  "MongoDB": {
    "ConnectionString": "mongodb://localhost:27017/cloud-
        soft",
    "DatabaseName": "cloudsoft"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information"
    }
  }
}
```

In development, this file alone is sufficient — the application connects to a MongoDB instance running locally on port 27017. In production, the same code must connect to a managed MongoDB cluster with credentials. Rather than editing the JSON file or adding an `appsettings.Production.json`, the deployment platform sets a single environment variable:

```
export MongoDB__ConnectionString="mongodb+srv://
        app:s3cret@cluster0.mongodb.net/cloudsoft"
```

When the application starts, the environment-variables provider sees `MongoDB__ConnectionString`, splits the double underscore into the path `MongoDB:ConnectionString`, and overrides the default value from `appsettings.json`. The compiled binary did not change, the JSON file did not change, but the runtime configuration tree now contains the production connection string. The companion exercise data layer walks through this exact pattern when wiring a controller to a MongoDB repository.

Command-line arguments sit at the end of the chain and override even environment variables. Passing `--MongoDB:ConnectionString="mongodb://staging-host:27017/test"` to `dotnet run` replaces whatever the environment variable supplied, which is convenient for one-off diagnostic runs.

10.6 Strongly-typed configuration with IOptions

Reading configuration through `configuration["MongoDB:ConnectionString"]` works but has two weaknesses. The key is a magic string — a typo compiles cleanly and fails only at runtime. The consumer must know the path of the section in the configuration tree, which couples it to the file layout. The **`IOptions<T>`** pattern fixes both issues by binding a section of configuration to a strongly-typed C# class and injecting it into services, decoupling the consumer from the configuration source and the path of the section in the underlying tree.

The pattern requires three pieces. A C# class with properties matching the configuration keys, a registration call in `Program.cs` that binds a configuration section to that class, and a constructor parameter of type `IOptions<T>` in the consumer:

```
public class MongoOptions
{
    public string ConnectionString { get; set; } =
        string.Empty;
    public string DatabaseName { get; set; } =
        string.Empty;
}

// Program.cs
builder.Services.Configure<MongoOptions>(
    builder.Configuration.GetSection("MongoDB"));

// Repository
public class SubscriberRepository : ISubscriberRepository
{
    private readonly MongoOptions _options;

    public SubscriberRepository(IOptions<MongoOptions>
        options)
    {
        _options = options.Value;
    }
}
```

The `Configure<MongoOptions>` call tells the dependency injection container to populate a `MongoOptions` instance from the `MongoDB` section of `IConfiguration` whenever a service requests `IOptions<MongoOptions>`. The repository receives the bound object and reads `_options.ConnectionString` as a typed property — a typo on the property name is a compile error, and the section path lives in exactly one place. Refactoring the file structure (renaming the `MongoDB` section to `Mongo`) requires changing one registration line, not every consumer.

`IOptions<T>` is a snapshot taken once at registration time and is safe for singleton services. Two related abstractions exist for cases where configuration may change at runtime: `IOptionsSnapshot<T>` re-binds per request, and `IOp-`

`tionsMonitor<T>` notifies subscribers of changes. The basic `IOptions<T>` is the right default for connection strings and other settings that do not change after startup.

10.7 The boundary between development and production secret stores

User-secrets and environment variables both keep credentials out of source control, but they differ in operational scope. User-secrets store values in a developer's home directory, accessible only to that user account on that machine. Environment variables on a production host are visible to anyone with shell access to the host, and platform configuration UIs that set them are typically protected by the same access controls as the deployment itself.

Neither mechanism is sufficient for production-grade secret management. Both store secrets in plaintext at rest, neither rotates them, and neither audits access. Production deployments use a dedicated secret store — Azure Key Vault, AWS Secrets Manager, HashiCorp Vault — that encrypts secrets at rest, controls access through identity-based policies, logs every read, and supports rotation without redeployment. The application receives a managed identity from the hosting platform, and the secret store grants that identity permission to read specific secrets. The full mechanism, including managed identities and Key Vault integration, is covered in Part V.

The configuration provider chain already accommodates this. A Key Vault configuration provider can be added to the chain alongside the JSON and environment-variables providers, and the secrets it loads appear as ordinary `IConfiguration` keys. Application code that reads `configuration["MongoDB:ConnectionString"]` does not change when

the source migrates from an environment variable to Key Vault — the abstraction is the same. The boundary is therefore an operational one, not an application-code one: development uses user-secrets, staging and production use the platform's secret store, and the consuming code is identical across all three.

PROVIDER	SCOPE	WHEN TO USE	LIVES WHERE
`appsettings.json`	All environments	Non-secret defaults	Source control
`appsettings.{Env}.json`	Specific environment	Non-secret per-env overrides	Source control
User-secrets	Development only	Local credentials, API keys	Developer home directory
Environment variables	Any environment	Per-deployment settings, production secrets	Process environment
Secret store (Part V)	Staging, Production	Production-grade secret management	Cloud-managed vault

10.8 Summary

Configuration in ASP.NET Core is loaded by a chain of providers — `appsettings.json`, `appsettings.{Environment}.json`, user-secrets, environment variables, and command-line arguments — that merge into a single hierarchical key-value tree exposed through `IConfiguration`. The `ASPNETCORE_ENVIRONMENT` variable selects which environ-

ment-specific JSON file is layered on top of the base file and gates developer-only features such as the user-secrets provider and the developer exception page. Hierarchical keys map between providers using a colon in JSON and a double underscore in environment variable names, so `MongoDB:ConnectionString` and `MongoDB__ConnectionString` address the same value. The `IOptions<T>` pattern binds a configuration section to a strongly-typed class, removing magic strings from consumer code and decoupling consumers from the source and path of their settings. User-secrets and environment variables keep credentials out of source control during development and routine deployment, but production-grade secret management — encryption at rest, access auditing, rotation — requires a managed secret store layered into the same provider chain, covered in Part V.

PART III · APPLICATION DEVELOPMENT

CHAPTER 11

Dependency Injection

Watch the presentation

Se presentationen på svenska

A controller that needs to send a newsletter cannot send the newsletter itself — it relies on a service to validate the subscriber, on a repository to persist the address, and on a logger to record the outcome. The naive way to obtain those collaborators is for the controller to construct them with `new` inside its own body. That choice locks the controller to one specific implementation, makes the class unusable in tests without spinning up real infrastructure, and silently spreads object-graph wiring through every layer. This chapter devel-

ops a different approach: declare the collaborators a class needs, and let a runtime container build the graph.

11.1 The problem with constructing collaborators directly

A class that creates its own collaborators carries two responsibilities at once: doing its own work, and assembling the objects it depends on. The two responsibilities pull in opposite directions. The work changes when the business rules change; the assembly changes when the implementation choices change. Mixing them together violates the separation of concerns principle that the three-tier architecture depends on.

Consider a controller written this way:

```
public class NewsletterController : Controller
{
    private readonly NewsletterService _service =
        new NewsletterService(new InMemorySubscriberRepos-
        itory());

    public IActionResult Subscribe(string email) =>
        _service.Subscribe(email) ? Ok() : BadRequest();
}
```

The controller now knows the concrete service type, the concrete repository type, and the order in which to construct them. Replacing the in-memory repository with one that talks to a database means editing the controller. Writing a unit test that does not actually store data means editing the controller. The controller is no longer focused on routing requests — it has become an object factory as well.

11.2 Inverting the dependency

The fix is to invert who controls construction. Rather than the class reaching out for its collaborators, the class declares what it needs and waits for them to be supplied. **Dependency injection** (DI) is a design pattern in which a class declares the services it needs through constructor parameters and a runtime container supplies the concrete implementations, decoupling the class from the lifecycle and choice of those services.

The same controller, written for DI, looks like this:

```
public class NewsletterController : Controller
{
    private readonly INewsletterService _service;

    public NewsletterController(INewsletterService service)
    {
        _service = service;
    }

    public IActionResult Subscribe(string email) =>
        _service.Subscribe(email) ? Ok() : BadRequest();
}
```

The controller no longer says how its service is built. It states a requirement: it needs something that satisfies `INewsletterService`. Whether the supplied object is a `NewsletterService` backed by a real database, an `InMemoryNewsletterService` used during development, or a `FakeNewsletterService` constructed in a unit test is a decision made elsewhere. The controller continues to work as long as the contract is honoured.

This style is called **constructor injection** — the dependency-injection style in which a class declares its required services as constructor parameters; the container resolves and

supplies them when it instantiates the class, making the dependencies explicit and immutable. The constructor is the only place a dependency enters, the field that holds it is `readonly`, and the rest of the class can assume the dependency is present and valid.

11.3 The ASP.NET Core DI container

The runtime piece that makes this work is the **DI container**. ASP.NET Core ships with one built in. At application startup, the container is told which interfaces map to which implementations and how long each instance should live. At runtime, when the framework needs to instantiate a controller, it inspects the controller's constructor, looks up each parameter type in the container, builds those dependencies (recursively resolving their own dependencies), and invokes the constructor with the resulting graph. The application code never calls `new` for any of these classes.

Registration happens through **`IServiceCollection`** — the ASP.NET Core builder used at startup (in `Program.cs`) to register services with the dependency injection container; each call associates a service type with a concrete implementation and a lifetime. Inside the project template generated by `dotnet new mvc`, `IServiceCollection` is exposed as `builder.Services`.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();
builder.Services.AddScoped<ISubscriberRepository,
        InMemorySubscriberRepository>();
builder.Services.AddScoped<INewsletterService, Newslet-
        terService>();

var app = builder.Build();
```

Three things are registered. The framework's MVC services come from the `AddControllersWithViews` extension. The application's repository is registered with the service type `ISubscriberRepository` mapped to the concrete `InMemorySubscriberRepository`. The application's service is registered with the service type `INewsletterService` mapped to the concrete `NewsletterService`. After `builder.Build()` runs, the container is sealed and ready to resolve graphs on demand.

When a request to `/Newsletter/Subscribe` arrives, the framework sees that the matched action lives on `NewsletterController`. It asks the container for a `NewsletterController`. The container reads the constructor parameter list — `INewsletterService` — and asks itself for one. The `NewsletterService` constructor in turn asks for an `ISubscriberRepository`, which the container supplies as an `InMemorySubscriberRepository`. The whole graph is built top-down without any explicit wiring code in the controller, the service, or the repository.

11.3.1 Interface-based vs concrete-type registration

Both registration styles are valid. Interface-based registration — `AddScoped<INewsletterService, NewsletterService>()` — is the common form for application code because it pre-

serves the abstraction the layers are built on. Consumers depend on the interface, the container produces the implementation, and the implementation can be swapped without touching the consumers.

Concrete-type registration — `AddScoped<NewsletterService>()` — registers a single class with no interface in front of it. It still gives the container responsibility for construction and lifetime, but it offers no substitution point. Concrete registration suits internal helper classes that have only one implementation and no value as a test seam. The general guideline is to register against an interface whenever the class crosses a layer boundary or might be replaced by a fake during testing.

11.4 Service lifetimes

Registration also declares how long an instance lives. A **service lifetime** controls how often the dependency injection container creates a new instance: **Singleton** lasts for the application lifetime, **Scoped** lasts for one request, and **Transient** creates a new instance every time the service is resolved.

The three lifetimes correspond to three registration methods on `IServiceCollection`:

METHOD	LIFE-TIME	WHEN THE CONTAINER CREATES A NEW INSTANCE
`AddSingleton<TService, TImpl>()`	Singleton	Once, the first time the service is resolved; reused for the entire application
`AddScoped<TService, TImpl>()`	Scoped	Once per HTTP request; reused for every resolution within that request
`AddTransient<TService, TImpl>()`	Transient	Every time the service is resolved, regardless of request

11.4.1 Singleton

A singleton instance is created once and shared by every component that depends on it for the lifetime of the process. Singletons fit services that hold expensive-to-build but immutable state — a parsed configuration, an in-memory cache that the application owns end-to-end, an HTTP client whose connection pool should be reused. Because the same instance is shared across concurrent requests, a singleton must be thread-safe; any mutable state inside it is a hazard.

Registering an in-memory repository as a singleton is appropriate when the in-memory store is the source of truth for the running process, and the underlying collection is built for concurrent access. The exercise version uses `ConcurrentDictionary` precisely so the repository can be shared safely.

11.4.2 Scoped

A scoped instance is created once per request and discarded when the request ends. Within a single request, every component that resolves the service receives the same instance, which is useful when several layers need to participate in the same logical operation. Most application services and repositories are scoped because they often hold per-request state — for example, an Entity Framework `DbContext` that tracks entities loaded during the current operation.

Scoped is the safe default for application code that talks to per-request resources. The lifetime aligns with the natural unit of work in a web application: one request in, one response out, no shared state crossing the boundary.

11.4.3 Transient

A transient instance is created every time the service is resolved, even within the same request. Transient suits lightweight stateless helpers — a small validator, a strategy object that is consulted once and thrown away. Transient is the most expensive lifetime in terms of allocations, so it should not be used for services that are heavy to construct or that intentionally cache work.

11.4.4 Captive dependencies

The lifetimes interact in a way that creates a real failure mode. A singleton lives for the whole application; a scoped service lives for one request. If a singleton declares a constructor parameter of a scoped type, the container resolves the scoped service the first time the singleton is built — and that scoped instance is then held by the singleton for the rest of the application's life. The scoped service has effectively been promoted to a singleton, even though its implementa-

tion may not be safe to share across requests. This is called a *captive dependency*.

The reverse is fine: a scoped service may depend on a singleton. The danger is always the longer-lived consumer holding the shorter-lived service. A repository that maintains per-request state but is captured inside a singleton service will leak state from one user's request into another's. ASP.NET Core can detect some of these cases and throw at startup when validation is enabled, but the responsibility ultimately rests with the registration choices in `Program.cs`.

11.5 A worked example: the newsletter service

The service layer exercise builds a small newsletter feature against this pattern. The data layer holds subscriber email addresses; the service layer enforces uniqueness and basic validation; the presentation layer exposes a controller action.

The repository contract describes what a subscriber store can do, without saying how it stores anything:

```
public interface ISubscriberRepository
{
    Task<bool> AddAsync(string email);
    Task<bool> ExistsAsync(string email);
}
```

The in-memory implementation backs the contract with a `ConcurrentDictionary` so that several concurrent requests can read and write safely:

```
public class InMemorySubscriberRepository : ISubscriberRe-
        pository
{
    private readonly ConcurrentDictionary<string, byte>
        _subscribers =
        new(StringComparer.OrdinalIgnoreCase);

    public Task<bool> AddAsync(string email) =>
        Task.FromResult(_subscribers.TryAdd(email, 0));

    public Task<bool> ExistsAsync(string email) =>
        Task.FromResult(_subscribers.ContainsKey(email));
}
```

The service contract describes the business operation, without referring to where data lives:

```
public interface INewsletterService
{
    Task<bool> SubscribeAsync(string email);
}
```

The implementation depends on the repository through its interface, validates the input, and delegates persistence:

```
public class NewsletterService : INewsletterService
{
    private readonly ISubscriberRepository _repository;

    public NewsletterService(ISubscriberRepository reposit-
        ory)
    {
        _repository = repository;
    }

    public async Task<bool> SubscribeAsync(string email)
    {
        if (string.IsNullOrWhiteSpace(email)) return false;
        if (await _repository.ExistsAsync(email)) return
        false;
        return await _repository.AddAsync(email);
    }
}
```

The controller accepts the service through its constructor. It does not know which repository the service uses, nor does the service know which controller called it:

```
public class NewsletterController : Controller
{
    private readonly INewsletterService _service;

    public NewsletterController(INewsletterService service)
    {
        _service = service;
    }

    [HttpPost]
    public async Task<IActionResult> Subscribe(string
        email) =>
        await _service.SubscribeAsync(email) ? Ok() : Bad-
        Request();
}
```

Wiring all three together happens in one place, `Program.cs`:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();
builder.Services.AddSingleton<ISubscriberRepository,
        InMemorySubscriberRepository>();
builder.Services.AddScoped<INewsletterService, Newslet-
        terService>();

var app = builder.Build();
```

The repository is registered as a singleton because the in-memory store *is* the application's data — a new instance every request would lose every subscriber. The service is registered as scoped because it carries no state of its own and pairs naturally with the per-request lifetime that real database-backed services will use later. The controller is not registered explicitly; ASP.NET Core's MVC integration registers controllers automatically and treats them as transient.

A swap to a database-backed repository later requires only one edit in `Program.cs`: replace `InMemorySubscriberRepository` with `SqlSubscriberRepository`, change the lifetime to scoped to match the database session, and the controller and service compile and run unchanged.

11.6 Dependency injection enables unit testing

The same indirection that lets the registration line change without touching the controller also lets a test substitute a fake at construction time. A unit test for the controller does not need a database, an HTTP server, or even the DI container

itself — it can hand-construct the controller with a fake service:

```
public class FakeNewsletterService : INewsletterService
{
    public bool ReturnValue { get; set; }
    public string? CapturedEmail { get; private set; }

    public Task<bool> SubscribeAsync(string email)
    {
        CapturedEmail = email;
        return Task.FromResult(ReturnValue);
    }
}

[Fact]
public async Task Sub-
        scribe_returns_BadRequest_when_service_rejects()
{
    var fake = new FakeNewsletterService { ReturnValue =
        false };
    var controller = new NewsletterController(fake);

    var result = await control-
        ler.Subscribe("dup@example.com");

    Assert.IsType<BadRequestResult>(result);
    Assert.Equal("dup@example.com", fake.CapturedEmail);
}
```

The test is fast, deterministic, and isolated. It exercises the controller's behaviour — translating a service result into an HTTP outcome — without exercising any real service logic, persistence, or framework plumbing. The same shape of test applies to the service: a fake repository in, the service under test, no infrastructure required. This testability is not an accidental benefit; it is the direct payoff for declaring dependencies through interfaces and constructor parameters.

11.7 Summary

Dependency injection moves construction out of the consumer and into a runtime container. A class declares the services it needs as constructor parameters; the ASP.NET Core DI container reads those parameters, resolves each one against its registrations, and supplies the graph. Registration happens once at startup through `IServiceCollection` calls in `Program.cs`, and each call binds a service type — usually an interface — to a concrete implementation and a lifetime. The three lifetimes are Singleton (one instance for the whole application), Scoped (one instance per request), and Transient (a new instance every time the service is resolved); registering a shorter-lived service inside a longer-lived consumer captures it and produces a captive-dependency bug. Constructor injection makes dependencies explicit and immutable, decouples each layer from the implementation choices of the layers beneath it, and turns unit testing into a matter of passing a fake into the constructor. The newsletter exercise applies the pattern end-to-end: a repository interface, a service interface, a single registration block, and a controller that knows nothing about how its collaborators are built.

PART IV

Data Access

Cloud applications stand on persistence. This Part covers the four ideas an application developer needs to write data-access code that holds up in production: how relational and document data models differ, how an ORM and the repository pattern keep query syntax out of business logic, how connections and transactions behave under concurrent load, and how object storage handles the binary blobs that databases handle badly.

Each chapter pairs with practical work in the Cloud Databases exercise chapter or the data layer exercise.

PART IV · DATA ACCESS

CHAPTER 12

Relational vs NoSQL Data Models

Watch the presentation
Se presentationen på svenska

Application data rarely arrives in one tidy shape. A banking ledger needs every cent accounted for and every transfer balanced. A product catalogue grows new attributes whenever the merchandising team launches a category. A telemetry feed writes millions of small records that are read back as time ranges. The choice of database model — how records are structured, queried, and kept consistent — determines how naturally the storage layer fits each of those workloads. The two dominant choices are the relational model and the docu-

ment model that dominates the NoSQL family, and the trade-offs between them shape every downstream decision a data layer makes.

12.1 How the two models structure records

The relational model splits each entity into a row inside a table. Every table has a fixed set of columns with declared types, and rows in different tables connect to each other through foreign-key columns whose values must reference an existing primary key. A purchase, for example, lives across an `orders` row, several `order_items` rows, a `customers` row, and a `products` row, stitched together with `customer_id` and `product_id` foreign keys. The structure is normalised — each fact lives in exactly one place, and queries reassemble the full picture by joining tables.

The document model, the most common variant of the NoSQL family, takes the opposite approach. A **document database** stores each record as a self-describing document (typically JSON or BSON), grouped into collections; queries match documents by field values without the join machinery a relational database uses, and individual documents may have heterogeneous shapes. The same purchase becomes a single `Order` document containing the customer's name, an array of line items with embedded product names, and the shipping address — everything the typical read needs, in one place. Where the relational design optimises for storing each fact once, the document design optimises for retrieving everything related to one entity in one round trip.

12.1.1 Schema as a contract

The structural difference is enforced through the **schema**. A schema is the formal description of the structure a data store

enforces — table and column definitions in a relational database, or the set of fields a document is expected to carry in a NoSQL store; relational schemas are rigid and require migrations to change, while document schemas are flexible and can vary per document. In PostgreSQL, the schema is declared with `CREATE TABLE` and enforced on every write — a row that violates a `NOT NULL` or foreign-key constraint is rejected before it touches disk. In MongoDB or Azure Cosmos DB for MongoDB, the schema lives in application code: the database accepts whatever document the client sends, and any structural rules are enforced by the application or by an optional validation expression attached to the collection.

This shift moves a category of bugs. With a rigid schema, the database catches type mismatches, missing fields, and orphan references at write time. With a flexible schema, those mistakes only surface when a query expects a field a particular document never had — typically inside a code path that has already been deployed. The trade-off is real, and it is the load-bearing decision in this chapter.

12.2 Schema rigidity and the cost of migration

Rigid schemas pay their cost upfront, in the form of *migrations*. Adding a column to a relational table is not a one-line code change — it is a coordinated operation that locks (or partially locks) the table, rewrites or backfills existing rows, and ships in lockstep with the application code that knows about the new column. Tooling such as Entity Framework migrations, Flyway, or Liquibase exists to make this disciplined: each schema change is a versioned script, applied in order, recorded in a migrations table the database itself owns.

Document databases avoid the lockstep deployment but do not eliminate the underlying problem. New code that writes a

new field starts producing documents shaped differently from the older ones already in the collection. The collection now contains a mixture, and any read path has to handle both shapes — usually with a default value in code, or with a back-fill job that rewrites the older documents. The migration has not vanished; it has moved into application code, where it is harder to track and easier to forget.

The deciding question is who pays the cost. A small team iterating quickly on a feature whose data shape is still being discovered benefits from the document model — the schema is allowed to drift while the design settles. A team operating a system whose invariants matter (a payments ledger, an inventory of physical goods, a regulatory reporting database) benefits from the relational model — the database refuses bad data, and the migration record is auditable.

12.3 Denormalization as the answer to the join problem

A relational query joins tables to assemble a complete view. A document query reads one document. To make that single read sufficient, document designs use **denormalization** — the deliberate duplication of data across documents or tables so that a single read can satisfy a query without joining; common in document databases, it trades write complexity (keeping copies in sync) for query simplicity.

Consider the order example again. A relational design stores the product's name once, in the `products` table, and joins it in whenever an order is displayed. A document design copies the product name into every `Order` document at the time the order is placed. The read becomes one trip to the database; the write — when a product is later renamed — has to find every order that copied the old name and decide

whether to update it. Sometimes the right answer is to update; sometimes (an order's snapshot of the product as it was sold) the right answer is to leave it. Denormalization is not a workaround; it is an explicit modelling choice that makes the trade-off visible.

The boundary of a document — what belongs inside it and what is referenced by id — is the central design decision in a document store. Embed too little and the application is reduced to fetching documents in a loop, recreating joins in client code with worse performance. Embed too much and a single document grows past practical size limits, becomes a write-contention hot spot, or forces unrelated updates to compete for the same record's lock.

12.4 ACID, CAP, and eventual consistency

Relational databases historically prioritise ACID guarantees: a transaction either commits all of its changes or rolls back all of them, and committed changes are durable. These guarantees are why a bank transfer feels safe — the debit and credit succeed together, or neither happens.

Distributed databases must contend with the CAP theorem, which states that under a network partition a system has to choose between consistency and availability. Many NoSQL stores choose availability: they keep accepting reads and writes on each side of a partition and reconcile afterwards. The model they expose is **eventual consistency** — the guarantee that all replicas of a piece of data will converge to the same value once writes stop, but at any single moment a read may return stale data; this is the consistency model many distributed NoSQL stores choose in exchange for availability under network partitions.

The application-visible effect is that a write the user just made may not appear in the next read served by a different replica. Cosmos DB exposes this as a configurable knob: the `Strong` consistency level serialises every read against the latest committed write at a single-region cost in latency, while `Eventual` accepts that some reads will be stale in exchange for the lowest latency and highest availability. Levels in between — `Bounded staleness`, `Session`, `Consistent prefix` — let an application name exactly how much staleness it is willing to tolerate.

Eventual consistency is acceptable for product catalogues, social feeds, telemetry, and analytics — workloads where a few seconds of staleness is invisible. It is not acceptable for ledgers, inventory of finite physical goods, or anywhere else two stale reads can produce a conflicting outcome that cannot be reconciled.

12.5 Worked example: a Cosmos DB collection for the MongoDB API

The companion exercise Portal Interface walks through provisioning an Azure Cosmos DB account configured for the MongoDB API. The act of creating the account never asks for a schema. Once the account exists, a database is created (a logical grouping), and inside it a collection — the document store's equivalent of a table. The collection demands one decision the relational equivalent does not: a *shard key*, the field Cosmos uses to distribute documents across partitions.

The follow-up exercise Connecting from Code uses the official MongoDB driver to read and write documents from .NET. A typical insert looks like this:

```
var client = new MongoClient(connectionString);
var database = client.GetDatabase("shop");
var orders = data-
        base.GetCollection<BsonDocument>("orders");

var order = new BsonDocument
{
    { "customer", new BsonDocument { { "name", "A. Lind-
        berg" }, { "email", "a@example.com" } } },
    { "items", new BsonArray
        {
            new BsonDocument { { "sku", "BK-042" }, {
        "title", "Domain-Driven Design" }, { "qty", 1 },
        { "price", 549 } }
        }
    },
    { "total", 549 },
    { "placedAt", DateTime.UtcNow }
};

await orders.InsertOneAsync(order);
```

The document carries the customer's name and email inline, and the line item carries the product title and price inline. Nothing in the database enforced that shape — the server accepted the document as written, and another insert against the same collection could carry a different set of fields. A read by customer email is a one-document fetch:

```
var filter = Builders<BsonDocument>.Filter.Eq("cus-
        tomer.email", "a@example.com");
var match = await
        orders.Find(filter).FirstOrDefaultAsync();
```

The same query in a normalised relational design would join `orders`, `order_items`, `products`, and `customers`. The document version is one round trip and one server-side index lookup; the trade-off is that the product's title is a copy,

and a later rename of the product has to be propagated explicitly if business rules require it.

12.6 A decision framework

The two models are not interchangeable, and the choice shapes downstream decisions about migrations, query design, and consistency. The following table captures the trade-offs that decide most cases:

CONCERN	RELATIONAL	DOCUMENT
Schema enforcement	At write time, by the database	In application code or optional validators
Shape evolution	Versioned migrations, lockstep with code	Mixed-shape collections, handled in code
Joins across entities	First-class, declarative	Avoided through denormalization
Transactions across many entities	Standard (multi-row, multi-table)	Limited; per-document atomicity is the norm
Consistency under partition	Typically strong, at the cost of availability	Often eventual, configurable in Cosmos DB
Best-fit workloads	Ledgers, inventory, regulated data	Catalogues, content, telemetry, user profiles
Best-fit team posture	Invariants matter; data lives longer than code	Schema is still being discovered; iteration speed matters

Several heuristics emerge from those trade-offs. Choose a relational database when the data has rich, stable relationships and the cost of an invalid row is high. Choose a document database when the dominant query reads one entity with all of its associated data, when the schema is genuinely evolving, or when horizontal scale and global distribution outweigh the convenience of joins. The cloud platforms make both choices managed services — Azure SQL Database and Azure Database for PostgreSQL on the relational side, Cosmos DB across document, key-value, and graph models on the NoSQL side — so the decision rests on data shape and consistency needs, not on operational capacity.

A frequent and reasonable answer is *both*. A retail system might keep its orders, products, and stock levels in PostgreSQL, where ACID transactions guarantee that a sold item is decremented from inventory atomically, and keep its product catalogue and search index in Cosmos DB, where flexible attributes and global reads matter more than transactional consistency. Picking a model per bounded context — rather than declaring one database the system's database — is usually the design that ages best.

12.7 Summary

Relational and document data models structure records differently, and that difference cascades through schema management, query design, and consistency guarantees. Relational schemas are enforced at write time and evolved through versioned migrations; document schemas are enforced in application code and evolve as a mixed-shape collection. Document designs use denormalization to make a single read sufficient, trading write-time complexity for read-time simplicity. Distributed NoSQL stores often expose even-

tual consistency as the price of remaining available under network partitions, and Cosmos DB makes that knob explicit. The right model depends on the shape of the data and the cost of staleness — and a system serving more than one workload will frequently use both.

PART IV · DATA ACCESS

CHAPTER 13

ORM and the Repository Pattern

Watch the presentation
Se presentationen på svenska

A web application that mixes SQL strings or MongoDB filter expressions directly into its controllers and services becomes hard to change. The query syntax leaks into request handlers, the same lookup gets rewritten in three places with subtle differences, and every test of the business logic has to spin up a database. Two patterns address this: an **ORM** translates between database rows and in-memory objects so the code stops dealing in raw SQL, and the **repository pattern** wraps whichever data-access library is in use behind a single inter-

face that the rest of the application calls. The two compose, and each carries trade-offs that decide when its abstraction cost is worth paying.

13.1 Why raw data access becomes a liability

A handler that posts a newsletter to subscribers might begin life as a few lines of MongoDB driver code embedded in the controller — open the collection, build a filter, run the query, deserialize the result. The same shape repeats in three or four places: the admin dashboard listing newsletters, the public archive page, the notification job. Each copy carries the database client's vocabulary into a part of the code that has nothing to do with persistence.

That coupling has measurable costs. Renaming a field in the schema means searching every file for the field name. Replacing MongoDB with a relational store would require rewriting every handler that touches data. Tests that exercise business rules need a real database because there is no seam to substitute a fake. And the rules themselves — what counts as a published newsletter, which subscribers are active — get smeared across the codebase next to query syntax, making them hard to find when they need to change.

The standard response to this is to push data access into a dedicated layer. The three-tier architecture chapter introduced that split: a presentation layer takes requests, a service layer enforces business rules, and a data layer talks to the database. The two patterns this chapter covers are the standard tools for building that data layer.

13.2 What an ORM does

An **ORM** (Object-Relational Mapper) is a library that translates between database rows and in-memory objects; the developer writes class definitions, and the ORM generates the SQL needed to load, save, and query those classes, often with change tracking that detects modified properties and writes only the deltas.

The translation runs in both directions. When a record is loaded, the ORM reads the row and populates the matching properties of an instance — column `published_at` becomes `Newsletter.PublishedAt`, foreign keys become navigation properties pointing at related entities. When the instance is saved, the ORM emits an `INSERT` or `UPDATE` statement that mirrors the changes. The application code never writes the SQL itself; it works with objects and method calls.

Three capabilities distinguish a full ORM from a simple mapper:

- **SQL generation** — the ORM produces the query text from a higher-level expression. Entity Framework Core in .NET, for example, takes a LINQ expression like `db.Newsletters.Where(n => n.PublishedAt != null).OrderByDescending(n => n.PublishedAt)` and emits parameterised SQL targeting the configured provider.
- **Change tracking** — once an entity is loaded, the ORM records its original values. On save, it compares current values against the snapshot and writes only the columns that changed. The application sets a property; the ORM works out whether to issue an `UPDATE` and what to put in the `SET` clause.
- **Identity management** — the ORM maintains one in-memory instance per primary key within a unit of work, so two queries that touch the same row return the same

object. Updates therefore stay consistent across the active session.

ORMs targeting document databases — sometimes called ODMs (Object-Document Mappers) — apply the same idea to documents. The schema lives in class definitions, and the library serialises objects to BSON for MongoDB or to JSON for other document stores. The mechanics differ (no joins, no foreign keys), but the principle is identical: the application works with typed objects, and the library handles the wire format.

The cost is indirection. Generated SQL is sometimes less efficient than a hand-tuned query. Change tracking adds memory and CPU per loaded entity. Complex queries can be awkward to express through the ORM's expression API and may force a drop down to raw SQL anyway. None of these are reasons to avoid an ORM in a typical web application — they are reasons to know what the ORM is doing on the developer's behalf.

13.3 The repository pattern

The **repository pattern** is a design pattern in which a single class (the repository) hides all data-access code behind an interface; service-layer code calls methods like `FindByIdAsync` or `InsertAsync` and the repository translates these into the underlying database client's calls, keeping query syntax out of business logic.

A repository exposes a small, deliberate vocabulary that matches the domain. Instead of `IMongoCollection<Newsletter>` with its `Find`, `Aggregate`, and `BulkWrite` methods, the rest of the code sees `INewsletterRepository` with methods like `GetPublishedAsync`, `FindBySlugAsync`, and `AddAsync`. The repository's job is to turn those

domain-shaped calls into whatever the underlying client requires.

That seam buys three things. First, the service layer stops importing the database driver — it depends on an interface, not on `MongoDB.Driver` or `Microsoft.EntityFrameworkCore`. Second, swapping the storage technology becomes a contained change: a SQL-backed implementation of the same interface can drop in beside the MongoDB one. Third, unit tests can substitute a fake repository in milliseconds, so the service layer can be tested without a database.

A repository typically supports the four **CRUD** operations — **Create** (insert new records), **Read** (query existing records), **Update** (modify records), and **Delete** (remove records). Repository interfaces typically expose one method per operation, often with additional domain-specific finders layered on top.

13.3.1 Generic versus domain-specific repositories

Two styles dominate. A *generic repository* is parameterised over the entity type — `IRepository<T>` — and exposes the CRUD operations that apply to any entity. A *domain-specific repository* declares an interface per aggregate — `INewsletterRepository`, `ISubscriberRepository` — and adds methods that only make sense for that type.

Generic repositories pay off when the application has many small entities with similar access patterns: load by id, save, list. Writing one `MongoRepository<T>` and registering it for each entity removes a lot of boilerplate. The cost is that domain-specific queries — “find all newsletters published in the last thirty days that have at least one open” — do not fit the generic interface. Forcing them through it tends to push

query construction back into the service layer, which is exactly what the repository was meant to prevent.

Domain-specific repositories carry the opposite trade-off. They contain every query relevant to the domain object, so the service layer's calls read like business operations. They also cost more code: each new aggregate gets its own interface and implementation, and shared CRUD methods get redeclared per repository unless an inherited base class supplies them.

The common compromise is a generic base — `IRepository<T>` with the universal methods — that domain-specific interfaces extend with their own finders. The base implementation supplies CRUD; the derived repository adds whatever the service layer actually asks for.

STYLE	BEST WHEN	TRADE-OFF
Generic `IRepository<T>`	Many small entities, mostly CRUD access	Domain-specific queries leak back into the service layer
Domain-specific `IXRepository`	Each aggregate has a distinct query surface	More interfaces and implementations to maintain
Generic base + domain-specific extension	Mix of universal CRUD and a few targeted queries	Slight extra setup; usually the right default

13.4 How ORM and repository compose

The two patterns operate at different layers, and a typical application uses both. The ORM (or ODM) sits at the bottom and handles the row-to-object translation. The repository sits

on top of the ORM and presents a domain-shaped API to the service layer.

Without the repository, services would talk to `DbContext` or `IMongoCollection<T>` directly. The query syntax would still leak across the codebase, just expressed in LINQ instead of SQL. With the repository in place, the ORM is an implementation detail of the data layer — visible inside `MongoRepository<T>` and `EfRepository<T>`, but not outside.

The reverse arrangement (a repository without an ORM) is also common. A MongoDB-backed repository typically calls the official driver directly rather than through an ODM; the driver already deserialises BSON into typed objects, which covers most of what an ODM would add. The repository wraps that driver behind the domain interface.

The composition matters because the patterns solve different problems. The ORM's job is to remove SQL or driver vocabulary from the data layer; the repository's job is to remove data-layer vocabulary from the service layer. Each one alone leaves the other concern unsolved.

13.4.1 Worked example: an `INewsletterRepository`

The companion exercise Data layer builds a repository over MongoDB for a newsletter application. The interface declares the operations the service layer needs:

```
public interface INewsletterRepository
{
    Task<Newsletter?> FindByIdAsync(string id);
    Task<IReadOnlyList<Newsletter>> GetPublishedAsync();
    Task<Newsletter?> FindBySlugAsync(string slug);
    Task AddAsync(Newsletter newsletter);
    Task UpdateAsync(Newsletter newsletter);
    Task DeleteAsync(string id);
}
```

A generic base supplies CRUD against any MongoDB collection:

```
public class MongoRepository<T> where T : class
{
    private readonly IMongoCollection<T> _collection;

    public MongoRepository(IMongoDatabase database, string
        collectionName)
    {
        _collection = database.GetCollection<T>(collection-
        Name);
    }

    public async Task<T?> FindByIdAsync(string id) =>
        await _collec-
        tion.Find(Builders<T>.Filter.Eq("_id",
        ObjectId.Parse(id)))
                        .FirstOrDefaultAsync();

    public Task AddAsync(T entity) =>
        _collection.InsertOneAsync(entity);

    public Task UpdateAsync(string id, T entity) =>
        _collec-
        tion.ReplaceOneAsync(Builders<T>.Filter.Eq("_id",
        ObjectId.Parse(id)), entity);

    public Task DeleteAsync(string id) =>
        _collec-
        tion.DeleteOneAsync(Builders<T>.Filter.Eq("_id",
        ObjectId.Parse(id)));

    protected IMongoCollection<T> Collection => _collec-
        tion;
}
```

The newsletter-specific repository extends the generic base with the domain finders:

```
public class NewsletterRepository : MongoRepository<News-
        letter>, INewsletterRepository
{
    public NewsletterRepository(IMongoDatabase database)
        : base(database, "newsletters") { }

    public async Task<IReadOnlyList<Newsletter>> GetPub-
        lishedAsync()
    {
        var filter = Builders<Newsletter>.Filter.Ne(n =>
        n.PublishedAt, null);
        var sort = Builders<Newsletter>.Sort.Descending(n
        => n.PublishedAt);
        return await Collec-
        tion.Find(filter).Sort(sort).ToListAsync();
    }

    public Task<Newsletter?> FindBySlugAsync(string slug)
        =>
        Collection.Find(Builders<Newsletter>.Filter.Eq(n
        => n.Slug, slug))
                  .FirstOrDefaultAsync();
}
```

Registration happens in `Program.cs` through dependency injection. The MongoDB client is registered as a singleton so the connection pool is shared across the application; the database handle is registered as a singleton too; the repository is registered behind its interface so the service layer can ask for `INewsletterRepository` and get the concrete implementation:

```
builder.Services.AddSingleton<IMongoClient>(sp =>
    new MongoClient(builder.Configuration.GetConnection-
        String("Mongo")));

builder.Services.AddSingleton(sp =>

        sp.GetRequiredService<IMongoClient>().GetDatabase("news-
        letter_app"));

builder.Services.AddScoped<INewsletterRepository, Newslet-
        terRepository>();
```

The service that publishes a newsletter now depends only on the interface:

```
public class NewsletterService
{
    private readonly INewsletterRepository _repo;

    public NewsletterService(INewsletterRepository repo)
        => _repo = repo;

    public async Task PublishAsync(string id)
    {
        var newsletter = await _repo.FindByIdAsync(id)
            ?? throw new NotFoundException(id);
        newsletter.PublishedAt = DateTime.UtcNow;
        await _repo.UpdateAsync(newsletter);
    }
}
```

The service contains no MongoDB types, no filter builders, no awareness of how `PublishedAt` becomes a BSON field. The repository owns those translations. Replacing MongoDB with a SQL store would mean writing a new `SqlNewsletterRepository` against the same interface and changing one DI registration; the service, the controller, and the tests stay the same.

13.5 When the abstraction is worth its cost

Adding the repository layer is not free. Every entity gets an interface and an implementation. Every domain query gets named twice — once on the interface, once in the implementation. New queries that the service layer needs cannot be expressed inline; they have to be added to the repository first. For a small application with a handful of entities and straightforward queries, this overhead can outweigh the benefit.

A useful rule of thumb: the repository pattern earns its cost when at least one of these is true.

- **The application has more than one consumer of the same data.** A controller, a background job, and a CLI tool all reading newsletters benefit from a single shared `GetPublishedAsync` rather than three near-identical implementations.
- **The service layer needs to be unit-tested without a database.** An interface seam is the cheapest way to substitute a fake.
- **The storage technology might change, or already differs across environments.** A staging environment on PostgreSQL and a production environment on a managed cloud SQL service can share a service layer if the repository hides the difference.
- **The team values keeping query syntax out of business logic on principle.** This is a real and defensible reason; it is also the reason most often cited and the one most likely to lead to over-abstraction when the other reasons do not apply.

If none of those apply — a single small service, queries used in one place, no testing without the database — then the layer of indirection mostly adds noise. A direct call to `DbContext` or `IMongoCollection<T>` from the service is fine, and the

repository can be introduced later when the costs of not having it start to show.

The same caution applies to over-generic repositories. A generic `IRepository<T>` with a fluent expression API, a specification pattern, and a custom query language tends to recreate the database driver one method at a time. At that point the abstraction is no longer hiding the driver; it is duplicating it. The interface is most valuable when it stays small and domain-shaped.

13.6 Summary

Mixing database-driver calls into application code couples business logic to a specific storage technology, scatters query syntax across the codebase, and makes the rules hard to test in isolation. An **ORM** removes the row-level coupling by translating between database rows and in-memory objects, generating SQL from typed expressions and tracking changes so only modified columns are written. The **repository pattern** removes the next layer of coupling by wrapping the data-access library — the ORM, the MongoDB driver, or whatever else — behind a domain-shaped interface that the service layer calls. Together they give the data layer a single seam: the ORM controls how data crosses the database boundary, and the repository controls what the rest of the application sees. The patterns repay their cost when the same data has multiple consumers, when the service layer needs to be testable without a database, or when the storage technology may change. They become a liability when they are applied to a single small service or when the repository grows generic enough to recreate the driver underneath it. The companion exercise Data layer implements `INewsletterRepository`

over MongoDB and registers it through dependency injection, exercising both patterns end-to-end.

PART IV · DATA ACCESS

CHAPTER 14

Connections, Pooling, and Transactions

Watch the presentation
Se presentationen på svenska

A web application that talks to a database has to do two things well: open the conversation cheaply, and finish each unit of work without leaving the data in a half-changed state. Naive code gets neither right — every request opens a fresh connection, every multi-step write scatters partial updates when something fails, and concurrent users overwrite each other's changes silently. Three mechanisms solve those prob-

lems together: the connection string that locates the database, the connection pool that keeps the conversation cheap, and the transaction — with its isolation level and locking strategy — that keeps each unit of work coherent.

14.1 The cost of opening a database connection

Opening a database connection is not a function call. It is a sequence of network and protocol steps that the application pays for every time it asks for a fresh connection.

A typical connection setup runs through:

1. A TCP three-way handshake to the database host (one round trip).
2. A TLS handshake if the link is encrypted (one or two more round trips).
3. A protocol-level handshake — for SQL Server, MongoDB, PostgreSQL, this includes capability negotiation and authentication.
4. Credential verification, often involving a password hash check or token validation against an identity provider.
5. Allocation of server-side resources — a session, a buffer, a worker thread.

On a low-latency local network this can take 5–50 milliseconds. Across regions or to a managed cloud database it can take 100 milliseconds or more. None of that work is the actual query — it is pure overhead before the first byte of useful data moves.

Code that opens a new connection per request scales badly under load. Each incoming HTTP request blocks for the handshake before it can read or write data, so request latency is dominated by connection setup rather than database work. A

burst of concurrent requests floods the database with handshakes, exhausting authentication threads on the server long before query throughput becomes the bottleneck. The same connection that just finished a 2-millisecond `INSERT` is then closed and discarded — the next request rebuilds it from scratch.

The fix is not to open fewer connections. The application still needs a connection per concurrent operation. The fix is to keep the connections open and hand them out on demand.

14.2 Connection pooling

A **connection pool** is a cache of open database connections that a client library reuses across requests, avoiding the cost of opening a new TCP connection and re-authenticating for each query; pool size is typically configured through the connection string. The pool turns connection setup into a once-per-process cost instead of a once-per-request cost.

The mechanics are straightforward. When the application asks the client library for a connection, the library checks the pool. If an idle connection is available, the library returns it immediately — no handshake, no authentication. If no connection is idle and the pool has not reached its maximum size, the library opens a new connection and adds it to the pool. If the pool is full, the request waits until another caller releases one. When the application finishes with the connection, the library does not close it; it returns it to the pool for the next caller.

14.2.1 Where the pool lives

The pool lives inside the database client library, in the application process. SQL Server's ADO.NET driver maintains a pool keyed by connection string. The MongoDB driver keeps a pool

inside each `MongoClient`. The Npgsql driver for PostgreSQL pools per process. The pool is not a separate service — it is a data structure inside the running application.

This placement matters for two reasons. First, every process has its own pool. Ten replicas of a web application each maintain their own pool of connections to the database; the database sees the sum of all pools. Second, the pool only helps if connections are returned to it. A client library that creates a new pool on every call (because the application keeps constructing new client objects) gets no benefit at all — every "pool" holds one connection and is then discarded.

14.2.2 Configuring the pool

Pool behaviour is configured through the **connection string**, the configuration value a client library uses to connect to a database, encoding the host, port, database name, credentials, and optional parameters such as pool size or timeout. Common pool parameters include:

- `MaxPoolSize` (or `maxPoolSize`) — the upper bound on simultaneous connections from this process. Default values vary: 100 for SQL Server's ADO.NET, 100 for the MongoDB driver, around 100 for Npgsql.
- `MinPoolSize` — connections kept open even when idle, to absorb sudden bursts without paying handshake cost.
- `ConnectionLifetime` (or `maxLifetimeMS`) — the maximum time a single connection stays in the pool before being closed and replaced. This protects against connections that have silently gone stale (e.g. through a load balancer's idle timeout) and is the mechanism that lets a fleet of clients gradually pick up DNS changes after a database failover.

- `ConnectionTimeout` — how long a request waits for a free connection before throwing.

A SQL Server connection string with explicit pooling parameters might look like the following.

```
Server=tcp:db.example.com,1433;Database=Newsletter;User
Id=app;Password=...;Min Pool Size=5;Max Pool Size=50;Con-
nection Lifetime=300;
```

A MongoDB connection string carries the same intent in URI form.

```
mongodb://app:...@cluster0.mongodb.net/?retry-
Writes=true&maxPoolS-
ize=50&minPoolSize=5&maxIdleTimeMS=60000
```

The numbers are not abstract. `MaxPoolSize` should be sized so the database can handle `pool_size × replica_count` simultaneous connections without exhausting its own connection limit. A managed database tier with a 200-connection cap and 8 application replicas leaves at most 25 pooled connections per replica before the database starts refusing connections. Sizing the pool above that limit causes failures under load that look like database outages but are really self-inflicted.

14.3 Reusing the client across the application

The connection pool only delivers its benefit if the same client object is reused across the application. Constructing a fresh `MongoClient`, `SqlConnection` factory, or `BlobServiceClient` per request defeats pooling entirely — each new client builds its own pool, uses it once, and is discarded.

The standard way to ensure reuse in ASP.NET Core is to register the client with the dependency injection container using the appropriate lifetime. For database clients that are thread-safe and hold a pool, the correct lifetime is Singleton — one instance for the lifetime of the process, shared by every request. The pool inside that single instance is then reused across all incoming work.

14.3.1 A worked example: registering MongoClient as a Singleton

The companion exercise Data Layer wires a `MongoClient` into the dependency injection container in `Program.cs`. The connection string comes from configuration through `IConfiguration`, keeping credentials out of source control.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<IMongoClient>(sp =>
{
    var connectionString = builder.Configuration.GetConnec-
        tionString("MongoDb");
    return new MongoClient(connectionString);
});

builder.Services.AddScoped<INewsletterRepository, MongoRe-
        pository<Newsletter>>();

var app = builder.Build();
```

The `IMongoClient` is registered as Singleton because `MongoClient` is documented as thread-safe and is intended to be created once per application. Internally it holds the connection pool, the topology monitor, and the server-selection state — all of which are expensive to set up and meant to be amortised. Repositories built on top of the client are registered as Scoped so each HTTP request gets its own

repository instance, but every repository in the process shares the same underlying client and therefore the same pool. This pairing — Singleton client, Scoped repository — is the standard pattern for document databases accessed through the repository pattern.

The same shape applies to other clients. `HttpClient` should be obtained from `IHttpClientFactory` rather than constructed per request. `BlobServiceClient` from `Azure.Storage.Blobs` is registered as Singleton. The Npgsql data source built with `NpgsqlDataSourceBuilder` is registered as Singleton and hands out pooled `NpgsqlConnection` objects on demand. The recurring rule: clients that own pools live for the process; objects derived from them live for the request.

14.4 Transactions

A connection pool keeps individual operations cheap, but applications rarely run a single operation in isolation. Transferring a balance between two accounts is two updates. Placing an order is an `INSERT` into the order table plus a stock-level decrement. Cancelling a subscription touches three tables. If any one of those steps fails, the others must not be left applied — the system has to either complete the whole sequence or behave as if none of it happened.

A **transaction** is a unit of work that the database treats atomically: either every operation in the transaction succeeds and the changes commit, or any failure causes all changes to roll back as if they never happened. Transactions are the database's expression of the ACID properties at the application boundary — atomicity, consistency, isolation, and durability all manifest in how a transaction is begun, what it sees while it runs, and how it commits or rolls back.

In code, a transaction follows a fixed shape:

```
using var transaction = connection.BeginTransaction();
try
{
    await DebitAsync(fromAccountId, amount, transaction);
    await CreditAsync(toAccountId, amount, transaction);
    transaction.Commit();
}
catch
{
    transaction.Rollback();
    throw;
}
```

The `BeginTransaction` call marks the start. The `Commit` call makes every change durable in one step. The `Rollback` call (or letting the `using` block dispose without committing) discards every change made since `BeginTransaction`. The database guarantees that no other transaction will ever observe a half-applied state — the changes are either all there or none of them are.

Two practical rules apply. First, transactions should be short. A transaction holds locks (or version snapshots) for its entire duration; a long-running transaction blocks other work and increases the chance of deadlock. Reading data, calling an external HTTP service, and only then committing is an anti-pattern — the HTTP call dominates the transaction lifetime. Second, transactions should be local to a single database where possible. Distributed transactions that span two databases (or a database and a message broker) require a two-phase commit protocol, which is fragile, slow, and frequently the wrong abstraction. Patterns like the outbox pattern replace distributed transactions with local transactions

plus eventually-consistent delivery, and are usually the better choice.

14.5 Isolation levels

Concurrency is where transactions get interesting. The database has to decide what each in-flight transaction is allowed to see of the others' uncommitted changes. The dial that controls this is the **isolation level**, which governs what concurrent transactions are allowed to see of each other's in-flight changes; common levels — Read Uncommitted, Read Committed, Repeatable Read, Serializable — trade performance for stronger guarantees against lost-update, dirty-read, and phantom-read anomalies.

The four standard SQL isolation levels form a ladder:

ISOLATION LEVEL	DIRTY READ	NON-REPEATABLE READ	PHANTOM READ	COST
Read Uncommitted	possible	possible	possible	lowest
Read Committed	prevented	possible	possible	low
Repeatable Read	prevented	prevented	possible	medium
Serializable	prevented	prevented	prevented	highest

A *dirty read* is reading data another transaction has written but not yet committed — and may still roll back. A *non-repeatable read* is re-reading a row inside the same transaction and getting a different value because another transaction

committed an update in between. A *phantom read* is re-running a range query and finding new rows that weren't there the first time, again because another transaction committed an insert.

Most relational databases default to **Read Committed**, which prevents dirty reads but allows non-repeatable reads and phantoms. This default suits typical web workloads: each request is short, conflicts are rare, and the application can tolerate a small chance of seeing a value that has just changed. **Serializable** gives the strongest guarantee — every transaction behaves as if it ran by itself — at the cost of either heavy locking or frequent retries on serialization failures. It is the right choice for financial workflows where any anomaly is unacceptable, and the wrong choice for a high-traffic feed query that just needs an approximate snapshot.

14.6 Optimistic and pessimistic locking

When two transactions touch the same row, the database has to decide what to do. The two strategies are named for the assumption they make about how often that happens.

Pessimistic locking prevents conflicts upfront by acquiring a database lock on a row before reading or modifying it; other transactions touching the same row block until the lock releases — useful when conflicts are common but expensive at scale. The lock is held for the duration of the transaction. Other readers and writers wait. The application gets a guaranteed-consistent view for the price of throughput: contended rows become a queue.

Optimistic locking detects conflicting updates by attaching a version field (or rowversion) to each record; an update succeeds only if the version still matches what was read, otherwise the application retries — useful when conflicts are rare

and locking would hurt throughput. No locks are held between read and write. The transaction reads the row and its version, does its work, and on update writes a `WHERE id = ? AND version = ?` clause. If another transaction has since incremented the version, zero rows match and the update silently fails — the application catches the failure and retries.

The choice depends on the conflict rate. Editing a user's profile page is almost never contended; optimistic locking pays nothing in the common case and only adds work when a true conflict happens. Decrementing a global counter (last seat on a flight, last unit in stock) is highly contended; every transaction collides with every other, optimistic retries thrash, and pessimistic locking — or a different design entirely, like a queue or an atomic decrement — wins.

STRATEGY	WHEN TO USE	TRADE-OFF
Pessimistic	High contention, short critical sections	Blocks other transactions; risk of deadlock
Optimistic	Low contention, longer transactions	Retry logic in the application; can starve under heavy contention

Some databases hide the choice. SQL Server with `READ COMMITTED SNAPSHOT` uses row versions for reads but locks for writes. PostgreSQL uses MVCC throughout, so readers never block writers and vice versa, and conflicts surface at commit time as serialization failures the application must retry. The vocabulary of optimistic versus pessimistic is still the right way to think about the problem even when the underlying mechanism is more nuanced.

14.7 Practical advice

Three rules summarise the chapter:

- Reuse client instances. Register database clients as Singleton in the dependency injection container. Treat constructing a new client per request as a bug. The pool is the asset.
- Keep transactions short. A transaction should cover the smallest possible unit of work that must be atomic. No HTTP calls, no waiting on user input, no long-running computation. Begin, change, commit.
- Avoid distributed transactions when possible. A single-database transaction is straightforward; a transaction spanning two databases is a system-design problem in disguise. Prefer local transactions plus eventually-consistent delivery (the outbox pattern) over two-phase commit.

The companion exercise Data Layer walks through registering a `MongoClient` as Singleton, configuring the connection string through `appsettings.json`, and observing that the same client serves every request without rebuilding its pool.

14.8 Summary

Opening a database connection is expensive — a TCP, TLS, and protocol handshake plus authentication — so naive code that opens a connection per request collapses under load. The connection pool, hosted inside the client library and configured through the connection string, amortises that cost across requests by reusing open connections. The pool only helps if the client object itself is reused; that is why `MongoClient`, `SqlConnection`-factory, and similar clients are registered as Singleton in the dependency injection container while repositories layered on top are Scoped per request.

Once connections are cheap, the next concern is making each unit of work coherent: a transaction wraps multiple operations into an atomic commit-or-rollback, the isolation level controls what concurrent transactions see of each other's in-flight changes (from Read Uncommitted to Serializable, trading throughput for stronger guarantees), and the locking strategy — optimistic with a version field for low-contention writes, pessimistic with row locks for high-contention ones — controls how conflicts are detected and resolved. Reuse client instances, keep transactions short, and avoid distributed transactions when possible.

PART IV · DATA ACCESS

CHAPTER 15

Object Storage and File Uploads

Watch the presentation

Se presentationen på svenska

Web applications routinely accept files from users — profile pictures, document attachments, generated reports, video uploads. Storing those bytes inside a relational table or document collection wastes the strengths of a database and exposes its weaknesses: query planners cannot index a JPEG, transactions hold locks while megabytes flush to disk, and backup windows balloon as the row count stays the same but the row size grows by a thousand. A database is a poor fit for any binary blob much past a few kilobytes, and the moment a

feature reaches profile pictures or attachments the application needs a different home for those bytes — somewhere designed for large payloads, accessed over HTTP and priced per gigabyte rather than per query.

15.1 Why databases struggle with large binary payloads

A database engine optimizes for many small operations: row-level locks, B-tree indexes, query plans that touch a handful of pages. Inserting a 50 MB image into a `BLOB` column or an embedded base64 field upends those assumptions. Each write moves a large value through the transaction log, the log replicates to standbys, and every backup carries the full payload along with the structured data the application actually queries. Read paths suffer too: a `SELECT` that does not project the binary column still pays for the row's expanded storage, and any column that does include the payload sends it across the wire even when the caller only needed a thumbnail URL.

The core mismatch is access pattern. Structured data is read by predicate: "the orders for customer 42." Binary payloads are read by identity: "the image at this URL." A system that excels at the first will be mediocre at the second, so cloud platforms separate the two — application data goes to a database, large binary data goes to an object store, and a simple foreign-key-like reference (a path or URL) ties them together.

Object storage was introduced in Part II as one of three primary storage architectures, alongside block storage and file storage. This chapter narrows the focus to how application code interacts with that storage: how blobs are organized, how access is delegated, and how an upload travels from a browser through a service method into the storage account.

15.2 Blobs and containers

A **blob** (Binary Large Object) is the unit of storage in an object store: an immutable byte sequence with a path-like name, optional metadata key-value pairs, and a content type; blobs are written and read whole through HTTP APIs rather than queried like database rows. A blob can be a 200-byte text file, a 4 GB video, or anything in between — the store treats them uniformly as opaque sequences of bytes addressed by name.

A **container** in object storage is the top-level grouping for blobs, somewhat like a bucket in S3 or a directory in a filesystem; access policies and access tiers are typically configured at the container level. A storage account holds one or more containers, and each container holds zero or more blobs. The container name is the first segment of every blob URL the account exposes:

```
https://clo25assets.blob.core.windows.net/profile-images/
user-42/avatar.png
                                                └─ container
─┘└──── blob name ────┘
```

The slash characters inside the blob name are conventional separators, not real directories. Object storage has a flat namespace per container; the SDK lets code list blobs by prefix (`user-42/`) which makes the namespace look hierarchical, but no `mkdir` operation exists and no rename moves directories. A blob named `user-42/avatar.png` is just a blob whose name happens to contain a slash.

15.2.1 The immutability model

Blobs do not support partial writes. Uploading a new version of `avatar.png` does not patch the existing blob — it replaces it. The previous bytes are gone the moment the upload com-

pletes, unless versioning or soft-delete is enabled at the account level. There is no `UPDATE` that changes byte 1024 of an existing blob; the smallest write is a complete rewrite of the blob (or, for block blobs, a coordinated replacement of one or more named blocks).

This immutability is what allows object storage to scale horizontally and to replicate aggressively across data centers. A row in a relational database has to be the single authoritative copy; a write must invalidate every cached read and propagate to every replica before the next read sees consistent data. A blob is content-addressable enough that the storage system can serve any cached copy until the new write completes, then atomically swap to the new version. Application code benefits from the simplicity (no merge logic, no partial updates) and pays for it in reasoning: if two requests upload the same blob name concurrently, one wins entirely and the other is overwritten entirely.

15.3 Access tiers as the cost-vs-latency knob

Not all stored data has the same access pattern. A user's current profile picture is read every time their profile renders; a five-year-old quarterly report is read once a year, if ever. Charging the same per-gigabyte storage rate for both wastes money on the rarely-read data and leaves no headroom to optimize the hot path.

An **access tier** is the storage class a blob is assigned to — Hot, Cool, or Archive in Azure Blob Storage — that trades retrieval latency for storage cost; Hot tier costs more to store but is read instantly, while Archive tier is cheap to store but takes hours to rehydrate. The tiers form a sliding scale:

TIER	STORAGE COST	READ COST	READ LATENCY	TYPICAL USE
Hot	Highest	Lowest	Milliseconds	Active assets, user uploads
Cool	Lower	Higher	Milliseconds	Backups, infrequent access
Archive	Lowest	Highest	Hours (rehydration)	Long-term retention, compliance

Hot and Cool blobs respond to GET requests immediately, like any HTTP endpoint. An Archive blob does not — a read against it returns an error until the blob is rehydrated by an explicit request, which moves the data back to Hot or Cool over a window measured in hours. The decision to archive is therefore not just about cost; it is about accepting that any read from this point forward will be a multi-hour operation.

Tiering can be set at the container level (default for new blobs) or per blob, and a lifecycle management policy can move blobs between tiers automatically based on age or access patterns. A common pattern is to upload to Hot, transition to Cool after 30 days of no access, and to Archive after 90 — the application code does not change, only the bill.

15.4 SAS tokens for delegated access

A storage account has a primary key. That key grants full read/write/delete on every blob in every container in the account, and embedding it in client-side code, mobile apps, or short-lived CI jobs is unsafe — the key is impossible to revoke without rotating it, which breaks every other con-

sumer at the same time. The standard mechanism for granting limited access is the SAS token.

A **SAS** (Shared Access Signature) is a time-limited, scope-limited URL signed with the storage account key that grants the bearer permission to read or write a specific blob or container without exposing the account key itself; SAS tokens are the standard way to delegate temporary access to clients. A SAS is just a query string appended to the blob URL — the storage service validates the signature, checks the expiry, and authorizes the operation if everything matches:

```
https://clo25assets.blob.core.windows.net/profile-images/
user-42/avatar.png
  ?sv=2023-11-03
  &sr=b
  &sp=r
  &se=2026-04-28T17:00:00Z
  &sig=...
```

The parameters encode permissions (`sp=r` means read-only), scope (`sr=b` means a single blob), expiry (`se=...` is the deadline after which the signature is invalid), and the signature itself. Anyone holding this URL can issue exactly one type of request against exactly one resource for a bounded window. After the expiry, the URL stops working with no server-side cleanup needed.

SAS tokens come in two flavors that matter for application code. A *service SAS* is signed directly with the account key and is the simpler form. A *user delegation SAS* is signed with a key that the storage service issues against an Entra ID identity, so revoking the identity revokes every SAS it has signed — the better choice when production code uses a managed identity to talk to storage. Either way, the client-facing pattern is the same: the application server generates a SAS scoped to one blob, returns it to the browser, and the browser uploads or

downloads directly to storage without the upload bytes flowing through the application server.

15.5 Streaming uploads from the SDK

A naive upload reads the entire file into memory, then writes the buffer to storage. This works for a 100 KB avatar; it fails for a 4 GB video on a server with 512 MB of RAM. The Azure Storage SDK sidesteps the problem with streaming uploads.

A **streaming upload** transfers a blob to the storage service in chunks rather than buffering the entire payload in memory first; the storage SDK handles chunking, parallelism, and retry, allowing the client to upload arbitrarily large files with bounded memory. The SDK breaks the source stream into blocks (typically 4 MB to 100 MB each), uploads them in parallel over multiple HTTP connections, and finally issues a `Commit Block List` operation that atomically assembles the blocks into a single blob. If a block fails, only that block retries — the entire upload does not restart.

For ASP.NET Core, the request body itself is a `Stream`, which means a controller can pipe the multipart form section directly into `BlobClient.UploadAsync` without ever materializing the file in memory. The pipe is the upload — bytes arrive from the browser, flow through the framework's pipeline, and land in storage in a single bounded-memory pass.

15.5.1 A worked example

The following service method, drawn from the data-layer exercise, uploads a file to a container and returns the blob's public URL. The service is registered with dependency injection and reads its connection string from IConfiguration:

```
public class ImageStorageService
{
    private readonly BlobContainerClient _container;

    public ImageStorageService(IConfiguration config)
    {
        var connectionString = config["AzureStor-
        age:ConnectionString"];
        var serviceClient = new BlobServiceClient(connec-
        tionString);
        _container = serviceCli-
        ent.GetBlobContainerClient("profile-images");
    }

    public async Task<Uri> UploadImageAsync(string file-
        name, Stream content, string contentType)
    {
        var blobClient = _con-
        tainer.GetBlobClient(filename);
        var headers = new BlobHttpHeaders { ContentType =
        contentType };

        await blobClient.UploadAsync(content, new BlobUp-
        loadOptions
        {
            HttpHeaders = headers
        });

        return blobClient.Uri;
    }
}
```

Several details carry weight. `BlobServiceClient` is created once per connection string and is safe to hold for the lifetime of the application — it is a thin wrapper around an HTTP client, registered as a Singleton in `Program.cs` and injected into services that handle file uploads. `GetBlobContainerClient` and `GetBlobClient` are lightweight; they do not make network calls, they just build URLs. The actual network traffic happens inside `UploadAsync`, which streams `content` to storage in blocks. Setting

`BlobHttpHeaders.ContentType` to the form upload's MIME type means the browser will later receive the correct `Content-Type` header when it fetches the blob, so an uploaded `image/png` renders as an image rather than downloading as `application/octet-stream`. If the same `filename` already exists, `UploadAsync` throws `BlobAlreadyExists` — adding `new BlobUploadOptions { Conditions = null }` and passing `overwrite: true` via the simpler overload (`await blobClient.UploadAsync(content, overwrite: true)`) makes the upload replace any existing blob of the same name, which is the right default when the filename includes a per-user prefix that is unique by construction.

The presentation layer (a controller action accepting a multipart form) calls this service without knowing anything about block sizes, retry policy, or the structure of a blob URL. The data layer hides the storage SDK behind a service method whose only contract is "stream in, URL out."

15.6 Content type and metadata

A blob carries more than its bytes. Two pieces of information ride alongside every upload and shape how the blob is later consumed.

The *content type* is an HTTP header — `Content-Type: image/png` — that storage records and replays on every download. Setting it correctly at upload is what lets a browser display the image inline, lets a PDF reader open the file, and lets a CDN apply MIME-aware compression. A blob uploaded without an explicit content type defaults to `application/octet-stream`, which most browsers treat as "download, do not preview."

Metadata is a dictionary of small string key-value pairs — typically a few hundred bytes total — that the application

attaches to the blob. Common uses include the original filename (the blob name often gets sanitized or randomized), the uploading user's ID, a checksum, or a workflow state ("pending-virus-scan"). Metadata does not influence routing or access; it is application-level annotation that travels with the blob and can be read back without downloading the payload. Searching by metadata is not supported — the metadata is per-blob, retrieved when the blob is fetched, and not indexed across the container. Cross-blob queries belong in a database that holds the blob URL plus searchable fields.

15.7 Choosing between blob storage and a database BLOB column

Both options can technically hold a JPEG. The decision is rarely about feasibility and almost always about access pattern, scale, and operations.

DIMENSION	BLOB STORAGE	DATABASE BLOB COLUMN
Cost per GB stored	Cents	Dollars (database-tier pricing)
Read pattern	URL fetch over HTTP	SQL/driver round-trip
Backup impact	Independent of database backups	Inflates database backup size
Browser-direct access	Yes (via SAS)	No (must go through app server)
Transactional with row	No	Yes
Partial reads	Range requests on HTTP	Driver-dependent
Maximum size	Terabytes per blob	Typically 1–4 GB, often less

Choose a database BLOB column when the binary content is small, tightly coupled to the row's transactional lifecycle, and never served directly to clients — for example, a 10 KB encrypted secret that must commit or roll back with the row that owns it. Choose blob storage in essentially every other case: any user-uploaded file, any media served to browsers, any artifact larger than a few hundred kilobytes, and any binary that benefits from CDN distribution or independent lifecycle management. The companion exercise Azure Blob Storage walks through provisioning a storage account and configuring the container; the Data Layer exercise then implements the upload service shown above in a working ASP.NET Core application.

15.8 Summary

Object storage exists because large binary payloads strain the assumptions databases make about row size, transaction logs, and query plans. A blob is an immutable byte sequence with a path-like name, a content type, and optional metadata, grouped into containers that scope access policies and access tiers. Access tiers (Hot, Cool, Archive) trade storage cost for retrieval latency, with Archive requiring an explicit rehydration before any read. SAS tokens delegate time-limited, scope-limited access without exposing the account key, enabling browsers and short-lived clients to talk to storage directly. Streaming uploads in the SDK chunk arbitrarily large files into bounded-memory transfers, and a service method that wraps `BlobClient.UploadAsync` keeps the upload mechanics out of the controller. The decision between blob storage and a database BLOB column comes down to access pattern: blob storage wins for anything served over HTTP or larger than the database is comfortable holding, while a database column is reserved for small, transactionally-coupled binary data.

PART V

Identity and Security

The Part the course returns to most often. Every web application needs to answer two questions on every protected request: who is asking, and are they allowed? This Part covers both questions, the building blocks the framework provides for each, and the cloud-side machinery for keeping the secrets those building blocks rely on.

The eight chapters move from the conceptual split (Ch 1) through the most common browser-facing pattern (cookies + Identity, Ch 2–4), into the API-facing patterns (JWT and API keys, Ch 5–6), out to delegated authentication (OAuth and OIDC, Ch 7), and end with secret management (Ch 8). Companion exercises include the

authentication and authorization exercise and the identity and user stores exercise.

PART V · IDENTITY AND SECURITY

CHAPTER 16

Authentication vs Authorization

Watch the presentation

Se presentationen på svenska

Web applications that serve more than one user must answer two questions on every secured request: who is making this call, and is that caller allowed to do what they are asking to do? The two questions sound similar, but they are answered by different mechanisms, at different points in the request pipeline, against different sources of truth. Conflating them produces fragile code that either trusts the wrong caller or denies the right one. The conceptual split between the two questions, the framework primitives that answer each one,

and the credential formats and decision rules built on that foundation are the through-line of Part V.

16.1 The two questions every secured request must answer

Authentication is the process of confirming who is making a request — verifying that a presented credential (password, token, certificate) maps to a known identity in the system. The output is an identity: a name, a user ID, sometimes a set of attached attributes such as email or tenant. Authentication does not decide whether the caller is allowed to do anything; it only establishes who they are.

Authorization is the process of deciding whether an authenticated identity is allowed to perform a requested action — a separate decision from authentication, and one that depends on the identity's roles, claims, or policy match. The input is an already-known identity and the resource being requested; the output is a yes-or-no answer to "may this caller proceed?"

The distinction matters because the two questions consult different sources of truth and fail in different ways. Authentication failure means the credential did not match — the response is `401 Unauthorized` and the standard remedy is "log in again." Authorization failure means the credential did match but the resulting identity lacks the necessary permission — the response is `403 Forbidden` and logging in again will not change the outcome. A system that returns the same status code for both, or treats both as the same kind of error, leaks information and confuses callers.

16.1.1 Why frameworks keep them separate

ASP.NET Core implements authentication and authorization as two distinct stages because the inputs, the trust model, and the rate of change are different. Authentication consults a credential store (a user database, a token issuer, a certificate authority) and tends to change rarely — adding a second authentication scheme is a deployment-level decision. Authorization consults the application's business rules — which roles can edit which resources, which claims gate which actions — and changes whenever the product changes. Separating the two means a feature team can adjust authorization rules without touching credential validation, and a platform team can add a new authentication scheme (cookies in a browser, JWT in an API) without rewriting controllers.

The separation also enforces a useful discipline: authorization code never sees a raw credential. By the time a controller asks "is this user an admin?", the framework has already validated whatever the caller presented, attached an in-memory identity to the request, and discarded the password or token. Business code reasons about identity, not credentials.

16.2 The principal — the identity attached to a request

Once authentication succeeds, the framework needs a place to put the validated identity so the rest of the pipeline can read it. That place is the principal.

A **principal** is the abstract identity attached to an authenticated request; in ASP.NET Core, the `ClaimsPrincipal` exposes the identity's name and claims to controller code via `HttpContext.User`. The `ClaimsPrincipal` is constructed by the authentication handler — the cookie handler, the JWT bearer handler, the certificate handler, or any other registered

scheme — and is the single object that authorization code reads from.

The principal carries a name (usually the username or user ID), one or more identities (in advanced multi-scheme scenarios), and a list of claims. A claim is a name-value pair such as `("role", "Admin")` or `("email", "anna@example.com")`. The cookie handler reads the encrypted authentication ticket and reconstructs a `ClaimsPrincipal` from it. The JWT handler parses the token's payload and projects each payload field as a claim. The credential format on the wire differs; the in-memory shape the rest of the application sees is the same.

This uniformity is the reason that swapping cookies for JWT — or supporting both at once — does not require rewriting controllers. A controller that reads `User.Identity.Name` or asks `User.IsInRole("Admin")` is talking to the principal, not to the cookie or the token. The transport changed; the contract did not.

16.3 Where each decision happens in the pipeline

ASP.NET Core processes a request as an ordered chain of middleware components, each of which can short-circuit the request or pass it to the next. Authentication and authorization sit at fixed positions in that chain.

The authentication middleware runs early. When `app.UseAuthentication()` is registered, every incoming request passes through it before reaching the routing or endpoint stages. The middleware inspects the request for a credential — a cookie, an `Authorization` header, a client certificate — and hands it to the configured authentication handler. If the credential validates, the handler builds a `Claims-`

`Principal` and assigns it to `HttpContext.User`. If no credential is present, `HttpContext.User` is set to an unauthenticated principal (the `IsAuthenticated` property is `false`). Crucially, the middleware does not reject the request when no credential is present — it merely records the absence. Public endpoints must remain reachable.

The authorization filter runs later, once routing has matched the request to a controller action. The framework checks for an `[Authorize]` attribute on the action or controller, and if one is present, it evaluates the requirement against `HttpContext.User`. Bare `[Authorize]` requires only that the user be authenticated. `[Authorize(Roles = "Admin")]` additionally requires a role claim with that value. `[Authorize(Policy = "ManagerOnly")]` evaluates a named policy registered at startup. If the requirement fails, the framework short-circuits the response with `401 Unauthorized` (when the user is not authenticated at all) or `403 Forbidden` (when they are authenticated but lack the required claim). If it succeeds, control proceeds to the action method.

The ordering is what makes the split work: by the time the authorization filter runs, `HttpContext.User` is already populated. Authorization never validates credentials and never queries the user database; it reads claims off the principal that authentication already produced.

The basic form of the `[Authorize]` attribute is part of the MVC framework — see the MVC pattern for the controller and routing context. The remaining sections describe what the attribute does once a request reaches it.

16.4 Credentials authentication accepts

Authentication is a family of mechanisms, not a single procedure. The handler that runs depends on the registered

scheme and on what the request actually carries. Each handler accepts a different credential format but produces the same `ClaimsPrincipal` shape on success.

A *password* is the credential a human user types into a login form. The server compares it to a stored hash, and on a match issues a session credential — typically an authentication cookie — so the user is not asked to retype the password on every request. Passwords are presented once per session; the cookie carries the established identity afterward.

A bearer token is a credential a client presents in the `Authorization: Bearer <token>` header on every API request. JSON Web Tokens are the most common bearer-token format: the token itself contains the claims, signed by the issuer, so the recipient can validate the token without contacting a session store. APIs prefer tokens over cookies because they are easier to issue, easier to revoke (by short lifetimes), and not subject to the cross-site cookie rules browsers enforce.

A *client certificate* is a credential presented during the TLS handshake itself, before the HTTP request body is read. The certificate's subject and issuer are validated against a trust chain, and the resulting identity is projected into a `ClaimsPrincipal`. Certificates suit machine-to-machine and high-assurance scenarios where deploying a private key to the client is acceptable.

Other forms exist — API keys for simpler service-to-service calls, OAuth tokens issued by external authorization servers, federated assertions from enterprise identity providers — and each is implemented as an authentication handler. The principle is consistent: the handler validates a credential and produces a principal.

16.5 Decisions authorization makes

Once a principal exists, authorization decides whether the principal may proceed. ASP.NET Core supports three styles of decision, each more flexible than the last.

A *role check* asks whether the principal belongs to a named group: `[Authorize(Roles = "Admin")]` or `User.IsInRole("Reviewer")`. Roles are coarse-grained — an identity either has the role or it does not — and they suit broad organizational categories such as administrator, reviewer, or candidate. The framework treats role membership as a special claim under the hood, but the syntax is shorter for the common case.

A *claim check* asks whether the principal carries a specific claim. A controller might require `User.HasClaim("tenant", currentTenant)` before allowing access to tenant-specific data. Claim checks are finer-grained than role checks and naturally encode multi-dimensional facts about the identity (tenant, department, security clearance, license tier).

A *policy evaluation* runs a named policy registered at startup against the principal. A policy bundles one or more requirements — "must be authenticated" plus "must have role Manager" plus "must have a `region` claim matching the request" — into a single named rule that controllers reference with `[Authorize(Policy = "ManagerInRegion")]`. Policies centralize business logic so that the rule lives in one place and every controller that opts in stays consistent. The full mechanics of roles, claims, and policies are developed in Roles, claims, and policies.

16.6 A worked example

Exercise 4 in the application-development track walks through the full arc on a hardcoded user list. The relevant fragment is a controller that renders a “Who Am I?” page only to authenticated callers.

```
[Authorize]
public class WhoAmIController : Controller
{
    public IActionResult Index()
    {
        var name = User.Identity?.Name ?? "anonymous";
        var role = User.FindFirst("role")?.Value ?? "(no
        role)";
        return View(new WhoAmIViewModel(name, role));
    }
}
```

When a request hits this controller, the framework runs the authentication middleware first. The middleware inspects the request for a cookie matching the configured cookie scheme; if one is present and validates, the cookie handler decrypts the authentication ticket and assigns the resulting `ClaimsPrincipal` to `HttpContext.User`. The authorization filter then sees the `[Authorize]` attribute, asks the principal `IsAuthenticated`, and either lets the request reach the action or short-circuits to the login page. By the time `Index()` runs, `User` is guaranteed to be an authenticated principal, and the action can read the name and role claims directly. No password validation happens in this controller; no user-database query happens here either. The controller reasons about identity, not credentials.

The companion exercise Authentication and Authorization develops this scenario through four steps: hardcoded credentials and a login form, attaching role claims, inspecting the

full `ClaimsPrincipal`, and adding CSRF protection. Each step layers on the same framework primitives this chapter introduced.

16.7 How the rest of Part V fits together

The remaining chapters drill into specific axes of the authentication-and-authorization split. Cookie-based authentication and sessions covers the credential format used in browser scenarios — how the authentication cookie is issued, validated, and protected from cross-site request forgery. ASP.NET Core Identity replaces the hardcoded user list with a managed user store, password hashing, and registration flows. Roles, claims, and policies develops the authorization side: how `ClaimsPrincipal` represents identity, how role checks and policy evaluation work, and when to choose one over the other. Bearer tokens and JWT replaces the cookie with a token format suited to APIs and explains how the same `ClaimsPrincipal` is reconstructed from a signed payload. API keys and machine-to-machine and OAuth 2.0 and OpenID Connect cover service-to-service and federated-identity scenarios. Secret management closes the Part with the operational side: how the cryptographic keys, connection strings, and signing secrets that all the previous chapters depend on are stored and rotated safely.

Each subsequent chapter reuses the conceptual frame this chapter establishes — the split between authentication and authorization, the principal as the in-memory identity, the middleware-then-filter pipeline order — and adds the specific mechanism that fills in one part of the picture.

16.8 Summary

Every secured request answers two questions: who is the caller, and may the caller proceed? Authentication answers the first by validating a credential — a password, a bearer token, a certificate — and producing a `ClaimsPrincipal` attached to `HttpContext.User`. Authorization answers the second by reading roles, claims, and policy results off that principal, never touching the credential itself. ASP.NET Core enforces the split by running authentication middleware early in the pipeline and the authorization filter later, after routing. Keeping the two stages separate lets credential formats change without rewriting business rules, and lets business rules change without revisiting credential validation. The chapters that follow develop each axis in turn — cookies, Identity, claims, JWT, API keys, OAuth, and secret management — on the conceptual foundation this chapter laid down.

PART V · IDENTITY AND SECURITY

CHAPTER 17

Cookie-Based Authentication and Sessions

Watch the presentation

Se presentationen på svenska

HTTP carries no memory between requests. Every request a browser sends to a web application arrives as if no previous request ever happened — the protocol itself does not record who is on the other end. A web application that lets a user sign in once and stay signed in across pages must therefore add its own mechanism for recognising the same browser across requests. Cookie-based authentication is the oldest

and still the most common answer to that requirement, and the mechanism every other browser-based authentication scheme on ASP.NET Core ultimately builds on.

The mechanism worth tracing end to end runs from the browser storing and replaying cookies, through the server turning a verified password into an encrypted authentication ticket, the sign-in and sign-out handshakes, the cookie attributes that harden the scheme against attack, and on to why cross-site request forgery (CSRF) is a problem only cookie-authenticated sites face. The pattern shown here is the foundation that ASP.NET Core Identity layers user persistence and password hashing on top of, and the contrast against which bearer tokens are introduced in Bearer Tokens and JWT.

17.1 How browsers handle cookies

A **cookie** is a small piece of data the server asks the browser to store and return on every subsequent request to the same domain; an **authentication cookie** carries an encrypted record of who the browser is signed in as, allowing the server to recognize the user without re-authenticating. The browser is the cookie store — the server only sees cookies it has previously asked the browser to remember.

The mechanism is defined by two HTTP headers. When the server wants to set a cookie, it returns a `Set-Cookie` response header naming the cookie, its value, and any attributes that govern lifetime and scope. The browser parses that header, stores the cookie in its cookie jar against the issuing domain, and on every later request to that domain attaches the cookie automatically in a `Cookie` request header. Application code never has to decide to send the cookie — the

browser sends it on every same-origin request, including image and script loads, until the cookie expires or is deleted.

For authentication, three properties of this behaviour matter:

- **The browser sends the cookie on every request to the domain**, including requests triggered by other sites (a `<form>` on `evil.example` that posts to `bank.example`). This is convenient for the legitimate user and the root cause of CSRF.
- **The cookie is bound to the domain that issued it**, not to the user account. If two users share a browser profile, they share the cookie jar; sign-out must therefore explicitly invalidate the cookie rather than merely forgetting it on the server.
- **The cookie's value is opaque to the browser**. The browser stores and replays bytes; it does not parse them. The server is free to put whatever encoded data in the cookie as long as the resulting size stays within the few-kilobyte budget browsers accept.

17.2 The authentication ticket inside the cookie

The temptation when first implementing cookie authentication is to store the user's name in the cookie and trust it on the way back. That is unsafe: the cookie sits on a machine the server does not control, and a hostile user can edit the value before sending it. ASP.NET Core's cookie-authentication middleware solves this by treating the cookie as the transport for an **authentication ticket** that is both encrypted and integrity-protected before it leaves the server.

The ticket is the serialised form of a `ClaimsPrincipal` (the principal representing the signed-in user) together with

metadata such as the issue time, expiration, and the authentication scheme that produced it. Before being placed in the `Set-Cookie` header, the ticket is run through the framework's data-protection API:

1. The serialised bytes are encrypted with a symmetric key held only on the server, so the cookie's contents are unreadable by the client and by anyone who steals it from the browser without also stealing the server's keys.
2. The ciphertext is signed with a message-authentication code, so any modification to the cookie value causes verification to fail when the cookie returns. A user who edits a single byte gets logged out, not promoted.
3. The result is base64-encoded so it survives the HTTP header transport.

The cookie is therefore opaque to the client and tamper-evident to the server. On every incoming request, the cookie middleware reads the cookie, verifies the signature, decrypts the ticket, reconstructs the `ClaimsPrincipal`, and assigns it to `HttpContext.User`. From the controller's perspective the user is signed in; the cookie machinery is invisible.

17.3 The sign-in handshake

Sign-in is the one-time exchange that converts a password into a cookie. The shape of the handshake is the same in every web framework; ASP.NET Core's specific names are shown for concreteness.

The user requests a protected page, the framework recognises that no authentication cookie is present, and it issues a redirect to a login route. That route renders a form whose action posts a username and password back to the server over HTTPS. A controller action receives the form data, looks

up the user record, and verifies the supplied password against the stored hash. If verification fails, the action re-renders the form with an error and no cookie is issued. If verification succeeds, the action constructs a `ClaimsPrincipal` describing the user and calls `HttpContext.SignInAsync`, which is the framework's instruction to the cookie middleware to emit a `Set-Cookie` response header carrying the encrypted ticket. The action then redirects the browser back to the originally requested page. On that follow-up request the browser attaches the new cookie, the middleware decrypts it, and the page renders for the now-authenticated user.

```
public class AccountController : Controller
{
    private readonly IUserDirectory _users;

    public AccountController(IUserDirectory users) =>
        _users = users;

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Login(LoginViewModel
        model, string? returnUrl)
    {
        if (!ModelState.IsValid) return View(model);

        var user = _users.FindByName(model.Username);
        if (user is null || !
        user.VerifyPassword(model.Password))
        {
            ModelState.AddModelError(string.Empty,
        "Invalid username or password.");
            return View(model);
        }

        var identity = new ClaimsIdentity(
            new[]
            {
                new Claim(ClaimTypes.Name, user.Username),
                new Claim(ClaimTypes.Role, user.Role),
            },
            CookieAuthenticationDefaults.AuthenticationS-
        cheme);

        await HttpContext.SignInAsync(
            CookieAuthenticationDefaults.AuthenticationS-
        cheme,
            new ClaimsPrincipal(identity));

        return LocalRedirect(returnUrl ?? "/");
    }
}
```

Two details in the action carry weight. The `ClaimsIdentity` is constructed with the cookie scheme name as its authentication type — this is the marker the middleware looks for when reconstructing the principal on later requests. The redirect uses `LocalRedirect` rather than plain `Redirect`, which refuses absolute URLs; this prevents an attacker from crafting a `?returnUrl=https://evil.example` link that bounces the freshly signed-in user (and any reflected query data) to an external site.

For the cookie middleware to know what scheme to look for and what cookie name to set, the application registers cookie authentication once at startup:

```
builder.Services
    .AddAuthentication(CookieAuthenticationDefaults.Authen-
        ticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Account/Login";
        options.LogoutPath = "/Account/Logout";
        options.AccessDeniedPath = "/Account/Forbidden";
        options.ExpireTimeSpan = TimeSpan.FromHours(8);
        options.SlidingExpiration = true;
        options.Cookie.Name = "CloudSoft.Auth";
        options.Cookie.HttpOnly = true;
        options.Cookie.SecurePolicy = CookieSecurePol-
        icy.Always;
        options.Cookie.SameSite = SameSiteMode.Lax;
    });

var app = builder.Build();
app.UseAuthentication();
app.UseAuthorization();
```

`AddAuthentication` declares the default scheme name; `AddCookie` registers the middleware that handles that scheme. The options block records every choice the applica-

tion makes about the cookie's lifetime and security attributes. Order matters in the pipeline: `UseAuthentication` must run before `UseAuthorization`, because the principal must be on `HttpContext.User` before any `[Authorize]` attribute is evaluated.

17.4 The sign-out handshake

Sign-out is symmetric. The user posts to a sign-out endpoint, the action calls `HttpContext.SignOutAsync` for the cookie scheme, and the middleware emits a `Set-Cookie` header that overwrites the existing cookie with an empty value and a past expiration date. The browser sees that the cookie has expired and removes it from the cookie jar. The next request arrives without an authentication cookie, the middleware leaves `HttpContext.User` unauthenticated, and any `[Authorize]`-protected page bounces back to the login route.

Because the cookie is self-contained — the server keeps no per-cookie session record by default — sign-out cannot revoke a cookie that has already been copied off the user's machine. This is the trade-off of stateless cookies: revocation requires either a server-side allow-list, short cookie lifetimes, or a key roll that invalidates every issued cookie at once. For most web applications the lifetime knob is sufficient.

17.5 Cookie attributes that matter for security

A cookie's attributes appear in the `Set-Cookie` response header alongside its value. Four attributes carry the security weight of the scheme.

ATTRIBUTE	PURPOSE	FAILURE MODE IF OMITTED
`HttpOnly`	Hides the cookie from JavaScript running in the page	A cross-site scripting bug exfiltrates the cookie via `document.cookie`
`Secure`	Sends the cookie only over HTTPS	A network attacker on plain HTTP captures the cookie verbatim
`SameSite=Lax` (or `Strict`)	Suppresses the cookie on most cross-site requests	A cross-site form post replays the user's session against the application
`Expires` / `Max-Age`	Bounds the cookie's lifetime in the browser	A stolen cookie remains valid indefinitely

`HttpOnly` and `Secure` are unconditional: an authentication cookie that is readable by JavaScript or sent over HTTP is broken regardless of the rest of the application's care. `SameSite` is the browser-level defence against CSRF and is covered in its own section below. The `Expires` attribute determines the cookie's persistence in the browser; if omitted the cookie becomes a *session cookie* that the browser deletes when it closes (the term refers to the browser session, not the application session).

17.5.1 Session lifetime versus sliding expiration

The cookie's expiration time is the application's primary lever for balancing usability against the consequences of a stolen cookie. Two strategies dominate.

A *fixed expiration* issues a cookie that is valid for a defined duration from the moment of sign-in — eight hours, for example — regardless of activity. The cookie goes away at the same wall-clock time whether the user worked continuously or stepped away for lunch. The simplicity is appealing, but active users get logged out mid-task.

A *sliding expiration* extends the cookie's lifetime on each request, capped at the configured maximum. ASP.NET Core implements this by re-emitting the cookie when more than half of `ExpireTimeSpan` has elapsed since the last issue. A user who works steadily through a six-hour day stays signed in; a cookie that is then stolen and unused for the full window expires before the attacker can exploit it. The cost is more `Set-Cookie` headers on the wire and a longer effective lifetime for active sessions, which makes the rotation question (how often the underlying authentication ticket is refreshed against the user store) more pressing.

A **session** is the period during which a particular client is recognized as the same authenticated user across multiple requests; in cookie-based authentication the session ends when the cookie expires, the user signs out, or the server invalidates the session record. Sliding expiration extends the session as the user works; sign-out terminates it explicitly; a key rotation on the server invalidates every outstanding session at once.

17.6 Why CSRF is a cookie-specific problem

Cross-site request forgery exploits the very property that makes cookies convenient: the browser attaches the authentication cookie to *every* request to the application's domain, including requests originating from other tabs and other sites. A hostile site can host a hidden form that posts to `https://bank.example/transfer` with attacker-chosen field values, and as long as the user is signed in to the bank in another tab, the browser cheerfully attaches the bank's cookie to the cross-site post. The bank sees a fully authenticated request and acts on it.

Bearer-token APIs are not exposed to this attack because tokens are not attached automatically — application code has to read the token from storage and place it in an `Authorization` header, and that code is bound by the same-origin policy. CSRF is therefore a problem peculiar to cookie-authenticated sites, and the defence has to live at the application layer.

The browser-level defence is the `SameSite` cookie attribute. With `SameSite=Lax` (the current default in evergreen browsers) the browser withholds the cookie from cross-site `POST`, `PUT`, `DELETE`, and `PATCH` requests, while still attaching it to top-level `GET` navigations so links into the application keep working. `SameSite=Strict` withholds the cookie from every cross-site request including top-level navigations, which is safer but breaks deep-linked sign-ins from email and chat. `Lax` is the right default for most applications.

The application-level defence is the **anti-forgery token** — a per-session secret embedded in HTML forms and validated on POST; it prevents a malicious site from causing the user's browser to submit a form to your site using the user's authentication cookie. ASP.NET Core implements it as a pair: a long-lived cookie carrying half of the secret, and a hidden form

field carrying the other half. The hostile site can cause the browser to send the cookie half (the browser attaches cookies automatically), but it cannot read the cookie or the hidden field across origins, so it cannot construct a matching form-field half. When the server's `[ValidateAntiForgeryToken]` attribute compares the two halves on POST and finds they do not match, the request is rejected with `400 Bad Request` before any controller code runs. The token is bound to the user's session, regenerated on sign-in, and invalidated on sign-out, which closes the loop with the cookie scheme described above.

The combination of `SameSite=Lax` and `[ValidateAntiForgeryToken]` is belt-and-braces: the browser refuses to send the cookie on most cross-site posts, and the server refuses to act on a post that arrives without a matching token even if a future browser bug or unusual configuration delivers the cookie anyway.

17.7 Putting the pieces together

The companion exercise series at `/exercises/10-webapp-development/4-authentication-authorization/` walks the entire mechanism in code: a hardcoded user list and a login form post in the first exercise, role-aware claims in the second, claims-based policies and the antiforgery middleware in the third. The same `Set-Cookie` flow described here drives every step; subsequent chapters of this Part replace pieces of it (the user store, the credential format, the third-party identity provider) without disturbing the cookie machinery itself.

17.8 Summary

Cookie-based authentication uses an HTTP cookie to carry an encrypted, signed authentication ticket between the browser and the server, so a user who signs in once is recognised on every later request without re-supplying a password. The sign-in handshake validates a credential, builds a `ClaimsPrincipal`, and calls `HttpContext.SignInAsync` to emit the cookie; sign-out emits an expired cookie that the browser then drops. The cookie's `HttpOnly`, `Secure`, `SameSite`, and expiration attributes are what keep the scheme safe against script-based theft, network-based theft, cross-site replay, and indefinite reuse. Sliding expiration extends an active session up to a configured maximum while letting idle sessions die. Cross-site request forgery is the structural cost of having the browser attach the cookie to every request, and the anti-forgery token plus `SameSite=Lax` is the standard defence. Every later chapter of this Part — Identity, claims and policies, bearer tokens, and external sign-in — either builds on this cookie scheme or contrasts itself with it.

PART V · IDENTITY AND SECURITY

CHAPTER 18

ASP.NET Core Identity

Watch the presentation

Se presentationen på svenska

A web application that lets users register, sign in, change passwords, and recover forgotten accounts has to solve a long list of small but unforgiving problems: storing passwords safely, generating reset tokens that cannot be guessed, locking accounts after repeated failures, and stitching all of this onto the cookie-authentication transport so the browser stays signed in. Writing this code from scratch is the wrong default — every shortcut becomes a security incident, and every team that tries it ends up reinventing the same data structures.

ASP.NET Core ships a framework component that already solves these problems and slots into a cookie-authenticated application without changing the request pipeline.

18.1 What ASP.NET Core Identity provides

ASP.NET Core Identity is the framework's user-management system: it stores user records (with hashed passwords), provides `UserManager<TUser>` and `SignInManager<TUser>` services, and integrates with cookie authentication to handle registration, sign-in, password reset, and lockout. It is a library, not a product — it ships as NuGet packages (`Microsoft.AspNetCore.Identity.EntityFrameworkCore` and friends) that an application opts into during service registration in `Program.cs`.

The component sits underneath the sign-in flow rather than alongside it. Cookie authentication answers the question *how does the server recognise this browser as already signed in?* — encrypted ticket, `Set-Cookie` header, middleware decoding the ticket on every request. Identity answers a different question: *given a username and password the user just typed, is there a matching record, and what does that record contain?* The two systems compose: `SignInManager` validates the credential against the user database, then asks the cookie-authentication handler to issue the ticket. Without Identity, an application can still use cookies; without cookies, Identity has no way to keep the user signed in. Identity provides the user database; cookie authentication provides the session transport.

18.1.1 The IdentityUser data model

Identity defines a base entity called `IdentityUser` that represents one row in the user table. Out of the box it carries fields that every credential-based system eventually needs: a

primary key (`Id`, a GUID by default), the username, a normalised version of the username for case-insensitive lookup, the email address (also normalised), the password hash, a security stamp that changes whenever the user's credentials are rotated, a phone number, two-factor-authentication state, lockout state, and an access-failure counter.

Most applications need to store more than this — a display name, a profile picture URL, a tenant ID, a created-at timestamp. The framework expects this and exposes a clean extension point: derive a class from `IdentityUser`, add the extra columns, and tell Identity to use the derived type. Figure 1 shows a minimal extension.

Figure 1: Extending `IdentityUser` with application fields

```
public class ApplicationUser : IdentityUser
{
    public string DisplayName { get; set; } = "";
    public DateTime CreatedAt { get; set; } = Date-
        Time.UtcNow;
}
```

The derived type flows through the rest of the framework as a generic parameter — `UserManager<ApplicationUser>`, `SignInManager<ApplicationUser>`, `IdentityDbContext<ApplicationUser>` — so the application's controllers see the full type, with the application-specific properties available, while the framework continues to operate on the base contract.

18.2 User stores: where the records live

A **user store** is the persistence backend ASP.NET Core Identity uses to read and write user records; common implementations include in-memory (development only), Entity Frame-

work Core against SQL, and custom implementations targeting other databases. The user store is the repository pattern applied to identity data: a thin abstraction (`IUserStore<TUser>`, `IUserPasswordStore<TUser>`, `IUserRoleStore<TUser>` and a handful of related interfaces) that the manager services call into without knowing how the data is actually stored.

This indirection matters because credential storage requirements vary. A unit-test suite wants an in-memory store that disappears between runs. A development environment wants a local SQLite file. A production deployment wants SQL Server, PostgreSQL, or a managed database service. A multi-tenant SaaS application might have a custom store that shards users across tenant-specific databases. The controller code does not change between these — `userManager.FindByNameAsync(name)` returns the user record regardless of where it is read from.

18.2.1 EF Core: the default store

The most common configuration uses Entity Framework Core against a relational database. The application defines a `DbContext` that derives from `IdentityDbContext<TUser>`, and the EF-backed user store reads and writes through that context. Identity's migrations create the tables — `AspNetUsers`, `AspNetRoles`, `AspNetUserRoles`, `AspNetUserClaims`, `AspNetUserLogins`, `AspNetUserTokens`, `AspNetRoleClaims` — with the columns required by the base entities and any extras introduced by derived types.

Service registration combines the two halves with one chained call:

Figure 2: Registering Identity with an EF Core store

```
builder.Services.AddDbContext<ApplicationDbContext>(options
        =>
    options.UseSqlServer(builder.Configuration.GetConnec-
        tionString("Default")));

builder.Services
    .AddDefaultIdentity<IdentityUser>(options =>
    {
        options.SignIn.RequireConfirmedAccount = true;
        options.Password.RequiredLength = 12;
        options.Lockout.MaxFailedAccessAttempts = 5;
    })
    .AddEntityFrameworkStores<ApplicationDbContext>();
```

`AddDefaultIdentity` registers `UserManager`, `SignInManager`, the cookie authentication handler used for the sign-in scheme, the password hasher, the validators, and several supporting services — using the framework's dependency-injection container. `AddEntityFrameworkStores` plugs in the EF-backed user store implementations. Configuration callbacks tune password rules, lockout thresholds, and sign-in requirements; everything else uses sensible defaults.

18.2.2 In-memory and custom stores

The EF Core InMemory provider is appropriate only for tests and short-lived demos: data lives in process memory and is lost when the application restarts. The Identity exercises start with this provider so students can register users and sign them in without setting up a database, then feature-flag a switch to SQLite so the same code persists across restarts.

When neither EF nor in-memory fits — for example, when users live in an existing legacy database with a different schema, or in an external directory service — the application implements `IUserStore<TUser>` and the related interfaces directly. The manager services do not change; they call the same interface methods, and the custom implementation

translates each call into the appropriate query against the target store. This is rarely the right starting point, but it is the escape hatch when the default store is wrong.

18.3 UserManager and SignInManager

Two services do most of the work, and the split between them is deliberate.

`UserManager<TUser>` is the user-database API. It exposes operations on the user records themselves — create a user, find a user by name or email, change a password, generate a password-reset token, validate that a token is still valid, add a user to a role, check email-confirmation status, count failed sign-in attempts. Every method is asynchronous and returns an `IdentityResult` carrying either success or a list of errors. `UserManager` does not know about cookies, sessions, or HTTP context; it is a pure data-and-rules service.

`SignInManager<TUser>` sits one layer above and orchestrates the sign-in flow. Its `PasswordSignInAsync(username, password, isPersistent, lockoutOnFailure)` method calls `UserManager.FindByNameAsync` to load the user, calls the password hasher to verify the supplied password against the stored hash, increments the failed-attempt counter on mismatch (and locks the account when the threshold is reached), and on success calls into the cookie-authentication handler to issue the encrypted ticket. The manager also handles sign-out, two-factor sign-in, external-login sign-in (the OAuth/OIDC entry point covered later in this Part), and re-validating the principal when the security stamp changes.

The reason these services are registered with dependency injection rather than exposed as static classes is that they depend on the user store, the password hasher, the option configuration, and the HTTP context — all of which are them-

selves DI-managed and request-scoped. A static `UserManager.FindByName` would have nowhere to find its store.

18.3.1 Password hashing

Password hashing transforms a plaintext password into a fixed-length value through a slow, salted one-way function (PBKDF2, bcrypt, Argon2), making it computationally infeasible to recover the original password from the hash; the user store keeps only the hash. Identity's default hasher uses PBKDF2 with HMAC-SHA-512, a per-user random salt, and a configurable iteration count (310,000 in the current default). The salt and iteration count are stored alongside the hash, so the same hasher can verify passwords created under previous parameter sets and produce a fresh hash with current parameters when the user signs in successfully.

The slowness is the point. A PBKDF2 verification takes a fraction of a second on the server — invisible to the user — but turns an offline attack against a stolen password column into a multi-year compute job. Combined with per-user salts, the hasher prevents rainbow-table attacks: each user's hash is unique even when two users picked the same password.

The application never sees the plaintext password after the request handler. `UserManager.CreateAsync(user, password)` accepts the plaintext, runs the configured password validators (length, complexity, banned-password lists), hashes the result, and stores only the hash. `SignInManager.PasswordSignInAsync` does the equivalent on the way back: it reads the hash, asks the hasher to verify the supplied plaintext against it, and discards the plaintext as soon as the comparison completes.

18.3.2 Lockout, registration, and password reset

Identity's lockout state is a small set of columns on the user record: a counter for consecutive failures, a `LockoutEnd` timestamp, and a flag for whether lockout is enabled at all. After `MaxFailedAccessAttempts` consecutive failures (default 5), `SignInManager` sets `LockoutEnd` to "now plus `DefaultLockoutTimeSpan`" (default 5 minutes). Subsequent sign-in attempts during the lockout window fail immediately without checking the password, even when the password is correct. A successful sign-in resets the counter. This blunts online password-guessing attacks: an attacker can try at most a few passwords per account per lockout window.

Registration is the inverse of sign-in: `UserManager.CreateAsync(user, password)` runs validators, hashes the password, persists the new row, and returns success or a list of validation errors. The application typically sends an email-confirmation link at this point, generated with `UserManager.GenerateEmailConfirmationTokenAsync` — a signed, time-limited token that the confirmation endpoint verifies before flipping the `EmailConfirmed` flag.

Password reset follows the same shape. `UserManager.GeneratePasswordResetTokenAsync` produces a token that is emailed to the user; the reset endpoint takes the token and a new password, calls `UserManager.ResetPasswordAsync`, and the framework verifies the token, runs the password validators, hashes the new password, and replaces the stored hash. Because the token is bound to the user's security stamp, any successful password change invalidates outstanding tokens — a useful defence when a previous reset link leaks.

18.4 A worked example: registering and signing in a user

The companion exercises in Identity and user stores build a full registration and sign-in flow on top of the cookie foundation from the previous chapter. Figure 3 shows the registration handler in its smallest useful form.

Figure 3: A registration controller using `UserManager`

```
public class RegisterController : Controller
{
    private readonly UserManager<IdentityUser> _userMan-
        ager;
    private readonly SignInManager<IdentityUser> _signIn-
        Manager;

    public RegisterController(
        UserManager<IdentityUser> userManager,
        SignInManager<IdentityUser> signInManager)
    {
        _userManager = userManager;
        _signInManager = signInManager;
    }

    [HttpPost]
    public async Task<IActionResult> Register(RegisterView-
        Model model)
    {
        if (!ModelState.IsValid) return View(model);

        var user = new IdentityUser
        {
            UserName = model.Email,
            Email = model.Email
        };

        var result = await _userManager.CreateAsync(user,
        model.Password);

        if (result.Succeeded)
        {
            await _userManager.AddToRoleAsync(user, "Can-
        didate");
            await _signInManager.SignInAsync(user, isPer-
        sistent: false);
            return RedirectToAction("Index", "Home");
        }

        foreach (var error in result.Errors)
            ModelState.AddModelError(string.Empty,
        error.Description);

        return View(model);
```

```
    }
}
```

The handler reads two services from constructor injection. `CreateAsync` validates the password against the configured rules, generates a per-user salt, runs PBKDF2, and persists the hashed row through the EF user store. `AddToRoleAsync` writes a row to the `AspNetUserRoles` join table. `SignInAsync` hands the freshly created user to the cookie-authentication handler, which builds a `ClaimsPrincipal` from the user record, encrypts a ticket containing it, and emits the `Set-Cookie` header. The browser keeps the cookie; every subsequent request reaches the application as an authenticated session, and `HttpContext.User` exposes the principal exactly as in the hand-rolled example from the previous chapter — but without any of the password-handling code in the application's own source.

18.5 When not to use Identity

Identity is the right default when the application owns its users — the user database belongs to this service, registration happens here, and password recovery flows live in this codebase. It is the wrong default in three situations.

SITUATION	BETTER CHOICE	WHY
Pure JSON API consumed by another service	JWT bearer authentication only	No browser, no sessions, no registration flow — Identity's machinery is dead weight.
Single-page app or mobile client federated to an external IdP	OpenID Connect (Microsoft Entra ID, Auth0, etc.)	The IdP owns the user database; the application validates tokens it produced.
Internal tool with no external users	Entra ID with conditional access	The organisation's existing directory holds the accounts; duplicating them in an app database is a liability.

Each of these alternatives is covered in later chapters of this Part. The decision tree in practice is short: if the application stores its own user records and accepts passwords from those users, use Identity; otherwise, use the framework component that matches the credential format the application actually receives.

18.6 Summary

ASP.NET Core Identity is the framework's user-management subsystem: it owns the user table, hashes passwords with PBKDF2 plus per-user salts, tracks lockout state, generates the tokens used for email confirmation and password reset, and exposes everything through two dependency-injected services — `UserManager<TUser>` for data and rules, `SignInManager<TUser>` for the sign-in flow. The user store interface decouples this logic from persistence, with EF Core against a relational database as the standard choice and custom implementations available when needed. Identity composes with cookie authentication rather than replacing it: cookies carry the session, Identity owns the database. Applications that store their own users almost always want Identity; APIs validating externally issued tokens, or systems federated to an external identity provider, do not.

PART V · IDENTITY AND SECURITY

CHAPTER 19

Roles, Claims, and Policies

Watch the presentation

Se presentationen på svenska

Once a request has been authenticated, the application still has to decide whether the caller may perform the requested action. That decision needs facts about the caller — which department they belong to, which roles they hold, how old they are, which tenant they signed in from — not merely confirmation that they signed in. ASP.NET Core represents those facts as claims hanging off a `ClaimsPrincipal`, and offers three layers of authorization built on that representation: role checks, named policies, and custom requirements with their

own handlers. Each layer earns its place at a different level of rule complexity, illustrated below by an admin-only controller and a side policy that reads a date-of-birth claim.

19.1 Claims as the unit of fact

A **claim** is a name-value pair carried by a `ClaimsPrincipal` that asserts something about the identity — name, email, role membership, tenant, scope; authorization decisions read claims off the principal rather than re-querying the user database. Each claim has a type (a URI-like string such as `"role"`, `"email"`, or a custom `"DateOfBirth"`) and a value (the asserted fact, such as `"Admin"` or `"1995-04-12"`). Authentication is what populates the principal with claims; authorization is what reads them.

This separation matters because it lets authorization run cheaply on every request. The cookie-authentication middleware decrypts the auth ticket and reconstructs the principal once; from then on the controller, the action filter, and the view all see the same claim set without touching the database. Claims travel with the request, not with the data store.

19.1.1 How a controller sees claims

Inside a controller, `User` is the `ClaimsPrincipal`. The principal exposes its claims through `User.Claims` (an enumerable) and `User.FindFirst(type)` (a lookup). The framework's [Authorize] attribute, introduced with the MVC pattern in Part III, hooks into the same principal: it asks the principal whether it satisfies the requirement attached to the attribute and short-circuits with a 401 or 403 if not.

```
public IActionResult Profile()
{
    string? name = User.Identity?.Name;
    string? email = User.FindFirst("email")?.Value;
    string? dateOfBirth =
        User.FindFirst("DateOfBirth")?.Value;
    return View(new ProfileViewModel(name, email, dateOf-
        Birth));
}
```

The action does not call into the user database. Every fact it displays was placed on the principal by the sign-in flow — either by the cookie middleware deserialising the auth ticket, or by a claim transformation step that ran when the principal was constructed.

19.2 Roles as a special claim

A **role** is a named group that an identity belongs to (Admin, Candidate, Reviewer); the framework treats role membership as a special claim and provides shorthand syntax such as `[Authorize(Roles = "Admin")]` to require it. Despite the convenience syntax, there is no magic: a role is a claim whose type is `ClaimTypes.Role` (the URI `"http://schemas.microsoft.com/ws/2008/06/identity/claims/role"`, also accepted as the short form `"role"`). The shorthand `User.IsInRole("Admin")` walks the principal's claims and returns true if any has type `Role` and value `"Admin"`.

Role-based authorization is coarse on purpose. It answers a single question — does the user belong to this group? — and it answers it quickly. For long-lived organizational categories that map directly to UI sections, that is exactly the right tool.

19.2.1 The admin-only controller

Consider a `CloudSoft Recruitment Portal` admin area: pages to approve candidates, retire job postings, and manage other administrators. None of these actions should be reachable by a regular candidate or even by a recruiter. A role check on the controller class enforces the boundary in one line:

```
[Authorize(Roles = "Admin")]
public class AdminController : Controller
{
    public IActionResult Dashboard() => View();

    public IActionResult ApproveCandidate(int id)
    {
        // ... business logic ...
        return RedirectToAction(nameof(Dashboard));
    }
}
```

Every action in `AdminController` now requires an authenticated principal whose claim set contains a `Role` claim with value `"Admin"`. An anonymous request bounces to the login page (401 unauthorized, redirected by the cookie scheme); an authenticated non-admin gets a 403 forbidden. The check applies uniformly without each action having to repeat itself.

The same attribute accepts a comma-separated list to widen the gate: `[Authorize(Roles = "Admin,Recruiter")]` admits anyone with either role. Multiple `[Authorize]` attributes stack with AND semantics — `[Authorize(Roles = "Admin")] [Authorize(Roles = "Recruiter")]` would require both, which is rarely useful.

19.3 When roles stop being enough

Role checks fail the moment the rule depends on something other than group membership. “Users may edit their own profile but not other people’s” depends on identity equality, not a role. “Users from the EU tenant must accept GDPR terms” depends on a tenant claim. “Users must be at least 18” depends on a date. Encoding these as roles works once and then collapses: every nuance becomes a new role, and the role list grows until it is no longer a list of groups but a list of policies in disguise.

Claim-based checks address the first half of the problem by reading specific claims directly: `User.HasClaim("tenant", "eu")`. They are flexible but ad hoc — the rule lives wherever it is needed, scattered across actions, with no single name to refer to. That makes the rules hard to audit and even harder to change consistently. The third layer, **policies**, gives the rule a name and a single registration point.

19.4 Authorization policies

An **authorization policy** is a named set of requirements registered at startup; controllers and actions opt in with `[Authorize(Policy = "Name")]`, and the framework evaluates each requirement against the principal’s claims to decide allow or deny. The policy is registered once in `Program.cs`, named once, and applied wherever the rule matters. If the rule changes, it changes in one place.

For simple cases, the policy can be expressed inline using the fluent builder on `AuthorizationOptions`:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("EuTenant", policy =>
        policy.RequireClaim("tenant", "eu"));

    options.AddPolicy("AdminOrRecruiter", policy =>
        policy.RequireRole("Admin", "Recruiter"));
});
```

A controller action then references the policy by name:

```
[Authorize(Policy = "AdminOrRecruiter")]
public IActionResult Inbox() => View();
```

The shift from `[Authorize(Roles = "Admin,Recruiter")]` to `[Authorize(Policy = "AdminOrRecruiter")]` looks cosmetic, but the indirection is the point. The action no longer encodes the rule; it asks for "the rule we call AdminOrRecruiter," and the rule's definition lives in `Program.cs`. Renaming a role, swapping the rule for "Admin OR holds a `recruiter-permission` claim," or routing the decision through an external system all become one-line edits at the registration point — the controllers do not change.

19.4.1 A custom requirement: minimum age

`RequireClaim` and `RequireRole` cover claim-equality checks. Anything more interesting — comparisons, computations, conditional logic — needs a custom requirement and a handler. The pattern has two pieces: an `IAuthorizationRequirement` (a marker carrying any parameters the rule needs) and an `IAuthorizationHandler` (the code that evaluates whether the principal satisfies the requirement).

```
public class MinimumAgeRequirement : IAuthorizationRequire-
        ment
{
    public int MinimumAge { get; }

    public MinimumAgeRequirement(int minimumAge) => Minimu-
        mAge = minimumAge;
}

public class MinimumAgeHandler : AuthorizationHandler<Min-
        imumAgeRequirement>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        MinimumAgeRequirement requirement)
    {
        Claim? dobClaim = context.User.FindFirst("DateOf-
        Birth");
        if (dobClaim is null || !
        DateTime.TryParse(dobClaim.Value, out var dob))
        {
            return Task.CompletedTask;
        }

        int age = DateTime.UtcNow.Year - dob.Year;
        if (dob.Date > DateTime.UtcNow.AddYears(-age))
        {
            age--;
        }

        if (age >= requirement.MinimumAge)
        {
            context.Succeed(requirement);
        }

        return Task.CompletedTask;
    }
}
```

The handler reads the principal's `DateOfBirth` claim, parses it, computes the user's age, and calls `context.Succeed(requirement)` only if the age clears the

threshold. It never calls `context.Fail()` — the framework's default is failure, so a handler that does not succeed leaves the requirement unmet. That convention lets multiple handlers contribute to a single requirement, succeeding under different conditions, without one silently vetoing another.

The requirement and handler are wired up at startup:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("MinimumAge18", policy =>
        policy.Requirements.Add(new
        MinimumAgeRequirement(18)));
});

builder.Services.AddSingleton<IAuthorizationHandler, Minim-
        umAgeHandler>();
```

A controller action gates itself on the policy without knowing how the rule is implemented:

```
[Authorize(Policy = "MinimumAge18")]
public IActionResult AdultContent() => View();
```

The action expresses intent ("require an adult"); the handler expresses mechanism (parse a date claim, compute years). Each can change without the other, and the policy name keeps the two halves connected.

19.5 A decision framework

Each authorization style has a sweet spot. Choosing well keeps rules readable and changeable; choosing poorly buries the real logic in scattered conditionals or in role names that have stopped describing groups.

MECHANISM	USE WHEN	TRADE-OFF
`[Authorize]` (no arguments)	Action requires any authenticated user	Says nothing about who; relies on a separate authentication step
`[Authorize(Roles = "X")]`	Rule is exactly "is the user in group X"	No way to express anything beyond group membership
`RequireClaim("type", "value")` policy	Rule is exactly "user has this fact"	Still equality-only; no comparisons or computations
Custom `IAuthorizationRequirement` + handler	Rule needs computation, comparison, or external lookup	More code; the rule lives in C#, not declarations

Two questions sharpen the choice. First: does the rule have a name a colleague would recognise? If yes, prefer a named policy regardless of how simple it currently is — the name itself is the long-term value. Second: can the rule be expressed as "the principal has claim X with value Y"? If yes, `RequireClaim` is enough; if not, a custom requirement is the honest expression.

Roles remain useful for the high-traffic, organization-shaped categories (Admin, Customer, Support). Claims-and-policies become essential the moment rules involve resource ownership, attributes of the resource itself, or anything time-dependent. Most production applications use both: roles for the broad strokes, policies for the nuance.

19.6 Where claims come from

A policy can only check claims that the principal actually carries. The cookie middleware reconstructs the principal from the encrypted auth ticket — which means whatever claims were placed on the principal at sign-in time are the claims policies will see. Adding a new claim type therefore has two costs: updating the sign-in code to issue it, and accepting that previously issued cookies will not contain it until users sign in again.

ASP.NET Core Identity, covered in its own chapter, provides a `IUserClaimsPrincipalFactory<TUser>` extension point: when `SignInManager` builds the principal, it asks the factory to layer additional claims on top of the user record's stored claims. That is where a `DateOfBirth` claim would be added in an Identity-backed application — the factory reads the field off the `ApplicationUser` and emits it as a claim every time the user signs in. Claim transformation, registered as `IClaimsTransformation`, runs even later (on every request) and is suited to short-lived enrichments — for example, looking up the user's current subscription tier without baking it into the cookie.

The companion exercise Authentication and Authorization builds these layers from the bottom: a hardcoded user list signs in with cookies, the principal is given a role claim, an `[Authorize(Roles = "Admin")]` page enforces it, and a final exercise issues a date-of-birth claim and gates an adult-content page on a `MinimumAge` policy. Walking through each step manually — without ASP.NET Core Identity in the way — makes the relationship between claim, role, and policy concrete.

19.7 Summary

Claims are the unit of fact authorization runs on, hanging off the `ClaimsPrincipal` that ASP.NET Core attaches to every authenticated request. Roles are a special-cased claim type with shorthand syntax, ideal for coarse organizational categories such as the admin-only `AdminController`. Named policies give rules a single registration point and a recognisable name, replacing scattered `User.HasClaim` calls with `[Authorize(Policy = "...")]`. When a rule needs computation, comparison, or anything beyond claim equality — a `MinimumAge` check against a date-of-birth claim is the canonical example — a custom `IAuthorizationRequirement` paired with an `IAuthorizationHandler` keeps the logic in one place while the controllers stay declarative. Roles for the broad strokes, policies for the nuance, custom handlers for the rules a single claim cannot express: each layer earns its place, and most applications use them together.

PART V · IDENTITY AND SECURITY

CHAPTER 20

Bearer Tokens and JWT

Watch the presentation

Se presentationen på svenska

Cookie-based sign-in works because the caller is a browser visiting pages on the same site. Once the caller becomes a mobile app talking to a JSON API, a single-page application calling its backend over CORS, or a backend service calling another backend service, cookies stop fitting. There is no browser to manage the cookie jar, no same-origin domain to scope it to, and often no server-side session store the operators want to maintain. APIs solve this with a different transport for the credential — a token the client carries explicitly in

the `Authorization` header — and with a self-describing token format that lets the receiving service validate the credential without touching a session database. Bearer tokens as a concept, the JWT structure that fills the bearer-token slot, the validation path the server runs, and the points at which the model breaks down are what the rest of the chapter develops.

20.1 Why cookies do not fit API calls

Cookie authentication works at the browser layer. When a server sends `Set-Cookie`, the browser stores the value, scopes it to the issuing domain, and attaches it to every subsequent request to that domain — see Cookie-based authentication for the full mechanics. That whole machinery lives in the browser. A native mobile app does not have a cookie jar shared with a browser; a Swift or Kotlin HTTP client must be told explicitly to attach a credential to each request. A single-page application calling an API on a different domain runs into CORS rules and the `SameSite` attribute, both of which were tightened precisely to stop cookies from leaking across origins. A scheduled background job calling a partner API has no browser at all.

What every one of these clients does have is the ability to set an HTTP header on the requests it makes. That is the slot bearer tokens fill. The client obtains a credential through some sign-in flow, then sends it on every API call as an `Authorization` header. The server validates the header, identifies the caller, and decides whether to serve the request — without ever asking the client to maintain a session cookie.

20.2 Bearer tokens

A **bearer token** is an opaque or structured credential a client sends in an HTTP request's `Authorization: Bearer <token>` header; whoever bears the token is treated as authenticated, so bearer tokens must be transmitted over TLS and kept short-lived. The wire format is deliberately simple:

```
GET /api/quotes HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5c-
CI6IkpXVCJ9...
```

The word "bearer" is load-bearing. The server does not check who is presenting the token, only that the token itself is valid. Any process that holds the token can use it. That is the design — it is what lets a mobile app, a server-side worker, and a CLI tool all use the same authentication scheme — but it is also the threat model. A leaked token is a usable credential until it expires, with no equivalent of "this is the wrong machine, reject it."

Two consequences fall out of this immediately. First, bearer tokens must travel over TLS. A token sent over plain HTTP can be observed by any intermediary on the network and replayed verbatim; with no possession check beyond holding the bytes, replay is sufficient. Production APIs reject HTTP entirely and accept tokens only over HTTPS. Second, bearer tokens should be short-lived. The shorter the token's lifetime, the smaller the window during which a leaked token is useful. Most API designs issue bearer tokens with lifetimes measured in minutes to a small number of hours, then refresh them through a separate flow.

Bearer tokens themselves do not specify a format. The header could carry a random opaque string that the server

looks up in a database. In practice, the structured-token format that has won is the JWT.

20.3 The JWT format

A **JWT** (JSON Web Token) is a structured bearer-token format consisting of a base64-encoded header, payload, and signature joined by dots; the payload carries claims (subject, issuer, audience, expiration), and the signature lets the recipient verify the token has not been tampered with. Decoded into its three parts, a JWT looks like this:

```
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCIsImtpZCI6IjEyMyJ9
.
eyJpc3MiOiJodHRwczovL2lkLmV4YW1wbGUuY29tIiwic3ViIjoiNDIiL-
CJhdWQiOiJhcGkuZXhhbXBsZS5jb20iLCJleHAiOjE3MTM5NjAwMDB9
.
TJ8x8KQ3vH9z...
```

The first segment is the *header*. Decoded as JSON, it tells the recipient which algorithm signed the token and, optionally, which key was used:

```
{ "alg": "RS256", "typ": "JWT", "kid": "123" }
```

The second segment is the *payload*, sometimes called the claim set. It is JSON describing who the token is about and the constraints under which it is valid:

```
{
  "iss": "https://id.example.com",
  "sub": "42",
  "aud": "api.example.com",
  "iat": 1713956400,
  "nbf": 1713956400,
  "exp": 1713960000,
  "role": "admin"
}
```

The third segment is the *signature*. It is the cryptographic output of signing `base64url(header) + "." + base64url(payload)` with the issuer's signing key. Verifying that signature with the matching key proves two things: the token came from someone who had the signing key, and not a single byte of the header or payload has been altered since.

An important non-property of the encoding deserves stating plainly: base64url is encoding, not encryption. Anyone who copies a JWT into a debugger can read the payload. The JWT format protects integrity, not confidentiality. Sensitive information — passwords, raw personal data, anything that would harm the user if disclosed — does not belong in a JWT payload.

20.3.1 The standard claims

The JWT specification reserves a small set of short claim names with defined meanings. They are the spine of every validation routine.

CLAIM	MEANING
`iss`	Issuer — the service that signed the token
`sub`	Subject — the identity the token is about, typically a user ID
`aud`	Audience — the service the token is intended for
`exp`	Expiration time as a Unix timestamp; the token is invalid after this instant
`nbf`	Not before — the earliest time at which the token may be used
`iat`	Issued at — when the token was minted

A JWT's **issuer** (`iss` claim) names the service that signed the token, and the **audience** (`aud` claim) names the service that should accept it; the validating service rejects tokens whose issuer or audience do not match its configured expectations, preventing tokens minted for one service from being replayed against another. These two claims do most of the heavy lifting in multi-service architectures. The token a sign-in service mints for the quotes API carries `aud: "https://api.example.com/quotes"`; the quotes API checks the audience and refuses any token where it does not match. A token leaked from a different API in the same fleet cannot be replayed against the quotes API, because its `aud` claim names a different service.

`exp`, `nbf`, and `iat` together bound the token's validity in time. `iat` records when it was issued (useful for logging and for revocation strategies based on "everything before this

timestamp is invalid"); `nbf` and `exp` define the window during which the token may be used. Beyond the standard claims, the payload can carry any application-specific claims the issuer wants the recipient to know — role memberships, tenant identifiers, scope strings.

20.4 Validating a JWT

When a request arrives at the API, the server pulls the token out of the `Authorization` header and runs a fixed validation routine before any controller code executes:

1. Split the token into its three segments.
2. Decode the header and read the `alg` and `kid` fields.
3. Look up the verification key for that `kid`.
4. Compute the signature over the first two segments and compare it to the third.
5. Decode the payload and check `iss`, `aud`, and the time-based claims.

If any step fails, the server responds with `401 Unauthorized` and the request never reaches the controller. If every step succeeds, the server constructs a `ClaimsPrincipal` from the payload and the controller sees an authenticated request — exactly as it would after cookie authentication.

The verification key depends on the signing algorithm. Two patterns dominate. *Symmetric signing* uses an HMAC algorithm such as `HS256`, where the same secret signs and verifies. This is straightforward when one service both issues and validates the tokens, but it forces every validating service to hold the signing secret — a problem when many services need to validate tokens from a single issuer. *Asymmetric signing* uses an algorithm such as `RS256` or `ES256`, where the issuer signs with a private key and validators verify with the match-

ing public key. The private key never leaves the issuer; the public key can be published openly, typically at a well-known URL such as `https://id.example.com/.well-known/jwks.json`. This is the model OAuth and OpenID Connect rely on, and the model ASP.NET Core's `JwtBearer` middleware expects when an authority is configured.

20.4.1 A worked example: the quotes API

The companion exercise REST API and DTOs walks through this pattern by building a small `CloudCiApi` quotes service and progressively adding a JWT bearer gate on top of it. Registering the middleware in `Program.cs` takes a handful of lines:

```
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationS-
        cheme)
    .AddJwtBearer(options =>
    {
        options.Authority =
        builder.Configuration["Jwt:Authority"];
        options.Audience  =
        builder.Configuration["Jwt:Audience"];
        options.TokenValidationParameters.ValidateIssuer
        = true;
        options.TokenValidationParameters.ValidateAudience
        = true;
        options.TokenValidationParameters.ValidateLifetime
        = true;
    });

builder.Services.AddAuthorization();
```

The `Authority` value points at the issuer (for example `https://id.example.com`). The middleware fetches `/.well-known/openid-configuration` from that URL on startup, reads the `jwks_uri` it advertises, and downloads the public keys it should use to verify signatures. The `Audience`

value is the string the middleware will require in the `aud` claim of every incoming token. Everything else — splitting the token, checking `exp` and `nbf`, comparing the signature — is handled automatically.

Controllers opt into the gate with the `[Authorize]` attribute, naming the JWT scheme so it is unambiguous which authentication scheme should validate the request:

```
[ApiController]
[Route("api/quotes")]
[Authorize(AuthenticationSchemes = JwtBearerDe-
        faults.AuthenticationScheme)]
public class QuotesController : ControllerBase
{
    [HttpGet]
    public IEnumerable<QuoteDto> List() => _ser-
        vice.GetAll();
}
```

A request without an `Authorization` header now returns `401 Unauthorized` with a `WWW-Authenticate: Bearer` response header. A request with a token that fails any validation step returns `401` as well; a request with a valid token reaches the action, and `User.Identity.IsAuthenticated` is true with the claims from the JWT payload available on `User.Claims`. From the controller's perspective, the only difference from cookie authentication is the credential transport — the authorization model on top is identical.

20.5 Lifetimes and refresh tokens

Bearer tokens are designed to be short-lived because possession alone authenticates. A common production lifetime is between five minutes and one hour. Anything longer enlarges

the window during which a leaked token is usable; anything much shorter forces the client to re-authenticate so often that user experience suffers.

Short lifetimes do not mean the user re-enters their password every five minutes. The standard pattern pairs a short-lived *access token* (the JWT presented to APIs) with a longer-lived *refresh token* (a separate, opaque credential the client keeps and presents only to the issuer). When the access token expires, the client sends the refresh token to the issuer's token endpoint and receives a new access token in return. The refresh token never goes to APIs and can be revoked server-side by removing it from the issuer's database. The split lets access tokens stay self-contained and verifiable offline while keeping a server-side handle on the long-lived credential.

20.6 When JWT is the wrong choice

The validate-without-asking property that makes JWTs efficient is also their main limitation. A cookie session can be killed by deleting one row from a database; the next request the client makes finds no session and is rejected. A JWT, once issued, is valid until it expires — there is no central place where the validating service asks "is this token still good?" Revocation strategies for JWT exist, but each one gives back something the format was designed to avoid:

- A revocation list (block-list of token IDs) requires every API to query a central store on every request, recreating the session lookup the format was meant to eliminate.
- Rotating signing keys invalidates every token signed with the old key, not just the ones for the compromised user.

- Shortening the lifetime to seconds approximates revocation by attrition but pushes load to the issuer's token endpoint.

Where revocation is critical — administrative bans, password resets that must take effect now, compliance scenarios that demand immediate session termination — server-side sessions or opaque tokens with central introspection are a better fit than JWTs. Where revocation can wait until the natural expiry of a short-lived token, JWTs offer scale and decentralisation that session stores cannot match. The decision is about how long an unwanted credential may remain valid, not about which technology is more sophisticated.

20.7 Summary

Bearer tokens move authentication out of the cookie jar and into the `Authorization` header, fitting the calling models cookies do not — mobile apps, single-page applications, and service-to-service calls. The format that fills the slot is the JWT: a base64-encoded header, payload, and signature whose payload carries standard claims (`iss`, `sub`, `aud`, `exp`, `nbf`, `iat`) and whose signature lets the recipient verify the token without contacting the issuer. ASP.NET Core's `JwtBearer` middleware, configured with an `Authority` and an `Audience`, handles the full validation path; controllers opt in with `[Authorize(AuthenticationSchemes = JwtBearerDefaults.AuthenticationScheme)]`. Short access-token lifetimes paired with longer-lived refresh tokens keep the leak window small without forcing constant re-authentication. The trade-off is revocation: a JWT is valid until it expires, so scenarios that demand immediate session termination remain better served by server-side sessions than by tokens.

PART V · IDENTITY AND SECURITY

CHAPTER 21

API Keys and Machine-to-Machine

Watch the presentation

Se presentationen på svenska

A backend service that calls another backend service has no human at a keyboard to type a password and no browser session to carry a cookie. The caller is a process; the callee still needs to decide whether to answer. The cheapest credential for that scenario is a long random string the two services agree on in advance, presented in a request header on every call. What such a string proves about the caller, how it travels with the request, where ASP.NET Core enforces the check,

and where the mechanism stops being enough are the questions that follow.

21.1 The machine-to-machine problem

Browser-based authentication has a user. The user types credentials into a login form, the server validates them against a user store, and the server issues a session credential — usually a cookie — that the browser attaches automatically to subsequent requests. Every chapter so far has built on that arc.

Service-to-service traffic has none of those pieces. A scheduled job calls a reporting API; a webhook handler in one service calls a fulfilment endpoint in another; a frontend BFF calls a downstream service it owns. There is no login form to render, no human to type a password, no browser to hold a cookie. The caller is a process started by an orchestrator, identified only by what it can prove on the wire.

This is the *machine-to-machine* (M2M) case, and the canonical credential for it is the API key. An **API key** is a long, opaque shared secret a client presents in a request header (commonly `X-Api-Key`) to identify itself to the server; unlike a JWT it carries no claims, and unlike a user password it identifies a calling application rather than a person. The server compares the presented value against a configured expected value and either accepts the request or rejects it. There is no user, no role, no expiration baked into the credential — only "the caller knows the shared key, or it does not."

That bare property is the entire security model. Everything else in this chapter follows from it.

21.2 How an API key travels with a request

An API key is just bytes — a string of letters, digits, and base64 punctuation, typically 256 bits or more of entropy. It travels in an HTTP header named by the service. The de facto convention is `X-Api-Key`, though some APIs use `Authorization: ApiKey <value>` or a custom name. A typical request looks like this:

```
GET /api/quotes HTTP/1.1
Host: ca-api-week6.azurecontainerapps.io
X-Api-Key: kQ8j+7Hf4N2lZtwBpY5RxV9aMcDeGhJkLnPqSt...
Accept: application/json
```

The server reads the header, compares it against the expected value, and either lets the request proceed or short-circuits with an error status. Two failure modes need separate codes. *No header at all* means the server cannot identify the caller, so the response is `401 Unauthorized` with a `WWW-Authenticate: ApiKey` header naming the expected scheme. *A header that does not match* means the server can identify the caller's claim of authentication and refuses it, so the response is `403 Forbidden`. RFC 7235 is precise about that split, and both numeric responses are mechanical for an API key: missing → 401, wrong → 403, right → 200.

Because the entire credential travels in a header on every request, two operational requirements follow. The transport must be TLS — anything that crosses the wire in plaintext puts the key in every intermediate router's logs. The key itself must be long enough that brute-forcing it across the network is implausible; 256 bits of entropy is the standard floor.

21.3 ASP.NET Core middleware as the enforcement point

ASP.NET Core processes a request as an ordered chain of middleware components. Each component receives an `HttpContext` and a delegate to the next link; it can inspect the request, write a short-circuit response, or call the delegate to hand off. Authentication, authorization, routing, response compression, and exception handling are all middleware. An API-key check is exactly that shape: read the header, compare it to a configured value, either short-circuit with 401/403 or call `await _next(context)`.

Custom middleware in ASP.NET Core is a class with two requirements: a constructor that takes a `RequestDelegate` (the next link in the chain), and an `InvokeAsync(HttpContext)` method that does the work. The class does not implement an interface — the framework discovers the shape by convention when `app.UseMiddleware<T>()` is called. Constructor parameters beyond `RequestDelegate` are resolved from dependency injection at construction time, which is fine for a configuration-bound value like the expected API key.

21.3.1 A worked example

The companion exercise Securing the API with an API Key Middleware walks the full pattern end to end. The middleware itself is short:

```
public class ApiKeyMiddleware
{
    private const string HeaderName = "X-Api-Key";
    private const string ConfigKey = "ApiKey:Value";

    private readonly RequestDelegate _next;
    private readonly string? _expectedKey;

    public ApiKeyMiddleware(RequestDelegate next, IConfig-
        uration configuration)
    {
        _next = next;
        _expectedKey = configuration[ConfigKey];
    }

    public async Task InvokeAsync(HttpContext context)
    {
        if (!
        context.Request.Headers.TryGetValue(HeaderName,
        out var providedKey)
            || string.IsNullOrWhiteSpace(providedKey))
        {
            context.Response.Headers.Append("WWW-Authentic-
        ate", "ApiKey");
            context.Response.StatusCode =
        StatusCodes.Status401Unauthorized;
            return;
        }

        if (string.IsNullOrEmpty(_expectedKey)
            || !string.Equals(providedKey, _expectedKey,
        StringComparison.Ordinal))
        {
            context.Response.StatusCode =
        StatusCodes.Status403Forbidden;
            return;
        }

        await _next(context);
    }
}
```

Three branches cover the three outcomes. The constructor reads `ApiKey:Value` from `IConfiguration` once at startup; the per-request method only compares strings. The comparison is `StringComparison.Ordinal` — culture-aware comparison can normalize Unicode in ways that accept the wrong key, and an API key is a byte-for-byte token, not human-readable text. Registration in `Program.cs` is one line, placed after routing-related middleware and before the endpoint terminus:

```
app.UseHttpsRedirection();
app.UseMiddleware<ApiKeyMiddleware>();
app.UseAuthorization();
app.MapControllers();
```

That position runs the check after routing has matched a path to an endpoint, so later evolutions can inspect endpoint metadata to exempt specific routes (a `/health` probe, the Swagger UI), and before any controller code runs on a rejected request.

21.4 Why API keys are weaker than JWT

An API key answers exactly one question: does the caller know the shared secret? Compare that to a JWT bearer token, and three weaknesses emerge.

A JWT carries an *expiration* in its `exp` claim. The validating service rejects tokens past their lifetime without any operational action. An API key has no built-in expiration. It is valid until somebody changes the configured value on both sides, which is a deployment, not a token-validation step.

A JWT carries claims — subject, roles, tenant, custom application data — signed by the issuer. The validating service can authorize on the contents without consulting another

database. An API key carries no claims. Every caller that holds the key looks identical to the server; the server cannot distinguish the scheduled job from the webhook handler from the developer running curl with the production key on a laptop. If different callers need different permissions, an API key cannot express the difference.

A JWT supports *rotation through key IDs*. The issuer can publish multiple signing keys with different `kid` headers; the validator accepts any token signed by any currently-published key. Old tokens stay valid until they expire; new tokens are signed with the new key; the cutover is invisible to clients. An API key has no equivalent. Rotating it means replacing the configured value on the server, which invalidates every client at once unless the server is built to accept multiple valid keys during a transition window.

PROPERTY	API KEY	JWT BEARER TOKEN
Expiration	None inherent	`exp` claim, validated automatically
Identity carried	None — only "knows the secret"	Subject, claims, scopes
Per-caller authorization	No — same key, same access	Yes — claims drive `[Authorize]` decisions
Rotation cost	Coordinated cutover for every client	New key ID; old tokens age out
Wire format	Opaque string in `X-Api-Key`	Signed `header.payload.signature` in `Authorization: Bearer`
Best fit	One trusted client, simple gate	Multiple callers, per-caller permissions

The right choice depends on what the gate actually needs to enforce. When the threat model is "keep the open internet from hammering this endpoint" and there is one trusted client behind it, an API key is sufficient and simpler. When the threat model is "different callers get different access" or "credentials must expire automatically," JWT is the next step.

21.5 Generating, storing, and rotating keys

The credential's strength depends entirely on operational discipline. Three practices carry most of that weight.

21.5.1 Cryptographically random generation

A predictable key is no key. The generator must be cryptographically random — no incrementing counters, no timestamp-derived strings, no UUIDs (which include non-random bits). On a development machine the canonical command is:

```
openssl rand -base64 48
```

That produces 48 random bytes (384 bits) of entropy, base64-encoded into a 64-character string. The base64 alphabet is wire-friendly: no spaces, no quoting issues, safe in HTTP headers and shell commands. The exact length above 256 bits is mostly insurance against future cryptanalysis; the practical exposure surface for an API key is leakage, not brute force. A key that ends up in a Slack channel or a screenshotted terminal window is compromised regardless of how many bits it had.

21.5.2 Storage out of source control

A key checked into the repository is no longer secret. Anyone who clones the repo — including everyone who ever forks it, plus every CI job, plus every static-analysis service that mirrors source — has the credential. Two correct homes exist for the configured value, depending on environment:

ENVIRONMENT	STORAGE	WHY
Local development	Non-secret placeholder in `appsettings.Development.json`, or `dotnet user-secrets` for real local credentials	Loopback traffic on a developer machine has no realistic attacker; a clearly labelled placeholder is self-documenting if it leaks
Production	Platform secret store, injected as an environment variable via a `secretref:` mechanism	Encrypted at rest, separate permission to read, never visible in `az containerapp show` or Portal blades

Production secrets belong in a secret store — Azure Key Vault for general-purpose secret management, or the Container Apps secret mechanism for Container Apps deployments. The application code reads the same `IConfiguration` key (`ApiKey:Value`) in both environments;

only the surrounding provider chain differs. That uniformity is the point of the configuration system: one read, three sources, one mental model.

21.5.3 Key rotation

Key rotation is the practice of issuing a new credential and retiring the old one on a schedule (or after a suspected compromise); rotation hardens systems against long-lived stolen credentials and is supported by API gateways and secret stores that allow multiple valid keys during a transition window.

For a single-key configuration the rotation is destructive: change the secret on the server, every client breaks until it picks up the new value, then traffic resumes. That is acceptable when the client and server are deployed together and one of them owns both ends. It is not acceptable when independent clients consume the API on schedules the server cannot coordinate.

For independent clients the configuration must accept two valid keys at once. The server reads both `ApiKey:Current` and `ApiKey:Previous` and accepts a match against either. Rotation then has three steps: write the new key as `Current` and demote the old one to `Previous`; let clients pick up the new value at their own pace; once telemetry shows no traffic still using the old key, remove `Previous`. This window pattern is what API gateways implement as a managed service, and what secret stores expose as secret versioning.

A rotation discipline is what distinguishes a real production deployment from a demo. Without rotation, the same key ends up in shell history, deployment scripts, debug logs, and an old laptop somewhere over a long enough timeline. With rotation, the blast radius of any individual leak is bounded by the rotation period.

21.6 When an API key is enough — and when it is not

The honest summary of the threat model is short. An API key proves that some client holds the shared secret; it says nothing about which client. Every client with the key looks identical to the server, and rotating the key invalidates every client at once unless the server runs the dual-key pattern.

That property is sufficient in three situations. A single trusted client calls the API, deployed alongside the server, both rotated together. A small number of trusted clients call the API and per-caller authorization is not needed — the gate exists to keep anonymous traffic out, not to differentiate callers. A development or internal API exposes data that is not sensitive enough to warrant per-user attribution, and the operational simplicity of one shared credential outweighs the audit cost.

The property is insufficient when per-caller attribution matters (which user did this?), when callers need different permissions (read versus write, scope A versus scope B), when credentials must expire automatically without operator action, or when the API is consumed by multiple independent organizations that must be revoked individually. For those cases the next step up is a JWT bearer token, possibly issued by an OAuth 2.0 authorization server. The transport mechanism is the same — a header on every request — but the credential carries identity and the validating service authorizes per-caller.

The architectural pattern is to start with the simplest gate that matches the threat model, and to escalate when a real requirement appears that the simpler gate cannot meet. An API key is rarely the wrong first step; it is often the wrong forever step.

21.7 Summary

A backend service calling another backend service has no user to authenticate, so it presents a long random shared secret in a request header — an API key — that identifies the calling application. The header conventionally named `X-Api-Key` travels with every request over TLS, and a small ASP.NET Core middleware enforces the check by reading the header, comparing it to a configured expected value, and short-circuiting with 401 (no header) or 403 (wrong header) when the comparison fails. API keys are weaker than JWT bearer tokens because they carry no expiration, no claims, and no built-in rotation — every caller with the key looks identical to the server. Operational discipline closes that gap: generate keys with a cryptographically random source of at least 256 bits, store them in a platform secret store rather than source control, and rotate them on a schedule using a dual-key window so independent clients can cut over without downtime. The mechanism is sufficient for one-trusted-client machine-to-machine traffic and insufficient when per-caller attribution or automatic expiration is required, at which point JWT is the natural next step.

CHAPTER 22

OAuth 2.0 and OpenID Connect

Watch the presentation
Se presentationen på svenska

A web application that needs to call a user's GitHub repositories, read their Google Calendar, or merely offer "Sign in with Microsoft" runs into the same problem: the application must act on behalf of the user without ever holding the user's password. Asking users to type their Google password into a third-party login form is both unsafe (the form could keylog the credential) and impractical (the third party has no good way to handle multi-factor prompts, account recovery, or revocation). OAuth 2.0 and OpenID Connect solve this prob-

lem by having a trusted third party — the authorization server — handle the credential and hand back a scoped token instead. The actor model, the browser-application flow, and the practical ASP.NET Core registration that wires Microsoft Entra ID into a webapp as the identity provider all build on that single shift in trust.

22.1 The delegated-access problem

A traditional login form asks the user to enter a password directly into the application that is checking it. That model works when the application owns the user database and the credential. It breaks the moment the application wants to *use* a service that belongs to a different security domain — a calendar API hosted by Google, an issue tracker hosted by GitHub, a directory hosted by Microsoft Entra ID. The application cannot ask for a Google password and store it; Google would not honor it for API calls anyway, since the API expects its own tokens. Even reusing a password for a single sign-in event leaks the credential to a third party that has no legitimate need to see it.

The same problem appears in a smaller form for plain login. An application that wants to offer "Sign in with Google" without managing passwords itself needs Google to perform the credential check, then somehow tell the application "yes, this browser belongs to anna@example.com." The application never sees the password; it only needs a trustworthy assertion of identity.

OAuth 2.0 is the delegated-authorization framework that lets a user grant a client application access to their resources on a third-party service without sharing the user's password with the client; the client receives an access token from an authorization server and presents it to the resource server.

OpenID Connect (OIDC) is an authentication layer on top of OAuth 2.0; in addition to an access token, the authorization server returns an *ID token* (a JWT) that asserts who the user is, allowing the client to use the third-party service as an identity provider. The two protocols are routinely deployed together, but they answer different questions. OAuth answers "may this client call that API on the user's behalf?" — an authorization question. OIDC answers "who is the user logging in?" — an authentication question. The authentication-versus-authorization split introduced earlier in the Part applies here across organizational boundaries.

22.2 The four actors

Both protocols define the same cast. Naming them precisely makes the message flow easier to follow.

The *resource owner* is the human user whose data or identity is being accessed. In a "Sign in with Google" flow, the resource owner is the person sitting at the browser. In a calendar-integration flow, the resource owner is the person whose calendar events are being read.

The *client* is the application requesting access. The OAuth specification uses "client" for any application that calls the protocol — a server-side webapp, a single-page application, a mobile app, a CLI tool. The client is identified by a `client_id` issued by the authorization server when the application is registered, and (for confidential clients) authenticated by a `client_secret`.

An **authorization server** is the OAuth/OIDC component that authenticates the user, asks for consent, and issues access (and ID) tokens to clients; Microsoft Entra ID, Google, GitHub, and Auth0 are common authorization servers. The authorization server owns the user directory, runs the login

UI, validates passwords or multi-factor prompts, and is the only party in the system that ever sees the credential.

The *resource server* hosts the API the client wants to call. It accepts access tokens issued by the authorization server, validates them, and serves protected data. Microsoft Graph is a resource server; the GitHub REST API is a resource server. For pure sign-in scenarios there is often no separate resource server — the client only needs the ID token to know who the user is.

The four actors run on different machines under different administrative control, which is precisely why a token-based protocol is needed: trust between them must be carried by signed tokens rather than shared databases.

22.3 Grant types at a high level

OAuth defines several *grant types* — message flows the client uses to obtain a token. Each grant type targets a different combination of client capability and trust. Four are commonly encountered.

The *authorization code* grant is the standard flow for interactive user logins. The client redirects the user's browser to the authorization server, the user authenticates there, and the authorization server redirects back to the client with a short-lived authorization code that the client exchanges for tokens. This is the only grant type covered in detail below, because it is the one a webapp uses for "Sign in with Microsoft."

The *client credentials* grant is for machine-to-machine calls where there is no user. A backend service authenticates with its own `client_id` and `client_secret` and receives an access token representing itself. A daily reporting job that queries Microsoft Graph for tenant-wide statistics uses this grant; no user is sitting at a browser.

The *device code* grant handles input-constrained devices — a smart TV, a CLI on a remote server — that cannot host a browser comfortably. The device displays a short code; the user opens a browser on a phone or laptop, enters the code, and authenticates there. The device polls the authorization server until the login completes. The Azure CLI's `az login` uses this grant when run in a remote shell.

The *refresh token* grant is not a stand-alone login flow; it lets a client trade a long-lived refresh token (issued alongside an access token in any of the above flows) for a new access token without re-prompting the user. Access tokens are deliberately short-lived (an hour is typical); refresh tokens carry the durability.

The original specification also defined an *implicit* grant for browser apps and a *resource owner password credentials* grant for legacy migrations. Both are now discouraged — implicit because it returns tokens directly in URL fragments where they leak through browser history, password credentials because it defeats the entire point of delegated access. Current guidance directs browser apps to authorization code with PKCE and treats password credentials as a stop-gap only.

22.4 Authorization code with PKCE

Authorization code with *PKCE* (Proof Key for Code Exchange, pronounced "pixie") is the recommended grant for any client running in a browser or on a user's device — public clients that cannot keep a `client_secret` truly secret. The flow runs in two halves separated by a browser redirect.

In the first half, the client generates a random `code_verifier` (a high-entropy string), hashes it into a `code_challenge`, and redirects the browser to the authorization

server's authorization endpoint with parameters that include the `client_id`, the `redirect_uri` the response should come back to, the requested `scope`, a `state` value, and the `code_challenge`. The authorization server presents its login UI, validates the user's credentials, possibly shows a consent screen listing the requested scopes, and redirects the browser back to the registered `redirect_uri` with a one-time `code` and the original `state`.

In the second half, the client makes a back-channel HTTPS call to the token endpoint, posting the `code`, the `redirect_uri` (for cross-checking), and the original `code_verifier`. The authorization server hashes the verifier, compares it to the stored challenge, and — if they match — issues an access token, optionally a refresh token, and (if OIDC was requested) an ID token. The first half runs through the user's browser and is observable to anyone who can intercept the redirect; the second half runs server-to-server and carries the actual tokens.

PKCE matters because a malicious app on the same device cannot redeem an intercepted authorization code — the attacker would need the original `code_verifier`, which never left the legitimate client. The flow turns the public client into something that can authenticate the second half of the exchange without holding a long-lived secret.

22.4.1 Redirect URIs and the state parameter

The `redirect_uri` is the location the authorization server sends the browser back to. Authorization servers require the URI to be pre-registered for the `client_id` and reject any request that does not match exactly. The check is strict — `https://app.example.com/auth/callback` and `https://app.example.com/auth/callback/` (trailing slash) are different URIs. This pre-registration prevents an attacker from sub-

stituting their own callback to harvest authorization codes intended for a legitimate app.

The `state` parameter is a random value the client generates per login attempt, sends in the outbound redirect, and validates on the inbound callback. If the value does not match what the client stored at the start of the flow, the callback did not originate from a login the client initiated and the request is rejected. The mechanism prevents cross-site request forgery against the callback endpoint: an attacker who tricks the user's browser into hitting the callback with a code chosen by the attacker cannot supply a `state` that matches a value the client is expecting.

22.5 Scopes — what the token may do

A *scope* is a string the client requests, and the authorization server includes in the issued token, that names a category of access. Scopes are the unit of consent: the consent screen the user sees lists scopes ("Read your email," "Edit your calendar"), and the resulting access token carries only those scopes the user agreed to.

For OIDC sign-in, the standard scopes are `openid` (required to trigger OIDC and receive an ID token), `profile` (request name, picture, basic profile claims), and `email` (request the user's email address). For API access, scope strings are defined by the resource server: Microsoft Graph uses scopes like `User.Read` and `Mail.Send`; GitHub uses `repo` and `read:user`. A client that asks for the smallest set of scopes that suffices for its function follows the principle of least privilege; users are also less likely to abandon a consent screen that requests three scopes than one that requests thirty.

22.6 What OIDC adds — the ID token

OAuth 2.0 by itself returns an access token that lets the client call an API. It does not, by design, tell the client *who* logged in. An access token is a permission slip, not an identity assertion; the resource server validates it but the client treats it as opaque.

OpenID Connect adds an ID token alongside the access token. The ID token is a JWT (the same structure introduced in Bearer tokens and JWT) signed by the authorization server. Its payload carries claims that name the user: `sub` (a stable subject identifier within the issuer), `name`, `email`, plus the issuer and audience claims that pin the token to a specific authorization server and a specific client. Because the token is signed, the client can verify the claims without calling the authorization server again — the signature, validated against the authorization server's public key, proves the issuer minted exactly these claims.

The ID token is for the *client* to consume, not for forwarding to a resource server. A common mistake is to send the ID token in `Authorization: Bearer` headers to APIs; the API is expecting an access token with appropriate scopes, and ID tokens have a different audience. Keep the two separate.

22.7 Authentication versus authorization, restated

OAuth and OIDC together cover both halves of the question Part V opened with. OAuth handles authorization across organizational boundaries — "may this client, acting for this user, call that API?" OIDC handles authentication across the same boundary — "who is this user?" The application code that follows looks much like the cookie-authentication code from earlier chapters: the framework hands controllers a

`ClaimsPrincipal` populated from the ID token's claims, and `[Authorize]` attributes work unchanged. The difference is where the identity came from. Instead of a local user database with hashed passwords, the identity came from a remote authorization server that already knows the user.

The same OIDC machinery is used in deployment pipelines for a different purpose: GitHub Actions can present an OIDC token to Azure to obtain temporary deployment credentials, replacing long-lived service-principal secrets. That flavor of OIDC is workload identity federation rather than user authentication and is covered in the DevOps & Delivery part of this book; the protocol is the same, but the resource owner is a workflow rather than a person.

22.8 Worked example — registering OIDC with Microsoft Entra ID

ASP.NET Core ships first-class support for OIDC. The `AddOpenIdConnect` extension method registers the handler, and the framework takes care of the redirects, code exchange, token validation, and `ClaimsPrincipal` construction. The application code that follows is the production-shape registration for signing users in against Microsoft Entra ID.

Figure 1: OIDC registration in `Program.cs`

```
builder.Services
    .AddAuthentication(options =>
    {
        options.DefaultScheme = CookieAuthentica-
        tionDefaults.AuthenticationScheme;
        options.DefaultChallengeScheme = OpenIdConnectDe-
        faults.AuthenticationScheme;
    })
    .AddCookie()
    .AddOpenIdConnect(options =>
    {
        options.Authority = $"https://login.microsofton-
        line.com/{tenantId}/v2.0";
        options.ClientId =
        builder.Configuration["Entra:ClientId"];
        options.ClientSecret =
        builder.Configuration["Entra:ClientSecret"];
        options.ResponseType = "code";
        options.UsePkce = true;
        options.Scope.Add("openid");
        options.Scope.Add("profile");
        options.Scope.Add("email");
        options.SaveTokens = true;
        options.GetClaimsFromUserInfoEndpoint = true;
    });
```

Two schemes are registered together. The cookie scheme is the default — once OIDC has signed the user in, the framework writes a local cookie so subsequent requests do not have to round-trip back to Entra. The OIDC scheme is the *challenge* scheme: when an unauthenticated request hits a controller marked `[Authorize]`, the framework redirects the browser to Entra, and the cookie scheme picks up afterwards. The `Authority` is the Entra tenant's OIDC discovery base URL; the framework fetches `Authority + "/.well-known/openid-configuration"` at startup to discover the authorization, token, and JWKS endpoints, so the code never hard-codes them. `ResponseType = "code"` and `UsePkce = true` select

authorization code with PKCE. The three scopes named line up with the OIDC standard set discussed above. `SaveTokens` keeps the access token in the auth properties so downstream code can call APIs as the signed-in user.

The companion exercise track Authentication and Authorization develops the cookie half of this picture from the bottom up; layering OIDC on top reuses the same `ClaimsPrincipal` model the exercises build, with Entra ID rather than a hardcoded user list as the identity source.

22.9 Summary

OAuth 2.0 lets a client application call an API on behalf of a user without ever seeing the user's password — the user authenticates with an authorization server, which issues a scoped access token the client can present to a resource server. OpenID Connect adds an ID token, a signed JWT that asserts who the user is, turning the same authorization server into an identity provider. Authorization code with PKCE is the recommended grant for browser-based clients; the redirect URI is pre-registered with the authorization server, the `state` parameter prevents callback CSRF, and PKCE binds the back-channel code exchange to the original front-channel request. Scopes name the units of access the user consents to; a client asks for the minimum it needs. ASP.NET Core's `AddOpenIdConnect` registers the handler, and the framework takes the response and produces the same `ClaimsPrincipal` that local cookie authentication would have produced — so authorization rules written against claims do not change when the identity source moves from a local database to Microsoft Entra ID.

PART V · IDENTITY AND SECURITY

CHAPTER 23

Secret Management

Watch the presentation
Se presentationen på svenska

A connection string committed to a public Git repository is the single most common pathway to production breaches. Once a credential enters version-control history, it is effectively permanent — public repositories are scraped within minutes, and even private repositories grant read access to a much wider circle than the production database itself. Application code therefore cannot embed secrets directly, and the configuration files that hold non-sensitive defaults cannot hold sensitive values either. The configuration system that Part III established extends naturally into production: secrets move out of files and environment variables and into a man-

aged secret store accessed through identity-based authentication.

23.1 Why production needs more than environment variables

The configuration and environments chapter introduced the provider chain that ASP.NET Core assembles at startup: `appsettings.json`, environment-specific JSON files, user-secrets in development, environment variables, and command-line arguments. That chain is sufficient for development and for non-sensitive deployment settings, but it falls short for production secrets along three axes.

Plaintext at rest is the first weakness. An environment variable set on a virtual machine or in a container's process environment is readable by anyone with shell access to the host. Platform configuration UIs that hold these values typically display them masked, but the underlying storage is unencrypted, and a compromised host exposes every secret the process can read.

Rotation is the second weakness. A leaked credential must be replaced quickly — within minutes for high-value secrets such as a database connection string or a payment-gateway API key. Environment variables are baked into a deployment manifest; rotating them requires editing the manifest and redeploying the application. The window between detecting a leak and completing the redeploy is the window during which the leaked credential remains valid.

Audit is the third weakness. After an incident, responders need to answer two questions: which secrets were readable from the compromised host, and was any of them actually read? Environment variables provide no audit trail. Reading

them is an ordinary file-system or memory operation that leaves no record.

Production secret stores address all three weaknesses by encrypting secrets at rest, supporting versioned rotation without redeployment, and logging every read.

23.2 Azure Key Vault as the production secret store

Azure Key Vault is a managed service that stores secrets, encryption keys, and certificates with audit logging and access control; applications retrieve secrets at runtime over HTTPS using a token, never embedding secrets in code or configuration files. A vault is a regional resource provisioned inside an Azure subscription and identified by a DNS name such as `cloudsoft-prod.vault.azure.net`. Inside the vault, three object types coexist: *secrets* (arbitrary strings, the focus of this chapter), *keys* (asymmetric and symmetric cryptographic keys whose private material never leaves the vault), and *certificates* (X.509 certificates with their private keys).

Each secret has a name, a current value, and a version history. Setting a new value does not delete the previous one — the vault retains the old version and assigns it a stable identifier, allowing applications to roll back if a rotation breaks something. The current version is the default when an application reads a secret by name, so rotation is transparent to consumers that follow the recommended pattern of always reading the latest version.

Reads happen over HTTPS to the vault's REST endpoint. The caller sends an OAuth-style bearer token in the `Authorization` header (the same `Authorization: Bearer ...` mechanism the bearer tokens and JWT chapter describes for application APIs), and the vault validates that token against

Microsoft Entra ID before returning the secret value. The secret never appears in URL paths, request bodies, or log output, and the TLS channel protects it in transit.

23.2.1 Versioning and rotation

A secret in Key Vault is identified by name and optionally by version. The URL `https://cloudsoft-prod.vault.azure.net/secrets/MongoConnection` returns the current version; appending a version identifier such as `/secrets/MongoConnection/d4e5f6...` returns that specific historical version. Borrowing the term **key rotation** from the API keys chapter, the vault makes rotation a one-line operation: setting a new value publishes a new version while leaving the previous version retrievable. Applications that read the current version pick up the new value the next time they refresh their configuration; applications that pin to a specific version continue using the old value until they are updated.

Rotation policy is therefore a deployment concern, not an application concern. The operational team rotates a database password by writing the new value to Key Vault and updating the database server to accept it; the application restarts (or refreshes its configuration) and begins using the new value without a code change.

23.3 Managed identity removes the bootstrap problem

Reading a secret from Key Vault requires authentication to Entra ID, which raises an obvious question: how does the application authenticate without storing a credential, when the entire point of Key Vault is to avoid storing credentials? The answer is **managed identity** — an Entra ID identity that Azure attaches to a compute resource (VM, App Service, Con-

tainer App, GitHub Actions runner via federation) and rotates automatically; the application authenticates to other Azure services as that identity without storing any credentials.

The mechanism works at the platform level. When an Azure Container App is provisioned with a system-assigned managed identity, the platform creates an Entra ID service principal whose lifecycle matches the container app, generates the underlying credentials, and exposes a local token endpoint inside the container's runtime. Application code calls that endpoint to obtain an access token for any Azure resource, and the platform handles the credential exchange. No secret ever appears in source code, in environment variables, or in deployment manifests.

The `Azure.Identity` library packages this mechanism behind a single class, `DefaultAzureCredential`, that probes a chain of credential sources in order. In production on Azure, the chain finds the managed identity and uses it. On a developer's laptop, the same chain finds the developer's `az login` session and uses that. The application code is identical in both environments — only the surrounding identity context differs.

23.4 RBAC role assignments grant access at the right scope

A managed identity is an authenticated principal but not an authorized one. Reading a secret from Key Vault requires an explicit grant, made through Azure's role-based access control system. An **RBAC role assignment** in Azure pairs a security principal (user, group, service principal, managed identity) with a role definition (built-in or custom) at a specific scope (subscription, resource group, individual

resource); the assignment grants exactly the actions the role allows on resources in that scope.

For Key Vault access from an application, the relevant built-in role is `Key Vault Secrets User`, which grants read access to secret values but not the ability to list, create, modify, or delete secrets. The role is assigned at the smallest scope that satisfies the application's needs — typically the individual vault, occasionally a resource group containing several vaults the application depends on, never the subscription. Granting `Key Vault Secrets User` at subscription scope would let the application read every secret in every vault under that subscription, which violates the principle of least privilege and turns a single compromised application into a subscription-wide breach.

Role assignments are auditable: Azure Activity Log records every assignment creation and deletion, with the principal that made the change, the time, and the affected scope. Combined with Key Vault's own access log (which records every secret read with the calling principal's identity), this produces an end-to-end trail from "who can read what" to "who actually read what, when."

ROLE	PERMISSIONS	WHEN TO ASSIGN
Key Vault Secrets User	Read secret values	Application identities reading secrets at runtime
Key Vault Secrets Officer	Read, create, modify, delete secrets	Operators rotating secrets
Key Vault Administrator	All vault management actions	Vault owners during initial setup
Key Vault Reader	List secrets but not read values	Auditors and inventory tools

23.5 Wiring Key Vault into the configuration chain

The configuration provider chain that Part III described accepts a Key Vault provider as one more link in the chain. Once registered, secrets stored in the vault appear as ordinary `IConfiguration` keys, and consumer code that reads `configuration["MongoDB:ConnectionString"]` does not know — and does not need to know — whether the value came from `appsettings.json`, an environment variable, or a vault.

Figure 1 registers Key Vault as a configuration provider in `Program.cs` using `DefaultAzureCredential`.

```
using Azure.Identity;
using Azure.Extensions.AspNetCore.Configuration.Secrets;

var builder = WebApplication.CreateBuilder(args);

var vaultUri = builder.Configuration["KeyVault:Uri"];
if (!string.IsNullOrEmpty(vaultUri))
{
    builder.Configuration.AddAzureKeyVault(
        new Uri(vaultUri),
        new DefaultAzureCredential());
}

builder.Services.AddControllersWithViews();

var app = builder.Build();
```

Figure 1: Registering Azure Key Vault as a configuration provider

The vault URI is itself a configuration value, supplied through an environment variable in production (`KeyVault__Uri=https://cloudsoft-prod.vault.azure.net/`) and absent in development so the block is skipped. `DefaultAzureCredential` resolves to the container app's managed identity at runtime; on a developer machine it falls back to `az login`. The `AddAzureKeyVault` call then enumerates every secret the identity has permission to read, normalizes the names by replacing `--` with `:` (the vault's character constraint translated back into the configuration key separator), and registers them as configuration keys.

A controller that needs the connection string reads it the same way it reads any other configuration value:

```
public class SubscriberController : Controller
{
    private readonly string _connectionString;

    public SubscriberController(IConfiguration configura-
        tion)
    {
        _connectionString = configuration["Mon-
        goDB:ConnectionString"];
    }
}
```

Figure 2: Reading a Key Vault-sourced secret through `IConfiguration`

The secret named `MongoDB--ConnectionString` in the vault becomes the key `MongoDB:ConnectionString` in `IConfiguration`. The controller is identical to the one shown in the configuration and environments chapter — only the source of the value changed. The companion exercise data layer walks through the connection string pattern; introducing Key Vault on top of that exercise requires adding the vault registration in `Program.cs` and writing the connection string into the vault rather than into an environment variable. The application code does not change.

23.6 Application secrets versus pipeline secrets

Two different categories of secrets exist in a typical deployment, and they belong in different stores. Application secrets are the values the running application needs at request time: database connection strings, API keys for downstream services, encryption keys for data the application owns. These belong in Key Vault, where the application's managed identity reads them through the configuration provider chain.

Pipeline secrets are the values the deployment pipeline needs at build and release time: container registry credentials, deployment-target subscription IDs, signing keys for build artifacts. These belong in the CI/CD platform's own secret store — for GitHub Actions, that means GitHub Actions secrets and, increasingly, OIDC federation rather than long-lived stored credentials.

OIDC federation eliminates the most dangerous pipeline secret entirely. Instead of storing an Azure service principal's client secret as a GitHub Actions secret, the workflow exchanges a short-lived OIDC token (issued by GitHub to the workflow run) for an Azure access token (issued by Entra ID to a federated credential). No long-lived credential ever exists, the token is scoped to the specific workflow run, and a compromised repository cannot leak a credential that was never created. The mechanism is covered in Part VIII (DevOps and Delivery); from the secret-management perspective, the relevant point is that pipeline secrets and application secrets are operationally separate concerns with separate stores.

23.7 Audit logging and incident response

Every read from Key Vault is logged. Each entry records the calling principal (the managed identity or user), the secret name, the timestamp, the source IP address, and whether the request succeeded. Logs flow into Azure Monitor and are retained according to the diagnostic settings configured on the vault.

The audit trail matters most during incident response. After a suspected breach, the first question is which secrets the compromised principal could have read; the second question is which it actually did. The role assignments answer the

first question — they enumerate exactly what the principal had permission to access. The Key Vault access log answers the second question — it shows which permissions were exercised and when. Together they let responders bound the blast radius precisely instead of conservatively assuming everything the principal could access was compromised.

23.8 A checklist for handling secrets

The technical mechanisms above prevent the most common failure modes, but they do not replace operational discipline. Four practices reinforce them:

- Never commit a secret to source control. Pre-commit hooks (`gitleaks`, `truffleHog`) and CI scans catch the obvious cases; a `.gitignore` entry for `appsettings.Production.json` and `secrets.json` catches the easy mistakes.
- Never log a secret. Connection strings and tokens that appear in log output land in centralized log systems where access controls are usually weaker than the secret store's. Structured logging libraries support redaction; use it.
- Never email or chat a secret. Mail and chat history are searchable by anyone with mailbox or workspace access, and forwarded messages multiply the exposure. When a secret must be shared, use a one-time-view tool or the vault's own access controls.
- Rotate on suspicion. If a secret may have leaked — a laptop was lost, an employee left, a log was over-shared — rotate immediately. The cost of an unnecessary rotation is a brief reconfiguration; the cost of skipping a needed rotation is a breach.

23.9 Summary

Secrets in source control are the most common production breach pathway, and neither configuration files nor environment variables provide encryption at rest, rotation, or audit. Azure Key Vault addresses these requirements by storing secrets, keys, and certificates in a managed service that authenticates callers through Entra ID and logs every access. Managed identities let Azure-hosted code authenticate to Key Vault without storing any credential — the platform attaches an identity to the compute resource and rotates the underlying credentials automatically. RBAC role assignments grant the identity the `Key Vault Secrets User` role at the smallest scope that satisfies the application, enforcing least privilege. Once registered as a configuration provider, Key Vault appears in `IConfiguration` exactly like any other source, so application code reads `configuration["MongoDB:ConnectionString"]` without knowing the value came from a vault. Pipeline secrets are a separate concern handled through GitHub Actions secrets and OIDC federation, covered in Part VIII. Audit logs and a short discipline checklist — never commit, never log, never email, rotate on suspicion — close the gap between technical controls and operational reality.

PART VI

Services and APIs

The ACD course's first deep dive into API design. Six chapters cover the conceptual ground (REST, resources, DTOs, status codes), the documentation layer (OpenAPI and Swagger), and the operational concerns that separate a toy API from a production one (pagination, idempotency, rate limiting).

The companion exercise is the REST API and DTOs chapter, which builds the same `CloudCiApi` quotes service through three identity models — anonymous, API key, JWT — while applying every principle covered here.

CHAPTER 24

REST Principles

Watch the presentation
Se presentationen på svenska

An HTTP API is a contract between two programs that have never met. The client was written in a different language, runs on a different machine, and may have shipped before the server it is now talking to existed. For that conversation to work, both sides need a shared vocabulary for what URLs mean, which methods do what, and what a response is allowed to look like. Without a shared style, every API would invent its own conventions and every client would need bespoke code per server. **REST** is the dominant style HTTP APIs follow to avoid that fate, and a small set of design constraints is what makes a service worth calling RESTful.

24.1 What REST is and why it exists

REST (Representational State Transfer) is an architectural style for HTTP APIs in which the server exposes named resources, the client manipulates representations of those resources using the standard HTTP methods, and each request carries everything the server needs to process it. The style was articulated by Roy Fielding in his 2000 dissertation as a description of how the web itself was already working — what made HTTP scale to millions of servers and billions of clients was a particular set of architectural constraints that any networked system could adopt.

REST is not a protocol. There is no `REST/1.0` specification to validate against, no schema language baked into the style, no lint that a service either passes or fails. REST is a set of constraints on top of HTTP, and a service is more or less RESTful depending on how many of those constraints it honours. That looseness is what makes REST useful — teams can adopt the parts that pay off for their use case and skip the parts that do not — but it is also what makes "is this REST?" a fuzzy question in code review.

The constraints exist for one reason: they make HTTP services that scale, evolve, and interoperate. A service that follows them can put a cache in front of it, swap its storage layer behind it, add new endpoints without breaking old clients, and let any HTTP-aware tool (a browser, `curl`, a CDN, a load balancer) participate in the conversation without special knowledge of the service.

24.2 The architectural constraints

Fielding's REST has six constraints. Five of them are load-bearing for an HTTP API; the sixth (code-on-demand) is optional and rarely applied. The five that matter are client-

server separation, statelessness, cacheability, the uniform interface, and a layered system. Each constrains the service in a way that buys back a property the network needs.

24.2.1 Client-server separation

The client and the server are independent programs that communicate only through the API. The client knows nothing about how the server stores data, which database it uses, or how it scales. The server knows nothing about how the client renders the response, which framework it uses, or whether the user is on a phone or a terminal. The only contract between them is the set of resources, methods, and representations the API exposes.

This separation is what lets the two halves evolve independently. A team can rewrite the server in a different language, swap the database, or move from one cloud region to another, and as long as the API contract holds, every client keeps working. The same API can serve a web frontend, a mobile app, a CLI, and a batch job — each is a different client of the same server, and none of them knows or cares about the others.

24.2.2 Statelessness

Statelessness is the REST constraint that each request from a client must contain all the information the server needs to process it; the server keeps no client session between requests, which makes the API trivially horizontally scalable but pushes session state into the client or into a separate token. The server is allowed to keep persistent state — quotes in a database, blobs in storage — but it must not keep *per-conversation* state in memory between calls.

The payoff is operational. Any replica of the service can serve any request, because no replica has special knowledge

about a particular client. A load balancer can round-robin requests across ten instances; if one instance falls over, the next request lands on another and works. Nothing has to migrate. Nothing has to be sticky.

The cost is that authentication context, cursor positions, and any other "where was this client" state has to ride along on every request. In practice that means an `Authorization` header on every call, identifiers in every URL, and pagination cursors echoed back to the server each time. The client gets bigger; the server gets simpler; the system scales.

24.2.3 Cacheability

Responses must declare whether they can be cached, and for how long. The server signals cacheability with HTTP headers — `Cache-Control`, `ETag`, `Last-Modified` — and any cache that sits between the client and the server (the browser, a CDN, a reverse proxy) is free to honour those signals without involving the application.

A cacheable `GET /quotes/42` response can be served from a CDN edge node in a few milliseconds without the request ever reaching the application, which is how a small backend can serve millions of clients. The constraint is that the application has to be honest about what is cacheable: a response that varies per user must say so (`Cache-Control: private` or `Vary: Authorization`), or every user will end up seeing the first user's data.

24.2.4 Uniform interface

The uniform interface is the constraint that makes REST recognizable. It has four sub-rules: resources are identified by URIs, those resources are manipulated through representations, messages are self-describing, and the application's state is driven by hypermedia (the much-debated HATEOAS

rule). In practice, this means an API exposes a set of nouns at predictable URLs, and clients act on those nouns using the standard HTTP methods.

A **resource** is anything an API addresses with a URI — a customer record, a list of orders, a single image — and that the API can return a representation of, accept updates to, or delete. The resource is the *concept*; the representation is the *bytes on the wire* — usually JSON, sometimes XML, sometimes a binary format. The same resource can have multiple representations: `/quotes/42` could return JSON to one client and XML to another based on the `Accept` header.

The standard HTTP methods carry conventional meaning across every REST API:

- `GET` retrieves a representation of a resource without changing it
- `POST` creates a new resource (typically against a collection URL)
- `PUT` replaces a resource with the supplied representation
- `PATCH` updates part of a resource
- `DELETE` removes a resource

The conventions matter because they let intermediaries reason about traffic without understanding the application. A cache knows `GET` is safe to store. A retry library knows `PUT` is idempotent and safe to retry, while `POST` is not. A monitoring tool can count `5xx` responses without knowing what the service does.

The fourth sub-rule, hypermedia as the engine of application state (HATEOAS), is the idea that a response should embed the URLs the client can follow next, so the client navigates the API the way a browser navigates the web — clicking links rather than knowing URL patterns in advance. Few production APIs implement HATEOAS strictly, because clients

almost always know the URL patterns from documentation rather than discovering them at runtime. The trade-off is real: HATEOAS makes URL refactoring safer at the cost of every response carrying link metadata the client may not use.

24.2.5 Layered system

A REST client cannot tell whether it is talking to the application server directly or to a reverse proxy, a load balancer, or a CDN that forwards to the application server. Each layer can add its own behaviour — caching, authentication, rate limiting, TLS termination — without the client knowing. The layered constraint is what lets a service insert Azure Front Door, an API Management gateway, or a Container Apps ingress in front of an application, and the application's clients keep working unchanged.

24.3 A worked example: GET /quotes/42

The companion exercise builds a tiny ASP.NET Core API called `CloudCiApi` whose `Quotes` resource exhibits every constraint above. A `GET` request for a single quote looks like this on the wire:

```
GET /api/quotes/42 HTTP/1.1
Host: cloudci-api.example.com
Accept: application/json
```

And the response:

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: public, max-age=60

{
  "id": 42,
  "author": "Edsger W. Dijkstra",
  "text": "Simplicity is prerequisite for reliability.",
  "createdAt": "2025-09-14T10:23:11Z"
}
```

Every constraint is visible in those few lines. The URL `/api/quotes/42` names a resource — quote forty-two — and the JSON body is one representation of it. The method `GET` is the standard verb for retrieval, so any cache or proxy in the path knows this response can be stored. The `Cache-Control` header makes that explicit: any layer is welcome to keep this response for sixty seconds. The request carries no session cookie and the server kept no record of the client between calls — this same request would work identically against any replica of the service, which is what statelessness buys. And the client did not need to know whether the request hit the application directly or went through Front Door, the Container Apps ingress, and a Kestrel listener — the layered system is invisible to it.

The companion exercise REST API and DTOs builds the controller that produces this response, alongside the entity-versus-DTO split that keeps the wire shape independent of the in-memory class.

24.4 The Richardson Maturity Model

The Richardson Maturity Model is a four-level scale, due to Leonard Richardson, that grades how thoroughly an API embraces the REST constraints. It is not a certification — it is

a way to talk about which constraints a given service honours and which it skips.

LEVEL	WHAT THE API DOES	TYPICAL EXAMPLE
0	One URL, one method (`POST /api`), action encoded in the body	XML-RPC, SOAP-over-HTTP
1	Multiple URLs (resources), but still typically `POST` for everything	Many early "REST" APIs
2	Resources plus the standard HTTP methods and status codes	Most production APIs called "REST" today
3	Level 2 plus HATEOAS — responses embed links to next actions	GitHub API, PayPal API

Most APIs that the industry calls "REST" sit firmly at Level 2: they expose resources, use `GET` / `POST` / `PUT` / `DELETE` for the obvious operations, and return meaningful HTTP status codes like `200`, `201`, `404`, and `409`. Level 3 is the strict-REST aspiration; it is rare in the wild because the cost of producing and consuming hypermedia rarely beats the cost of documenting URL patterns.

For a working API, Level 2 is usually the sweet spot. Resources, standard methods, and meaningful status codes give clients enough predictability that any HTTP-aware tool participates correctly, without forcing every team to adopt link-driven navigation.

24.5 When REST is not the right choice

REST is the default style for an HTTP API because it composes well with everything else on the web — caches, proxies, load balancers, browser developer tools, OpenAPI tooling. It is not the only choice, and for some workloads it is genuinely the wrong one.

gRPC uses HTTP/2 and Protocol Buffers rather than JSON over HTTP/1.1. It produces smaller payloads and supports streaming in both directions, which makes it a better fit for service-to-service traffic inside a backend where latency and throughput matter and the clients are programs, not browsers. The trade-off is that gRPC traffic is opaque to the HTTP toolchain — a cache cannot inspect it, a browser cannot make it directly without a translation layer, and `curl` cannot be used to debug it.

GraphQL lets the client specify exactly which fields it wants from a single endpoint, which is a better fit when one screen needs data from many resources and the client wants to avoid a fan-out of REST calls. The trade-off is that caching becomes much harder (every query is potentially unique), the server has to defend against expensive queries, and the simplicity of `GET /quotes/42` is gone.

For an HTTP API that public clients consume, that has predictable resources, and that benefits from CDN caching, REST is almost always the right starting point. For high-throughput service-to-service traffic or for client-driven query shapes, the alternatives are worth considering, alongside the REST API that fronts the system from the outside.

24.6 Summary

REST is an architectural style for HTTP APIs that constrains how clients and servers communicate, with the goal of produ-

cing services that scale, cache well, and evolve without breaking clients. The five load-bearing constraints — client-server separation, statelessness, cacheability, uniform interface, and a layered system — each trade a degree of freedom for an operational property that the network needs. Most production APIs implement REST loosely, sitting at Richardson Level 2 with resources, standard HTTP methods, and meaningful status codes, but skipping the strict HATEOAS requirement of Level 3. REST is not the only style — gRPC and GraphQL solve problems REST does not — but for an HTTP API consumed by external clients, REST remains the default, and understanding its constraints is what lets a service team know which conventions to honour, which to bend, and what each compromise costs.

CHAPTER 25

Resource Modeling and URIs

Watch the presentation
Se presentationen på svenska

The hardest part of designing an HTTP API is not writing the controller code; it is deciding which things the API exposes and what they are called. Once a URI is published, clients depend on it, so any rename or restructure breaks them. Resource modeling is the practice of choosing those names deliberately — picking the nouns the API addresses, deciding which of them are lists and which are single items, and laying them out under URIs that other developers can guess without reading the docs.

25.1 The resource as the unit of API design

A REST API does not expose functions; it exposes resources. The first design decision for any new endpoint is therefore not "what verb describes this operation?" but "what noun does this operation act on?" If the answer is a noun the API already addresses, the new operation is another HTTP method on the existing URI. If the answer is a noun the API has not modelled yet, a new URI is needed. This shift from verb-thinking to noun-thinking is what gives REST APIs their shape.

The two URI shapes that carry the bulk of an API are the collection and the item. A **resource collection** is a URI that addresses a list of resources of the same type (`/quotes`, `/orders`); it is conventionally manipulated with `GET` to list, `POST` to create, and `DELETE` only when removing the entire collection makes sense. A **resource item** is a URI that addresses one specific resource by its identifier (`/quotes/42`, `/orders/abc-123`); manipulated with `GET` to retrieve, `PUT` or `PATCH` to modify, and `DELETE` to remove. Almost every endpoint in a typical API falls into one of these two shapes.

The discipline is what makes the API predictable. A developer who has called `GET /api/customers` and `GET /api/customers/{id}` can guess that creation is `POST /api/customers` and deletion is `DELETE /api/customers/{id}` without consulting the documentation. That guessability is not an accident — it is the payoff of modeling the API as resources rather than as a list of named operations.

25.2 Naming conventions

Resource URIs use **plural nouns** for collections. `/quotes`, `/orders`, `/customers` — never `/quote`, `/order`, or `/cus-`

`tomer`. The plural reads correctly in both directions: `GET /quotes` returns the list, and `GET /quotes/42` reads as "quote forty-two from the quotes collection." Mixing singular and plural across an API forces clients to memorise which is which, and the inconsistency surfaces immediately the first time someone autocompletes a URL.

Verbs do not appear in URIs. `GET /api/quotes` lists quotes; there is no `/api/getQuotes`, `/api/listQuotes`, or `/api/quotes/list`. The HTTP method is the verb. A URI that contains a verb either duplicates the method (`/getQuotes` plus `GET`) or contradicts it (`POST /api/deleteQuote/42`), and neither helps clients. The only legitimate exception is operations that genuinely do not fit a CRUD shape — `POST /api/quotes/42/publish` for a state transition that has no obvious resource of its own — and even there the noun-first preference (`POST /api/quotes/42/publications`) usually reads better.

URIs are case-sensitive in the spec but conventionally lowercase. Words are joined with hyphens, never underscores or camelCase: `/customer-orders`, not `/customerOrders` or `/customer_orders`. The hyphen choice matches how URLs render in browser address bars and search engines and avoids the shift-key cost of underscores. The path segments stay short — `/quotes` beats `/quote-records`, and `/customers/{id}/orders` beats `/customer-management/customers/{customer-id}/customer-orders`.

25.2.1 Identifiers in item URIs

The identifier in a resource-item URI is whatever the server uses internally to address that resource. Integer ids (`/quotes/42`) are common when the database is the source of truth and the id is auto-generated. GUIDs (`/orders/3f2a8c91-...`) are used when ids are generated client-side

or when sequential ids leak business information (an order id of `1024` tells a competitor roughly how many orders the business has processed). Slugs (`/articles/getting-started-with-rest`) are used when the URI doubles as something humans read or share. The choice trades off readability, leak resistance, and uniqueness guarantees; what matters for resource modeling is that whatever identifier is chosen becomes part of the URI contract and cannot change.

25.3 Collection and item URIs

Each of the two URI shapes has its own conventional set of HTTP methods. The combination of shape and method is what gives a REST API its grid of operations.

URI SHAPE	METHOD	CONVENTIONAL MEANING
`/quotes`	`GET`	List the collection
`/quotes`	`POST`	Create a new item; `Location` header points at the new item
`/quotes`	`DELETE`	Remove the entire collection (rare)
`/quotes/{id}`	`GET`	Retrieve one item
`/quotes/{id}`	`PUT`	Replace the item
`/quotes/{id}`	`PATCH`	Modify part of the item
`/quotes/{id}`	`DELETE`	Remove the item

`PUT` on an item URI replaces the resource — the request body must contain every field the resource has. `PATCH` modifies part of it — the request body contains only the fields that change. `PUT` is the simpler default; `PATCH` matters once a

resource has enough fields that always sending the whole thing is wasteful or risks overwriting concurrent changes.

`POST` on a collection URI is the conventional create operation. The 201 response carries a `Location` header pointing at the newly-created item URI, so the client receives the canonical address of the resource it just created. ASP.NET Core's `CreatedAtAction` helper builds this header from a named action; the framework computes the URL, so a route refactor does not break clients (the REST controllers exercise demonstrates the pattern).

What does *not* go on a collection URI is per-item operations. `DELETE /quotes` does not delete one quote — it deletes the whole list. `PUT /quotes` does not update one quote — it replaces the entire collection. Both are valid HTTP, but they almost always indicate a design mistake; the intended operation belongs on the item URI instead.

25.4 Hierarchical relationships

When one resource belongs to another, the URI hierarchy can express the relationship: `/customers/{id}/orders` addresses the orders that belong to a specific customer. The customer is the parent; the orders are sub-resources scoped under that customer. The hierarchy reads as a path: "customer 42's orders," "customer 42's order number 7." A client listing `GET /customers/42/orders` cannot accidentally see customer 99's orders, because the URI itself excludes them.

Hierarchy is a design choice, not a database mirror. A relational database has a `customer_id` foreign key on the `orders` table; that does not mean the API has to expose the

relationship as a sub-resource. Two URI shapes are available for the same data:

- **Sub-resource:** `GET /customers/42/orders` — list orders belonging to customer 42.
- **Flat with filter:** `GET /orders?customerId=42` — list all orders, filtered by customer.

Both work; both can return identical JSON. The difference is what the URI emphasises. The sub-resource shape says "orders are something a customer has." The flat shape says "orders are an independent thing that happen to reference a customer." The right choice depends on whether the parent owns the child conceptually and on how clients want to reach the data.

25.4.1 When to use which

DECISION FACTOR	SUB-RESOURCE (`/CUSTOMERS/{ID}/ORDERS`)	FLAT WITH FILTER (`?CUSTOMERID=42`)
Child cannot exist without parent	Yes — orders only make sense per customer	Less natural
Most queries are scoped to one parent	Yes — the URI carries the scope	Adequate, but the filter is mandatory
Cross-parent queries are common	Awkward — needs a separate flat route	Yes — `?status=pending` across all customers
Authorization scope matches parent	Yes — easy to enforce "this caller can only see their own customer's orders"	Possible, but the check is per query
The child has a global identity	Less natural — `/customers/42/orders/7` plus `/orders/7` is duplication	Yes — `/orders/7` is canonical

Some APIs expose both — `/customers/42/orders` for scoped listing and creation, `/orders/{id}` for direct access by global id. That is fine when a clear convention separates

the two: the sub-resource is the *list* under a parent, and the flat item URI is the *canonical* address of one resource. Mixing them inconsistently — sometimes scoped, sometimes flat, with no rule — is what creates confusion.

25.5 Filtering, pagination, and sub-resources at a glance

Query parameters carry options that modify how a collection is read but do not change which resource it addresses. Filtering is the most common use: `GET /api/quotes?status=published&author=knuth` returns the subset of the collection that matches. The collection URI is still `/api/quotes` — the filter narrows the response, it does not address a different resource. This separation matters because `/quotes?status=published` and `/quotes?status=draft` are two views of one resource, not two distinct resources.

Sorting (`?sort=createdAt`), field selection (`?fields=id,author`), and search (`?q=optimization`) follow the same rule. They are query parameters, not path segments, because they describe how to project the collection, not which collection to address.

Pagination is the same idea applied to size: a request like `GET /api/quotes?page=3&size=20` returns one slice of a long list. The strategies (offset, page, cursor) and the response-envelope shape are detailed in pagination, idempotency, and rate limiting. What matters for resource modeling is that pagination is a query-parameter concern on the collection URI, not a separate resource — `/quotes/page/3` is a URI mistake.

Sub-resources extend the path. The conventional shape `/quotes/{id}/comments` reads as "the comments belonging to quote 42." `GET` lists them, `POST` creates one, and the result-

ing comment is addressable at `/quotes/{id}/comments/{commentId}` or, if comments have global ids, at `/comments/{id}`. The hierarchy depth tracks the conceptual ownership; URIs deeper than three levels (`/customers/42/orders/7/items/3/discounts`) become unwieldy and usually indicate that the deeper resources deserve their own top-level URI.

25.6 URI stability — the contract

A published URI is a contract. Once a client has called `GET /api/quotes/42`, that URI is part of the system the client depends on. Renaming it to `/api/cool-quotes/42` does not "improve" the API — it breaks every client that wired the old URI into its code, configuration, bookmarks, or external integrations. Resource modeling is therefore not a thing that happens once and then evolves freely; the model the API ships is the model it lives with for as long as the URI exists.

The remedy when a model genuinely needs to change is API versioning — the `/v1/quotes` path stays exactly as it was, while a parallel `/v2/quotes` introduces the new shape. Old clients keep working against `v1`; new clients adopt `v2`. The version number is the explicit contract boundary that says "everything under here will not be silently restructured." Versioning has its own trade-offs and is detailed in status codes, versioning, and error responses; what matters for resource modeling is that versioning is the alternative to changing a URI in place, and changing a URI in place is what the contract forbids.

This is why the early naming decisions matter so much. A plural-vs-singular slip-up, a verb left in a URI, or a relationship modeled as a sub-resource that should have been flat (or vice versa) — each of these is cheap to fix on day one and expensive to fix on day three hundred. The ten minutes spent

at the whiteboard deciding `/quotes` vs `/quote` saves the breaking change six months later.

25.7 Worked example — the `CloudCiApi` quote endpoints

The companion exercise scaffolds an ASP.NET Core Web API called `CloudCiApi` whose only resource is the `Quote`. The three endpoints it exposes are exactly the conventional shapes for a collection plus an item:

```
GET    /api/quotes          → 200, JSON array of QuoteDto
GET    /api/quotes/{id}     → 200 with the QuoteDto, or 404
if no such id
POST   /api/quotes          → 201 Created, Location: /api/
Quotes/5, body = the new QuoteDto
```

The collection URI is `/api/quotes` — plural, lowercase, no verb, no version segment. `GET` lists; `POST` creates. The item URI is `/api/quotes/{id}` — same plural noun followed by the integer identifier the in-memory store assigns. `GET` retrieves; the controller returns `404` when the id does not match a stored quote. The `[HttpGet("{id:int}")]` route constraint pins the segment to integers so `/api/quotes/abc` does not even reach the action — the framework returns `404` from routing.

The `POST` response is the place where resource modeling and HTTP semantics meet. The status is `201 Created`; the `Location` header points at the new item URI; the body carries the freshly-created `QuoteDto`. The controller never hard-codes the URL — `CreatedAtAction(nameof(GetById), new { id = quote.Id }, dto)` asks the framework to build it from the named action. If the route template ever changed, the

framework would still produce the correct `Location` value, and clients reading the header would still find the resource at the right address. The naming convention (one plural noun, one HTTP method per shape) and the framework feature (route generation by action name) reinforce each other — both are aimed at keeping the URI stable as the implementation evolves.

What the exercise does *not* yet have is a `PUT`, `PATCH`, or `DELETE`. Those would each go on `/api/quotes/{id}` if added, and each would slot into the grid above without changing any of the three URIs already published. The next exercise in the chapter adds an API-key gate; it does not change the resource model — only the conditions under which clients are allowed to hit it. That is the resource model paying off: a security change is a middleware change, not a URL change.

25.8 Summary

A REST API exposes resources, not functions, and the URIs that name those resources are the API's primary contract. Collections are addressed by plural nouns (`/quotes`); items are addressed by the same plural noun plus an identifier (`/quotes/42`). Each shape has a conventional set of HTTP methods — `GET`/`POST` on collections, `GET`/`PUT`/`PATCH`/`DELETE` on items — and matching that grid is what makes an API guessable. Hierarchical relationships can be expressed either as sub-resources (`/customers/{id}/orders`) or as flat resources with a foreign-key filter (`/orders?customerId=42`); the choice depends on ownership semantics and query patterns, not on the database schema. Filtering and pagination ride on the collection URI as query parameters, never as new path segments. URIs are case-sensitive, lowercase, hyphenated, verb-free, and — once published — stable.

When the model genuinely needs to change, the right tool is a new API version, not a renamed URI. The `CloudCiApi` quote endpoints in the companion exercise are the smallest concrete instance of these rules: one collection URI, one item URI, three methods, no verbs, no surprises.

PART VI · SERVICES AND APIS

CHAPTER 26

DTOs vs Entities

Watch the presentation
Se presentationen på svenska

An HTTP API exposes a contract that clients depend on, and a database that internal code depends on. The temptation is to use one class for both — a single `Quote` type that represents a row in storage and the JSON that flows over the wire. That choice feels economical for a week and expensive for a year. The shape of a stored row and the shape of a wire payload answer different questions, evolve at different rates, and break for different reasons. Keeping the two shapes separate from day one — with explicit types for each role and a small mapping function in between — is what lets each layer evolve on its own clock.

26.1 Why entities make poor wire shapes

An **entity** is the in-memory class representing a row in the database (or a document in a document store); its shape is driven by persistence concerns — primary keys, navigation properties, audit columns — that the API client neither needs nor should see. Returning that entity directly from a controller action turns it into the public contract. Three forces pull against that arrangement.

26.1.1 Over-exposure of internal fields

Entities accumulate fields the API has no business publishing. A `Quote` row in a real schema gains a `CreatedBy` user id, a `ModifiedAt` timestamp, a `RowVersion` concurrency token, perhaps a `IsDeleted` soft-delete flag. None of those are part of the contract a public client cares about, but a default JSON serializer prints every public property it sees. Adding the column ships the column. The first sign of trouble is usually a security review that finds an internal user id in a response body, weeks after the column was added for an unrelated migration.

The defensive workaround — sprinkling `[JsonIgnore]` attributes on the entity — works, but it inverts the safe default. A new property is exposed unless someone remembers to hide it. A separate response type inverts that default back: a new property is hidden unless someone explicitly adds it.

26.1.2 Leaked persistence concerns

Persistence frameworks decorate entities with concerns that make sense to the database and noise to the client. Entity Framework navigation properties like `public Author Author { get; set; }` resolve through a relationship; serialised naively, they either trigger a lazy-load query mid-response or produce a deeply nested JSON tree

the client never asked for. `Id` columns carry the database's choice of integer, GUID, or composite key — a wire shape might prefer a string, a slug, or no id at all on creation. A `RowVersion` byte array exists to support optimistic concurrency on writes, not to be base64-encoded into every read.

Coupling the wire format to these decisions means changing the database hurts clients. Switching from integer keys to GUIDs becomes a breaking API change. Adding an EF relationship adds an unbidden nested object. The wire shape should describe what the API exposes, not how the storage layer happens to keep it.

26.1.3 Tight coupling between layers

When the entity is the wire shape, the API contract and the database schema become the same artifact. Schema migrations turn into client-visible changes. Renaming a column in storage requires a coordinated client release. Adding a column ships the column to every client whether they asked for it or not. Over time the team stops doing harmless internal refactors because every refactor is a public contract change.

The fix is to decouple the two layers explicitly with a type boundary. The entity evolves with persistence; the wire shape evolves with the contract; a small mapping function in between absorbs the difference.

26.2 What a DTO is and how to design it

A **DTO** (Data Transfer Object) is a class shaped specifically for travelling over the wire between client and server; it carries only the fields the API contract exposes, and is decoupled from the persistence model so that database changes do not break clients and API changes do not require database migrations. The defining property of a DTO is that it has no beha-

viour and no persistence attributes — it is a flat record of fields, their types, and their validation rules.

Designing a DTO starts from the question "what does the client need to do its job?" rather than "what does the database happen to store?" That reframe leads to two practical decisions. First, the DTO's properties are a deliberate subset, not a copy. Second, the DTO is split by direction: one type for what the server accepts, another for what the server emits.

26.2.1 Request DTOs for input validation

A *request DTO* describes what a client is allowed to send. The defining trait is that it omits every field the server controls. A client creating a quote does not choose its `Id` — the server assigns that on insert. The client does not choose `CreatedAt` either; the server stamps it when the row is written. Putting those fields in the request type would let a malicious or buggy client supply them, and the server would have to remember to throw the supplied values away.

Validation attributes attach naturally to the request DTO. Annotations like `[Required]`, `[StringLength]`, `[Range]`, and `[EmailAddress]` describe what a valid request looks like at the boundary. The `[ApiController]` attribute makes ASP.NET Core return a `400 Bad Request` automatically when a request fails validation, before any controller code runs. Keeping these annotations on the request DTO rather than the entity prevents two unrelated rules — request validation and domain invariants — from sharing the same attributes and changing for the same reasons. A request might cap an author's name at 100 characters because that is what the form allows; the entity might enforce a 200-character ceiling because that is what the column allows. Those are different rules even when the numbers happen to match.

26.2.2 Response DTOs for selective exposure

A *response DTO* describes what the server returns. Its job is to expose exactly what the client needs and nothing else. Internal columns stay out by construction — there is no property to fill, so there is nothing to leak. A response DTO can also reshape the data: flatten a nested entity, rename a confusing column, format a `DateTimeOffset` as an ISO-8601 string with a known time zone, or compose multiple entities into a single resource view.

Splitting the response DTO from the request DTO has a second benefit: idempotent fields surface automatically. The response naturally carries server-assigned values like `Id` and `CreatedAt` because the server has them by the time it returns. The request naturally lacks them because the client cannot supply them. The compiler enforces the split.

26.3 Mapping between entities and DTOs

A **mapper** is the code that converts between an entity and a DTO; it can be hand-written extension methods, an automated library such as AutoMapper, or constructor-style records that take an entity and emit a DTO. The mapper is small and almost always uninteresting, but choosing where it lives and how it is implemented shapes the experience of working in the codebase.

26.3.1 Where mapping happens

Three locations are common, each with a different cost.

LOCATION	STRENGTH	TRADE-OFF
Inline in the controller action	Visible at the call site; trivial for one or two endpoints	Duplication grows with the resource count; controllers fill with shape-shuffling code
Dedicated mapper class or extension methods	One place to find the conversion; testable in isolation	Requires a small extra file; one more place to look
Library (AutoMapper, Mapster)	Zero hand-written code per type; convention-driven	Hidden behaviour; debugging a missing field means reading library config rather than code

The companion exercise places the mapping in a private static method on the controller — `private static QuoteDto ToDto(Quote q)`. That is the lightest viable form for a single resource. A real codebase with five or ten resources usually graduates to one of the other two: a mapper class per aggregate, or extension methods grouped by entity. The library option is an optimisation that pays off only when the mapping count is high enough that hand-written code becomes a maintenance tax.

26.3.2 Worked example

The exercise REST Controllers, DTOs, and Swagger defines the three types side by side. The entity captures the domain shape, including audit columns that may be added later. The two DTOs cover the two directions of traffic.

```
// Entity — domain shape, may grow internal columns over
        time.
public class Quote
{
    public int Id { get; set; }
    public string Author { get; set; } = string.Empty;
    public string Text { get; set; } = string.Empty;
    public DateTimeOffset CreatedAt { get; set; }
    // Future internal columns: ModifiedBy, RowVersion,
        IsDeleted
}

// Request DTO — what the client is allowed to send.
public class CreateQuoteRequest
{
    [Required]
    [StringLength(100, MinimumLength = 1)]
    public string Author { get; set; } = string.Empty;

    [Required]
    [StringLength(500, MinimumLength = 1)]
    public string Text { get; set; } = string.Empty;
}

// Response DTO — what the server returns.
public class QuoteResponse
{
    public int Id { get; set; }
    public string Author { get; set; } = string.Empty;
    public string Text { get; set; } = string.Empty;
    public DateTimeOffset CreatedAt { get; set; }
}
```

Three observations follow from the shapes. The request omits `Id` and `CreatedAt` because the server owns both — the controller calls `_store.Add(request.Author, request.Text)` and the store stamps the timestamp. The response carries every public field a client legitimately needs, and only those fields. The validation annotations live on the request type, so they describe the contract a client must sat-

isfy without leaking onto the in-memory entity that downstream code uses after validation has already passed.

The mapper in the controller is one method:

```
private static QuoteResponse ToDto(Quote q) => new()
{
    Id = q.Id,
    Author = q.Author,
    Text = q.Text,
    CreatedAt = q.CreatedAt,
};
```

When a future migration adds `ModifiedBy` to the entity, this method does not change and the response shape does not change. When the response gains a new computed field — say a `Slug` derived from the author and text — the change is local to the mapper and the DTO. Each layer evolves on its own clock.

26.4 The cost of the split

The trade-off is real: more types means more typing. A small API with three resources gains three entity types, three or more request DTO types, three response DTO types, and a mapper per resource. A code generator or a library lowers the per-line cost, but the conceptual cost — two parallel hierarchies that have to stay roughly aligned — does not go away.

The cost is worth paying when any of the following holds. The API is public-facing or has clients outside the team's release control, so a change to the wire format requires a coordinated rollout. The persistence layer is non-trivial and likely to acquire internal columns. The validation rules differ from the domain invariants. The response shape differs from

the storage shape — even slightly, like flattening one nested entity.

The cost is harder to justify when the API is a thin private endpoint inside a single deployable unit, the database schema is essentially the public contract, and the wire shape is provably identical to the storage shape forever. That description fits some internal admin tools and very little else. For most APIs that survive past their first release, the split pays for itself the first time the entity grows a column the wire should not see.

26.5 Summary

Entities and DTOs answer different questions and should be kept as different types. The entity captures the domain shape that internal code reasons about, including persistence concerns like primary keys, navigation properties, and audit columns. The DTO captures the wire contract — exactly what the API exposes, with validation attributes describing what valid inputs look like. Splitting the DTO further into request and response types makes the asymmetry between client-supplied and server-controlled fields explicit at the type level, so the compiler enforces what discipline alone would not. The mapper between them is small, frequently boring, and the price of admission for letting the persistence layer and the API contract evolve independently. The cost — more types, more boilerplate — is the cost of a stable contract; it is paid back the first time storage changes without breaking clients, or the API contract changes without forcing a database migration.

CHAPTER 27

Status Codes, Versioning, and Error Responses

Watch the presentation
Se presentationen på svenska

A REST API speaks two languages at once. The body carries the data the client asked for or sent in, but the *outcome* of the call is communicated by the response code, the headers, and — when something goes wrong — a structured error body. Clients written by different teams, in different languages, on different release schedules need a way to know whether to retry, give up, prompt the user, or fall over to a different code path;

reading prose from a freeform `message` field is not it. Three things an API author has to get right let every client answer "what just happened?" mechanically: the status code returned on each path, the standard shape of an error body, and the discipline that lets the contract evolve without breaking the clients that depend on it.

27.1 Status codes as the primary signal

The general meaning of HTTP status codes — the 1xx through 5xx ranges, the canonical names, the cacheability rules — is developed in HTTP fundamentals. What matters here is the narrower set an API author actually reaches for and the conventions for choosing among them. A REST API is a finite vocabulary of resource manipulations, and a small, well-disciplined subset of status codes is enough to describe every outcome a client needs to reason about.

The principle is that the status code is the *primary* signal. The body adds detail, but the client should be able to branch on the code alone. A `200` followed by an error message in the body, or a `500` whose body says "everything is fine," forces every client to read prose to find the truth. An API that always tells the truth in the status line is one that any HTTP library, monitoring agent, or load balancer can interpret without a custom parser.

27.1.1 The success codes

Three codes cover almost every successful response.

`200 OK` is the unmarked default. The request succeeded; the body holds whatever the client asked for. `GET /api/quotes` returns `200` with a JSON array; `GET /api/quotes/42` returns `200` with one object; a successful `PUT` that over-

wrote an existing resource returns `200` with the updated representation.

`201 Created` is the response to a `POST` that created a new resource. Two things are mandatory: the body should be the new resource (so the client does not have to make a follow-up `GET`), and the response must include a `Location` header pointing at the canonical URI of the new resource. The `Location` header is the wire-level answer to "where did it go?" — clients that need to bookmark, share, or later fetch the resource read it from there rather than constructing the URL themselves.

`204 No Content` signals success with no body. The typical caller is a `DELETE` that succeeded — there is nothing meaningful to return — or a `PUT` that the client does not need a response payload for. Returning `204` rather than `200` with an empty body is more than cosmetic: it tells the client there is nothing to parse, and intermediaries (caches, proxies) treat it accordingly.

27.1.2 The client-error codes

The 4xx range tells the client "you got something wrong; do not retry without changing the request." Five codes carry almost all the load.

`400 Bad Request` is the catch-all for malformed input. The JSON did not parse, a required field was missing, a value was out of range. ASP.NET Core's `[ApiController]` attribute returns `400` automatically when model validation fails — the controller action never runs.

`401 Unauthorized` means the request lacked a valid credential. The name is historical; "unauthenticated" would be more accurate. The convention is that `401` invites the client to retry with credentials, and the response should include a

`WWW-Authenticate` header naming the scheme expected (`Bearer`, `Basic`, an API key header).

`403 Forbidden` means the credential was valid but the caller is not allowed to perform this action. The user is signed in but lacks the role; the API key is recognised but not entitled to the endpoint. `403` is terminal in a way that `401` is not — retrying with the same credential will give the same `403`.

`404 Not Found` means the resource at this URI does not exist. The route matched, but the entity is not there — `GET /api/quotes/9999` when there is no quote 9999. `404` is also the conventional response when the caller is not entitled to *know* whether the resource exists; revealing the difference between "not found" and "forbidden" can leak information.

`409 Conflict` means the request collided with the current state of the resource. Two clients edited the same record and the second `PUT` arrived with a stale version; a `POST` tried to create a resource with a unique key that another resource already holds. `409` invites the client to re-fetch, reconcile, and try again.

27.1.3 Validation: 400 versus 422

The 4xx codes draw a line between *malformed* and *semantically invalid*. **`422 Unprocessable Entity`** was introduced to express the second case: the JSON parsed, the field types were correct, but the values violate a business rule (a quote `Author` longer than 100 characters, an order with a negative quantity). `400` is the older, broader code, and many APIs use it for both cases without confusing clients. `[ApiController]` defaults to `400` for model validation failures; either choice is defensible as long as it is consistent.

27.1.4 Throttling and server errors

`429 Too Many Requests` is the response when the client has exceeded a rate limit. The response should include a `Retry-After` header naming how many seconds the client must wait before trying again. Rate limiting itself is developed in the pagination and rate limiting chapter; what matters here is that `429` is a distinct signal from `503` ("server unavailable") and from `403` ("not allowed at all").

`500 Internal Server Error` is the catch-all for unexpected failures. The action threw an exception the application did not handle, the database is unreachable, a downstream dependency timed out. `500` is the only code in this list whose response body must not contain a stack trace, exception message, or any other internal detail — error responses returned to anonymous clients should never expose information that helps an attacker map the system.

27.2 Problem details: a standard error shape

When a request fails, the client needs more than a status code. It needs to know *which* field was wrong, *why* the conflict occurred, *what* the rate-limit window is. The temptation is to invent a custom JSON envelope per service — `{ "error": "...", "code": 1234, "details": [...] }` — and every team has done it once. The result is that every client that talks to two APIs has to write two parsers.

Problem details (RFC 7807) is the standard JSON shape for HTTP error responses: an object with `type`, `title`, `status`, `detail`, and `instance` properties, returned with `Content-Type: application/problem+json`; ASP.NET Core emits this shape from `ProblemDetails` and `Validation-`

`ProblemDetails` helpers. The five fields have stable meanings:

- `type` — a URI identifying the *kind* of error. Stable across calls, useful as a key for client-side switch logic.
- `title` — a short human-readable summary of the kind. Stable across calls; safe to log or show to a developer.
- `status` — the HTTP status code, repeated in the body for clients that have already lost the response line.
- `detail` — a human-readable explanation specific to this occurrence. May include the offending value.
- `instance` — a URI identifying *this specific* occurrence, useful for correlating with server-side logs.

A typical `404` response from an ASP.NET Core API following the convention looks like this:

```
{
  "type": "https://tools.ietf.org/html/
        rfc9110#section-15.5.5",
  "title": "Not Found",
  "status": 404,
  "detail": "No quote exists with id 9999.",
  "instance": "/api/quotes/9999",
  "traceId": "00-7f3c...-01"
}
```

The shape is open: properties beyond the five standard ones are allowed. ASP.NET Core adds `traceId` so that an error in production can be matched to a specific request in the logs without the client having to invent a correlation header.

27.2.1 `ProblemDetails` and `ValidationProblemDetails` in ASP.NET Core

ASP.NET Core ships two strongly-typed helpers that produce this shape. `ProblemDetails` is the base class with the five standard fields; `ValidationProblemDetails` extends it with an `errors` dictionary that maps field names to arrays of validation messages. Controllers reach them through `ControllerBase.Problem(...)` and `ControllerBase.ValidationProblem(...)`.

The framework also wires them in by default. `[ApiController]` returns a `ValidationProblemDetails` body automatically when model binding fails, and `app.UseExceptionHandler("/error")` paired with a controller that returns `Problem(...)` produces a `ProblemDetails` body for unhandled exceptions. The author writes the happy path; the framework writes the error path.

27.3 A worked example: create and validation failure

The companion exercise REST Controllers, DTOs, and Swagger builds a `QuotesController` whose `Create` action illustrates both signals — the `201 Created` with a `Location` header on success, and the `400` with a `ValidationProblemDetails` body on validation failure.

```
[HttpPost]
public ActionResult<QuoteDto> Create([FromBody] Cre-
        ateQuoteRequest request)
{
    if (request.Text.Contains("forbidden-word"))
    {
        return ValidationProblem(
            detail: "The quote text contains a disallowed
        term.",
            statusCode: StatusCodes.Status422Unprocess-
        ableEntity);
    }

    var quote = _store.Add(request.Author, request.Text);
    var dto = ToDto(quote);

    return Created($"/api/quotes/{quote.Id}", dto);
}
```

Three response paths leave this method. A request with valid input runs the last two lines: `Created(...)` produces a `201`, sets the `Location` header to `/api/quotes/<id>`, and returns the DTO as the body. A request whose `Text` is missing or longer than 500 characters never reaches the method body — `[ApiController]` runs model validation first and returns `400` with a `ValidationProblemDetails` body listing the offending fields. A request that *parses* but trips a business rule reaches the method, and `ValidationProblem(...)` returns `422` with the same problem-details shape. Three different outcomes, three different status codes, one consistent error envelope. A client that checks the status code first and falls through to the `errors` dictionary on `400` and `422` handles every case the same way.

27.4 API versioning

Once an API has clients in production, the contract is frozen — at least the parts those clients use. Adding fields to a response is usually safe; renaming a field, removing one, changing a status code, or restructuring an error body is not. **API versioning** is the practice of letting old clients keep working when the API contract changes; common strategies are URL-path versioning (`/v1/quotes`), media-type versioning (`Accept: application/vnd.api+json; v=2`), and query-string versioning (`?api-version=2`).

Three families of strategy show up in production APIs. Each is a defensible answer; the trade-offs are in routing, caching, and client ergonomics.

STRATEGY	EXAMPLE	STRENGTHS	TRADE-OFFS
URL path	`/v1/quotes`, `/v2/quotes`	Visible in logs, in browser history, in `curl`; trivial to route	Couples versioning to the URI, which conflicts with the REST view that the URI names a resource, not a representation
Header (media type)	`Accept: application/vnd.acme+json; v=2`	Keeps the URI clean; aligns with content negotiation	Invisible in logs and bookmarks; awkward to test from a browser; harder to cache by URL
Query string	`/quotes?api-version=2`	Easy to pin per request; visible in logs	Mixes versioning with normal query parameters; clutters cache keys

URL-path versioning is what most teaching examples use and what most public APIs ship: it is unambiguous, easy to explain, and tooling friendly. Header-based versioning is purer and what the hypermedia community prefers. The choice matters less than picking one and applying it consistently across every endpoint.

27.4.1 Breaking-change discipline

Versioning is the escape hatch, not the default. The discipline that keeps APIs healthy is treating *most* changes as additive: a new optional field on a response, a new optional query parameter, a new endpoint at a fresh URI. Additive changes do not need a version bump because no existing client breaks.

Some changes are unavoidably breaking: removing a field, changing its type, narrowing what the server accepts, repurposing a status code. The rule for those changes is to introduce them under a new version, keep the old version running long enough for clients to migrate, and document the deprecation window — a header like `Sunset: Wed, 31 Dec 2025 23:59:59 GMT` lets the server tell the client when the old version stops working. Removing a version before its sunset is the failure mode that makes "we have versioning" feel like a lie to the teams that depend on it.

27.5 Summary

The status code is the primary signal an API gives its clients: a small disciplined vocabulary (200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500) is enough to describe every outcome that matters. The body of an error response should follow RFC 7807 problem details — `type`, `title`, `status`, `detail`, `instance` — with `Content-Type: application/problem+json`, so every client written against any API can use the same parser; ASP.NET Core ships `ProblemDetails` and `ValidationProblemDetails` plus `[ApiController]` to produce this shape automatically. API versioning is the escape hatch for the changes that cannot be made additively: pick one strategy (URL path is the safest default), keep additions backward compatible, and document the sunset date when an old version retires. The combined effect is a contract that any

client can interpret mechanically and that the server team can evolve without breaking the callers it has already shipped to.

PART VI · SERVICES AND APIS

CHAPTER 28

OpenAPI and Swagger

Watch the presentation

Se presentationen på svenska

An HTTP API without machine-readable documentation forces every client integrator to read source code or guess. That guessing scales badly. Two integrators reach two different conclusions about what `POST /quotes` accepts; a third writes a client that works against the current contract and breaks the next time a property is renamed; a fourth gives up and emails the team for a Postman collection. The fix is a contract document the server itself emits — written in a format both humans and tools understand — and a browser

explorer that renders it as a navigable, executable view of the API. The two pieces that fill those roles in ASP.NET Core are the **OpenAPI** specification and **Swagger UI**.

28.1 The contract problem

A REST API exposes resources, operations, request shapes, response shapes, and authentication schemes. Each of those is information a client needs before it can issue a single useful request. Without a contract document the information lives in three unreliable places: the source code of the server, the documentation site somebody hand-wrote last quarter, and the institutional memory of whoever built the API.

All three drift. Source code is authoritative but unreadable to non-developers and inaccessible to clients written in other languages. Hand-written docs go stale the moment a controller changes. Institutional memory leaves with the people. The cost is paid every time a new client integrates: hours of reading code, slack threads asking for examples, and bug reports about responses that look nothing like what the docs promised.

The remedy is to derive the contract from the code automatically and publish it in a format that tools can consume. That format is OpenAPI.

28.2 The OpenAPI specification

OpenAPI is the standard format (JSON or YAML) for describing an HTTP API's endpoints, request and response shapes, parameters, and authentication schemes; tooling consumes the OpenAPI document to generate documentation, client SDKs, and contract tests. It started life as the Swagger Specification at SmartBear, was donated to the Linux Foundation

in 2015, and became OpenAPI 3.0 in 2017. Most production APIs target OpenAPI 3.0 or 3.1.

An OpenAPI document is one file. It declares the API's metadata (title, version, description), the servers it lives on, the security schemes it accepts, and a list of paths. Each path lists its operations — `get`, `post`, `put`, `patch`, `delete` — and each operation declares its parameters, request body, and possible responses. Reusable shapes go under `components/schemas` so a `Quote` defined once can be referenced from every operation that produces or consumes one.

A trimmed JSON document for a quotes API looks like the following.

```
{
  "openapi": "3.0.1",
  "info": { "title": "CloudCiApi", "version": "v1" },
  "paths": {
    "/api/Quotes": {
      "get": {
        "tags": ["Quotes"],
        "responses": {
          "200": {
            "description": "Success",
            "content": {
              "application/json": {
                "schema": {
                  "type": "array",
                  "items": { "$ref": "#/components/schemas/
        QuoteDto" }
                }
              }
            }
          }
        }
      }
    }
  },
  "components": {
    "schemas": {
      "QuoteDto": {
        "type": "object",
        "properties": {
          "id": { "type": "integer", "format": "int32" },
          "author": { "type": "string" },
          "text": { "type": "string" },
          "createdAt": { "type": "string", "format": "date-
        time" }
        }
      }
    }
  }
}
```

That document is the entire contract. A client developer reads it and knows what `GET /api/Quotes` returns. A code generator reads it and emits a typed C#, TypeScript, or Python client. A contract-testing tool reads it and asserts that the running server's responses still match. The single artifact serves humans and machines.

28.2.1 Paths, operations, and schemas

The three building blocks repeat throughout. A **path** is a URI template (`/api/Quotes/{id}`), the parameterised address of a resource or resource collection. An **operation** is the combination of an HTTP method and a path — `GET /api/Quotes/{id}` is one operation, `POST /api/Quotes` is another. Each operation has its own parameters, request body schema, and response schemas. A **schema** describes the shape of data — usually a JSON object with named, typed properties, but also arrays, primitives, enums, and references to other schemas.

Schemas are where the DTO discipline pays off. The OpenAPI document references `QuoteDto` and `CreateQuoteRequest` — the wire types — never the entity. A reader of the contract sees only what the API speaks, never the persistence model.

28.2.2 Security schemes

OpenAPI describes authentication as well as data. A document that secures every operation behind a bearer token declares a security scheme:

```
{
  "components": {
    "securitySchemes": {
      "Bearer": {
        "type": "http",
        "scheme": "bearer",
        "bearerFormat": "JWT"
      }
    }
  },
  "security": [{ "Bearer": [] }]
}
```

An API key scheme reads as `"type": "apiKey"` with `in` set to `header` or `query` and a `name` for the header. Once a security scheme is declared, tooling treats it as a first-class part of the contract — Swagger UI grows an **Authorize** button, code generators emit a parameter for the credential, and contract tests fail loudly if a route that should require auth doesn't.

28.3 Swagger UI as the browser explorer

The OpenAPI document is machine-readable but not pleasant to read by hand. **Swagger UI** is the browser-rendered explorer that reads an OpenAPI document and displays each endpoint with its parameters and request/response examples, with a "Try it out" button that issues real requests; in ASP.NET Core it is added by the Swashbuckle package and served at `/swagger`.

The page renders one tag per group of related operations (typically one per controller), each expandable into its operations. Expanding `POST /api/Quotes` shows the request body schema with example JSON, the possible responses, and a **Try it out** button. Clicking the button turns the operation

into a form: the user fills in the request body, hits **Execute**, and the page shows the actual `curl` command, the response status, the response headers, and the response body.

For an API still being built this is invaluable. The same page that documents the contract is also the simplest test client. A backend developer adds an action, refreshes `/swagger`, and the new operation is right there to exercise — no Postman setup, no `curl` arguments to remember, no separate test harness. For a deployed API it is a discovery surface: a colleague asks "what does the quotes API do?" and the answer is a URL, not a screen-share.

The trade-off is exposure. Swagger UI reveals every route, parameter, and response shape — fine for a known public contract, reconnaissance for an attacker against an internal admin API. The conventional middle ground is to ship the OpenAPI JSON at `/swagger/v1/swagger.json` for tooling but gate or remove the interactive UI in production, or keep both behind authentication.

28.4 Swashbuckle generates OpenAPI from the controller code

ASP.NET Core does not produce OpenAPI by hand. The **Swashbuckle.AspNetCore** package introspects the controller assembly at runtime, walks every action method, and emits an OpenAPI document derived from what it finds. The route attribute becomes the path. The HTTP method attribute becomes the operation. The parameters of the action method become the operation's parameters and request body. The action's return type becomes the success response schema.

Two service registrations and two middleware calls are everything an out-of-the-box setup needs.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.MapControllers();
app.Run();
```

`AddEndpointsApiExplorer` registers the metadata source that Swagger reads — without it, `AddSwaggerGen` has nothing to introspect and the resulting document is empty. `AddSwaggerGen` registers the document generator. `UseSwagger` serves the document at `/swagger/v1/swagger.json`. `UseSwaggerUI` serves the browser explorer at `/swagger`. Both middleware calls must register before `MapControllers` — the endpoint routing terminates the pipeline, and middleware downstream of it never runs.

That is the whole contract for a basic API. From there, every additional piece of metadata in the controller — DTO types, return-type attributes, XML doc comments — flows into a richer document with no extra wiring.

28.4.1 Annotations that improve the document

The default-generated document is correct but sparse. The action's declared return type is the only response shape Swagger sees, so an action returning `ActionResult<QuoteDto>` documents `200 OK → QuoteDto` and nothing about the `404` it might return. `[ProducesResponseType]` fills in the gaps:

```
[HttpGet("{id:int}")]
[ProducesResponseType(typeof(QuoteDto),
        StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
public ActionResult<QuoteDto> GetById(int id)
{
    var quote = _store.GetById(id);
    return quote is null ? NotFound() : Ok(ToDto(quote));
}
```

Each attribute adds one response entry to the operation's `responses` map. The `200` carries a `QuoteDto` schema; the `404` carries no body, just a status. A reader of the contract now knows both code paths exist.

XML documentation comments add prose. With `<GenerateDocumentationFile>true</GenerateDocumentationFile>` in the `.csproj` and `c.IncludeXmlComments(xmlPath)` in `AddSwaggerGen`, the `<summary>` on an action becomes the operation's description in Swagger UI, and the `<remarks>` becomes the long-form text. Property comments on a DTO become the schema's property descriptions. The same triple-slash comments developers already write for IntelliSense flow into the contract document.

28.4.2 Exposing authentication so "Authorize" works

A controller decorated with `[Authorize]` rejects unauthenticated requests at runtime, but Swashbuckle does not know how the authentication is supposed to happen — bearer token, API key, OAuth flow, mutual TLS. The author tells it explicitly with `AddSecurityDefinition` and `AddSecurityRequirement`:

```
builder.Services.AddSwaggerGen(c =>
{
    c.AddSecurityDefinition("Bearer", new OpenApiSecurityS-
        cheme
    {
        Name = "Authorization",
        Type = SecuritySchemeType.Http,
        Scheme = "bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
        Description = "Paste a JWT obtained from the token
        endpoint."
    });

    c.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference
                {
                    Type = ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            },
            Array.Empty<string>()
        }
    });
});
```

Two effects follow. The OpenAPI document gains the security scheme and the operation-level requirement, so generated clients know to attach an `Authorization: Bearer <token>` header. Swagger UI gains the **Authorize** button at the top of the page; clicking it opens a dialog where the user pastes a token, and from that moment every “Try it out” call against a secured operation includes the header automatically. A second `AddSecurityDefinition` for an API key — `Type = SecuritySchemeType.ApiKey, In = ParameterLoca-`

`tion.Header, Name = "X-Api-Key"` — produces a second authentication option in the same dialog.

This is what makes Swagger UI a real test client for a secured API. Without the security definition, every secured operation returns `401` from inside the explorer, and developers have to fall back to `curl` or Postman to exercise auth-protected routes.

28.5 The toolchain payoff

The OpenAPI document is a one-time effort that pays out in many directions.

Auto-generated client SDKs. Tools like NSwag, OpenAPI Generator, and Kiota read the OpenAPI document and emit a typed client in the target language. A C# project consumes the API by adding a generated class with methods like `GetQuotesAsync()` and `CreateQuoteAsync(CreateQuoteRequest)` — no hand-written `HttpClient` plumbing, no JSON serialisation boilerplate, and a compile-time error if the contract changes in a breaking way. A TypeScript front end gets the same treatment with the same source document.

Contract tests. A test runner like Schemathesis or Dredd reads the OpenAPI document and exercises every operation against the running service, asserting that responses match the declared schemas. A response that drops a property, changes a field type, or returns an undocumented status code fails the test. The contract is no longer aspirational — it is enforced.

Postman and Insomnia imports. Both tools accept an OpenAPI document and produce a complete request collection from it, with all routes, example bodies, and authentica-

tion wired up. A new team member is one URL away from a working request library.

Mock servers. Tools like Prism start an HTTP server from an OpenAPI document and return example responses for every operation. A front-end team builds against the mock while the back end is still being implemented; both sides agree on the contract first, then implement against it independently.

USE OF THE OPENAPI DOCUMENT	WHAT IT PRODUCES
Swagger UI	Browser-rendered explorer with "Try it out"
Code generator (NSwag, Kiota, openapi-generator)	Typed client SDK in C#, TypeScript, Python, etc.
Contract tester (Schemathesis, Dredd)	Pass/fail assertions that responses match schemas
Postman / Insomnia import	A complete request collection
Mock server (Prism)	A stand-in HTTP server for parallel development

Each consumer reads the same document. Generating it once from the controller code — and keeping it accurate with `[ProducesResponseType]` and security definitions — unlocks all five.

28.6 Worked example tied to the exercise

The companion exercise REST Controllers, DTOs, and Swagger builds a `CloudCiApi` Web API around a `Quotes` resource and wires Swashbuckle exactly as described above. The Swagger setup in that exercise is intentionally minimal — `AddEnd-`

`pointsApiExplorer`, `AddSwaggerGen`, `UseSwagger`, `UseSwaggerUI` — because the controller, the route attributes, and the DTO types carry all the information Swashbuckle needs to produce a usable document on day one. The exercise also leaves Swagger enabled in production, making the deployed `/swagger` page the API's demo URL. Subsequent exercises in the same chapter add an API key middleware and a JWT bearer scheme; each of those exercises extends the Swashbuckle configuration with the corresponding `AddSecurityDefinition` so the **Authorize** button in Swagger UI works for the new scheme.

The takeaway from running through that exercise is that OpenAPI is not a deliverable a team writes after the API is built. It is a by-product of writing the controller carefully — naming the action, declaring the route, returning a DTO, annotating the response types, configuring the security scheme. Swashbuckle then generates the document for free, and Swagger UI renders it.

28.7 Summary

OpenAPI is the standard JSON or YAML format for describing an HTTP API's endpoints, request and response shapes, parameters, and authentication schemes. Swagger UI is the browser-rendered explorer that reads an OpenAPI document and turns it into a navigable, executable surface. In ASP.NET Core, the Swashbuckle package introspects controllers, DTOs, attributes, and XML comments to generate the document at runtime — `AddSwaggerGen`, `UseSwagger`, and `UseSwaggerUI` are the entire wire-up. `[ProducesResponseType]` documents non-success responses; `AddSecurityDefinition` exposes bearer and API key schemes so Swagger UI's **Authorize** button can attach credentials to "Try it out"

calls. The same OpenAPI document feeds code generators, contract testers, Postman imports, and mock servers, turning one piece of metadata into a whole toolchain. The contract becomes a living artifact — derived from the code, consumed by tools, and trustworthy because it cannot drift away from what the server actually does.

PART VI · SERVICES AND APIS

CHAPTER 29

Pagination, Idempotency, and Rate Limiting

Watch the presentation
Se presentationen på svenska

A REST API that runs against a real database eventually faces three operational pressures that the happy-path design ignored. The first is size: a `GET /quotes` that grew from ten rows in development to a million rows in production becomes hostile to every client that calls it. The second is duplication: a mobile client retries a `POST /orders` after a flaky network and the server has no way to tell whether it is seeing a new

order or the second copy of the previous one. The third is abuse: a single client running a hot retry loop or a misconfigured cron job can saturate a backend that was sized for the average case. Pagination, idempotency, and rate limiting are the three contractual mechanisms that turn a working API into one that is also safe to expose. Each of them addresses one of the three pressures, and the worked example at the end ties them together with a paginated controller and a token-bucket policy in `Program.cs`.

29.1 Why returning everything is not an option

A `GET` against a resource collection is the natural first endpoint to write. It is also the one most likely to collapse under its own weight. A collection with a million rows serialises to hundreds of megabytes of JSON, occupies a controller worker thread for the entire duration of the database read, and forces the client to receive, parse, and store all of it before it can render the first row. A connection that drops at the 90% mark wastes everything transferred so far, because there is no way to resume.

Returning everything is a contract decision, not a coding decision. Once the API promises that `GET /quotes` returns the entire collection, every client is entitled to that response forever, and the server has to keep finding the memory and bandwidth to produce it. Reversing the decision later means breaking clients. The remedy is to commit to returning only a slice from the very first version.

Pagination is the practice of returning a slice of a large collection per request rather than the whole collection; common strategies are offset-based (`?offset=20&limit=10`), page-based (`?page=3&size=10`), and cursor-based (a server-issued opaque token that is opaque to the client). All three

strategies cap the per-request response size, but they differ in how they identify the next slice and in how they behave when the underlying collection is changing while the client is paging through it.

29.1.1 Offset and limit

The offset-based strategy reads as plain English: "skip the first 20 rows, then give me 10". It maps cleanly onto the SQL `OFFSET 20 LIMIT 10` clause that most databases support natively, which is why it is the default in many starter tutorials. The trade-off is that the database still has to count and discard the skipped rows on every request, so deep pagination (`?offset=950000`) gets slower as the offset grows, even though the response size stays the same. The strategy also drifts under concurrent writes: if rows are inserted before the current offset between two page requests, the client either sees the same row twice or skips a row entirely.

29.1.2 Page and size

Page-based pagination (`?page=3&size=10`) is offset-based pagination in friendlier clothes. The server computes `offset = (page - 1) * size` and runs the same query. The advantage is that page numbers are easier to render in a UI ("page 3 of 47") and easier for humans to reason about than raw row counts. The trade-offs are identical to offset-based: deep pages are slow, and the boundary drifts under concurrent writes. The strategy works best on collections that are mostly stable and where users are expected to land on a page directly rather than scroll through the whole list.

29.1.3 Cursor-based pagination

Cursor-based pagination replaces the numeric offset with an opaque token that the server issues alongside each page. The

client treats the cursor as a black box and passes it back unmodified on the next request. Internally, the cursor encodes the position of the last row returned — typically the primary key plus a sort field — so the server can resume the scan with `WHERE id > :cursor` instead of skipping rows. This makes deep pagination cheap (no skipping) and stable under inserts (a new row at the start of the collection does not shift the cursor's meaning). The trade-off is that cursors do not support arbitrary jumps: a client cannot ask for "page 47" because the cursor for that page has not been issued yet. Cursors also tie the client to the server's sort order, since the cursor is meaningless if the sort changes.

The three strategies map onto different read patterns:

STRATEGY	STRENGTH	TRADE-OFF
Offset / limit	Easy to implement, supports arbitrary jumps	Deep pages slow, drifts under concurrent writes
Page / size	UI-friendly, intuitive for humans	Same trade-offs as offset / limit
Cursor	Stable under writes, deep pages cheap	No arbitrary jumps, server controls sort order

29.1.4 The response envelope

Once the response is paginated, the client needs enough information to fetch the next slice. Two conventions dominate. The first is a JSON envelope that wraps the data in an object with two top-level fields:

```
{
  "data": [
    { "id": 1, "author": "Hopper", "text": "..." },
    { "id": 2, "author": "Knuth", "text": "..." }
  ],
  "pagination": {
    "nextCursor": "eyJpZCI6Mn0=",
    "limit": 20
  }
}
```

The second is the GitHub-style `Link` header, where the response body is a plain JSON array and the next-page URL is carried in an HTTP header (`Link: <https://api.example.com/quotes?cursor=...>; rel="next"`). The envelope shape is easier for typed clients (the `pagination` object maps cleanly to a DTO), while the `Link` header keeps the response body shape symmetrical between paginated and non-paginated endpoints. Both are correct; consistency within an API matters more than the choice between them.

29.2 Idempotency as a contract

A request is **idempotent** when sending it twice has the same observable effect as sending it once. The HTTP specification fixes this property for some methods and leaves it open for others, and clients rely on those guarantees when they retry on failure.

Idempotency is the property that calling the same operation with the same input multiple times produces the same observable result as calling it once; `GET`, `PUT`, and `DELETE` are idempotent by HTTP definition, while `POST` is generally not — for at-least-once delivery to be safe, an `Idempotency-Key` header lets the server detect retries.

`GET` is idempotent because reading a resource does not change it. `PUT` is idempotent because it replaces the resource with the request body, so applying the same body twice leaves the resource in the same state as applying it once. `DELETE` is idempotent because the second delete finds nothing to delete and returns `404` or `204` — the resource is gone either way. The HTTP method (defined in Part III HTTP fundamentals) carries the contract; client libraries and proxies are entitled to retry idempotent requests on transient failures without asking the application.

`POST` is the outlier. Posting `{ "amount": 100 }` to `/orders` twice creates two orders. This is correct semantics — the second `POST` is a different operation, even if the body is byte-identical — but it makes retries dangerous. A mobile client that times out on an order submission has no way to know whether the server received the request and the response was lost, or the request never arrived. Retrying could double-charge the customer; not retrying could lose the order.

29.2.1 The Idempotency-Key header

The pattern that resolves this is the **`Idempotency-Key`** request header. The client generates a unique value (typically a UUID) before the first attempt, attaches it to the request, and reuses the same value on every retry. The server stores the key alongside the response it generated and, on a subsequent request with the same key, returns the cached response instead of executing the operation again.

```
POST /api/orders HTTP/1.1
Idempotency-Key: 9c3f2b1e-0d7a-4e8a-9c01-f7c2a4d3b2a1
Content-Type: application/json

{ "amount": 100, "currency": "USD" }
```

The server's part is bookkeeping: persist the key with a TTL (24 hours is common), record the resulting status code and body, and return the cached response on a hit. Stripe popularised this pattern for payment APIs, and it is now the standard mechanism for safe `POST` retries. The trade-off is the storage cost of the key cache and the requirement that clients generate keys consistently — a client that generates a fresh UUID on every retry defeats the mechanism.

29.3 Rate limiting as a backstop

A well-behaved client paginates and uses idempotency keys. A misbehaving or misconfigured client does neither and hammers the API with requests until something gives way. **Rate limiting** is the server-side practice of capping how many requests a client can make in a window; common algorithms are fixed window, sliding window, and token bucket; ASP.NET Core ships a rate-limiting middleware that returns `429 Too Many Requests` when a client exceeds its quota.

Rate limiting protects shared backend resources from abuse and from accidental hot-loops. A single client running a `while (true) { GET /quotes }` loop can exhaust the database connection pool, starve every other client, and crash the service. The rate limiter caps the damage that any single identity can do per unit of time, returning a `429` status code (defined in Part III HTTP fundamentals) once the cap is reached. The response carries a `Retry-After` header that tells the client how long to wait before its next attempt.

29.3.1 The three algorithms

The **fixed window** algorithm divides time into equal buckets (say, one minute each) and counts requests per identity per bucket. The bucket resets when the next minute starts. The algorithm is cheap to implement — a single counter per identity — but allows a burst of `2 × limit` requests around the bucket boundary, since a client can spend its full quota in the last second of one minute and the first second of the next.

The **sliding window** algorithm smooths out the boundary effect by counting requests in the last N seconds rather than per fixed bucket. It is more accurate but more expensive: the server has to keep a list of timestamps per identity, or maintain a weighted average across two adjacent buckets.

The **token bucket** algorithm models the limit as a bucket that holds up to N tokens and refills at a steady rate (say, 10 tokens per second). Each request consumes one token; if the bucket is empty, the request is rejected. The bucket allows short bursts up to its capacity (a client can spend all 100 tokens at once after an idle period) while still enforcing the long-run average rate. This shape — burst tolerance plus a steady refill — fits most real client behaviour better than either fixed or sliding windows, which is why it is the algorithm most APIs default to.

ALGORITHM	BURST BEHAVIOUR	COST	USE WHEN
Fixed window	Allows 2× limit at the boundary	Single counter per identity	Cap is loose, simplicity matters
Sliding window	Smooth, no boundary spike	Timestamp list or weighted counter	Cap must be enforced strictly
Token bucket	Allows configurable bursts, enforces long-run rate	Two integers (tokens, last-refill timestamp)	Real client behaviour is bursty

29.4 A worked example

The chapter's running example is a quotes API — the same `Quote` domain (id, author, text, createdAt) used in the REST API and DTOs exercise. The list endpoint paginates with cursors, and the whole API sits behind a token-bucket rate limiter.

```
[HttpGet]
public async Task<ActionResult<PagedResponse<QuoteDto>>>
        GetQuotes(
    [FromQuery] string? cursor,
    [FromQuery] int limit = 20)
{
    limit = Math.Clamp(limit, 1, 100);
    var afterId = DecodeCursor(cursor);

    var quotes = await _db.Quotes
        .Where(q => afterId == null || q.Id > afterId)
        .OrderBy(q => q.Id)
        .Take(limit + 1)
        .ToListAsync();

    var hasMore = quotes.Count > limit;
    var page = quotes.Take(limit).Select(q =>
        q.ToDto()).ToList();
    var nextCursor = hasMore ? EncodeCursor(page[^1].Id) :
        null;

    return Ok(new PagedResponse<QuoteDto>(page, new Pagina-
        tion(nextCursor, limit)));
}
```

The action clamps the requested limit to a sensible ceiling (the client cannot ask for ten thousand rows), decodes the cursor, fetches one row more than the limit to detect whether more pages exist, and returns a `PagedResponse` envelope with `data` and `pagination` properties. The cursor is a base64-encoded id; the client treats it as opaque and passes it back unchanged. Sorting by `Id` ascending makes the cursor stable under inserts (a new quote always gets a higher id, so it appears on a future page rather than shifting an existing one).

The token-bucket policy lives in `Program.cs`:

```
builder.Services.AddRateLimiter(options =>
{
    options.AddTokenBucketLimiter("quotes", config =>
    {
        config.TokenLimit = 100;
        config.TokensPerPeriod = 10;
        config.ReplenishmentPeriod =
        TimeSpan.FromSeconds(1);
        config.QueueLimit = 0;
        config.AutoReplenishment = true;
    });
    options.RejectionStatusCode =
        StatusCodes.Status429TooManyRequests;
    options.OnRejected = async (context, _) =>
    {
        context.HttpContext.Response.Headers.RetryAfter =
        "1";
        await con-
        text.HttpContext.Response.WriteAsync("Rate limit
        exceeded.");
    };
});

app.UseRateLimiter();
```

The bucket holds 100 tokens and refills at 10 tokens per second, so a client can issue a burst of 100 requests after an idle period and then settle into a steady 10 requests per second. When the bucket empties, the middleware short-circuits the pipeline, returns `429`, and writes a `Retry-After: 1` header so the client knows when to try again. The `[EnableRateLimiting("quotes")]` attribute on the controller (or on individual actions) opts the endpoint into the named policy.

The companion exercise REST API and DTOs does not yet wire pagination or rate limiting in, but the controller it produces is the natural place to bolt them on once the basic shape works.

29.5 Summary

A list endpoint that returns the entire collection works in development and fails in production; pagination is the contract that keeps responses bounded. Offset, page, and cursor strategies all cap the response size but trade off differently on deep pages, stability under writes, and support for arbitrary jumps. Idempotency is a property of HTTP methods — `GET`, `PUT`, and `DELETE` are idempotent by definition, `POST` is not — and the `Idempotency-Key` header lets clients retry `POST` safely by giving the server a deduplication handle. Rate limiting is the backstop that protects shared resources from abuse and from accidental hot-loops, and the token-bucket algorithm fits most real client behaviour because it allows controlled bursts on top of a steady long-run rate. ASP.NET Core's rate-limiter middleware enforces the cap and returns `429 Too Many Requests` with a `Retry-After` header when a client crosses it. Together, these three mechanisms turn a working REST API into one that is also safe to expose.

PART VII

Containers

Containers replaced the virtual machine as the standard deployment unit for stateless application code. This Part covers the model from the bottom up: how containers differ from VMs, what an image actually is, how a Dockerfile constructs one, how a single image targets multiple CPU architectures, how Compose runs several containers as one local environment, and how registries distribute images to deployment targets.

The companion exercise is the Docker chapter — five exercises that build a multi-stage .NET image, push it to Docker Hub, run it in Compose alongside a database, and rebuild it for both amd64 and arm64.

PART VII · CONTAINERS

CHAPTER 30

Containers vs Virtual Machines

Watch the presentation
Se presentationen på svenska

An application that runs cleanly on a developer's laptop often fails on the test server, then fails again on the production host. The cause is rarely the code — it is the surrounding environment: a different operating system patch level, a missing system library, an interpreter at version 6.0.404 instead of 6.0.418, an environment variable nobody documented. The phrase "works on my machine" describes a real engineering cost: hours of debugging that produce no new functionality. Containers attack this problem at its root by

packaging the application together with the exact runtime environment it requires, so the same artifact runs identically wherever it lands. The sections below develop how containers achieve that property, how they differ from the older virtual-machine approach to the same problem, and when each model is the right choice.

30.1 The reproducibility problem

Two strategies have dominated server-side reproducibility. Both isolate workloads from one another and from the host, but they isolate at different layers of the stack and pay very different costs to do so.

The older strategy, **virtualization**, emulates an entire computer in software. Each workload runs on its own simulated hardware with its own operating system. The newer strategy, **containerization**, isolates processes within a single shared operating system, packaging only what the application itself needs. Both work; they just sit at different points on the trade-off curve between isolation strength and runtime overhead.

Understanding the difference begins with revisiting how a virtual machine builds its illusion of a separate computer.

30.2 How a virtual machine virtualizes hardware

A virtual machine (VM) emulates a physical computer in software, complete with its own operating system, its own kernel, its own simulated CPU, memory, disk, and network interface. The component that creates this illusion is the **hypervisor** — a layer that sits between physical hardware and the virtual machines, allocating real CPU time, real memory pages, and

real I/O bandwidth to each VM while keeping them strictly isolated from each other.

Part II treats the VM layer in depth, including the distinction between Type 1 (bare-metal) and Type 2 (hosted) hypervisors and how virtual CPUs, memory, and disks map to physical resources. For the present discussion, two consequences matter.

First, every VM carries the weight of a complete operating system. A minimal Linux VM image is hundreds of megabytes; a Windows Server VM is several gigabytes. Booting that VM means booting the kernel, initialising device drivers, starting system services, and only then launching the application. Startup time is measured in tens of seconds at best, often minutes.

Second, isolation is hardware-grade. Two VMs on the same host share nothing above the hypervisor: not the kernel, not the filesystem, not the process table. A kernel exploit inside one VM does not directly compromise another, because each VM runs its own kernel. This is the strongest isolation a single physical host can provide.

30.3 How a container isolates processes on a shared kernel

A **container** is a lightweight, isolated process running on a shared kernel; it bundles an application and its dependencies into a standardized unit that starts in milliseconds, provides process and network isolation, and runs identically across development, testing, and production environments. The crucial phrase is *shared kernel*. There is no second operating system inside the container — only the application, its libraries, and the files it expects. The container reuses the kernel of the host machine.

What stops a container from seeing the rest of the host? Two Linux kernel features do the work, and a third executes them.

Isolation in containers is the kernel-enforced separation of processes, filesystems, and networks from the host and other containers; process namespaces hide other processes, network namespaces provide separate network stacks, and cgroups limit CPU and memory to prevent one container from starving others. *Namespaces* are the kernel's way of giving a process its own private view of system resources: its own process tree, its own network interfaces, its own mount points, its own user IDs. Inside the container, process ID 1 looks like the only process on the system, even though dozens of other processes are running on the host. *Cgroups* (control groups) are a separate kernel mechanism that places hard ceilings on resource consumption — this container may use at most 0.5 CPU and 512 MB of memory, period. Together, namespaces hide and cgroups limit.

A **container runtime** is the software that executes containers on a host machine; Docker is the most common runtime, using the Linux kernel's containerization features (namespaces and cgroups) to enforce isolation without the overhead of a full virtual machine. The runtime is what reads an image, sets up the namespaces and cgroups, mounts the container's filesystem, and starts the application process. The kernel does the actual isolation work; the runtime orchestrates it.

This architectural choice — reusing the host kernel rather than booting a new one — is the single decision that drives every other difference between containers and VMs. **Kernel sharing** is the architectural difference between containers and virtual machines: containers all use the same host operating system kernel (but isolated via namespaces), whereas

virtual machines each run their own full kernel, trading isolation for size and startup overhead.

30.4 What each model gives up and gains

Sharing the kernel makes containers small and fast. There is no second operating system to ship, so a typical container image is tens of megabytes rather than gigabytes, and starting it means starting a single process rather than booting a kernel. The cost is that isolation is now software-enforced inside one shared kernel rather than hardware-enforced across two separate ones. A serious kernel vulnerability is a problem for every container on the host, in a way it would not be for VMs.

The virtual machine pays the opposite price. Each VM duplicates the kernel, the init system, the system libraries, and the OS-level services, regardless of whether the application needs them. That duplication buys stronger isolation and the ability to run completely different operating systems side-by-side — a Windows VM and a Linux VM on the same physical host, both behaving exactly as their respective operating systems should.

The portability story also splits. A VM image is portable in the sense that any hypervisor of the right type can run it, and what runs is a complete operating system the team chose and configured. A container image is portable in a stricter sense: the same image runs on any host with a compatible kernel and a container runtime, and the surface area exposed to environment drift is reduced to the kernel ABI. Containers cannot, however, change their kernel — a container built to run on Linux cannot run natively on Windows, and a container built for the AMD64 architecture cannot run natively on an ARM64 host without emulation.

30.5 Density and startup time in practice

The architectural difference shows up most visibly in two operational metrics: how many workloads fit on a host, and how quickly a workload becomes ready.

Consider a single physical host with 32 CPU cores and 128 GB of RAM. Running ten VMs, each with 4 vCPUs and 12 GB of RAM, fills the box and leaves headroom for the hypervisor — and each VM permanently consumes the memory of its own kernel and OS-level services, perhaps 1–2 GB before the application has loaded a single byte. The same host can comfortably run a hundred or more containers of equivalent applications, because each container only carries the application's own working set. Density on the order of 10× is routine.

Startup time tells the same story. A VM cold-starts in 30 seconds to several minutes: BIOS, bootloader, kernel, init, services, application. A container cold-starts in milliseconds to a few seconds: the runtime sets up namespaces and cgroups, mounts the image's filesystem, and execs the application. The difference is not a tuning question — it is structural. A VM has a kernel to boot and a container does not.

30.5.1 A worked example: dotnet run vs the same app as a VM

The CloudSoft Recruitment app from the Docker exercise track is a small ASP.NET Core service. Run it directly on a developer machine:

```
$ time dotnet run --project src/CloudSoft.Web
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000

real    0m0.213s
```

The .NET runtime starts, the application initialises, and the HTTP listener is ready in roughly 200 ms. Wrap that same application as a container image and `docker run` it: the runtime mounts the image's filesystem, the application process starts under its namespaces, and the container is serving requests in roughly the same 200 ms — the dotnet startup time dominates and the container overhead is negligible.

Now provision the same application as an Azure VM. The VM creation completes in about a minute; first boot of the operating system takes another 30–60 seconds; configuration management installs the .NET runtime and copies the application; finally the application starts in its 200 ms. Wall-clock time from "go" to "serving requests" is two to three minutes for a fresh VM, versus a fraction of a second for a container on a host that already has the image. Both deployments end up running the exact same compiled application — the difference is entirely in what surrounds it.

30.6 When each model fits

Neither model is universally better. The right question is which trade-off matches the workload.

DIMENSION	VIRTUAL MACHINE	CONTAINER
Isolation boundary	Hardware-virtualised, separate kernel	Kernel-enforced namespaces, shared kernel
Image size	Gigabytes (full OS)	Tens to hundreds of MB (app + libs)
Startup time	Tens of seconds to minutes	Milliseconds to seconds
Density per host	Tens of VMs	Hundreds of containers
Guest OS flexibility	Any compatible OS, including Windows	Must match host kernel family
Per-instance overhead	Dedicated kernel, init, OS services	Application process only
Failure blast radius	One VM, contained by hypervisor	One container, contained by namespaces

A VM is the right choice when the workload needs its own operating system. Legacy Windows desktop applications that depend on the Win32 API run as Windows VMs, not containers, on a Linux host. Workloads with strict regulatory isolation requirements may demand the hardware-grade boundary that only a separate kernel provides. Database engines tuned for specific kernel parameters, or workloads that need direct access to specialised hardware drivers, often live in VMs because the abstraction layer of containers hides what they want to reach.

A container is the right choice when the workload is a stateless service that should start fast, scale horizontally, and ship

as a self-contained artifact. A web API serving HTTP requests, a background worker consuming a message queue, a scheduled job that processes a file and exits — all are textbook container workloads. The application is small, the dependencies are well-defined, and operational value comes from running many cheap instances rather than a few expensive ones.

The two models also compose. Cloud platforms run containers *on top of* VMs: a managed Kubernetes cluster, or Azure Container Apps, provisions VMs as the underlying compute and packs containers onto them. The VM provides the hardware-grade tenant boundary between customers, and the container provides the lightweight unit of deployment within that boundary. Neither model has to win for both to be useful.

30.7 Summary

Virtual machines and containers solve the same problem — running isolated workloads on shared hardware — at different layers. A VM virtualises hardware: a hypervisor allocates physical resources to guests, each running a full operating system, paying gigabytes of image size and tens of seconds of boot time in exchange for hardware-grade isolation and full OS flexibility. A container shares the host kernel: namespaces and cgroups isolate a single process tree, its filesystem, and its network, and a container runtime such as Docker executes the resulting unit. Containers start in milliseconds, pack an order of magnitude denser per host, and ship as compact images, but they cannot mix kernel families and they share a kernel-vulnerability blast radius. The fit follows the trade-off: VMs for legacy desktop OSes, regulated isolation, or hardware-specific workloads; containers for stateless services that want fast starts, high density, and a single artifact that runs identically everywhere. The remaining chapters of this

Part build on the container model — how images are layered, how Dockerfiles describe them, how Compose orchestrates them locally, and how registries distribute them.

CHAPTER 31

Images and Layers

Watch the presentation
Se presentationen på svenska

A container is an isolated process, but a process needs files to execute: a binary, shared libraries, configuration, and a directory layout the application expects to find. Containers do not conjure that filesystem on demand. They start from a prebuilt, immutable template that defines the entire environment down to the last shared object. That template is the image, and understanding how images are constructed, distributed, and cached is the difference between a container build that completes in seconds and one that drags through every dependency on every push.

31.1 What an image is

A **Docker image** is an immutable, layered template that defines an application and its runtime environment; it specifies a base OS, dependencies, files, and a default command, and is stored in a registry for distribution; containers are running instances of images. The image itself never executes — it is a passive artifact, a snapshot of a filesystem plus a small bundle of metadata. The container runtime takes that snapshot, wraps it in process and network isolation, and starts the configured entrypoint. Two containers launched from the same image see identical filesystems at the moment of start.

An image is more than the files it carries. The metadata records the default command (`ENTRYPOINT` and `CMD`), the working directory, environment variables, exposed ports, and the user the process runs as. Pull an `nginx` image and the metadata says: run `/docker-entrypoint.sh nginx -g 'daemon off;'`, exposing port 80, as the `root` user. Pull `mcr.microsoft.com/dotnet/aspnet:10.0` and the metadata configures `DOTNET_RUNNING_IN_CONTAINER=true`, sets the working directory, and prepares the runtime to execute a published .NET assembly. The image bundles both the bits and the instructions for how to run them.

Images are immutable. Once built, the bytes do not change, and a fingerprint (the manifest digest) covers every file, every metadata field, and every layer. Modifying anything — a dependency, a configuration line, a single byte — produces a new image with a new digest. The previous image still exists, unchanged, addressable by its old digest. This immutability is what makes container deployments reproducible: a digest pinned in production today refers to the same bits that ran in staging last week.

31.2 The layer model

An image is not a single tarball of files. It is a stack of **layers**, where a layer is a read-only filesystem snapshot created by one or more Dockerfile instructions; layers are stacked (like a union filesystem) to form the complete image filesystem, and Docker caches unchanged layers during rebuilds to avoid redundant work. Each instruction in a Dockerfile — `FROM`, `COPY`, `RUN`, `ADD` — produces one layer. Metadata-only instructions like `ENV`, `EXPOSE`, and `WORKDIR` mutate the image configuration without adding filesystem content, but they still participate in the cache key.

When a container starts, the runtime stacks the image's layers in order using a union filesystem (`overlay2` on most Linux hosts). The lowest layer is the base; each subsequent layer overlays the one below, with later files masking earlier ones at the same path. The container then receives a thin, writable layer on top — a scratch space that absorbs any modifications the running process makes. When the container stops, that writable layer can be discarded; the underlying image layers are untouched.

This stacking has a subtle consequence: deletion does not actually remove bytes. A `RUN rm -rf /var/cache/apt/*` produces a new layer that masks the cache directory, but the bytes still sit in the layer below, traveling with the image to every host that pulls it. This is why image-size optimization usually means avoiding the bytes in the first place — running the install and the cleanup as a single `RUN` command so both happen in the same layer — rather than removing them after the fact.

31.2.1 Content addressing

Each layer is identified by a SHA256 hash of its contents. Two builds that produce byte-identical layer content produce

identical hashes, regardless of who built them or when. This is content addressing, and it is the mechanism that makes layer caching and sharing possible. A registry storing one hundred images that all use `mcr.microsoft.com/dotnet/aspnet:10.0` as their base stores the runtime layer exactly once. A host pulling its hundredth ASP.NET image downloads only the application-specific layers; the runtime is already on disk from the first pull.

The same logic powers the build cache. **Image cache** is Docker's optimization that reuses layers from previous builds; when a Dockerfile instruction has not changed and its context is identical, Docker skips re-execution and uses the cached layer, reducing build time significantly. The cache key for a layer combines the parent layer's digest, the instruction text, and (for `COPY` and `ADD`) the hashes of the files being copied. If any input changes, the cache misses and Docker re-executes the instruction. If nothing changed, the existing layer is reused untouched.

31.3 Why layer ordering matters

Cache hits cascade top-down: as soon as one instruction misses the cache, every instruction below it must also re-execute, even if its inputs are identical. The cached `dotnet publish` from yesterday is useless if the layer feeding into it is new. This is why Dockerfile authors order instructions from least to most volatile — stable, slow-to-rebuild layers first, fast-changing application code last.

Consider an ASP.NET project. The .NET SDK base image and the NuGet dependencies change weekly at most. The application source code changes on every commit. A naive Dockerfile that copies the entire source tree before restoring packages forces a `dotnet restore` on every build, even

when only a controller method changed. A well-ordered Dockerfile copies the `.csproj` files first, runs `dotnet restore` in its own layer, then copies the rest of the source. The restore layer caches across hundreds of source-only commits; a fresh build with no dependency changes finishes in seconds rather than minutes.

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

COPY *.csproj ./
RUN dotnet restore

COPY . ./
RUN dotnet publish -c Release -o /app
```

The discipline is to ask, for each instruction: how often does its input change? Put the rare changes near the top of the Dockerfile; put the constant changes near the bottom. This is not a stylistic preference — it is the difference between a CI pipeline that finishes in 30 seconds and one that takes 5 minutes on every push.

31.4 The base image

Every image chain starts with a **base image**, the initial image specified in a Dockerfile's `FROM` instruction; it provides the operating system, runtime, and package manager for the rest of the image (e.g., `FROM mcr.microsoft.com/dotnet/aspnet:latest` provides the .NET runtime). The base image determines the kernel-compatibility expectations, the available shell, the libc flavor (glibc vs. musl), the package manager, and the security posture of everything built on top.

The choice of base image is the single most consequential decision in a Dockerfile. An ASP.NET application can be built `FROM mcr.microsoft.com/dotnet/aspnet:10.0` (Debian-based, ~220 MB, includes a full .NET runtime and the dependencies the runtime expects), or `FROM mcr.microsoft.com/dotnet/aspnet:10.0-alpine` (Alpine-based, ~110 MB, uses musl libc), or `FROM mcr.microsoft.com/dotnet/runtime-deps:10.0-alpine` paired with a self-contained publish (smallest possible, but the application carries its own runtime).

Smaller images pull faster, attack surfaces are smaller, and start-up is quicker. The trade-off is operational: Alpine's musl libc occasionally surfaces compatibility issues with native dependencies compiled against glibc; some performance counters and globalization features in .NET behave differently under Alpine. Distroless images go even further — no shell, no package manager — at the cost of debugging convenience. The rule of thumb is to pick the lightest base that does not introduce more friction than it saves, and to pin a specific tag (`10.0` rather than `latest`) so the base does not silently shift under the build.

31.5 The image manifest

When an image lands in a registry, its identity is described by a manifest. An **image manifest** is the JSON metadata that describes an image: it lists layers, their digests and sizes, the image configuration, and platform architecture; a **manifest list** extends this to map multiple platform-specific manifests to a single tag, enabling multi-platform support. The manifest is small — a few kilobytes of JSON — but it is the linchpin of the distribution model.

A pull operation begins by fetching the manifest, not the image. The client reads the layer digests, checks which ones are already on disk, and downloads only the missing layers. The manifest itself has a digest, the SHA256 of its JSON bytes, and that digest is what `docker pull mcr.microsoft.com/dotnet/aspnet@sha256:abc123...` resolves to. Tags like `10.0` are mutable pointers into the registry's manifest store; digests are immutable.

This separation matters in production. A deployment pipeline that pulls `aspnet:10.0` today and `aspnet:10.0` tomorrow may receive two different images if Microsoft pushed a patch release in between. A pipeline that resolves the tag once, captures the digest, and deploys by digest receives the same bits every time, regardless of subsequent tag movements.

31.6 Storage and pull efficiency

Layer sharing is not just a build optimization; it is the mechanism that keeps registries and hosts from drowning in duplicated bytes. A team that builds twenty microservices, all from the same `mcr.microsoft.com/dotnet/aspnet:10.0` base, stores the runtime layer once on the registry and once on each host. The only per-service cost is the application-specific layers — typically tens of megabytes — even though each service's image, taken in isolation, would appear to be 250 MB.

The same effect speeds up incremental deployments. A new build that only touches the application binary produces a new top layer; every layer below — base image, dependency restore, framework configuration — is unchanged and already present on the target host. The host pulls the manifest, sees that it is missing only the new top layer, downloads a few

megabytes, and starts the new container. Hosts that have never seen the image at all pay the full cost the first time and the marginal cost every time after.

31.7 Worked example: pulling and inspecting an image

The companion exercise Run Your First Container starts with a single command:

```
docker pull mcr.microsoft.com/dotnet/aspnet:10.0
```

The output streams a list of layer pulls, one per line, each identified by a 12-character hash prefix. Some layers download in parallel; others marked `Already exists` are skipped because they were pulled by an earlier image. Once the manifest reconciliation completes, the local Docker daemon has the full image and prints the resolved digest.

To see the structure of the pulled image, `docker history` walks the layer stack:

```
docker history mcr.microsoft.com/dotnet/aspnet:10.0
```

```
IMAGE          CREATED        CREATED
BY                                        SIZE
sha256:abc...  3 days ago     ENTRYPOINT
["dotnet"]                           0B
<missing>      3 days ago     ENV ASPNET-
CORE_HTTP_PORTS=8080                  0B
<missing>      3 days ago     COPY /dotnet /usr/share/
dotnet                  175MB
<missing>      3 days ago     RUN apt-get update && apt-get
install -y ...    32MB
<missing>      3 weeks ago    /bin/sh -c #(nop)  CMD
["bash"]                 0B
<missing>      3 weeks ago    /bin/sh -c #(nop) ADD
file:... in /             80MB
```

Reading from the bottom: the `ADD` of the Debian rootfs is the base layer, contributing 80 MB. The next `RUN` installs runtime dependencies (libicu, libssl) for another 32 MB. The `COPY` lays down the .NET runtime itself at 175 MB. Above that, several metadata-only entries (`ENV`, `ENTRYPOINT`) carry zero bytes but still appear in the history — they are configuration changes baked into the manifest. The `<missing>` IMAGE column is expected; intermediate layer IDs are not stored locally for pulled images, only the final digest. The bottom-up reading reveals exactly what the image is made of and where the size comes from, which is precisely the information needed to decide whether a smaller base image or a multi-stage build would shrink the artifact.

31.8 Summary

A Docker image is the immutable, layered filesystem template a container starts from, packaging both the bits the application needs and the metadata that describes how to run them. The image is built as a stack of read-only layers, one per Dockerfile instruction, each identified by a content hash that

lets Docker cache unchanged layers and registries share them across images. The base image fixes the OS, libc flavor, and runtime that everything else builds on, making it the single most consequential choice in a Dockerfile. Layer ordering — stable inputs first, volatile inputs last — determines whether the build cache delivers seconds or minutes. The image manifest is the JSON record that names the layers and their digests; pulls fetch only the layers a host is missing, which is what makes container distribution efficient enough to be practical at scale.

PART VII · CONTAINERS

CHAPTER 32

Dockerfiles and Multi-Stage Builds

Watch the presentation

Se presentationen på svenska

Shipping an application to production means shipping a unit of deployment that another machine can run identically to the developer's laptop. The Docker image fills that role, but an image is only as reproducible as the recipe that builds it. A Dockerfile is that recipe — a plain-text script that turns source code into an immutable image. The sections below develop the Dockerfile format, the build context that feeds it, the layer cache that makes rebuilds fast, and the multi-stage pattern that keeps production images small.

32.1 The Dockerfile as a build recipe

A **Dockerfile** is a plain-text file containing a sequence of instructions (`FROM`, `COPY`, `RUN`, `ENTRYPOINT`, etc.) that define how to build a Docker image; each instruction creates a new layer. Docker reads the file from top to bottom, executes each step against an intermediate container, and snapshots the resulting filesystem as a read-only layer. The final image is the stack of those layers plus a small JSON configuration that records the entrypoint, exposed ports, and environment.

The file is committed alongside source code, which means the build process itself is version-controlled. Anyone with the repository and a Docker daemon can produce the same image, byte for byte, without manually installing compilers or copying configuration. That reproducibility is the whole point: a container is only useful as a unit of deployment if the image behind it is deterministic.

32.1.1 Dockerfile instructions

An **instruction** in a Dockerfile is a command such as `FROM`, `COPY`, `RUN`, `ENV`, `EXPOSE`, or `ENTRYPOINT` that performs an operation (e.g., adding a file, running a shell command, setting an environment variable) and typically creates a new image layer. Each instruction sits on its own line, written as the instruction name in upper case followed by its arguments.

A working Dockerfile usually combines a small set of instructions:

- `FROM` declares the base image — the starting point for everything that follows. Every Dockerfile begins with `FROM`.

- `WORKDIR` sets the working directory inside the image. Subsequent `COPY` and `RUN` instructions resolve relative paths against this directory, and the container starts in it.
- `COPY` adds files from the build context into the image. `COPY src dst` copies the host path `src` to the image path `dst`.
- `RUN` executes a shell command during the build. `RUN dotnet restore` downloads NuGet packages; `RUN apt-get install -y curl` installs a system package. The result is captured as a new layer.
- `EXPOSE` documents which TCP port the application listens on. It does not actually publish the port — that happens at `docker run` time with `-p` — but it is metadata that tools and humans rely on.
- `ENV` sets an environment variable that persists in every container started from the image. `ENV ASPNETCORE_URLS=http://+:8080` is a common .NET pattern.
- `ARG` declares a build-time variable that can be passed with `docker build --build-arg`. Unlike `ENV`, `ARG` values do not survive into the running container — they exist only during the build.
- `ENTRYPOINT` and `CMD` define what the container runs when it starts. The two interact in a specific way unpacked under ENTRYPOINT versus CMD.

These instructions cover most real Dockerfiles. The full reference contains a few more (`USER`, `LABEL`, `HEALTHCHECK`, `VOLUME`), but the seven above carry almost all the weight in application images.

32.2 The build context and .dockerignore

When `docker build .` is invoked, the trailing `.` is not just a path — it is the **build context**. A **build context** is the directory (and its contents) passed to `docker build` from which the Dockerfile can `COPY` files into the image; it excludes files listed in `.dockerignore`, similar to `.gitignore`. The Docker CLI archives the entire context and ships it to the Docker daemon over a socket. Only files inside the context are visible to `COPY` instructions — a Dockerfile cannot reach above its context to grab a file from a parent directory.

This matters for two reasons. First, the context determines build speed: a 50 MB context uploads in a fraction of a second, while a 5 GB context (with a `node_modules` directory, a `.git` history, and a `bin/` folder full of compiled output) makes every build painful. Second, anything in the context can end up baked into the image by accident — secrets in a `.env` file, local credentials, cached test artifacts.

A **.dockerignore** file lists patterns of files to exclude from the build context, preventing unnecessary files (node_modules, .git, build artifacts) from being sent to the Docker daemon and reducing image size. The syntax mirrors `.gitignore`: one pattern per line, glob wildcards permitted, leading `!` to re-include. A reasonable starting point for a .NET project looks like this:

```
bin/
obj/
.git/
.vs/
*.user
appsettings.Development.json
**/.env
```

The first three lines exclude build output and version-control metadata that the build does not need. The last two protect against committing local developer secrets into a shared image. Treat `.dockerignore` as a default-deny list when secrets are involved — it is much easier to reason about which files the build *can* see than to chase down which files leaked.

32.3 The layer cache and instruction order

Each instruction produces a layer, and Docker fingerprints that layer based on the instruction text and the files it touched. The image cache reuses layers from previous builds when the instruction and its inputs have not changed; what matters here is how Dockerfile authoring interacts with it.

The cache invalidates on the first instruction whose inputs differ from the previous build. From that point down, every later instruction reruns. Order therefore decides whether a one-line code change triggers a 30-second rebuild or a 10-minute one.

The canonical example is dependency installation. Consider a Node.js project where `npm install` takes two minutes. A naive Dockerfile copies everything first:

```
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "server.js"]
```

Every time any source file changes, `COPY . .` invalidates, and `npm install` runs again. The fix is to copy the manifest first, install against it, and only then copy the rest:

```
FROM node:20
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install
COPY . .
CMD ["node", "server.js"]
```

Now `npm install` only reruns when `package.json` or `package-lock.json` change. Source edits invalidate the final `COPY . .` layer, which is fast. The same pattern applies to .NET (`COPY *.csproj` then `RUN dotnet restore`), Python (`COPY requirements.txt` then `RUN pip install`), and Go (`COPY go.mod go.sum` then `RUN go mod download`). The principle is the same: copy what changes rarely, install against it, then copy what changes often.

32.4 ENTRYPOINT versus CMD

The last load-bearing instruction defines what the container does when it runs. Two instructions cooperate here: **ENTRYPOINT** defines the main executable that always runs when a container starts, while **CMD** provides default arguments; `ENTRYPOINT ["app"]` and `CMD ["--help"]` combine so `docker run myimage` runs `app --help`, and `docker run myimage --version` runs `app --version`.

The mechanics matter. `ENTRYPOINT` is fixed: it is the executable that runs no matter what. `CMD` is a default argument list that can be overridden by passing arguments after the image name on `docker run`. If only `CMD` is set, the entire command can be replaced — `docker run myimage bash` drops into a shell. If `ENTRYPOINT` is set, `docker run myimage bash` passes `bash` as an argument to the entrypoint, which is usually not what was intended.

For application images — a web server, a worker, an API — `ENTRYPOINT` is almost always the right choice. The image has a single job, and the container should run that job. Pinning the entrypoint to the application binary prevents the container from accidentally being run as a shell when an operator types something wrong, and it makes the image self-documenting: `docker run myimage` does the obvious thing. Tooling images (a `.NET` SDK image, a `kubectl` image, a `psql` image) are the opposite case — they are meant to be invoked with arbitrary commands, and `CMD` or no entrypoint at all is more flexible.

Both forms accept either a JSON array (`["dotnet", "App.dll"]`) or a shell string (`dotnet App.dll`). Always prefer the array form. The shell form wraps the command in `/bin/sh -c`, which means the application does not receive Unix signals correctly — `SIGTERM` from `docker stop` goes to the shell, not to the application, and the shell ignores it until Docker times out and sends `SIGKILL` ten seconds later. The array form runs the application directly as PID 1, and signals reach it.

32.5 The multi-stage pattern

Building an application requires tools the running application does not need. A .NET service needs the SDK to compile but only the ASP.NET runtime to run. A Go service needs the compiler and its module cache during build but ships as a single static binary. A Java service needs Maven or Gradle and a JDK to build but only a JRE to serve traffic. Bundling the build tools into the production image bloats it (the .NET SDK alone adds about 700 MB) and widens the attack surface — every package manager, compiler, and shell utility in the image is a potential vulnerability.

A **multi-stage build** is a Dockerfile pattern using multiple `FROM` instructions; earlier stages can execute build-time tools (compilers, package managers) and later stages copy only the compiled artifacts, producing smaller final images without build tools. Each `FROM` starts a new stage with a clean filesystem; a later stage uses `COPY --from=<stage>` to pull files out of an earlier one. Only the final stage becomes the published image; the intermediate stages are discarded after the build completes.

32.5.1 Worked example: .NET multi-stage Dockerfile

The CloudSoft Recruitment exercise (see /exercises/20-docker/, specifically the third sub-exercise) refactors a single-stage .NET image into a two-stage build. The single-stage version produces a roughly 1 GB image carrying the SDK, the source tree, and the compiled output. The multi-stage version produces an image of around 200 MB containing only the runtime and the published artifacts.

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

COPY *.csproj ./
RUN dotnet restore

COPY . .
RUN dotnet publish -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app

COPY --from=build /app/publish .

EXPOSE 8080
ENV ASPNETCORE_URLS=http://+:8080
ENTRYPOINT ["dotnet", "CloudSoft.Recruitment.dll"]
```

The first stage is named `build` (via `AS build`) and uses the SDK image, which weighs in at over 800 MB but contains the C# compiler, MSBuild, and the NuGet client. It restores dependencies from the project file first — a cacheable step — then copies the source and publishes a release build into `/app/publish`. The second stage named `final` starts fresh from the much smaller `aspnet` runtime image, copies the publish output across with `COPY --from=build`, declares port 8080, and pins the entrypoint to the application DLL. The SDK, the source code, the NuGet cache, and the intermediate `bin` and `obj` directories never reach the final image — they live and die in the discarded `build` stage. The result is a production image that contains exactly what is needed to run the application: the .NET runtime and the published artifacts.

32.6 Summary

A Dockerfile is the version-controlled recipe that turns source code into a Docker image. Each instruction (`FROM`, `WORKDIR`, `COPY`, `RUN`, `EXPOSE`, `ENTRYPOINT`, `CMD`, `ARG`, `ENV`) creates a layer, and the build context is the directory of files those instructions can reach. A `.dockerignore` file keeps the context small and protects against leaking secrets or local artifacts into the image. Instruction order matters because the layer cache invalidates on the first changed step — copying dependency manifests before source code keeps `npm install` or `dotnet restore` cached across edits. `ENTRYPOINT` pins the executable a container runs and is the right default for application images; `CMD` supplies overridable arguments. The multi-stage pattern places build-time tooling in an early stage and copies only the compiled artifacts into a small runtime stage, producing images that are smaller, faster to pull, and carry a smaller attack surface.

PART VII · CONTAINERS

CHAPTER 33

Multi-Platform Builds

Watch the presentation

Se presentationen på svenska

A developer on an Apple Silicon laptop builds a container image, pushes it to a registry, and deploys it to an Azure VM running on Intel hardware. The deployment fails with `exec format error`. The image was compiled for ARM64; the VM runs AMD64. Single-platform images quietly assume the build host and the runtime host share an architecture, and that assumption breaks the moment a team mixes Apple Silicon with x86 cloud VMs, or a CI runner with a developer workstation. The sections below cover how container images

encode CPU architecture, why a single image cannot serve both architectures, and how `docker buildx` produces a single tag that resolves to the correct binary for whichever host pulls it.

33.1 What a platform means for an image

A Docker image is not a portable bundle of source code. It contains compiled binaries — system libraries, the application's runtime, and the application itself — that target a specific instruction set. A **platform** in container terms specifies the CPU architecture (amd64, arm64, arm/v7) for which an image is compiled; images compiled for one platform cannot run natively on a different architecture without emulation.

Two facts make platform a first-class concern:

- The CPU executes only the instruction set it was designed for. An ARM CPU cannot decode AMD64 opcodes, and an AMD64 CPU cannot decode ARM opcodes. There is no runtime translation layer in the kernel that papers over the difference.
- Container images bundle every binary the application depends on. Even if the source code is portable, the binaries inside the image are not. An nginx binary compiled for ARM64 is a different file from the nginx binary compiled for AMD64, with different machine code.

A platform identifier in Docker's notation has three components: `os/architecture/variant`. For server containers, the OS is almost always `linux`. The architecture is the load-bearing field — `amd64` (also called `x86_64`) and `arm64` (also called `aarch64`) cover the overwhelming majority of cases.

The optional variant distinguishes sub-versions of an architecture, most commonly for 32-bit ARM (`arm/v6`, `arm/v7`).

33.1.1 Why architecture mismatches surface late

A developer rarely sees the architecture of their host explicitly. The shell command `uname -m` reveals it: `x86_64` on most Intel and AMD machines, `arm64` on Apple Silicon Macs and Raspberry Pi 4. A `docker build` on either host produces an image targeted at the host's architecture by default, with no warning that the result is non-portable.

The mismatch surfaces only at runtime, on a machine with a different architecture, when the kernel rejects the binary with `exec format error` or `standard_init_linux.go: ... no such file or directory`. Both error messages mean the same thing: the kernel found the image's executable but cannot decode its instructions.

33.2 Buildx and the multi-architecture pipeline

The standard `docker build` command produces one image for one platform — the host's. **Buildx** is a Docker CLI extension that enables building images for multiple platforms and architectures; it uses BuildKit and QEMU emulation to cross-compile and push multi-platform manifests to registries.

Buildx ships with recent Docker Desktop installations and adds a parallel `docker buildx build` command. The relevant differences from plain `docker build`:

- `--platform linux/amd64,linux/arm64` accepts a comma-separated list of target platforms. Buildx executes the Dockerfile once per platform and produces one image per entry.

- `--push` uploads results directly to a registry. Multi-platform builds cannot terminate in the local image store, because that store has no way to represent multiple architectures under a single tag — only a registry can.
- The build runs inside a dedicated builder instance (created with `docker buildx create`), separate from the default Docker engine, and can use BuildKit features that the legacy builder cannot.

33.2.1 How QEMU enables cross-architecture builds

A laptop with an ARM64 CPU has no native way to execute AMD64 instructions. **QEMU** is a user-space emulator that can decode and execute foreign instruction sets, translating each AMD64 opcode to an equivalent sequence on the host CPU. Docker Desktop registers QEMU with the Linux kernel through the `binfmt_misc` mechanism, so the kernel transparently routes foreign binaries to QEMU when they are invoked.

The practical effect: a `RUN apt-get install nginx` instruction in a Dockerfile being built for `linux/amd64` on an ARM64 host runs the AMD64 `apt-get` binary under QEMU, which in turn invokes AMD64 versions of every helper it needs. The result is a fully AMD64 image filesystem, produced entirely on an ARM64 host.

The trade-off is speed. QEMU adds a translation cost to every instruction the foreign binary executes. A build that takes 30 seconds natively can take 5 to 10 minutes under emulation, and CPU-heavy steps (compiling C extensions, running test suites) suffer the most. For occasional builds this is acceptable. For frequent builds — every commit on a CI pipeline, for example — teams typically use **native builders**: a CI matrix with one runner per architecture, each building its native image, with a final step that combines the results into a

multi-platform tag. GitHub Actions and most other CI systems support this pattern, and it is the standard approach for production pipelines.

33.3 The manifest list as the indirection layer

If a single tag in a registry can refer to multiple architecture-specific images, the registry needs a way to record that mapping. The mechanism is the **manifest list** — registry metadata that maps a single image tag to multiple platform-specific image digests; when a user pulls an image, the Docker client consults the manifest list and downloads the correct image for their architecture.

A normal image in a registry has an image manifest — a JSON document listing the layers, their digests, and a configuration object describing the image. A multi-platform image has one manifest per architecture, plus an additional document — the manifest list — that points to each per-architecture manifest by digest and labels it with its platform.

Conceptually, the structure looks like this:

```
Tag: myimage:1.0
  -> Manifest list (digest sha256:aaaa...)
       - linux/amd64 -> Image manifest sha256:bbbb...
            - layers, config (AMD64 binaries)
       - linux/arm64 -> Image manifest sha256:cccc...
            - layers, config (ARM64 binaries)
```

When `docker pull myimage:1.0` runs on an ARM64 host, the Docker client first fetches the manifest list, scans it for an entry matching its own platform (`linux/arm64`), and then pulls the layers referenced by that entry's manifest. The AMD64 manifest and its layers are never downloaded on an ARM64 host. The same `myimage:1.0` command on an

AMD64 host pulls the AMD64 layers instead. The tag is identical; the bytes that arrive on disk differ.

33.3.1 How the registry stores the bytes

Registries store layers content-addressably: each layer is a blob identified by the SHA256 of its contents. If the AMD64 and ARM64 images share an identical layer — for example, a layer containing only platform-neutral configuration files — the registry stores it once and both manifests reference the same blob. Most layers, however, contain compiled binaries and therefore differ between architectures. A multi-platform image typically occupies close to the sum of its per-architecture sizes, with only modest deduplication.

Manifest lists are part of the OCI Image Specification (and the older Docker Image Manifest V2 Schema 2). Docker Hub, Azure Container Registry, GitHub Container Registry, and every other compliant registry support them without special configuration.

33.4 A worked example

The companion exercise Multi-Platform Builds walks through the full sequence; the core build command is short:

```
docker buildx create --name multiarch --use
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t username/myimage:1.0 \
  --push .
```

The first command creates a builder named `multiarch` and selects it as the active builder. This builder runs in a separate container and brings BuildKit and QEMU into the build path.

The second command builds the Dockerfile in the current directory (`.`) for both `linux/amd64` and `linux/arm64`, tags the result `username/myimage:1.0`, and pushes the resulting manifest list and per-architecture layers to the registry. On an ARM64 laptop, the AMD64 build runs under QEMU emulation and takes noticeably longer than the native ARM64 build; the two builds proceed in parallel where possible.

After the push, `docker buildx imagetools inspect username/myimage:1.0` prints the contents of the manifest list, including the digest and platform of each entry. A pull from any host — `docker pull username/myimage:1.0` — then downloads only the layers for that host's architecture, transparently.

33.5 Trade-offs and when multi-platform matters

Multi-platform builds add cost. Build time grows roughly linearly with the number of platforms when emulating, so a two-platform build takes roughly twice as long as a single-platform one. Image storage in the registry grows proportionally — two platforms occupy roughly twice the storage of one. Pulls remain single-architecture, so users do not pay a runtime cost.

APPROACH	BUILD COST	CACHE HIT RATE	WHEN TO USE
Single-platform (default)	Native speed	High	All teammates and all targets share one architecture
Buildx with QEMU emulation	Slow on emulated platforms	Medium	Occasional builds; small teams; first-time setup
Native CI matrix builders	Native speed per arch	High	Frequent builds; production pipelines

Multi-platform builds matter when the build host's architecture differs from any target host's architecture. The most common drivers:

- A team mixes Apple Silicon (arm64) developer laptops with cloud VMs that default to amd64.
- An application targets edge or IoT devices on ARM as well as servers on AMD64.
- A production deployment uses ARM-based cloud instances (such as AWS Graviton or Azure Ampere) for cost reasons while CI runs on amd64.

When every build host and every deployment target share an architecture, single-platform builds remain the simpler choice. Multi-platform tooling is the answer to architectural diversity, not a default to apply universally.

33.6 Summary

Container images embed compiled binaries that target a specific CPU architecture, so an image built for AMD64 will not

run on ARM64 and vice versa. The platform identifier `os/architecture/variant` makes this explicit, and `uname -m` on the host reveals which architecture local builds target by default. `docker buildx` is the multi-platform builder: it accepts a `--platform` list, executes the Dockerfile once per target, and uses QEMU to emulate non-native architectures during the build. The registry side of the story is the manifest list — a single tag points to a list of per-architecture image manifests, and the Docker client picks the right one at pull time. Buildx with QEMU is convenient but slow; production pipelines typically run native builders in a CI matrix and combine the results. Reach for multi-platform builds when the team's architectures diverge from the deployment target's, and stick with single-platform builds when they do not.

PART VII · CONTAINERS

CHAPTER 34

Docker Compose

Watch the presentation
Se presentationen på svenska

A real application is rarely a single container. A web application talks to a database, the database persists data to disk, and a queue or cache often sits between them. Running each piece by hand with `docker run` quickly becomes a sequence of long, fragile commands: a network has to be created first, the database needs a volume mount, the application needs the right environment variables and the correct hostname for the database. Compose collapses all of that into one declarative file. The sections below develop the compose model — services, named networks, volumes, environment variables, and

dependency ordering — and end with a worked example that mirrors the companion exercise at /exercises/20-docker/.

34.1 The problem Compose solves

A web application that uses MongoDB needs three things to run end-to-end on a developer laptop: the application process, the database process, and a network between them. Started by hand, the sequence is something like:

```
docker network create appnet
docker volume create db-data
docker run -d --name db --network appnet -v db-data:/data/
        db mongo:7
docker run -d --name web --network appnet -p 8080:8080 \
  -e MongoDB__ConnectionString=mongodb://db:27017 myapp:dev
```

Five commands, in the right order, with the right flags, every time the laptop reboots. A second developer joining the project has to learn that exact incantation, and any drift between developers becomes a “works on my machine” bug. **Docker Compose** is a tool that reads a `docker-compose.yml` file and orchestrates multi-container applications; it automates networking, volume management, and environment setup for local development and testing. The five commands above collapse into one file checked into the repository and one command (`docker compose up`) that any developer can run.

Compose targets the local development workflow. The same file can also drive integration tests in a CI pipeline and one-off staging environments, but Compose is not a production orchestrator. Production scaling, rolling updates, and self-healing belong to platforms covered later in this Course

Book — Azure Container Apps and Kubernetes are previewed at the end of this chapter and developed in Part VIII.

34.2 The compose file

A **compose file** (docker-compose.yml) is a YAML document that declares services, volumes, networks, and environment variables for a multi-container application; it serves as the source of truth for the application's topology. The file lives at the root of the repository, alongside the `Dockerfile` for the application. Compose accepts both `docker-compose.yml` and `compose.yaml` as the default name; the latter is the form recommended by the current Compose specification.

The file has four top-level keys that matter for most projects:

```
services:   # one entry per container
volumes:    # named volumes that outlive containers
networks:   # named networks the services share
```

`services` is mandatory; `volumes` and `networks` are optional. When `networks` is omitted, Compose creates a default network for the project automatically — every service on it can reach every other service by service name. That implicit network is the most important convenience Compose provides, and the worked example below leans on it.

A fourth top-level key, `version`, used to be required. Current Compose releases ignore it, and recent documentation removes it entirely. New files should not include a `version` line.

34.3 Services

A **service** (in Compose context) is a named, containerized application; multiple services communicate via a Docker Compose-managed network, each addressable by its service name as a hostname (e.g., the `web` service reaches `mongodb` at hostname `mongodb`). Each service entry under `services:` becomes one container when Compose runs the file. The service name is the key in the YAML map and doubles as the DNS hostname inside the project network.

A service has either a `build:` directive (Compose builds an image from a local Dockerfile) or an `image:` directive (Compose pulls a pre-built image from a registry), but not both meaningfully — `build` produces an image that `image` can name. The minimum useful service definition specifies one of those two, plus any ports, environment variables, or volume mounts the container needs:

```yaml
services:
  web:
    build: .
    ports:
      - "8080:8080"
  db:
    image: mongo:7
```

Two services, one built locally from the Dockerfile in the current directory, the other pulled from Docker Hub. Compose places both on the project's default network and assigns them DNS names `web` and `db`. Inside the `web` container, the hostname `db` resolves to the database container's IP — no `localhost`, no hardcoded address, no manual `--link`.

34.3.1 Ports and exposure

`ports` publishes a container port to the host. The form `"8080:8080"` maps host port 8080 to container port 8080, making the service reachable from a browser at `http://localhost:8080`. Service-to-service traffic does not need `ports` — services on the same network can talk to each other on any port the target container listens on, regardless of whether that port is published to the host. The database in the example above is reachable from the web container at `db:27017` even though no `ports` entry exposes 27017 to the laptop.

This distinction matters for security. Publishing the database port to the host is convenient for connecting a GUI client, but it also opens the database to anything else running on the developer machine. Leaving it unpublished keeps the database reachable only from the project network.

34.4 Named networks

A **named network** is a Docker network created and managed by Compose; services connected to the same named network can resolve each other by hostname via embedded DNS, enabling service-to-service communication without hard-coded IPs. Compose creates one named network per project automatically — its name is `<project>_default` where `<project>` is the directory name of the compose file. Every service joins that default network unless told otherwise.

For most projects, the default network is enough. Larger compositions sometimes split services across multiple networks to enforce isolation: a `frontend` network where the web service is reachable from the host, and a `backend` network where only the web and database services live. A compose file can declare networks explicitly:

```yaml
services:
  web:
    networks: [appnet]
  db:
    networks: [appnet]

networks:
  appnet:
```

Naming the network gives it a stable, predictable name (`<project>_appnet`) and documents the topology in the file. Beyond that, behavior is identical to the implicit default network.

34.4.1 DNS-based service discovery

Compose runs an embedded DNS server inside the project network. When the web container resolves the hostname `db`, the DNS server returns the database container's current IP address. If the database container is restarted and gets a new IP, the next DNS lookup returns the new address — service discovery survives container churn without any application change. This is what makes a connection string of the form `mongodb://db:27017` work without modification across every developer machine and every CI run.

34.5 Volumes

A **volume** in Docker is a directory managed by the Docker daemon (or host-mounted) that persists data beyond a container's lifetime; in Compose, `volumes: { data: {} }` declares a named volume and `service: volumes: [data:/path]` mounts it into the container. Containers are designed to be ephemeral — `docker compose down` removes them and any data written inside the container's writable layer disap-

pears with them. A database container without a volume loses every record on shutdown.

A named volume is a directory the Docker daemon manages on the host filesystem. It survives container removal, can be reattached to a new container at the same path, and is portable across container restarts. In Compose, declaring and mounting a named volume takes two lines:

```
services:
  db:
    image: mongo:7
    volumes:
      - db-data:/data/db

volumes:
  db-data:
```

The top-level `volumes:` key declares the volume; the service's `volumes:` list mounts it at `/data/db` inside the container, which is where MongoDB writes its data files. Bringing the stack down with `docker compose down` removes the containers but leaves the volume intact. Bringing it back up with `docker compose up` reattaches the volume and the database starts with the previous data still present. Removing the volume requires the explicit `docker compose down -v` flag.

Bind mounts are a different form: they map a host directory into the container directly. Bind mounts are appropriate during development for source code (so file edits on the host are visible inside the container) but inappropriate for database state, because they spread filesystem semantics and permissions across host and container in ways that can corrupt data files.

34.6 Environment variables and .env files

Containerized applications read configuration from environment variables rather than baked-in config files, which keeps the same image deployable across development, staging, and production. Compose supports environment variables in two complementary ways.

The `environment:` key on a service sets variables inside that container:

```
services:
  web:
    environment:
      - MongoDB__ConnectionString=mongodb://db:27017
      - ASPNETCORE_ENVIRONMENT=Development
```

The double-underscore syntax (`MongoDB__ConnectionString`) is the .NET configuration convention for nesting — the application reads this as `MongoDB:ConnectionString` in its configuration tree. The value is a connection string that uses the service hostname `db`, not `localhost`.

A `.env` file in the same directory as the compose file is read automatically; its values are interpolated into the compose file using `${VAR}` syntax:

```
services:
  db:
    image: mongo:7
    environment:
      - MONGO_INITDB_ROOT_USERNAME=${DB_USER}
      - MONGO_INITDB_ROOT_PASSWORD=${DB_PASSWORD}
```

The `.env` file is conventionally gitignored, so secrets stay out of the repository while a committed `.env.example` docu-

ments the expected variables. Compose substitutes the values at `up` time; the running container only sees the resolved environment.

34.7 Dependency ordering

A **dependency** in Compose (specified via `depends_on`) declares that one service should start after another; however, it does not guarantee the dependency is ready, so production-grade services use retry logic or readiness checks instead. A web service that connects to a database at startup needs the database container to exist first, but `depends_on` only guarantees the database container has been started — not that the database process inside it has finished initializing and is accepting connections.

The fully specified form combines `depends_on` with a `healthcheck` on the dependency:

```
services:
  db:
    image: mongo:7
    healthcheck:
      test: ["CMD", "mongosh", "--eval", "db.runCom-
        mand({ping:1})"]
      interval: 5s
      timeout: 3s
      retries: 10
  web:
    build: .
    depends_on:
      db:
        condition: service_healthy
```

With `condition: service_healthy`, Compose holds the web container's start until the database's healthcheck reports healthy. The healthcheck runs `mongosh` inside the database

container every five seconds and counts the database as healthy once the ping command succeeds. This is the closest Compose comes to true readiness ordering.

Even with healthchecks, applications should retry their initial database connection. A network glitch, a long-running migration, or a transient lock can fail the first connection attempt long after the container reports healthy. Retry logic in the application code (with backoff, capped at a sensible number of attempts) is the production-grade fix; `depends_on` is the development-time convenience.

34.8 Worked example: web and database

The companion exercise at /exercises/20-docker/4-docker-compose-local-development-stack/ builds a complete `compose.yaml` for an ASP.NET Core application backed by MongoDB. A condensed version captures the load-bearing pieces:

```yaml
services:
  web:
    build: .
    ports:
      - "8080:8080"
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - MongoDB__ConnectionString=mongodb://db:27017
      - FeatureFlags__UseMongoDB=true
    depends_on:
      db:
        condition: service_healthy
    networks:
      - appnet

  db:
    image: mongo:7
    volumes:
      - db-data:/data/db
    healthcheck:
      test: ["CMD", "mongosh", "--eval", "db.runCom-
        mand({ping:1})"]
      interval: 5s
      timeout: 3s
      retries: 10
    networks:
      - appnet

volumes:
  db-data:

networks:
  appnet:
```

Reading top-down, the file declares two services on a shared named network `appnet`. The `web` service builds from the local Dockerfile, publishes port 8080 to the host, and reads its database connection string from an environment variable that uses the hostname `db` — Compose's embedded DNS resolves that hostname to the database container's IP.

The `web` service waits for the `db` healthcheck to pass before starting. The `db` service mounts a named volume `db-data` at MongoDB's data directory, so records persist across `docker compose down` and `docker compose up` cycles. A feature flag `FeatureFlags__UseMongoDB=true` toggles the application's use of MongoDB without code change, demonstrating how environment variables drive runtime behavior.

Running `docker compose up --build` from the repository root builds the web image, pulls the MongoDB image, creates the `appnet` network and `db-data` volume, starts both containers in dependency order, and streams their logs to the terminal. `docker compose down` stops and removes the containers and the network but leaves the volume; `docker compose down -v` removes the volume too.

34.9 Compose and production orchestration

Compose is a local-development tool. It runs containers on a single host (the developer laptop or a CI runner) with no horizontal scaling, no rolling updates, and no self-healing across multiple machines. Production environments need more.

TOOL	SCOPE	USE CASE
Docker Compose	Single host	Local dev, integration tests, simple staging
Azure Container Apps	Managed multi-host	Production for most web/API workloads
Kubernetes	Multi-host cluster	Production at scale, complex topologies

The compose model — services, networks, volumes, environment variables, dependency ordering — translates conceptually to both production targets, even though the file format

and runtime are different. A service in Compose maps to a Container App in Azure Container Apps and to a Deployment + Service in Kubernetes. A named volume maps to managed storage. The environment variables move into a configuration store or secret manager. Part VIII of this Course Book develops the production side of this story; the skills built here in Compose carry over directly.

34.10 Summary

Docker Compose is the local-development orchestrator for multi-container applications: a single YAML file declares services, networks, volumes, and environment variables, and a single command brings them all up. Each service becomes a container on a Compose-managed network where service names resolve as hostnames via embedded DNS, eliminating hardcoded addresses. Named volumes persist state that must outlive containers, while environment variables and `.env` files drive configuration without rebuilding images. `depends_on` with a `healthcheck` orders startup, but applications should still retry their initial connections. The compose file is checked into the repository, becomes the source of truth for the application's local topology, and forward-translates conceptually to Azure Container Apps and Kubernetes for production.

CHAPTER 35

Container Registries

Watch the presentation
Se presentationen på svenska

A built image is only useful if the machines that need to run it can get hold of it. Local builds live in the daemon's storage on a single laptop; deployment targets — a teammate's machine, a CI runner, a production cluster — never see them. Distributing an image therefore needs a server that can store images and serve them to anyone with the right credentials. That server is a container registry, and the operations a registry supports — naming images, addressing them by tag or by digest, controlling who can push and who can pull — shape day-to-day container workflows from local development through production deployment.

35.1 What a registry is

A **container registry** is a server that stores and distributes Docker images; Docker Hub is public and free, while Azure Container Registry (ACR) is private and integrated with Azure identity for enterprise teams. Both implement the same underlying protocol — the OCI Distribution Specification — so the Docker CLI, the BuildKit toolchain, and orchestrators like Kubernetes interact with any compliant registry through the same commands.

A registry is best understood as a content-addressable store. Each image layer is an immutable blob identified by a SHA256 hash; an image manifest lists the layer hashes plus configuration; the image as a whole is identified by the hash of that manifest. The registry exposes a small REST API: clients ask for a manifest, then pull each blob the manifest references, then assemble them locally into a usable image. Pushing reverses the flow — the client uploads any blobs the registry does not already have, then uploads the manifest last. Because blobs are content-addressed, the registry can deduplicate them across images: two images that share a Debian base image store that base image's layers exactly once.

35.1.1 Repositories and the reference syntax

Within a registry, images are organized into **repositories**. A repository holds many versions of the same logical image — typically the build history of one application. The fully qualified reference to an image takes the form `host/repository:tag`, for example `mycr.azurecr.io/cloudsoft/recruitment:1.0`. The host segment identifies the registry; the repository segment names the application (and optionally a project or team prefix); the tag identifies the version.

When the host is omitted, the Docker client assumes Docker Hub: `docker pull nginx:latest` is shorthand for `docker pull docker.io/library/nginx:latest`. This implicit default makes Docker Hub feel native to the toolchain, but any registry can be addressed by including its hostname in the reference. A reference can also use a digest in place of a tag, written `repository@sha256:...`; the use of digests is covered below.

35.2 Tags versus digests

An **image tag** is a human-readable label for a version of an image (e.g., `latest`, `v1.0`, `main`); tags are mutable, meaning the same tag can be reassigned, unlike image digests which are immutable. A tag is just a pointer that the registry maintains in a small piece of metadata: it can be moved to point at a different image at any time, and rebuilding an image with the same tag and re-pushing it overwrites the old pointer.

An **image digest** is a SHA256 hash of an image's manifest; it uniquely and immutably identifies an image (e.g., `sha256:abc123...`) and remains the same even if the tag is reassigned. Because the digest is computed from the manifest's bytes, two different images can never share a digest, and any change — a single byte in any layer — produces a different digest.

This distinction matters for production deployments. A workflow that pulls `myapp:latest` is implicitly trusting that the tag has not been moved between the moment the image was tested and the moment it is pulled into production. If a teammate pushes a new build to `:latest` between those moments, the deployment runs an untested image. Pinning the deployment to a digest — `myapp@sha256:9f86d0...` —

eliminates that race entirely, because the digest can only refer to the exact bytes that were tested. CI pipelines therefore tend to record the digest produced by the build step and pass that digest to the deploy step, while still publishing a friendly tag for humans to read. A common convention is to push both `:1.0` and `:latest` for every release, and to deploy by digest while documenting the corresponding tag for traceability.

35.3 Push and pull

Push is the operation to upload a locally built image to a container registry; `docker push username/myimage:latest` sends the image and its layers to the registry, making it available for others to pull. The client first checks which layer blobs the registry already has — if a base image like `mcr.microsoft.com/dotnet/aspnet` is already present, those layers are skipped — and uploads only the new ones, then finally writes the manifest under the requested tag.

Pull is the reverse: the operation to download an image from a container registry to the local machine; `docker pull username/myimage:latest` fetches the image manifest and layers from the registry for local use. The client retrieves the manifest, downloads any layers it does not already have cached, and reconstructs the image. Both push and pull are resumable at the layer level: a connection drop in the middle of a 500 MB layer means that one layer must restart, but layers already transferred are not retransmitted.

35.3.1 Worked example

Pushing a locally built application to a private Azure Container Registry uses the same two-step pattern regardless of

registry: tag the image with the registry's hostname, then push.

```
docker tag myapp:local mycr.azurecr.io/myapp:1.0
docker push mycr.azurecr.io/myapp:1.0
```

The `docker tag` command does not copy or rebuild the image — it adds a second name to the same image bytes already in the local daemon. The new name happens to include the registry hostname, which tells `docker push` where to send the upload. The push then streams the layers up to `mycr.azurecr.io`, skipping any blobs the registry already has. Once it completes, any other machine authenticated to that registry can run `docker pull mycr.azurecr.io/myapp:1.0` and receive the same bytes. The companion exercise Build and Push Your First Image walks through this flow against Docker Hub, where the registry hostname is implicit.

35.4 Public and private registries

Registries split broadly into public and private along two axes: who can pull (read access) and who can push (write access).

Docker Hub is the default public container registry hosted by Docker; it stores millions of images (official images like nginx and ubuntu, community images, and user-pushed images), and anyone can pull images without authentication. It is the place where a Dockerfile's `FROM nginx:alpine` ultimately resolves. Free accounts can also push their own images, which become publicly readable by default. Docker Hub sits at the centre of the open-source container ecosys-

tem, and many teams use it as their default registry because everything else does.

The pitfall to know about is rate limiting. Anonymous pulls from Docker Hub are throttled per source IP — at the time of writing, 100 pulls per six hours — and authenticated free accounts are throttled at 200 per six hours. A CI pipeline that pulls `node:20` for every build, run from a shared cloud runner pool that shares an outbound IP with thousands of other tenants, can hit the limit unpredictably. The fix is either to authenticate the CI runner against Docker Hub with a paid plan, to mirror the upstream image into a private registry the team controls, or — increasingly common — to consume official images from a registry like Microsoft's MCR (`mcr.microsoft.com/dotnet/aspnet`) which does not throttle.

Azure Container Registry (ACR) is Microsoft's managed private container registry; it stores images securely, controls access via Azure identity, integrates with Azure Container Apps and CI/CD pipelines, and supports geo-replication for global availability. An ACR instance has a hostname like `myregistry.azurecr.io` and lives inside an Azure subscription, billed by the storage and bandwidth it uses. Images pushed to ACR are private by default — pulling requires authentication — which makes ACR the natural choice for proprietary application images that should not be world-readable.

A useful default for an Azure-hosted system is to push every application image to ACR, even if the team also uses Docker Hub for shared base images. ACR's rate limits are governed by the registry's chosen pricing tier, not by anonymous-IP throttling, so production pulls remain predictable. Geo-replication can mirror an image automatically across

Azure regions, so a workload in a different region pulls from a local replica rather than across the public internet.

35.4.1 Choosing a registry

REGISTRY	VISIBILITY	PULL AUTHENTICATION	COMMON USE
Docker Hub (public)	World-readable	None for public images	Open-source distribution, base images
Docker Hub (private repo, paid)	Authenticated	PAT or username/password	Small teams, mixed open-source workflow
Azure Container Registry	Private	Azure identity	Production deployments on Azure
MCR (Microsoft Container Registry)	Public, no rate limit	None	Pulling Microsoft official images (.NET, SQL Server)

The choice is rarely binary. Most production setups consume base images from Docker Hub or MCR (because that is where the upstream publishers push them) and push application images to a private registry like ACR (because that is where deployment targets pull from). The Dockerfile's `FROM`

line and the CI pipeline's `docker push` target are independent decisions.

35.5 Authenticating to a registry

Every push requires authentication, and every private pull does too. The registry never stores plaintext credentials — it accepts a credential, mints a short-lived bearer token, and uses that token for the actual blob and manifest operations — but the entry point that produces the credential is what teams have to configure.

For Docker Hub, the entry point is a username and a Personal Access Token (PAT). The PAT is generated in the Docker Hub web UI with a chosen scope (read-only, read-write, or admin) and an optional expiration. The CLI command `docker login -u <username>` prompts for the PAT and stores a credential reference in `~/.docker/config.json`. Treating a PAT like a password is essential: it is a secret, it should never appear in a repository, and it should be revoked when an engineer leaves the team.

For ACR, several authentication paths exist, and the right one depends on what is doing the authenticating. For interactive use from a developer laptop, `az acr login --name myregistry` exchanges the developer's Azure login for a short-lived ACR token and configures the Docker CLI automatically — no long-lived secret is stored. For an Azure compute resource pulling an image — a Container App, a VM, an AKS node pool — the right path is **managed identity**: the resource has its own Azure identity, the registry grants that identity the `AcrPull` role, and the platform handles credential rotation transparently with no secret involved. Managed identities are explained in detail under Identity and Security. For a CI pipeline running outside Azure, the preferred path is

OIDC federation: the pipeline presents a short-lived OIDC token issued by the CI provider, ACR validates it against a configured trust relationship, and grants access without a stored secret. The Part VIII chapters on CI/CD cover OIDC for registry login in depth.

The pattern across all three is the same: avoid long-lived secrets where possible, prefer identity-based authentication, and when an API key or PAT is unavoidable, store it in a secret manager rather than in source control or a `.env` file.

35.5.1 Registry security scans

Most managed registries run a vulnerability scanner against pushed images, comparing each layer's installed packages against public CVE databases. ACR integrates with Microsoft Defender for Cloud to surface findings; Docker Hub offers Snyk-powered scanning on paid plans; standalone scanners like Trivy can run against any registry as part of a CI pipeline. The output is a list of CVEs by severity, attached to a specific image digest. Security scanning does not replace careful base-image selection — a vulnerability discovered after the scan still ships — but it raises the floor by catching obviously stale images before they reach production. A common policy is to fail a deployment if the image carries unfixed Critical CVEs and to warn but proceed for High or below, with the rules tuned to the team's risk tolerance.

35.6 Summary

A container registry is a content-addressable server that stores image blobs and manifests, organizing them into repositories addressed as `host/repo:tag`. Tags are mutable pointers that humans use to refer to versions; digests are immutable SHA256 hashes that uniquely identify exact image

bytes, and production deployments should pin to digests to avoid running an image other than the one that was tested. Docker Hub is the default public registry and the home of most upstream open-source images, but its anonymous rate limits make it a poor primary registry for CI pipelines; Azure Container Registry is the private counterpart for Azure-hosted systems, authenticated via Azure identity rather than a stored secret. Push and pull move layers in and out of the registry, deduplicated by content hash so that shared base layers transfer only once. Authentication ranges from PATs (Docker Hub) through managed identities (ACR from Azure compute) to OIDC federation (ACR from external CI), with the trend across all paths being to eliminate long-lived stored secrets.

PART VIII

DevOps and Delivery

The Part that closes the loop between writing code and running it. Seven chapters cover the philosophy that motivates DevOps, the practices it relies on (CI, CD, pipelines as code, gated tests), the operational choices that make releases safe (deployment strategies, federated pipeline identity), and one concrete deployment target — Azure Container Apps — that ties the previous Parts together.

The companion exercise is the CI/CD to Container Apps chapter, three exercises that progress from public registry + manual deploy through private registry + automated deploy to passwordless OIDC federation.

CHAPTER 36

The DevOps Philosophy

Watch the presentation
Se presentationen på svenska

Software organizations historically split the work of building applications and the work of running them between two separate groups. Developers wrote features and threw releases over a wall to operations staff, who installed, monitored, and patched them. The handoff itself became the most expensive step in delivery: each release required coordination meetings, change tickets, and late-night cutover windows, and when something broke the two groups blamed each other instead of fixing the system. This chapter intro-

duces the philosophy that replaced that arrangement, the metrics that tell an organization whether it has succeeded, and the rest of Part VIII previews how pipelines, gates, and deployment strategies put the philosophy into practice.

36.1 The legacy split and its handoff cost

For most of the 1990s and 2000s, a typical enterprise release looked the same. A development team finished a quarterly release, packaged the binaries, and produced a deployment runbook. An operations team scheduled a maintenance window, often on a weekend, and worked through the runbook step by step. If the deployment failed, operations rolled back to the previous version and filed a ticket. Developers learned about the failure on Monday morning, debugged what they could from log fragments, and queued a fix for the next quarterly release.

The cost of this arrangement was hidden in two places. The first was the size of each release. Because deployments were rare and risky, every release accumulated months of changes. A failure in production could trace back to any one of dozens of merged feature branches, and isolating the cause took days. The second was the incentive structure. Operations staff were measured on uptime, so they resisted change. Developers were measured on feature velocity, so they pushed for change. The two groups optimized for opposing goals, and the system as a whole optimized for neither.

This pattern produced predictable failure modes. Production incidents took hours or days to resolve because the people who understood the code were not the people with access to the servers. Releases were postponed because operations could not absorb the risk on top of normal maintenance load. Hotfixes required heroics — pulling engineers out

of meetings, opening emergency change tickets, and skipping the test suites that the regular pipeline would have run. None of these failures came from individual incompetence; they came from a structure that placed a wall between writing code and running it.

36.2 What DevOps changed

DevOps is a cultural and technical movement that bridges development and operations teams; it emphasizes automation, measurement, and sharing to shorten the feedback loop between writing code and running it in production, enabling faster, safer releases. The label became common after 2009, when conference talks and books started naming the pattern that high-performing internet companies had quietly adopted. The cultural shift came first, the tooling second, and the metrics last.

The cultural shift centred on shared ownership. A team that builds a service also runs it: it is on call for production incidents, it sees the operational metrics every day, and it has both the authority and the responsibility to change the system when something hurts. Shared ownership eliminates the handoff because there is nothing to hand off — the same people sign the commit and watch the deploy.

A second cultural change is the **blameless post-mortem**. When an incident happens, the investigation focuses on what the system allowed to go wrong, not on which engineer typed the wrong command. The goal is to find the missing guardrail — the pre-flight check, the automated rollback, the alarm that should have fired earlier — and add it to the system. Blame discourages reporting, and unreported incidents cannot be learned from, so a culture that punishes individual mistakes ends up with worse outages, not fewer.

A third cultural change is **automation as the default**. Manual steps in a release are treated as defects to be fixed, not as features to be preserved. Anything done twice is scripted; anything scripted is committed; anything committed runs in a pipeline. The point is not to remove humans from the loop but to remove humans from the boring, error-prone steps so that they can spend attention on the steps where judgment actually matters.

These cultural changes are what make the tooling work. A pipeline without shared ownership becomes a wall in YAML form, and a metric without a blameless culture becomes a stick to beat individuals with. The order matters: culture, then tooling, then measurement.

36.3 The four DORA metrics

The DevOps Research and Assessment program, run by Nicole Forsgren, Jez Humble, and Gene Kim and now part of Google Cloud, measured thousands of teams over multiple years and found that four metrics together separate high-performing organizations from low-performing ones. Two metrics measure throughput — how fast value moves to production — and two measure stability — how reliably it lands.

Lead time is the elapsed time from when a developer commits a code change to when that change is live in production; it measures how long the organization takes to deliver value and is a key metric for DevOps maturity. Lead time captures everything between `git push` and a user seeing the change: code review, build, test, packaging, deployment, gate approvals. A team with hours of lead time can fix a critical bug the same morning it is reported. A team with weeks of lead time cannot, no matter how skilled its engineers are.

Deployment frequency is how often code changes reach production (e.g., once per week, multiple times per day); high deployment frequency correlates with lower lead time and lower change-failure rate, indicating a mature CI/CD capability. Deployment frequency is the easier metric to game, because a team can technically deploy daily by deploying nothing of substance, but in practice the metric is honest because nobody runs an empty pipeline. Frequent deployments force every supporting practice — fast tests, automated rollback, small batches, feature flags — to work, because nothing else can survive that cadence.

Mean time to recovery (MTTR) is the average time required to restore service after a production incident; organizations that automate deployment and testing can revert or fix problems faster, reducing MTTR and limiting business impact. MTTR matters because deployment frequency and lead time both push more change into production, and more change means more incidents. The DORA research found, counter-intuitively, that the same teams that deploy most often also recover fastest — not despite the change rate but because of it. Teams that practise deploying every day have practised rolling back every day; teams that deploy once a quarter have not rolled back since the previous quarter.

Change-failure rate is the percentage of deployments that result in a production incident or rollback; a low change-failure rate indicates reliable releases and reflects the maturity of testing, code review, and deployment automation practices. Change-failure rate disciplines the throughput metrics: a team can drive deployment frequency to the moon by skipping tests, but the change-failure rate will rise to meet it. The four metrics are read together, never in isolation.

The DORA reports group teams into bands — “low,” “medium,” “high,” and “elite” — and the gap between the

bands is wide. Elite teams measure lead time in hours and MTTR in minutes; low-performing teams measure both in months. The metrics are not vanity numbers; they predict business outcomes, including time-to-market and the team's own retention rate, because a team trapped in a slow release cycle is also a team that loses its best engineers.

36.4 The value stream and where waste hides

A **value stream** is the sequence of steps (from idea to running code) that an organization must complete to deliver value to users; DevOps practices aim to shorten and optimize this stream by removing bottlenecks and automating manual work. Mapping the value stream is a diagnostic exercise: list every step from a feature idea entering a backlog to the resulting code running for users, mark how long each step typically takes, and mark how much of that time is the step itself versus waiting for the next step to start.

Waiting time dominates the typical value stream. A code change might compile in 30 seconds, but wait 18 hours for a code reviewer to be available. A merged pull request might pass tests in 4 minutes, but wait two days for a release manager to schedule a deployment window. A failed deployment might be fixable in 10 minutes, but the rollback procedure requires a change ticket that takes 2 hours to approve. Each of these waits is invisible in any single engineer's day but ruinous when summed.

Two patterns expose waste reliably. First, **batch size**: large batches hide problems and slow feedback. A release that bundles 200 commits is harder to test, harder to diagnose, and harder to roll back than 200 releases of one commit each, even though the total work is the same. Second, **manual handoffs**: every place where one human waits for another human

creates queue time. The fix is automation that closes the handoff, not heroics that work through it.

Optimizing the value stream is what the rest of Part VIII is about. The next chapters cover continuous integration and deployment as the practices that compress the build-test-deploy waiting times, pipelines as code as the artefact that captures the optimization in version control, and deployment strategies as the techniques that reduce the risk per release so that the gates between commit and production can be removed without raising change-failure rate.

36.5 Worked example: a team's two-year arc

Consider a small team running an e-commerce checkout service. At the start of the period, the team's metrics looked like this:

- **Deployment frequency**: once per quarter, in a Saturday-night maintenance window.
- **Lead time**: 11 weeks from commit to production.
- **MTTR**: 8 hours, because incidents required pulling the on-call developer out of bed, getting them VPN access, and walking them through the operations runbook.
- **Change-failure rate**: 30 percent — roughly one in three releases caused a customer-visible incident in the first 24 hours.

Two years later, the same team measured:

- **Deployment frequency**: 4–6 times per day, on demand, no maintenance window.
- **Lead time**: 90 minutes from commit to production.
- **MTTR**: 12 minutes, because the on-call engineer can roll back to the previous container revision from a phone.

- **Change-failure rate**: 5 percent — and the failures that do happen are caught by the smoke test before they reach customers.

What changed culturally and in the tool chain is worth listing side by side, because neither half works alone. Culturally, the team adopted shared ownership of the production service, ran blameless post-mortems after every incident, and shifted on-call from "the operations team" to "whichever developer is on rotation." Tool-chain-wise, the team built a GitHub Actions pipeline that ran tests on every pull request, packaged the application as a container, and deployed it to Azure Container Apps via a smoke-tested workflow. A failed smoke test rolled back automatically by directing all ingress traffic to the previous revision.

The cultural changes made the tools usable. Without shared ownership, no developer would have agreed to be on call for the pipeline they wrote. Without blameless post-mortems, the team would have stopped deploying on Friday afternoons after the first incident, and lead time would have crept back up. The tool changes made the culture sustainable. Without an automated rollback, the on-call engineer would have dreaded every page; without smoke tests, every deploy would have required the on-call engineer to watch for an hour.

The exercise /exercises/3-deployment/9-cicd-to-container-apps/ walks through the same arc in miniature: the first sub-exercise has a pipeline that builds and pushes an image but leaves deployment manual, the second closes the gap with an automated, smoke-tested deploy, and the third replaces the long-lived secret with OIDC federation. Each step takes one cell of the worked-example table from the "before" to the "after" column.

36.6 How the rest of Part VIII operationalizes the philosophy

The remaining chapters in this Part are not separate topics. Each one operationalizes a part of the philosophy.

The chapter on continuous integration and continuous deployment defines the two practices that drive lead time and deployment frequency down. Pipelines as code is where the practices live in version control, so that the pipeline itself can be reviewed, tested, and rolled back. Build, test, and smoke gates are the mechanisms that hold change-failure rate down even as deployment frequency rises. Deployment strategies — blue-green, canary, rolling — are the techniques that reduce the blast radius of any single release, which is what allows MTTR to stay low even when releases accelerate. Pipeline secrets and OIDC federation are the security primitives that make pipelines safe to run with real production credentials. Azure Container Apps is the deployment target this Part uses because it makes revisions, ingress, and rollback first-class concepts that the pipeline can drive directly.

A reader who has internalized this chapter should expect each later chapter to answer one specific question: which of the four DORA metrics the practice moves, and at what cost.

36.7 Summary

DevOps replaces the historical wall between development and operations with shared ownership, blameless post-mortems, and automation as the default. The four DORA metrics — lead time, deployment frequency, mean time to recovery, and change-failure rate — measure whether the philosophy has actually taken hold, with the first two measuring throughput and the second two measuring stability. The value stream is the diagnostic tool that exposes where waste hides, and most

of the waste is queue time between manual handoffs rather than the work itself. The rest of Part VIII operationalizes the philosophy through CI, CD, pipelines as code, gates, deployment strategies, secrets management, and a managed deployment target. The arc from a team that deploys once a quarter to one that deploys several times a day is achievable, but only when culture, tooling, and measurement move together.

CHAPTER 37

Continuous Integration vs Continuous Deployment

Watch the presentation
Se presentationen på svenska

The acronym “CI/CD” hides three distinct practices that arrived at different points in software history and solve different problems. Continuous integration addresses the pain of merging long-lived branches. Continuous delivery addresses the pain of release-day rituals. Continuous deployment goes

further still and removes the human approval entirely. Treating them as one undifferentiated blob makes it impossible to reason about what a team should adopt next, because each step assumes the previous one is in place.

This chapter separates the three. It also names the branching model and the human checkpoint that have to be present for any of them to work in practice — without those, the pipeline becomes theatre rather than engineering.

37.1 The pain that motivated continuous integration

Consider a team where each developer works on a feature branch for two or three weeks before merging back to `main`. By the time the merge happens, the trunk has moved on. Files have been renamed, function signatures have changed, dependencies have been upgraded. The merge surfaces dozens of conflicts at once, and the integration takes a full day of fragile manual work. Worse, the branches do not just collide on text — they collide on assumptions. Two developers may have independently written the same helper function, or relied on database fields that the other one removed.

This pattern was the norm in the late 1990s, and it produced a predictable failure mode: integration risk grew nonlinearly with branch age. A one-week branch was annoying to merge. A four-week branch was an event. A six-month branch became a separate product that no longer fit the trunk at all.

Continuous integration (CI) is a practice where developers integrate code changes into a shared repository frequently (multiple times per day); each integration is automatically built, tested, and verified to catch integration errors early. The word “continuous” is doing real work here — it is not a synonym for “automated.” A team that runs builds on a

nightly cron is not doing CI. A team that integrates each developer's work hourly, automatically, and rejects integrations that break the build is doing CI.

The motivating insight is that integration pain compounds. Three small daily merges are dramatically easier than one merge holding three days of divergent work, even though the line counts are identical. CI forces the small daily merges and pays down the integration cost continuously instead of letting it accumulate.

37.2 What CI actually requires

CI is often described as "running tests on every push," but that description omits the parts that make it work. Three properties have to hold at the same time:

1. **Every commit triggers a build and test run.** Not just the commits a developer chooses to test — every commit, including the small ones, including the ones that "just" change a comment. The pipeline runs unconditionally. This is what makes CI a property of the repository rather than a habit of the developer.
2. **Every commit eventually reaches a shared trunk.** A pipeline that runs on a feature branch for six weeks before the branch merges has not actually integrated anything. The integration happens at merge time, and a passing CI on a stale branch tells you very little about whether the merge will succeed.
3. **A broken build blocks everyone.** If `main` is red, no further work can merge until it is green again. The team treats a broken build as the highest-priority interruption — not because of moralism, but because every additional commit on top of a broken trunk makes the eventual fix harder.

Property three is what most teams get wrong. It is comfortable to merge "just one more thing" while the build is broken, on the grounds that it is unrelated. The cost of doing so is that the next failure is now ambiguous — it could be the original break, the new commit, or an interaction. Discipline around the red build is the difference between CI as a practice and CI as a job that happens to run.

37.3 The spectrum: integration, delivery, deployment

The three practices form a spectrum, each one strictly building on the previous. Skipping a step does not work.

37.3.1 Continuous integration

The trunk is always buildable. Tests pass on `main`. Any developer can pull `main` at any time and have a working starting point. The pipeline runs on every commit. Whether the artifact gets deployed anywhere is a separate question — at this stage, deployment can still be a quarterly ritual handled by an operations team.

37.3.2 Continuous delivery

Continuous delivery is a practice where code changes are automatically built, tested, and packaged so they are always in a releasable state; deployment to production is automated but gated by a manual approval step. The artifact is ready for production at all times. The pipeline could deploy it within minutes, but a human chooses when. The gate exists for non-technical reasons: a marketing launch window, a compliance review, a customer communication that must precede the change.

The crucial property is that the readiness is real. A team practicing continuous delivery should be able to point at any green commit on `main` and say "this could be in production by lunch." If the path from green build to production involves a two-week QA cycle, the team is not yet doing continuous delivery — they are doing CI plus a heavy release process.

37.3.3 Continuous deployment

Continuous deployment is a practice where code changes that pass automated tests are automatically deployed to production without manual gates; every commit to the main branch can reach users. The human gate is gone. A green merge to `main` reaches production within minutes, automatically, without anyone clicking a button.

This requires a higher bar of automated quality assurance than continuous delivery, because no human will catch the problem the tests miss. It also requires deployment strategies that limit the blast radius of a bad change — feature flags, canary releases, fast rollback paths. Continuous deployment without those safety mechanisms is reckless rather than mature.

The progression matters. A team that adopts continuous deployment without first establishing continuous integration is automating the delivery of unreliable software at high frequency, which is worse than the slow manual process they replaced.

37.4 Trunk-based development

A branching model that fits CI must allow daily integration. The model that does so is **trunk-based development** — a branching strategy where developers work on short-lived feature branches that are frequently merged to the main branch

(the "trunk"); this enables frequent integration and reduces merge conflicts and slow feedback loops.

"Short-lived" usually means hours to two days. A branch that lives a week is already drifting. The expectation is that a developer cuts a branch from `main`, makes a focused change, opens a pull request, gets it merged, and deletes the branch — all within the same working day if possible.

Two patterns make this practical for changes that are too large to ship in a single small commit:

- **Vertical slicing.** A feature is decomposed into independently mergeable slices — a database migration first, then a backend endpoint that uses it, then a frontend that calls the endpoint. Each slice merges separately, and the trunk is never inconsistent.
- **Feature flags.** A larger change is merged in pieces hidden behind a runtime configuration switch. The code is in `main` and going through CI on every commit, but users do not see it until the flag is enabled. This separates the act of deploying from the act of releasing — two things that the older release-day model conflated.

The contrast is **GitFlow** — a model with long-lived `develop`, `release`, and `hotfix` branches. GitFlow encodes the assumption that releases are events. Trunk-based development encodes the opposite assumption: that releases are continuous and the trunk is always production-shaped.

37.5 The pull-request gate

Continuous integration on the trunk does not mean the trunk has no quality bar. The bar moves from "release day" to the moment of merge. The mechanism is the **pull-request gate** — an automated check (build, linting, tests) that must pass

before a code review can approve and merge a pull request; it prevents broken or low-quality code from reaching the trunk.

The pull-request gate is the human checkpoint in an otherwise automated flow. It runs on every PR and typically includes:

- A successful build (compilation, type checks).
- All unit tests passing.
- Static analysis (linting, security scans) reporting clean.
- At least one peer reviewer's approval.

The gate is not the same as a deployment gate. A deployment gate decides whether tested code reaches production; a PR gate decides whether code reaches the trunk at all. Conflating them is a common mistake — teams that put their main quality controls in a "pre-prod" gate after the merge end up with a red trunk, because the merge has already happened by the time anyone notices.

A well-tuned PR gate finishes in minutes. If it takes 45 minutes, developers will start batching up several days of work into a single PR to avoid waiting for the pipeline three times — which destroys the property that made CI work in the first place. Pipeline speed is therefore not a vanity metric; it is a precondition for trunk-based development.

37.6 A worked example

Two teams ship the same feature: adding a "favourite" button to a product list.

Team A — long-lived feature branch. A developer cuts `feature/favourites` from `main`. The branch lives for nine days. Over that period, three other developers merge unrelated changes to `main` — a refactor of the data access layer, an upgrade of the ORM, and a new authentication

helper. By the time the favourites branch is ready to merge, the data access calls in the branch use the old API, the ORM upgrade has changed how queries are written, and the new authentication helper would have replaced 40 lines of boilerplate that the branch still contains. The merge takes most of a day. Two integration bugs surface in QA the following week. The change reaches production 14 days after the original commit.

Team B — flag-protected trunk-based. A developer cuts `feat/favourites-table` from `main` for an hour, adds the database migration, opens a PR, gets it reviewed and merged the same morning. The next morning, a second branch adds the backend endpoint behind a feature flag `favourites_enabled=false`. It merges by lunch. Over the next three days, three more small PRs add the API client, the UI component, and the wiring — each of them merged to `main` on the day it was opened, each gated by the same PR pipeline, each going to production immediately because the team practices continuous deployment. None of them are visible to users because the flag is off. On day four, the flag is flipped to `true` for 5% of users via a canary, then expanded to 100% by the end of the day. The change reaches production four days after the original commit, and the production switch was a configuration change, not a deployment.

The two flows ship the same feature. The second one carries less integration risk, has shorter feedback cycles, and decouples the deployment from the release — which is what makes it possible to roll back the favourites button instantly without redeploying anything.

37.7 Choosing where on the spectrum to land

The choice between continuous delivery and continuous deployment is not primarily a technical one. The technical bar is similar — automated tests, deployable artifacts, monitored production. The deciding factors are usually external:

FACTOR	FAVOURS CONTINUOUS DELIVERY	FAVOURS CONTINUOUS DEPLOYMENT
Regulatory regime	Audit trails require human approval per change	No mandated approval step
Customer communication	Releases must be announced or coordinated	Users tolerate frequent silent updates
Test confidence	Test suite cannot fully replace human judgement	Test coverage is high and trustworthy
Rollback speed	Rollbacks are slow or destructive	Rollback is automated and seconds-quick
Blast radius control	No feature-flag or canary infrastructure	Mature deployment strategies in place

A team that ships an embedded firmware update to medical devices will not adopt continuous deployment regardless of how good their tests are — the rollback cost is too high and the regulatory environment forbids it. A team that ships a consumer web feature where the worst case is “the favourites

button is broken for an hour" can adopt continuous deployment safely, provided the rollback path is automated.

A useful rule: do not adopt continuous deployment until rollback is faster than diagnosis. If a bad change takes three minutes to roll back and 30 minutes to diagnose, automated deployment is fine — the worst case is a three-minute outage. If rollback takes 30 minutes and diagnosis takes three minutes, a manual gate provides real value, because a human can catch the problem before it ships.

37.8 Where this connects to the exercises

The exercises in `/exercises/3-deployment/9-cicd-to-container-apps/` walk through this spectrum on a working pipeline. The first exercise stops at CI plus a manual deployment step (the developer clicks "Create new revision" in the Azure Portal — a stand-in for a manual approval gate). The second exercise extends it into continuous deployment: a green build automatically updates the Container App revision and a smoke test confirms the new version is responding before the pipeline reports success. The third exercise replaces the stored credential with OIDC federation but keeps the same delivery shape — the same continuous deployment, with a stronger trust model.

The mining ground for this chapter is therefore not theoretical. The same code travels from "build and push" to "build, push, deploy, verify" across three iterations of the same pipeline, and the differences between them are exactly the differences between CI, continuous delivery, and continuous deployment.

37.9 Summary

Continuous integration is the practice of integrating code into a shared trunk multiple times a day, with every commit automatically built and tested and broken builds treated as blockers for the whole team. Continuous delivery extends CI by keeping every green commit in a releasable state, gated by a human approval. Continuous deployment removes the human gate and ships every green commit to production automatically. Trunk-based development is the branching model that makes any of this possible, and the pull-request gate is the human checkpoint that protects the trunk from regressions before they merge. The choice between continuous delivery and continuous deployment depends on regulatory context, rollback speed, and the maturity of the deployment safety net — not on technical capability alone. The shared origin of all three practices is the recognition that integration risk and release risk both compound non-linearly, and that paying them down continuously is cheaper than paying them down in lumps.

PART VIII · DEVOPS AND DELIVERY

CHAPTER 38

Pipelines as Code

Watch the presentation

Se presentationen på svenska

A team that ships code every day cannot afford a release process that lives only in someone's head or in a sequence of clicks inside a portal. Build steps configured through a portal UI cannot be code-reviewed, cannot be diffed, and cannot be reproduced when the portal redesigns its menus. The remedy is to express the entire build-and-release process as a file that lives in the repository alongside the application code. This chapter introduces the vocabulary and mechanics of that approach in GitHub Actions, the platform used by the companion exercise.

38.1 Why the pipeline belongs in the repository

A portal-driven release process accumulates configuration in a place that is invisible to most of the team. A developer who joins the project six months later cannot read the production deployment logic by browsing the source tree — they have to know which portal to open, which project to inspect, and which tabs hide the relevant settings. When a build breaks, there is no commit history to bisect. When the release machinery itself needs to change, no pull request gates the change against review. And when the portal's vendor renames a tab or removes a feature, the team's institutional knowledge silently rots.

The phrase **pipeline as code** describes the alternative: the pipeline definition is stored in the same repository as the application, in a plain-text file, and is applied by reading that file. The same review process that gates application changes now gates release changes. A revert of a bad pipeline change is a `git revert` away. A new team member reads the pipeline by reading the file. The pipeline becomes a first-class artifact of the project rather than a configuration that exists in a separate, lossy medium.

This shift carries an obvious cultural implication. Pipeline edits are now visible to everyone with read access to the repository, and breaking the pipeline now leaves a fingerprint in the commit log. That visibility is the point. It also nudges teams toward the continuous integration (CI) practice the previous chapter introduced — small, frequent merges to the trunk become realistic only when the integration check is automated, defined in code, and reliably triggered on every push.

38.2 The GitHub Actions vocabulary

Different platforms (GitLab CI, Azure Pipelines, CircleCI, Jenkins) all converge on roughly the same conceptual model, but the specific names differ. The remainder of this chapter uses GitHub Actions terminology because that is what the companion exercise uses, and because GitHub Actions is the most common starting point for new projects.

A **pipeline** is a sequence of automated stages (build, test, deploy) that takes source code and produces a running application; each stage depends on the previous one succeeding, and failure at any stage halts the pipeline. The word "pipeline" is the platform-neutral term used in the DevOps literature.

A **workflow** (in GitHub Actions) is an automated process defined in a YAML file (`.github/workflows/*.yml`) that triggers on events (push, pull request, schedule) and orchestrates jobs to build, test, and deploy code. In GitHub Actions terminology, the workflow is the file; the pipeline is the abstract concept the file expresses. A repository can hold many workflow files, and each one is independent — a `ci.yml` for build-and-test, a `release.yml` for production deploys, a `nightly.yml` for scheduled long-running tasks.

A **job** (in GitHub Actions) is a set of steps that run on the same runner; jobs can run in parallel or sequentially, and a workflow often contains multiple jobs (e.g., build job, test job, deploy job). Jobs are the unit of parallelism: a workflow with three independent jobs runs them concurrently on three separate runners by default. A job that needs the output of another job declares the dependency with `needs:`, and the platform schedules them in the right order.

A **step** (in GitHub Actions) is a single command or action within a job; it runs sequentially in the order defined, and failure in a step can halt the job unless explicitly ignored.

Steps within a job share the same filesystem and environment, so files written in one step are visible to the next. Steps across different jobs do not share a filesystem — each job starts on a fresh runner, which is why artifacts (covered below) exist.

A **runner** is a machine (GitHub-hosted or self-hosted) that executes the steps of a GitHub Actions job; GitHub-hosted runners are provided by GitHub (Ubuntu, Windows, macOS), while self-hosted runners are managed by the organization. A runner is provisioned for the duration of a single job and discarded afterward. The fresh-machine guarantee is part of why the pipeline is reproducible: there is no state from a previous run to pollute the next one. The trade-off is that anything the job needs (the source code, dependencies, build tools) must be installed by the steps themselves.

An **action** (in GitHub) is a reusable unit of code published to the GitHub Marketplace; actions encapsulate common tasks (checkout code, set up a runtime, deploy) and are called by name in workflow steps (e.g., `actions/checkout@v4`). An action is the GitHub-specific equivalent of a shell function — a named, parameterised block that someone else has written and that the workflow can invoke without copying its implementation. Most workflow steps are either a shell command (`run:`) or an action invocation (`uses:`).

The vocabulary fits together hierarchically: a workflow contains jobs, a job runs on a runner and contains steps, and a step is either a shell command or an action call.

38.3 Triggers: when a workflow runs

A workflow runs only when an event triggers it. The `on:` field at the top of the workflow file lists the events the workflow listens for. Four triggers cover almost every common case:

- **`push`** — fires whenever a commit lands on a specified branch. The default CI trigger; every push to `main` rebuilds and re-tests.
- **`pull_request`** — fires when a pull request is opened, updated, or reopened. The pull-request gate runs the build against the proposed merge before any human review begins.
- **`workflow_dispatch`** — adds a manual "Run workflow" button in the GitHub UI. Useful for production deploys that should not happen automatically, or for re-running a flaky job without pushing a fake commit.
- **`schedule`** — fires on a cron expression (`0 2 * * *` for 02:00 every day). Useful for nightly integration tests, dependency-update checks, or housekeeping tasks.

A single workflow can listen for several triggers at once, and triggers can be filtered by branch, by path, or by label, so the same file can express "build on every PR, but deploy only on push to main."

38.4 Code review for the pipeline itself

Once the pipeline is a file in the repository, every change to it flows through the same pull-request review the team already uses for application code. A reviewer reading a diff of `ci.yml` sees exactly what is changing: a new step being added, a runner image being upgraded, a deploy command being modified. The reviewer can ask the same questions they would ask of an application change — is this safe, is this

tested, does this match our conventions — and can block the merge until the answers are satisfactory.

This matters most when a pipeline change is risky. Adding a step that writes to production, rotating a credential reference, or changing the order in which gates run — these are the changes that break releases at 02:00. Subjecting them to review is the cheapest insurance the practice provides. It also creates a paper trail: the commit history of `.github/workflows/` is the audit log of every release-process change the team has made.

38.5 Artifacts: passing files between jobs

A **workflow run** is a single execution of a workflow, triggered by a single event. Each job in the run starts on a fresh runner with an empty filesystem, which means a job cannot read files produced by an earlier job by reading the disk directly — the disk does not carry over. The mechanism for moving files between jobs is the **artifact**.

An **artifact** (in CI) is a file or collection of files (compiled binaries, Docker images, test reports) produced by a pipeline job that can be stored, passed to later jobs, or published for download. A `build` job uploads its compiled output as an artifact; a downstream `test` job downloads the same artifact and runs against it. The artifact is also retained on the workflow run page in the GitHub UI for a configurable period (90 days by default), which makes it easy to download a build that shipped six weeks ago for forensic inspection.

Artifacts and Docker images serve overlapping but distinct purposes. An artifact is a workflow-internal mechanism — a way to move files between jobs of the same run. A Docker image is an external mechanism — a portable, content-addressed package pushed to a registry, intended to be pulled

by other systems (a Container App, another pipeline, a developer's laptop). A pipeline that builds a container often does both: it uploads the build log and test report as workflow artifacts, and it pushes the production image to a registry as the actual deliverable.

38.6 Worked example: a minimal CI workflow

The companion exercise CI/CD to Container Apps walks through three increasingly capable pipelines. The first pipeline is the smallest one that does useful work: it builds a .NET app, runs its tests, and produces a Docker image. Figure 1 shows the file in full.

Figure 1: `.github/workflows/ci.yml` — minimal build-test-image pipeline

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '10.0.x'
      - run: dotnet build --configuration Release

  test:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '10.0.x'
      - run: dotnet test --configuration Release

  docker-build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - uses: docker/build-push-action@v5
        with:
          context: .
```

```
          push: true
          tags: myorg/myapp:${{ github.sha }}
```

The `on:` block declares two triggers. The `jobs:` block defines three jobs that run in sequence: `build` produces compiled binaries, `test` runs the test suite, and `docker-build` packages the application into a container image and pushes it to Docker Hub. The `needs:` keyword chains the jobs — `test` will not start until `build` finishes successfully, and `docker-build` will not start until `test` passes. Each job runs on a fresh `ubuntu-latest` runner, which is why every job repeats `actions/checkout@v4` and `actions/setup-dotnet@v4`: the previous job's runner no longer exists. The `${{ secrets.DOCKERHUB_TOKEN }}` reference reads a value injected from the repository's secret store, never visible in the file or the logs.

This pipeline is small enough to read end-to-end in under a minute, which is the property that matters most. A reviewer can see at a glance what builds, what tests, and what ships. The next chapter (Build, Test, and Smoke Gates) layers gates and result publishing on top of this skeleton; the chapter on secrets and OIDC replaces the long-lived `DOCKERHUB_TOKEN` with a short-lived federated credential.

38.7 The trade-off: YAML readability degrades

The pipeline-as-code practice has one durable downside: the file format. YAML is forgiving about whitespace until it isn't, expression substitution syntax (`${{ ... }}`) accumulates quickly, and a workflow that started at 30 lines often grows past 300 as the team adds matrix builds, conditional steps, and deploy logic. At that size the file stops being scannable.

The discipline that keeps the practice working is the same discipline that keeps application code working: refactor early. GitHub Actions provides three mechanisms for keeping workflows small. **Reusable workflows** (`workflow_call`) let one workflow call another, the way a function call factors out shared logic. **Composite actions** package a sequence of steps into a single named action that lives in `.github/actions/` inside the repository. **Matrix strategies** collapse "the same job for each of these versions" from a copy-paste of N jobs into one job definition with a list of variants.

A pipeline that is hard to read is a pipeline that hides bugs. The cost of a poorly factored workflow is paid every time someone tries to add a step, debug a failure, or onboard a new engineer. Treat the workflow file with the same refactoring instincts the team applies to the application code, and the practice scales. Treat it as throwaway scripting, and the file eventually becomes the same opaque artifact the portal-driven approach was meant to escape.

38.8 Summary

A pipeline expressed as code lives in the repository, is reviewed like any other code change, and reproduces deterministically on a fresh runner each time it executes. GitHub Actions structures the work as workflows containing jobs, jobs containing steps, and steps invoking either shell commands or pre-published actions. Triggers (`push`, `pull_request`, `workflow_dispatch`, `schedule`) determine when a workflow runs, and artifacts move files between jobs that otherwise share no filesystem. The companion CI/CD exercise builds an increasingly capable pipeline on this foundation, starting with a minimal build-test-image workflow and ending with passwordless deployment to Azure Con-

tainer Apps. The single durable cost of the practice is YAML drift: workflows grow, and the team that does not refactor them ends up with the same opaque release machinery the practice was meant to avoid.

CHAPTER 39

Build, Test, and Smoke Gates

Watch the presentation
Se presentationen på svenska

A pipeline that compiles code and pushes an image is a build script with a fancy trigger. What turns automation into continuous integration is the set of checks that decide whether the result is safe to ship. Each check is a stage in the pipeline, and each stage either passes the artifact along or stops the run. This chapter develops the three gates that earn their keep on every CI/CD pipeline a CLO student will write — the build stage, the test layers, and the smoke test against the

deployed application — and explains how their ordering controls cost, signal, and feedback time.

39.1 From build script to continuous integration

Continuous integration (CI) only delivers value when each commit is verified against a battery of automated checks. Without those checks, the pipeline is a publishing mechanism: it takes whatever the developer pushed and produces an artifact, regardless of whether the artifact works. The verification layer is what gives CI its name. A change is only "integrated" once the pipeline has demonstrated, mechanically and reproducibly, that it still compiles, still passes its tests, and still runs once deployed.

The vocabulary for these checks is the **gate (CI)** — a stage or condition that must be satisfied before the pipeline proceeds to the next stage. Gates prevent broken or untested code from reaching later stages. The gate is not the test itself but the structural decision that the pipeline halts when the test fails. A test suite that prints failures but lets the pipeline continue is not a gate; the same suite wired so the pipeline exits nonzero on any failure is.

Three gates appear in almost every pipeline that ships container workloads to a managed platform: the build gate, the test gate, and the smoke gate. They differ in cost, in the kind of failure they catch, and in how close to production they execute.

39.2 The build stage

A **build stage** is the first phase of a CI pipeline that compiles source code into executable binaries or artifacts; it fails fast if

the code does not compile, preventing broken code from proceeding to tests or deployment. For a .NET application this stage runs `dotnet restore` and `dotnet build`. For a TypeScript application it runs `npm ci` and `tsc`. For a Java application it runs `mvn compile`. The exact commands differ by ecosystem; the role does not.

The build stage answers the cheapest question the pipeline can ask: "Does this source code form a valid program?" A missing semicolon, a misspelled package name, a renamed method that broke a caller — all of these surface here, before a single test runs. The cost is small, usually under a minute on a hosted runner, and the signal is binary. If the build fails, no later stage can produce useful information.

Linting often rides along with the build stage. A linter (`dotnet format --verify-no-changes`, `eslint`, `ruff`) checks style and common error patterns without executing the code. Including lint in the build gate keeps stylistic regressions out of the trunk and catches a class of bugs (unused variables, unreachable branches) that compilers may not flag. The runtime is small enough that bundling lint with build pays for itself.

The artifact emitted by the build stage — compiled binaries, a published directory, a container image — is what later stages consume. In a containerized pipeline the build stage frequently produces a Docker image and uploads it to a registry; subsequent stages then pull that image rather than recompiling. This pattern is described in pipelines as code, where the artifact is the unit of work passed between jobs.

39.3 Test layers

A passing build proves the code is valid. It proves nothing about whether the code is correct. Tests close that gap, and

they do so in layers, each catching a different class of defect at a different cost.

39.3.1 Unit tests

A **unit test** is an automated test that verifies a single function or method in isolation; it runs quickly and is the first line of defense against bugs, catching logical errors before code integration. A unit test for a discount calculator constructs the calculator, calls `Apply(orderTotal, couponCode)`, and asserts the returned total. Nothing else runs — no database, no HTTP server, no message queue. Dependencies that the unit collaborates with are replaced by test doubles (mocks, fakes, stubs).

The defining property of a unit test is speed. A well-structured suite of several hundred unit tests should complete in seconds. That speed comes from isolation: with no external dependencies to start or wait for, the test exercises only the logic under examination. Unit tests catch the kinds of mistakes that live inside a method — incorrect branching, off-by-one errors, mishandled null inputs, wrong arithmetic. They cannot catch mistakes that emerge from how units combine.

39.3.2 Integration tests

An **integration test** is an automated test that verifies how multiple units (functions, services, databases) interact together; it runs slower than unit tests because it sets up more dependencies, but it catches bugs that unit tests miss. An integration test for the same discount calculator might start a real PostgreSQL container, load the coupon table from a fixture, and call the calculator through the same service entry point a controller would use. The point is to exercise the wiring: the SQL the data layer emits, the JSON the service deserializes, the configuration the application binds.

These tests find the failures that mock-based unit tests cannot reach. A SQL query that returns the wrong column. A JSON contract that drifted between client and server. A migration that added a non-nullable column without a default. None of these surface in unit tests because the unit-test boundary deliberately excludes the systems where the bug lives. They surface as soon as a real version of the dependency runs end-to-end.

The cost is the runtime. Spinning up a database container, applying schema, seeding data, and running queries adds seconds per test. A suite of fifty integration tests can take several minutes. That cost is why integration tests sit later in the pipeline — they should not run if the build is broken or unit tests are red, because their failure would be redundant with cheaper signals.

39.3.3 Smoke tests

A **smoke test** is a lightweight, high-level verification that the deployed application is alive and responding; it typically makes an HTTP request to a public endpoint and checks for a successful response, providing a quick confidence check that deployment succeeded. The name comes from electronics: power up the device and check that no smoke comes out. The test does not verify functionality in depth — that is what unit and integration tests are for. It verifies that the deployment landed at all.

A smoke test runs against the deployed artifact, not against locally built code. It targets the production URL (or a staging URL, depending on the deployment target) and confirms the live process is reachable, the routing is correct, the configuration loaded, and the dependencies the application needs at startup are wired. A passing smoke test does not promise the

application is bug-free; a failing smoke test promises the deployment is broken.

39.4 Why each layer earns its keep

The three layers — unit, integration, smoke — are not redundant. Each catches failures the others miss, and each runs at a different cost. The economics is what makes the layered approach pay off.

LAYER	SPEED	CATCHES	COST PER FAILURE CAUGHT
Unit	Seconds	Logical bugs in a single function	Lowest
Integration	Minutes	Contract drift between components	Medium
Smoke	Seconds	Deployment, configuration, networking	Lowest at deploy time

A unit test cannot detect that the production database is unreachable. A smoke test cannot pinpoint which method computed the wrong total. Treating tests as a single mass and asking “do we have enough tests?” is the wrong question. The right question is whether each layer is present in proportion to the failures it is meant to catch.

The pipeline's contribution is to enforce ordering. A failing unit test should stop the pipeline before integration tests run. A failing integration test should stop the pipeline before the deployment job runs. A failing smoke test should stop the pipeline before traffic shifts to the new revision. Each gate trades a small additional runtime for protection against a larger downstream cost.

39.5 Gate ordering and fail fast

A pipeline that runs every check in parallel maximizes feedback speed but burns runner minutes on jobs whose result is no longer interesting. A pipeline that runs every check sequentially is cheap but slow. The compromise is to order gates by cost, cheapest first.

The principle is **fail fast**: a broken build should stop the pipeline before it spends twenty minutes building containers, twenty minutes running integration tests, and twenty minutes deploying to a staging environment. The earlier a failure surfaces, the cheaper it is to fix and the less downstream work the pipeline wastes.

The standard ordering for a containerized pipeline:

1. **Build** — compile and lint. Fast, deterministic, binary signal.
2. **Unit test** — isolated tests. Fast, executes against the freshly built binaries.
3. **Integration test** — tests with real dependencies. Slower, but still pre-deployment.
4. **Image build and push** — produce the deployment artifact. Only runs if all earlier gates passed.
5. **Deploy** — update the running service.

6. **Smoke test** — verify the deployed service responds. Final gate before declaring success.

Each gate is a `needs:` dependency on the previous one in GitHub Actions, or a `dependsOn:` clause in Azure Pipelines. The pipeline halts at the first failing stage; nothing downstream runs. The runtime saved is real — a broken build that would have triggered ten more jobs now triggers one.

39.6 Test-result publishing

A test that fails inside a pipeline is invisible unless the pipeline surfaces the result. The job log will record a non-zero exit code, but reading raw logs to find which test failed and why is a poor developer experience. A **test-result publisher** is a tool or action in a CI pipeline that captures, formats, and displays test results (pass/fail counts, coverage, failures) in a human-readable report; it provides visibility into test quality across builds.

For .NET tests, `dotnet test --logger "trx"` emits a structured TRX file. A GitHub Actions step like `dorny/test-reporter@v1` consumes the TRX and renders an inline summary on the pull request — pass and fail counts, the names of failing tests, the assertion diffs. The PR reviewer sees the test outcome without expanding the log. For TypeScript tests, JUnit-format XML serves the same role; for Python, a JUnit reporter from pytest.

Test-result publishing matters because the gate is only as useful as its visibility. A failing test that buries its diagnostic in a 5000-line log slows the response. A failing test whose name and stack trace appear on the PR page enables an immediate fix. The publisher does not change what the test verifies; it changes how quickly the developer learns what went wrong.

39.7 The smoke test as deployment gate

The build and test stages happen before deployment. The smoke test happens after. That timing gives it a different role: it is the final check that decides whether the deployment is declared successful, and it is the only check that runs against the running production (or staging) environment.

A worked example, drawn from the pipeline built in Exercise 3.9 — CI/CD to Azure Container Apps, shows the pattern. The pipeline first runs `dotnet test` in a job named `test`. If any test fails, the job exits non-zero and the dependent deploy job never runs. If all tests pass, a deploy job updates the Container App revision. A final smoke job then verifies the live endpoint:

```yaml
test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-dotnet@v4
      with:
        dotnet-version: '10.0.x'
    - run: dotnet test --logger "trx" --results-directory
        TestResults
    - uses: dorny/test-reporter@v1
      if: always()
      with:
        name: dotnet tests
        path: TestResults/*.trx
        reporter: dotnet-trx

deploy:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: azure/login@v2
      with:
        creds: ${{ secrets.AZURE_CREDENTIALS }}
    - run: az containerapp update --name $APP --resource-
        group $RG --image $IMAGE

smoke:
  needs: deploy
  runs-on: ubuntu-latest
  steps:
    - run: |
        curl --fail --max-time 10 \
          "https://${CONTAINER_APP_FQDN}/healthz"
```

The smoke job is short — a single `curl --fail` against the health endpoint. The `--fail` flag tells `curl` to exit non-zero on any HTTP status outside 2xx, which propagates as a job failure. That non-zero exit fails the workflow run, marks the commit red on GitHub, and signals the team that the deployed revision is unhealthy. A more thorough smoke

check inspects the response body for a version cue (a build SHA, a release tag) to confirm the new revision actually serves traffic, not the previous one.

The reason a smoke test belongs in the pipeline rather than in a monitoring tool is timing. Monitoring detects unhealthy services minutes after they degrade. The smoke gate detects them seconds after the deployment finishes, when the change that caused the regression is still on the developer's screen. The smoke gate is also the gate that decides whether the pipeline reports success — without it, the workflow would turn green the moment `az containerapp update` returned, regardless of whether the new revision could actually serve a request.

39.8 Summary

The gates that separate a pipeline from a build script are the build stage, the test layers, and the smoke test. The build stage answers the cheapest question and emits the artifact later stages consume. Unit tests verify logic in isolation and run in seconds; integration tests verify how units combine and run in minutes; smoke tests verify the deployed artifact responds and run against the live URL. Ordering gates by cost — cheapest first — surfaces failures early and saves runner time, the principle known as fail fast. A test-result publisher renders the gate's outcome where developers can act on it. The smoke test, run as the final stage against the deployed application, is the gate that decides whether the workflow declares success or red-flags the release.

CHAPTER 40

Deployment Strategies

Watch the presentation
Se presentationen på svenska

The moment a new build replaces the running version is the riskiest moment in the life of a service. Tests have passed, the artifact is signed, and the pipeline is ready to ship — but the new code has never carried real traffic. A bug that the test suite missed will surface here, in front of users, with the previous version already gone. Different deployment strategies exist precisely to mitigate this risk in different ways: some make the cutover instant and reversible, some bleed the change in slowly while watching metrics, some keep a human

in the loop. The choice shapes how a release feels to the team and to users — and which failures stay invisible.

40.1 Why a strategy is needed

A **deployment strategy** is a method for rolling out code changes to production (e.g., all-at-once, blue-green, canary, rolling); the choice of strategy affects risk, rollback speed, and user impact. The naive approach — stop the old version, start the new one — works for a hobby project. It fails for a production service for two related reasons. The first is downtime: every user hitting the service during the swap sees an error. The second is blast radius: if the new build is broken, every user sees the breakage at once, and rollback means another stop-start cycle that compounds the outage.

Strategies differ along three axes. The first is **cutover duration** — a blue-green flip lasts seconds, while a rolling deployment can take many minutes. The second is **blast radius** when the new version is broken: a canary exposes 1% of users, a rolling deployment exposes a growing fraction, and an all-at-once swap exposes 100%. The third is **reversal speed** — blue-green keeps the old environment idle, so rollback is one traffic flip, whereas rolling deployment requires reversing the rollout step by step.

These axes do not move independently. A faster cutover usually means a larger blast radius. A safer rollout usually means more infrastructure to maintain. The deployment strategy is the team's pre-committed trade among these costs: which of them is acceptable in exchange for which protections.

40.2 The manual gate

The simplest production-grade strategy is also the oldest. The pipeline builds, tests, and packages the code, then stops. A human reviews the result and clicks **Approve** to deploy. This is **continuous delivery** without **continuous deployment** — the artifact is always shippable, but a person decides when it ships.

A **manual gate** is a point in a deployment process where a human must explicitly approve before proceeding; it adds a safety checkpoint but increases lead time if the approver is not available. Manual gates show up in three common forms:

- **Approve to deploy** — the pipeline reaches a `deploy` job that requires a reviewer's click before running.
- **Approve to promote** — the build deploys to staging automatically, and a click promotes the same artifact to production.
- **Click to flip traffic** — the new version is fully deployed but receives no traffic until a human routes it there. The companion exercise's first variant uses this form: the GitHub Actions pipeline pushes a new image, but a developer must open the Azure Portal and click **Create new revision** before users see the change.

The trade-off is bluntly mechanical. Manual gates catch the failure modes a human would notice — a release on a Friday afternoon, a deploy during a customer demo, a build that has the right number but suspicious test output. They do not catch failures that only show up under traffic, because no traffic is hitting the new version yet. They also tax lead time: if the approver is asleep, on vacation, or in a meeting, the change waits. Many teams keep a manual gate for production while letting staging deploy automatically — a useful middle

ground that captures the safety value without paying the lead-time cost on every deploy.

40.3 Rolling deployment

A **rolling deployment** is a strategy where instances of the old version are progressively replaced with the new version, a few at a time; the application remains available throughout, but the rollout takes time and rollback is more complex than blue-green. The mechanics are straightforward when the workload is stateless and runs on multiple replicas. The orchestrator (Kubernetes, Container Apps, an autoscaling group) takes one or two replicas out of the load-balancer pool, replaces them with the new version, waits for them to pass a readiness probe, and puts them back. It then repeats for the next batch until all replicas run the new version.

Two settings shape a rolling deployment. `maxUnavailable` is how many replicas can be missing from the load balancer at once — too low, and the rollout stalls if a new replica is slow to start; too high, and an in-progress rollout under heavy load drops requests. `maxSurge` is how many extra replicas can run during the rollout — a higher surge keeps capacity stable but costs more during the rollout window.

Rolling deployments suit stateless services that can run mixed versions side by side for several minutes. The application remains available throughout, and capacity is preserved within the surge budget. The trade-off is the half-state in the middle: while the rollout is running, requests land on either version depending on which replica the load balancer picks. If the new version introduces a backwards-incompatible change to a shared resource — a database schema, a cache key format, a session cookie — the mixed-version window will surface bugs that a same-version deployment would not. Roll-

back is also slow: reversing a rolling deployment means rolling back batch by batch, while users continue to hit the broken version until the reversal completes.

40.4 Blue-green deployment

Blue-green deployment is a strategy where two identical production environments (blue and green) are maintained; the new version is deployed to the inactive environment, tested, and then traffic is switched over, allowing instant rollback to the previous version if problems occur. At any moment, one environment serves all production traffic and the other sits idle. To deploy, the team installs the new build on the idle environment, runs smoke tests against it through an internal hostname, and then flips a single switch — usually a DNS record, a load-balancer rule, or a traffic-split setting — to point users at the new environment.

The defining property is the cutover. It is atomic: a request is either served by the old version or the new one, never half-and-half. There is no mixed-version window, so backwards-incompatible changes are easier to handle than under rolling deployment. Rollback is equally atomic: flipping the switch back routes all traffic to the still-running old environment in seconds. Until the team is confident in the new release, the old environment is kept warm and ready.

The cost is duplication. Blue-green requires roughly twice the infrastructure of a single environment, at least during the deploy window. For stateless workloads on elastic platforms, this cost is short-lived and small. For stateful workloads, the picture is harder — both environments cannot own the database at once, so blue-green is usually paired with backwards-compatible schema migrations applied before the cutover, with the database itself remaining shared.

40.5 Canary deployment

Canary deployment is a strategy where a new version is deployed to a small percentage of production traffic first (the "canary"); if metrics look good, the rollout continues; if issues arise, traffic reverts to the stable version, limiting user impact. The new version runs alongside the stable version, but the load balancer routes only 1–5% of requests to it. The team watches metrics — error rate, latency, business KPIs — for a fixed window. If the canary stays healthy, the percentage ramps up: 1% → 10% → 50% → 100%. If a metric breaches a threshold, the load balancer routes traffic away from the canary and the deployment is rolled back.

A canary catches failures that only surface under real production traffic — load-dependent bugs, edge cases in real user data, third-party-API timeouts, latency regressions invisible at low scale. It catches them on a small slice of users, not on everyone. The blast radius at 1% traffic is roughly 1% of what an all-at-once deploy would have produced.

The price is operational sophistication. A canary needs traffic-splitting infrastructure, a metric-driven rollout controller, and clear go/no-go thresholds defined before the deploy. It also requires that the stable and canary versions can run in parallel without sharing state in incompatible ways. Teams that adopt canaries usually combine them with **automated promotion** — a controller (Argo Rollouts, Flagger, a custom controller in Container Apps) advances the percentage when metrics stay green and rolls back when they go red, without a human clicking through each step.

40.6 Feature flags

A **feature flag** is a configuration switch in an application that enables or disables a feature at runtime without redeploying

code; flags allow features to be deployed to production but hidden from users until ready, and enable quick rollback if issues arise. Feature flags are not a deployment strategy in the same sense as blue-green or canary — they sit at a different layer. Where deployment strategies decide which version of the code is running, feature flags decide which features inside that version are reachable.

This separation is the key idea. **Deploying** code means installing a new build; **releasing** a feature means making it visible to users. Feature flags decouple the two. A team can deploy a half-finished feature behind an off-by-default flag, run it in production with no user impact for weeks, and then flip the flag for internal users, then for 1% of customers, then for everyone — long after the deployment that introduced the code has shipped.

Flag-based rollouts complement deployment strategies. A canary controls the version mix at the infrastructure layer; a feature flag controls the feature mix at the application layer. The two combine well: a build deployed via canary can include several flagged features, each released independently of the deployment that delivered them. Rollback also gets safer — turning off a flag is faster than redeploying, and avoids the mixed-version window of a rolling rollback.

40.7 Choosing a strategy

The strategies are not mutually exclusive, but each one fits a different risk profile.

STRATEGY	CUTOVER	BLAST RADIUS IF BROKEN	ROLL-BACK SPEED	INFRA-STRUC-TURE COST	BEST FIT
Manual gate	Whatever the underly-ing deploy is	Same as underly-ing	Same as under-lying	None extra	Low-fre-quency releases; humans add the missing signal
Rolling	Several minutes	Grows with the rollout	Slow (reverse the rol-lout)	None extra	State-less ser-vices with many replicas; back-wards-compat-ible changes
Blue-green	Seconds	100% during cutover, but cutover is short	Instant (flip back)	Roughly 2× during the window	Cutover-sensit-ive ser-vices; back-wards-incom-patible changes with prior data

STRATEGY	CUTOVER	BLAST RADIUS IF BROKEN	ROLL-BACK SPEED	INFRA-STRUC-TURE COST	BEST FIT
					migra-tion
Canary	Long (hours)	1–5% during canary phase	Fast (route traffic away)	Slight surge + observ-ability	Services with rich metrics; rollouts that need real-traffic valida-tion
Feature flag	Instant	Whatever the flag scope is	Instant (flip the flag)	A flag service or lib-rary	Decoup-ling release from deploy; gradual feature expos-ure inde-pendent of deploy cadence

The choice depends on how much the team trusts the test suite, how rich the production metrics are, how much extra infrastructure is acceptable, and how compatible successive

versions are with each other. Teams that release rarely tolerate a manual gate; teams that release dozens of times per day need automated canaries to keep up. Teams running stateless container workloads default to rolling or blue-green; teams running customer-facing experiments lean on feature flags.

40.8 A worked example with Container Apps

The companion exercise CI/CD to Azure Container Apps builds the manual-gate strategy explicitly. The pipeline ends with `docker push` to a registry, but the Container App keeps serving the previous **revision** — the immutable version unit in Container Apps. A developer opens the Portal, clicks **Create new revision**, picks the new image tag, and only then does the new code reach users. The gap between "image is built" and "users see it" is something a developer should feel at least once.

A pipeline that automates the same flow with a smoke-tested cutover replaces the manual click with two CLI calls. The first creates a new revision in single-revision mode, which Container Apps treats as a one-step blue-green flip:

```
az containerapp update \
  --name myapp \
  --resource-group rg-prod \
  --image mycr.azurecr.io/myapp:${{ github.sha }} \
  --revision-suffix "${{ github.run_number }}"

FQDN=$(az containerapp show \
  --name myapp \
  --resource-group rg-prod \
  --query properties.configuration.ingress.fqdn -o tsv)

curl --fail --silent --max-time 10 "https://${FQDN}/
        health" \
  || { echo "Smoke test failed"; exit 1; }
```

The first command pushes a new revision with a deterministic suffix and, in single-revision mode, retires the previous revision once the new one is healthy. The second probes the public FQDN with a `--fail` flag that turns any non-2xx response into a non-zero exit code — the pipeline's failure signal. If the smoke test fails, the workflow exits non-zero and a follow-up step or manual intervention can roll the revision back.

For a true canary, Container Apps can run in multi-revision mode with a `traffic` block that splits requests across revisions by weight. The pipeline pushes a new revision at 5% weight, observes for a window, then issues a second update that ramps the weight to 100% and retires the old revision. The same primitive — a revision plus a traffic weight — supports both the manual-gate and the canary forms of the same release.

40.9 Summary

A deployment strategy is the team's plan for the riskiest moment in a service's life. The manual gate keeps a human in

the loop and trades lead time for an extra signal. Rolling deployments preserve availability on stateless services but expose a mixed-version window. Blue-green deployments make the cutover atomic and rollback instant at the price of duplicate infrastructure. Canary deployments validate a release against real traffic on a small slice of users before extending to all of them. Feature flags decouple deploy from release entirely and operate at the application layer rather than the infrastructure layer. Each strategy matches a different risk profile, and most production systems combine several — a canary deploy of a build that contains several flagged features, with a manual gate guarding the production environment as a whole.

CHAPTER 41

Pipeline Secrets and OIDC Federation

Watch the presentation
Se presentationen på svenska

A pipeline that deploys to a cloud account needs an identity. The build step compiles the code; the deploy step has to call Azure, AWS, or GCP and tell it to update something — and the cloud provider will only honour that call if the request carries proof that the caller is allowed to make it. One approach is to store a long-lived password in the CI provider's secret store and let the pipeline use it on every run. The stronger approach is to let the CI provider mint a short-lived token for each individual workflow run and have the cloud

provider trust that token directly, with no shared password anywhere. This chapter covers both approaches, the failure mode that drove the shift from one to the other, and the configuration mechanics that connect a GitHub Actions workflow to an Azure tenant without a stored credential.

41.1 Why pipeline credentials are dangerous

A continuous integration pipeline is, by design, a script that runs unattended on someone else's hardware with the permissions needed to change production. That combination — automation, third-party infrastructure, production access — is exactly the threat model that made stored cloud credentials the most common cause of cloud breach reports for years. A credential that lives in a secret store is a credential that can be extracted. A credential that is rotated annually is a credential that an attacker can use for up to a year. A credential that grants broad permissions to "deploy" almost always grants enough permissions to exfiltrate data or destroy resources at the same time.

The defence is layered. The pipeline should authenticate as a non-human identity scoped to exactly what the deploy needs, not as a developer's personal account. The credential should be stored encrypted at rest, masked in logs, and never readable through the UI after creation. And — the focus of the second half of this chapter — the credential should ideally not exist at all as a long-lived value, replaced instead by short-lived tokens that the CI provider mints on demand for a single workflow run.

41.2 The GitHub secret store

A **GitHub secret** is a confidential value (API key, access token, password) stored encrypted in a GitHub repository; secrets are injected into workflows as environment variables and are masked in logs to prevent accidental exposure. Secrets can be defined at three scopes — repository, environment (e.g., a `production` environment), and organization — and a workflow step references them through the `${{ secrets.NAME }}` expression syntax. Once a secret is created, the value is never shown again in the GitHub UI; the only operations are overwrite and delete. The plaintext is encrypted with a public key whose private half lives in the GitHub Actions runner control plane, and only an actively running job authorized to use that secret can decrypt it.

Two practical constraints follow from how the store works. First, secrets are exposed only to workflow runs that GitHub considers authorized — typically runs originating from branches in the repository, but explicitly not from forks of public repositories opening pull requests. This default prevents an outside contributor from writing a workflow change that exfiltrates the secret on a CI run. Second, GitHub automatically masks secret values in workflow logs; a secret that accidentally appears in `echo` output is rendered as `***` rather than the plaintext. Masking is best-effort — a base64-encoded value is masked, but a value passed through `tr` or `cut` may slip through — so the safe pattern is to never echo a secret at all.

The store is the right primitive for any credential that the pipeline cannot avoid carrying. Examples include a Docker Hub Personal Access Token (PAT) for pushing images to a public registry, a third-party API key for a service the pipeline integrates with, or — the case this chapter shrinks — an Azure

service principal client secret. Storing one of these is necessary; making it unnecessary is better.

41.3 The service principal model

A **service principal** is an Azure-managed identity that represents a non-human principal (a CI pipeline, a scheduled job) and can be granted Azure RBAC roles; it enables automation without storing human passwords in code. Mechanically, a service principal is an instance of an Entra ID **application** plus a credential, with one or more **role assignments** that grant the application permissions on Azure resources. Creating one with the Azure CLI looks like this:

```
az ad sp create-for-rbac \
  --name "github-deploy-recruitment" \
  --role contributor \
  --scopes /subscriptions/<sub-id>/resourceGroups/recruit-
        ment-rg \
  --json-auth
```

The command emits a JSON document containing four fields the pipeline needs: `clientId` (the application's identifier), `clientSecret` (the long-lived password), `tenantId` (the Entra tenant), and `subscriptionId` (the Azure subscription the role assignment lives in). The `--json-auth` flag formats the output for direct consumption by the `azure/login` action in GitHub Actions.

The pipeline then stores the entire JSON blob — or just the four fields individually — as repository secrets, and an `azure/login` step reads them at runtime to obtain an Azure access token. The role assignment is `contributor` scoped to a single resource group, which is wide enough to update a

Container App and narrow enough to keep the blast radius small if the secret leaks.

This model is workable. It is also the failure mode the next section eliminates: the `clientSecret` is a long-lived password sitting in GitHub. If GitHub is compromised, if a maintainer's account is phished, if a workflow change accidentally prints the secret to a log that escapes masking, if the secret is rotated lazily — any of these turns into an attacker with `contributor` rights on a real resource group.

41.4 The federated-credential model

A **federated credential** is a trust relationship between a service principal in Azure and an external identity provider (e.g., GitHub); it allows the external provider to mint short-lived tokens that Azure validates, eliminating the need for a stored password. Instead of GitHub holding a copy of the secret, GitHub holds a private signing key and Entra holds the corresponding public key. When a workflow runs, GitHub mints a JWT signed with that private key — describing exactly which workflow run is asking for credentials — and the workflow exchanges that token with Entra for a real Azure access token.

OIDC federation (workload) is the practice of using OpenID Connect tokens issued by a CI/CD provider (GitHub Actions, GitLab CI) to authenticate to cloud providers (Azure, AWS, GCP); it replaces long-lived stored credentials with short-lived, automatically rotated tokens. The "workload" qualifier matters: the OIDC flow used here is structurally similar to the user-facing OIDC flow described in OAuth and OIDC, but the principal being authenticated is a CI pipeline, not a human at a browser. The user-flavour flow exists to log a person in; the

workload-flavour flow exists to log a workflow run in. Both rely on JWTs, but they solve different problems.

The exchange happens in three steps inside the pipeline:

1. The GitHub Actions runtime, on request from the workflow, generates a **short-lived token** — a bearer token issued by a CI/CD provider for a specific workflow run; it expires after a short duration (minutes to hours) and is automatically rotated on each run, reducing the blast radius of a compromise. The token's payload describes the run: the repository, the branch or tag, the workflow file, and the environment name.
2. The workflow presents this JWT to Entra ID's token endpoint, asking for an Azure access token in exchange.
3. Entra validates the JWT against the federated credential's trust configuration. If the JWT's claims match the configured **subject** string and the signature verifies against GitHub's published public keys, Entra issues a normal Azure access token, and the workflow uses it to call Azure as the configured service principal.

No password is ever stored. The pipeline carries no Azure credential between runs; the credential is reborn on every run from a JWT that itself only lives for a few minutes.

41.4.1 The federation subject string

The subject string is the load-bearing part of the trust configuration, because it controls which workflow runs Entra will accept tokens from. It encodes which repository, which kind of ref, and which specific ref the run must be on:

```
repo:cloud-developers/recruitment:ref:refs/heads/main
```

That subject says: trust GitHub-issued JWTs that come from the `cloud-developers/recruitment` repository, where the workflow ran on a push to the `main` branch. A run from any other branch presents a different subject and is rejected. The same applies to tags (`refs/tags/v1.0`), pull requests (`pull_request`), and named environments (`environment:production`). Each variant is its own federated credential entry — a project that deploys from `main` and from version tags needs two credentials configured, one per subject.

The subject is also where almost all OIDC misconfigurations live, because the format is unforgiving and the failure mode is silent. Common typos:

- `ref:refs/head/main` instead of `ref:refs/heads/main` (the trailing `s` is part of the standard Git ref path).
- `repo:cloud-developers/Recruitment:...` when the actual repo name is lowercased — the match is case-sensitive.
- `ref:refs/heads/main` when the workflow actually runs from a feature branch — the deploy is gated by the subject, so a workflow running from a `dev/...` branch will fail its login even if everything else is correct.
- `environment:production` configured but the workflow's job does not declare `environment: production` — the JWT will not include the environment claim, and Entra will reject the exchange.

The error from `azure/login` in any of these cases is a generic "AADSTS70021: No matching federated identity record found", which sends the operator hunting through the wrong end of the configuration. The fix is almost always at the federated-credential level: the subject string in Azure must literally match the JWT GitHub will issue, and any drift between the two is a deploy failure.

41.5 Comparison: long-lived vs federated

The two models are not equivalent. Federated credentials require more setup but eliminate the largest single category of CI/CD security incident.

PROPERTY	LONG-LIVED SERVICE PRINCIPAL SECRET	OIDC-FEDERATED SERVICE PRINCIPAL
Stored credential	`clientSecret` in GitHub secret	None
Credential lifetime	Until rotated (months to years)	Single workflow run (minutes)
Blast radius if leaked	Full RBAC scope, until detected	None — the JWT is already expired
Rotation effort	Manual; must update GitHub secret	Automatic; GitHub mints fresh JWT each run
Setup complexity	One CLI command, one secret	Service principal + federated credential per subject
Branch/tag scoping	None — any workflow run can use the secret	Built in via subject string
Audit trail	"Service principal X was used"	"Workflow run R used service principal X"

The federated model is the default the exercise on CI/CD to Azure Container Apps lands on by its third stage, and it is the

recommended pattern for any new CI integration with Azure. The long-lived secret model is still valid for pipelines whose CI provider does not support OIDC federation, but for GitHub Actions deploying to Azure there is no reason left to use it.

41.6 Worked example: azure/login with OIDC

The pipeline-side configuration to use a federated credential is small. The workflow declares `id-token: write` permission so the runner is allowed to mint OIDC tokens, and the `azure/login@v2` step receives only the public identifiers — no client secret.

```
permissions:
  id-token: write
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4

      - name: Azure login (OIDC)
        uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: $
        {{ secrets.AZURE_SUBSCRIPTION_ID }}

      - name: Deploy revision
        run: az containerapp update --name recruitment --
        resource-group recruitment-rg --image
        mycr.azurecr.io/recruitment@sha256:...
```

Three things deserve note. First, the three values passed to `azure/login` are public identifiers, not secrets — putting them in GitHub secrets is convention, not security. Leaking a `client-id` is harmless without a corresponding federated credential to back it. Second, the `permissions: id-token: write` line is mandatory and is the most common omission; without it, the runner refuses to mint the JWT and `azure/login` fails before it even reaches Entra. Third, the `environment: production` declaration does double duty: it gates the job behind whatever protection rules the environment has (manual approvers, branch restrictions), and it adds an `environment` claim to the OIDC token, which lets the federated credential's subject string be scoped tighter than just the branch.

The same workflow could equally well authenticate to a container registry using OIDC, push an image, and only then run the deploy step. The pattern generalizes: any cloud action that can be wrapped in an Entra-issued access token can be performed without a stored secret if the federation is configured.

41.7 A note on managed identity

Inside Azure, the equivalent of OIDC federation for non-Azure workloads is the managed identity attached directly to a VM, Container App, or function. A pipeline running on a self-hosted GitHub runner inside Azure can use the runner's managed identity directly and skip OIDC federation entirely. For pipelines running on GitHub-hosted runners (the common case), OIDC federation plays the equivalent role: an identity for a workload that lives outside Azure but needs to act inside it.

41.8 Summary

A CI pipeline that deploys to Azure needs an identity, and the choice of how that identity is provided governs the pipeline's security posture. The traditional approach — a service principal whose client secret lives as a GitHub secret — is functional but exposes a long-lived password to the largest attack surface in the system. The federated-credential approach configures Entra to trust short-lived JWTs minted by GitHub Actions for individual workflow runs; the pipeline carries no password, the token expires within minutes of issue, and the trust relationship is scoped tight to a specific repository, branch, and environment via the federation subject string. The setup mechanics are small — a service principal, a federated credential per subject, and an `azure/login@v2` step with `id-token: write` permission — and the failure modes cluster around typos in the subject string, which presents as a generic "no matching federated identity" error. For new GitHub Actions integrations with Azure, OIDC federation is the recommended default; the long-lived secret model remains a fallback only for CI providers that do not support it.

CHAPTER 42

Azure Container Apps as a Deployment Target

Watch the presentation
Se presentationen på svenska

A pipeline that builds, tests, and pushes an image is only half of a delivery story. The other half is the platform that receives the image and turns it into a running, addressable, scalable service. Plain virtual machines can do the job, but at the cost of patching, autoscaling, load-balancing, and certificate rotation that the team must manage by hand. A full Kubernetes cluster solves those concerns but introduces its own

operational tax — node pools, ingress controllers, RBAC, etcd backups — that is hard to justify for a handful of stateless web services. Azure Container Apps occupies the gap between these two extremes: it accepts an image, runs it as a managed service with autoscaling and HTTPS ingress already wired up, and exposes only the few primitives that an application team actually needs to reason about. This chapter explains those primitives and how they compose with the pipeline pieces from earlier in this Part.

42.1 Why a managed runtime for stateless containers

Most application services in a typical web system are stateless: they hold no durable state between requests, scale horizontally by adding replicas, and recover from a crash by being restarted on a different host. Running such services on Kubernetes works, but the operator carries the entire cluster lifecycle — upgrading the control plane, patching nodes, tuning the cluster autoscaler, configuring ingress, wiring observability — for the sake of a workload that only needs "run this container, give me a URL, and add replicas when traffic rises."

Azure Container Apps is a managed, serverless container service that runs Docker images without requiring container orchestration expertise; it handles scaling, networking, and revision management, making it ideal for deploying containerized applications without operational burden. The platform runs on top of Kubernetes internally, but the cluster is invisible: there are no nodes to size, no kubelets to patch, no ingress controllers to configure. The team interacts with three primitives — environments, container apps, and revisions — and lets Azure handle everything below.

The trade-off is reduced control. Container Apps imposes opinions about scaling behaviour, ingress, and lifecycle that a hand-rolled Kubernetes deployment could override. For services that fit those opinions — stateless HTTP APIs, web frontends, background workers driven by a queue — the saved operational cost is large. For services that need a custom CNI plug-in, a service mesh sidecar, or persistent volumes with strict topology requirements, AKS or another full orchestrator is still the right answer.

42.2 The Container Apps model

The platform is built from three layered concepts: the environment, the container app, and the revision.

A **Container App environment** is the shared infrastructure and network namespace in which one or more Container Apps run; it provides a virtual network boundary, shared logging, and a common internal DNS domain for service-to-service communication. An environment is the unit that the pipeline provisions once and then ignores. Apps inside the same environment can address each other by short DNS names; logs from every app in the environment flow into a single Log Analytics workspace; the environment's virtual network controls what can reach those apps from outside.

A **container app** is the deployable unit that runs an image. The app holds the metadata that defines a service: which image to run, what environment variables to inject, what scale rules to apply, and what ingress to expose. It belongs to exactly one environment. Most teams treat one container app as one microservice — `cloudci-api`, `cloudci-worker`, `cloudci-frontend` — and the environment as the runtime that hosts all of them.

A **revision** (in Azure Container Apps) is an immutable version of a Container App; when you deploy a new image or change configuration, a new revision is created automatically, and traffic can be split between revisions for canary or blue-green deployments. The revision is the layer where deployments actually happen. The container app is conceptually the long-lived service `cloudci`; the revision `cloudci--rev-3` is the specific version of `cloudci` that ran image tag `1.0.2` with a particular set of environment variables. Every change of substance — a new image, a changed env var, a scaled CPU limit — produces a new revision rather than mutating an existing one.

This three-level model maps cleanly to how a CI/CD pipeline thinks. The environment is provisioned by the platform team once; the container app is created when a service first ships; the revision is what the pipeline produces every time a new image moves through the build, test, and deploy stages.

42.3 Single and multiple revision modes

Container Apps offers two policies for how revisions live alongside each other, and choosing between them is the same conversation as choosing a deployment strategy.

In **single-revision mode**, every deployment replaces the active revision: traffic moves to the new revision, the old revision is deactivated, and only one revision serves users at any time. This matches the rolling-deployment model — fast, simple, low-overhead — and is the right default when an in-place replacement is acceptable.

In **multiple-revision mode**, the previous revision keeps running after a new one is deployed, and traffic is split between them according to a configured weight (e.g., 90% to

the old revision, 10% to the new one). This is the substrate for canary and blue-green deployments on Container Apps: a pipeline can publish a new revision at 0% traffic, run smoke checks against its dedicated FQDN, gradually shift traffic to it, and roll back instantly by reweighting if metrics degrade. The cost is the operational care of remembering to retire old revisions; the platform keeps up to 100 revisions per app, and stale revisions consume neither replicas nor money once their traffic weight is zero, but they do clutter the revision list.

42.4 Ingress

Stateless services are useless without a way for traffic to reach them, and a managed platform earns its keep by making that wiring trivial.

Ingress (in Azure Container Apps) is the network configuration that exposes a Container App to incoming traffic; it defines the external hostname (FQDN), port, transport protocol (HTTP, gRPC), and optionally internal-only or public visibility. When ingress is enabled on an app, the platform assigns a stable FQDN of the form `cloudci.<env-id>.<region>.azurecontainerapps.io`, terminates HTTPS using a managed certificate, and routes incoming requests to whichever revision the traffic-split policy points at. The team does not configure load balancers, certificates, or DNS — the platform owns all three.

Three ingress capabilities are worth knowing about explicitly. **HTTP routing** sits at Layer 7: the platform inspects the host header and the path and chooses a revision accordingly, which is what makes weighted traffic splits possible without IP-level games. **Custom domains** (e.g., `api.cloudci.example.com`) are bound to an app by adding a CNAME to the platform-issued FQDN and uploading or letting

the platform issue a certificate; from then on, the custom domain serves as the public face of the service. **mTLS** (mutual TLS) can be required on the environment level, so that every internal call between apps in the environment is authenticated and encrypted with platform-managed client certificates — useful for tightening service-to-service trust without adding sidecars.

Ingress can also be marked internal-only, in which case the FQDN is reachable only from inside the environment's virtual network. A common pattern is to expose an API-gateway app publicly and keep every backend service internal, so that the only entry point from the public internet is the gateway.

42.5 Scale rules

The reason to use a serverless container platform rather than a fixed-size deployment is autoscaling, and Container Apps surfaces it as a small set of declarative rules.

A **scale rule** (in Azure Container Apps) is a declarative policy that automatically adjusts the number of container replicas based on a metric (CPU, memory, HTTP requests per second, custom metrics); it enables horizontal autoscaling without manual intervention. The app declares a minimum and maximum replica count and one or more rules; the platform watches the chosen metric and adjusts the replica count between those bounds.

The built-in rule kinds cover the common cases. **HTTP concurrency** scales out when the number of simultaneous in-flight HTTP requests per replica crosses a threshold, which is the right choice for request-driven web services. **CPU and memory** scale on observed resource pressure, which suits CPU-bound workers more than chatty I/O-bound services. **Custom KEDA scalers** plug in any of the dozens of triggers

from the KEDA project — scale on Azure Service Bus queue depth, on Kafka consumer lag, on a cron schedule — for workloads driven by events rather than HTTP traffic. KEDA is the same autoscaler used by Kubernetes-native installations; Container Apps just exposes it without the cluster.

The minimum-replicas setting controls **scale to zero**. Setting `--min-replicas 0` lets the app shrink to no replicas when no traffic is flowing, in which case the team pays nothing for compute until a request arrives and the platform spins up a replica. The trade-off is a cold-start latency on the first request after idleness — typically a few seconds — which is fine for low-traffic internal tools and a problem for user-facing APIs with strict tail-latency requirements. Setting `--min-replicas 1` keeps one replica warm at all times.

SCALE RULE	BEST FOR	TRADE-OFF
HTTP concurrency	Request-driven web APIs	Needs realistic concurrency target per replica
CPU / memory	CPU-bound workers, batch	Lags behind traffic spikes
KEDA queue length	Event-driven workers	Requires queue-length metric to be cheap to read
Cron (KEDA)	Predictable load schedules	Does not react to unexpected demand

42.6 The deployment update flow

The end-to-end flow that makes Container Apps a useful CI/CD target is short, and most pipelines come down to a single command.

```
az containerapp update \
  -n cloudci \
  -g rg-cloudci \
  --image mycr.azurecr.io/cloudci:1.0
```

This command updates the container app `cloudci` in resource group `rg-cloudci` to run image `mycr.azurecr.io/cloudci:1.0`. The platform does not patch the running revision; instead it creates a new immutable revision, e.g. `cloudci--rev-7`, with the new image and the existing configuration. Under single-revision mode, traffic moves to `rev-7` as soon as it reports healthy and `rev-6` is deactivated. Under multiple-revision mode, both revisions stay active and the existing traffic-split weights apply until the pipeline (or an operator) re-assigns weights.

Observing the new revision afterwards is a one-liner:

```
az containerapp revision list \
  -n cloudci \
  -g rg-cloudci \
  --query "[].{name:name, active:properties.active,
        traffic:properties.trafficWeight, image:proper-
        ties.template.containers[0].image}" \
  -o table
```

The output makes the revision sequence concrete: each row is one immutable snapshot of the app, identified by its revision name and the image digest it ran. A pipeline's final smoke-test step typically reads this list to confirm that a new revision exists and is active before reporting the deployment as successful. The companion exercise CI/CD to Azure Container Apps walks through this update flow end-to-end against a real Container App.

42.7 Pulling images from ACR with managed identity

The naive way to let a Container App pull a private image is to give it a registry username and password as configuration. That is also the way that turns a CI repository into a place where ACR credentials live, where they leak when the repository is forked, and where they need to be rotated by hand. The platform-native alternative borrows the container registry chapter's identity-based authentication and removes the secret entirely.

A managed identity assigned to a container app attaches an Azure-managed principal to every replica at runtime. Granting that identity the `AcrPull` role on the registry — `az role assignment create --assignee <identity-id> --role AcrPull --scope <registry-id>` — and then pointing the app's registry configuration at the identity instead of at a username/password, lets the runtime fetch a short-lived ACR token whenever it needs to pull an image. No password is stored anywhere; rotation is handled by Azure; the same primitive is reused by other Azure services that need to read from ACR. Managed identities are the recommended default for any Container App that pulls from a private registry it controls.

42.8 Composing with the rest of the Part

The Container Apps primitives plug into the earlier chapters of this Part with little ceremony. The pipeline builds and pushes the image; the smoke gate curls the app's FQDN after `az containerapp update` to confirm the new revision is healthy; the deployment strategy decides whether the pipeline runs in single-revision mode (replace) or multiple-revision mode (canary or blue-green); the pipeline-secret

chapter explains how the workflow itself authenticates to Azure to call `az containerapp update` without a stored secret. Container Apps does not require any of these to be in place — a developer can deploy a revision from a laptop with `az` — but the platform was designed around them, and the primitives were chosen so that a CI/CD pipeline can drive every operation through the CLI.

The result is a deployment target that occupies a deliberate middle ground: enough abstraction to disappear from the operator's day-to-day workload, enough hooks to integrate cleanly with a real pipeline, and enough flexibility to support the deployment strategies that production traffic actually demands.

42.9 Summary

Azure Container Apps runs Docker images as a managed serverless platform, removing the operational burden of nodes, ingress controllers, and certificate rotation that a hand-rolled Kubernetes deployment carries. Its model has three layers: a Container App environment provides the network and observability boundary; a container app is the deployable service; a revision is the immutable snapshot produced every time the image or configuration changes. Single-revision mode replaces the active revision on each deploy, while multiple-revision mode keeps old revisions running so that traffic can be split for canary or blue-green strategies. Ingress wires up HTTPS with managed certificates and supports custom domains and mTLS without bespoke load-balancer configuration. Scale rules — HTTP concurrency, CPU, custom KEDA scalers — adjust replica counts automatically, and a minimum of zero replicas turns the platform into a true serverless billing model at the cost of cold-start latency. A

pipeline drives all of this through `az containerapp update --image`, which creates a new revision atomically, and through a managed identity that grants the app `AcrPull` on its registry so that no credentials need to be stored anywhere.

PART IX

Operations and Observability

Once an application is deployed, it becomes a black box unless its operators have made it observable. This Part covers the signals that reveal what an application is doing in production and the tooling that turns those signals into answers: the three pillars of observability, structured logging, Application Insights, KQL queries against Log Analytics, health checks, and alerts.

The companion exercise is the Logging and Monitoring chapter, which wires up the same `CloudCi` application progressively from container-stdout-only logging

through Application Insights to KQL-based dashboards and alerts.

CHAPTER 43

The Three Pillars: Logs, Metrics, Traces

Watch the presentation
Se presentationen på svenska

A deployed application is a black box. Once the binary leaves a developer's laptop and starts serving users in the cloud, the only visibility into its behaviour is whatever the application chooses to emit. If it emits nothing, the operator can confirm the process is alive — and almost nothing else. The request that returned a 500 error to a user this morning is, from the operator's seat, indistinguishable from a request that succeeded. The instinct to attach a debugger does not survive the transition to production: there is no debugger to

attach, no breakpoints to set, and no second chance to reproduce a fault that already happened.

This chapter develops the three signal types that close that gap — logs, metrics, and traces — and shows how each one answers a different question. The companion exercise Logging and Monitoring is where these pillars become concrete on a running .NET application; the prose here establishes the vocabulary and the trade-offs that the exercises assume.

43.1 From monitoring to observability

The older discipline of monitoring asks a fixed set of questions of a system: is the CPU above 80%? Is the disk full? Has the website returned 5xx errors more than three times in the last minute? Each question is encoded as a check, each check has a threshold, and the dashboard lights up when a threshold is crossed. Monitoring is excellent at telling you that something broke. It is poor at telling you why.

Observability is the ability to infer the internal state of a system from the outputs it produces — logs, metrics, and traces. A system is observable when the questions asked of it can be answered without adding new instrumentation, relying instead on the signals the system already emits. The shift from monitoring to observability is a shift from a fixed list of dashboards to an open-ended capacity to ask new questions about behaviour the developers did not anticipate. Monitoring catches the failure modes you predicted; observability lets you investigate the ones you did not.

The collective term for the data that makes observability possible is **telemetry** — the structured logs, metrics, traces, and events that a system emits for external collection and analysis. Telemetry enables visibility into running applications without changing the code to add logging statements or

metrics collection. The application emits a stream; a backend collects, indexes, and queries that stream. The investigator's tools sit on the backend, not on the running process.

43.2 Logs: the narrative of what happened

Logs are timestamped, structured records of discrete events that occurred in the system; they form the narrative of what happened, in what order, with what context attached. Unlike metrics, logs capture the full detail of a single event (the error message, the exception stack, the request ID); unlike traces, logs exist within a single process and do not automatically cross service boundaries.

A well-written log line for a checkout request might record the user ID, the cart total, the payment provider, the response code, and a correlation ID that ties all log lines from the same request together. Replaying that line three months later, the investigator can reconstruct what the application was doing at that exact moment for that exact user. No other signal type carries this level of per-event detail.

Logs excel at narrative reconstruction — "what exactly happened to user 42 at 14:03 yesterday?" — and at exception analysis, where the stack trace is the load-bearing payload. They are also the signal that survives the developer's intuition of what to instrument: a `Warning` log line written eighteen months ago for a once-in-a-blue-moon edge case is still there when the edge case finally fires.

The cost of this richness is volume. Every event becomes a row in a log store. A modest web application emits tens of thousands of log lines per minute; a busy one emits millions. Log Analytics, Datadog, Splunk, and other backends charge per gigabyte ingested and per gigabyte retained. Aggregating logs to answer "what was the average response time over the

last hour?" forces the query engine to scan and group every relevant row — expensive, and slower than asking the same question of a metric.

43.3 Metrics: numbers over time

Metrics are numeric measurements sampled at regular intervals (e.g., every second or minute) that describe the state or behaviour of a system; examples include CPU usage, request count per minute, and response time percentiles. Metrics are cheap to aggregate and store, making them ideal for answering "how is the system performing in aggregate" but poor at answering "what was the exact sequence of events."

A metric is an aggregate by construction. The series `http_requests_total` does not record individual requests; it records the count of requests in each one-minute bucket, possibly split by status code or route. The series `http_request_duration_seconds_p95` does not record any individual request's latency; it records the 95th percentile of all request durations in each bucket. The aggregation is the data — there is no underlying detail to drill into.

This shape makes metrics dramatically cheaper to store and query than logs. A counter that increments a billion times per day still produces only a few thousand data points (one per minute, or one per ten seconds for higher-resolution series). A dashboard rendering a week of CPU usage scans a few thousand rows; the equivalent in logs would scan billions.

Metrics are the signal of choice for dashboards, alerts, and capacity planning. A graph of error rate over the last hour, a threshold that fires when latency exceeds 500 ms, a long-term trend that shows traffic doubling year-over-year — these are metric problems. Metrics are also the foundation of service-level objectives (SLOs): the SLO "99.9% of requests suc-

ceed over a 30-day window" is computed from a request-success counter, not from a search of the log store.

The trade-off cuts the other way for diagnosis. A metric tells you the error rate spiked at 14:03; it cannot tell you which user was affected, what they were trying to do, or what the stack trace looked like. For that, the investigator needs a different signal.

43.4 Traces: end-to-end flow across services

Traces (in the distributed tracing sense) are causal chains of operations that flow through multiple services, tied together by a shared correlation ID (operation ID); they answer "how did a single user request flow through the system and where did time get spent." A trace includes one or more spans, each representing a unit of work (an HTTP request, a database query, a message consumption).

A **span** is a named unit of work within a distributed trace, representing a single operation; it carries a start time, end time (duration), a status (success or error), and any events or attributes that occurred during its execution. Multiple spans from different services are linked by a shared trace ID and parent-span ID to form a complete trace. The result is a tree: a root span for the inbound HTTP request, child spans for each downstream call (the SQL query, the call to a payment provider, the message published to a queue), and grandchild spans for whatever those services in turn called.

Traces close the gap that logs and metrics leave open: they make a request's journey across service boundaries visible. In a system of three services where a checkout call traverses an order service, a payment service, and a shipping service, a log line in the shipping service does not automatically know that it belongs to the same user request as a log line in the

order service. A trace ties them together. The W3C Trace Context standard defines the header (`traceparent`) that propagates the trace ID across HTTP boundaries; instrumented frameworks read and write it automatically, so the correlation survives every hop.

The cost of traces is instrumentation. Every service must emit spans, and every cross-service call must propagate the trace context header. Auto-instrumentation libraries — for ASP.NET Core, for HttpClient, for Entity Framework, for Azure SDK — cover most cases without code changes; custom logic still needs explicit spans for the work that matters. At high volume, traces are usually sampled (1% of requests, or 100% of failed requests) to keep ingestion costs bounded, with statistical reweighting to preserve aggregate accuracy.

43.5 How the three pillars combine

No single pillar is sufficient. Logs without metrics turn every dashboard question into an expensive query. Metrics without logs reduce every incident to a graph that confirms the failure but explains nothing. Traces without either narrate the flow of one request but cannot answer “is this request typical?”

PILLAR	BEST AT	FALLS SHORT WHEN
Logs	Narrative reconstruction; exception detail	Aggregate questions over high volume
Metrics	Dashboards, alerts, SLOs, trends	Per-request detail; root-cause investigation
Traces	Cross-service request flow; latency attribution	Storage cost at full sampling; frameworks must be instrumented

A mature observability stack uses each for what it does best. Metrics drive the dashboards on the wall and the alerts on the pager. Traces drive the latency investigation when the dashboards report a slow endpoint. Logs drive the root-cause analysis when the trace identifies the failing span. The three pillars share a correlation key (operation ID, request ID) so that an investigator who starts in one can pivot to the others without losing the thread.

43.5.1 Worked example: a single failing request

A user reports that submitting a cart fails intermittently around 14:00. The on-call engineer opens the observability stack and walks through the three pillars.

Metric — the aggregate signal. The dashboard shows the metric `http_requests_failed_total{route="/checkout"}` spiking from a baseline of 0–1 per minute to 47 per minute around 14:03, then settling back to baseline ten minutes later. The metric establishes the shape of the incident: a brief failure window, several dozen affected requests, route-spe-

cific. It does not say which users, which requests, or which downstream call.

```
http_requests_failed_total{route="/checkout"}
  14:01  1
  14:02  0
  14:03 47   <-- spike
  14:04 38
  14:05 12
  14:06  0
```

The metric is the right tool for noticing the incident and characterising its scope, but the investigator now needs detail.

Trace — the request-flow signal. The engineer pivots to the trace store, filters to traces with status 500 on the `/checkout` route in the spike window, and opens one. The trace shows a 4.8-second root span for the inbound POST, with most of the time spent in a single child span: a call to the payment provider's `/charge` endpoint that timed out at 5 seconds.

```
POST /checkout                    4823ms  ERROR
├── SQL SELECT cart_items           12ms  OK
├── HTTP POST payment.example      5000ms  ERROR (timeout)
└── (no further spans — request aborted)
```

The trace identifies the failing dependency: the payment provider, not the database, not the application code. It also identifies the symptom: timeouts, not 4xx responses, suggesting the provider was unresponsive rather than rejecting requests.

Log — the per-event detail. The engineer copies the operation ID from the trace and pivots to the log store, querying for that ID. Three log lines appear: an `Information` line at the start of the request, a `Warning` line when the HttpClient cancellation token fired, and an `Error` line with the full excep-

tion stack trace from the `TaskCanceledException`. The error log line includes the user ID, the cart contents, the request body, and the correlation ID — enough context to call the user and confirm the cart was not double-charged.

The same incident, seen through three signals, yields three different parts of the answer. Metrics said *something is wrong*. Traces said *the payment provider is the bottleneck*. Logs said *here is the exact failure for this exact user*. None of the three would have produced this complete picture alone.

43.6 Cost trade-offs in practice

Each pillar carries a different cost shape, and a working observability budget allocates spend accordingly.

Logs are the most expensive per question answered. Storage scales with event volume, and queries scale with the time range scanned. Teams reduce log cost by sampling at the source (drop debug logs in production), retaining recent data hot and older data cold, and pushing aggregate questions to metrics instead.

Metrics are the cheapest at scale. A pre-aggregated counter costs almost nothing to store and almost nothing to query, regardless of underlying request volume. Metric cost grows with cardinality — the number of distinct label combinations — not with raw event count. A counter labelled by `status_code` is cheap; a counter labelled by `user_id` is catastrophic, because every user creates a new series.

Traces sit between the two. A fully sampled trace store has logs-like cost characteristics; a 1%-sampled store has metric-like cost with logs-like detail on the sampled subset. Adaptive sampling — keep all errors and slow requests, drop a fraction of the rest — gives most of the diagnostic value at a fraction of the price.

A reasonable starting allocation for a small ACD-scale application: metrics on every signal that drives a dashboard or alert (cheap, always on); logs at `Information` level for business events and `Warning`/`Error` for failures (medium cost); traces auto-instrumented on framework calls with adaptive sampling at higher volume (medium cost, full detail on errors).

43.7 Summary

A deployed application emits three kinds of signal that, together, make it observable. Logs record discrete events with full per-event context, ideal for narrative reconstruction and exception detail but expensive at high volume. Metrics record numeric measurements aggregated into time buckets, cheap and ideal for dashboards, alerts, and SLOs but unable to describe individual requests. Traces record the end-to-end flow of a single request across services as a tree of spans, ideal for latency attribution and cross-service investigation. None of the three is sufficient alone: metrics notice an incident, traces locate the failing dependency, logs explain the exact failure for the affected user. The remaining chapters of this Part build on this foundation — structured logging with `ILogger<T>`, Application Insights as the .NET telemetry sink, KQL over Log Analytics, health checks, and alerts grounded in service-level objectives — each one developing a specific capability of the three-pillar model on a running .NET application.

CHAPTER 44

Structured Logging with ILogger

Watch the presentation
Se presentationen på svenska

A single application running on a developer laptop produces a few hundred log lines a day, and a human can read them. The same application running across several replicas in production produces hundreds of thousands per hour — most of them irrelevant most of the time, and the few that matter buried inside free-text strings that no machine can usefully filter. The discipline that closes this gap is structured logging: writing log calls so that the framework retains both the rendered text for humans and a set of named fields for

machines. This chapter develops what structured logging means, how `ILogger<T>` and message templates implement it in ASP.NET Core, when each log level applies, and how scopes and enrichment attach context without cluttering the call sites.

44.1 From free-text strings to queryable data

A traditional log line is a string. Code such as `Console.WriteLine($"User {userId} signed in from {ip}")` produces output like `User 4129 signed in from 10.0.2.5`. The values are present, but they are concatenated into the message. To find every sign-in by user 4129, an operator must search for the substring `User 4129` and hope no other message uses the same prefix. To count sign-ins per IP address, the operator must extract the address with a regular expression, hoping the message format never changes.

Structured logging is the discipline of writing log lines with constant message templates containing named placeholders (e.g., `"User {UserId} signed in at {Timestamp}"`) rather than string interpolation; the placeholders preserve semantic fields that can be queried and aggregated downstream, transforming logs from searchable text into queryable data. The same call now emits two things in parallel: a rendered string for humans (`User 4129 signed in from 10.0.2.5`) and a structured record where `UserId = 4129` and `IpAddress = 10.0.2.5` are first-class fields. Downstream stores like Log Analytics index those fields as columns. A query for “all sign-ins by user 4129 in the last hour” becomes a filter on a column, not a substring search across a text blob.

The key insight is that logs are most valuable when their structure is preserved end-to-end. Every transformation that

flattens the structure into prose loses information. The motivation for the rest of this chapter is to keep that structure intact from the call site to the query.

44.2 How `ILogger<T>` works

ASP.NET Core builds on top of an abstraction called Dependency injection, and logging is one of the services it ships with by default. The host registers an open generic mapping from `ILogger<>` to the framework's `Logger<>` implementation. Any class that asks for `ILogger<SomeType>` in its constructor receives a logger that has already been configured with output sinks, filters, and formatters.

`ILogger<T>` is the generic logging abstraction in ASP.NET Core; it is registered by the host and injected into controllers, services, and middleware. The type parameter `T` (typically the class being logged) becomes the log category, enabling per-category filtering through configuration. `ILogger<T>` enforces message templates at compile time through overloads and is designed for structured logging with named fields.

The category matters because filtering happens per category. A `NewsletterController` logger has the category `Acme.Web.Controllers.NewsletterController`. A configuration block like `"Logging": { "LogLevel": { "Acme.Web.Controllers": "Debug" } }` raises the verbosity of every controller without affecting framework noise from `Microsoft.AspNetCore.*`. Category matching is hierarchical: the framework looks for an exact match first, then walks up the namespace prefixes, and finally falls back to `Default`.

44.2.1 Message templates

A **message template** is a constant string containing named placeholders (e.g., `"Order {OrderId} shipped to {Address}"`) passed as the first argument to an `ILogger` method; the values corresponding to each placeholder are passed as trailing arguments. The template and values are kept separate by the logging framework, enabling structure preservation even when the final formatted message is rendered as text.

Two calls illustrate the difference. The first uses string interpolation:

```
logger.LogInformation($"User {userId} signed in from {ipAd-
        dress}");
```

The second uses a message template:

```
logger.LogInformation("User {UserId} signed in from {IpAd-
        dress}", userId, ipAddress);
```

Both produce the same human-readable string. The interpolation form, however, hands the framework a string that has already been collapsed: the values are no longer separable from the surrounding text. The template form hands the framework two things it can preserve independently — the constant template `User {UserId} signed in from {IpAd-dress}` (which becomes a stable identifier for this kind of event) and the values `{ UserId = ..., IpAddress = ... }` (which become structured fields).

Placeholder names are PascalCase by convention because they become field names in the output. The names matter; positions do not. The framework matches placeholders to arguments by left-to-right order, but the field name is taken from the template. A field consistently called `UserId` across

the codebase aggregates cleanly in queries; one called `userId` in some calls and `user_id` in others does not.

A subtle but important consequence: never call `logger.LogInformation(message)` where `message` is itself an interpolated or concatenated string built at the call site. The first argument is a constant template, not a payload. Tools like `LoggerMessage.Define` and the source-generated `[LoggerMessage]` attribute help enforce this at compile time.

44.3 Log levels

A **log level** is a severity classification assigned to a log message; ASP.NET Core defines six levels in order of severity: Trace, Debug, Information, Warning, Error, and Critical. Per-category filtering through `appsettings.json` controls which levels are emitted, allowing developers to tune the verbosity of each component independently without redeploying code.

Each level has a distinct use:

- **Trace** is for the most fine-grained signals, often values that change inside a tight loop. Trace output is normally disabled outside targeted debugging. It can include data that should not leave the developer's machine.
- **Debug** is for diagnostic detail useful while developing or troubleshooting a specific incident. Decision branches taken, payload sizes, intermediate identifiers. Off by default in production for cost and noise reasons.
- **Information** is the default level for the application's normal flow: a request started, a job completed, a user signed in. These are the events an operator wants to confirm "the system is doing the expected thing."
- **Warning** flags something unexpected but recoverable: a configuration value missing and falling back to a default, a

retry that succeeded on the second attempt, a slow query that finished. The system continued; a human should know it happened.

- **Error** marks an operation that failed in a way the user noticed: a request that returned 500, a database write that threw, a background job that aborted. One log per failed operation, with the exception attached.
- **Critical** is for system-level failure that requires immediate intervention: data loss, security boundary violation, the process about to terminate. These should be rare and always actionable.

The discipline is to choose one level per call and stick to its meaning. A codebase that logs every loop iteration at Information drowns the operator in noise; one that logs every failure at Warning understates the severity and erodes trust in alerts. Production environments typically run with Information enabled for the application namespaces and Warning or higher for framework namespaces.

44.4 Logging scopes

A request enters a controller, passes through several services, queries a database, and writes a response. Each layer logs something. Without correlation, the operator sees a flat sequence of unrelated lines and cannot tell which lines belong to which request. The fix is to attach a per-request identifier to every line emitted while that request is being processed.

A **logging scope** is a context established with `ILogger.BeginScope(...)` that attaches key-value pairs to every log call within that context; scopes are composable (nested) and commonly used to attach a correlation ID to all log lines

from a single HTTP request or background job without changing each individual log call.

The pattern is a `using` block:

```
using (logger.BeginScope(new Dictionary<string, object>
{
    ["CorrelationId"] = correlationId,
    ["UserId"] = userId,
}))
{
    logger.LogInformation("Loading newsletter {Newslet-
        terId}", id);
    var newsletter = await repository.GetAsync(id);

        logger.LogInformation("Newsletter {NewsletterId}
        loaded ({ItemCount} items)",
        id, newsletter.Items.Count);
}
```

Both `LogInformation` calls inside the `using` block emit not only their own templated fields (`NewsletterId`, `ItemCount`) but also the scope fields (`CorrelationId`, `UserId`). The fields appear at the structured-data level — they are not concatenated into the message text. Scopes nest: a request-level scope might wrap a tenant-level scope, and both sets of fields attach to every inner log line.

Scopes only function when a logging provider opts in. The default console provider has scopes disabled by default; enabling them requires `IncludeScopes = true` in configuration. The JSON console formatter and most production sinks (Application Insights, Serilog, OpenTelemetry) include scope fields automatically.

44.5 Enrichment

Some context belongs on every log line emitted by the application, regardless of which request triggered it: the machine name, the process ID, the application version, the deployment environment. Adding this on each call site is repetitive and error-prone.

Log enrichment is the practice of adding contextual fields to log records automatically — either globally (e.g., machine name, process ID via an initializer) or per-request (e.g., correlation ID via a scope). Enrichment keeps log calls focused on business logic while infrastructure details are injected by the framework.

Enrichment is configured once at host setup, typically in `Program.cs`. Application Insights, for example, exposes telemetry initializers that run for every emitted item and can stamp it with `cloud_RoleName`, `cloud_RoleInstance`, and the build SHA. Serilog and OpenTelemetry expose similar pipelines. The result is that every log line — without any change at the call site — carries the host context that an operator needs to filter by deployment, replica, or version.

A useful split is to think of enrichment in two layers. Global enrichment adds fields that never change for the lifetime of the process: machine name, build SHA, environment name. Per-request enrichment adds fields that change per request: correlation ID, authenticated user ID, tenant ID. Global enrichment is configured at startup; per-request enrichment is added by middleware that opens a scope at the start of the request and disposes it at the end.

44.6 A worked example

Consider a controller for a newsletter service. Constructor injection takes an `ILogger<NewsletterController>`; the

action wraps its work in a per-request scope and emits an Information line at the start and end of the operation.

```
public class NewsletterController : Controller
{
    private readonly ILogger<NewsletterController> _logger;
    private readonly INewsletterRepository _repository;

    public NewsletterController(
        ILogger<NewsletterController> logger,
        INewsletterRepository repository)
    {
        _logger = logger;
        _repository = repository;
    }

    [HttpPost]
    public async Task<IActionResult> Subscribe(string
        email)
    {
        var correlationId = HttpContext.TraceIdentifier;

        using (_logger.BeginScope(new Dictionary<string,
        object>
        {
            ["CorrelationId"] = correlationId,
        }))
        {
            _logger.LogInformation(
                "Subscribe requested for {Email}", email);

            var result = await _repository.AddSubscriberA-
        sync(email);

            if (result.AlreadyExisted)
            {
                _logger.LogWarning(
                    "Subscribe noop — {Email} already sub-
        scribed", email);
                return Ok();
            }

            _logger.LogInformation(
                "Subscribe completed — {Email} added as
        {SubscriberId}",
                email, result.SubscriberId);
```

```
            return Ok();
        }
    }
}
```

Three concerns are visible in the snippet. The category is bound to `NewsletterController` through the generic parameter, so configuration can dial logging for this controller independently from the rest of the app. The scope adds `CorrelationId` to every inner log line without each call having to mention it. The templates use PascalCase placeholders (`Email`, `SubscriberId`), which become structured fields in the output. A query that asks "how many Subscribe noops did user X cause yesterday" answers itself: filter the table by the constant template, group by `Email`, count rows.

A note on the `Email` field in this example: an email address is personally identifiable information. In a real implementation it would either be hashed before logging, replaced with the surrogate `SubscriberId`, or omitted entirely. The example shows the mechanism; the operational discipline follows.

44.7 Operational practices

Structured logging unlocks queryable data, but it does not by itself impose discipline on what gets logged. Three practices matter most:

No PII in logs. Email addresses, phone numbers, full names, government identifiers, payment instruments — none of these belong in log streams. Logs are typically retained longer, replicated more widely, and exposed to more roles than the production database. Replace identifiers with surrogates (`SubscriberId` instead of `Email`), or hash sensit-

ive values at the boundary so the log captures correlation without exposure.

No secrets in logs. API keys, connection strings, bearer tokens, and signed URLs all leak via logs more often than via code repositories. Treat logging as a sink that any operator with workspace read access can query, and assume any field written to it has been disclosed. Configuration values, request bodies, and exception messages all need scrubbing before they reach the logger.

Sample at high volume. A handler that emits a hundred log lines per request is fine in development; it costs significant ingestion budget once the application sees serious traffic. Drop chatty Debug calls in production. Use level filters per category to silence the framework's per-request lifecycle traces. For high-frequency events that still matter (e.g., per-cache-miss), consider sampling — log one in N — rather than every occurrence.

A fourth, smaller practice closes the loop: keep the message template constant. Every time the template changes, the structured event identity changes, and dashboards keyed on the old template stop counting. Add or rename fields, but resist the urge to "improve the wording" of a template that downstream queries already depend on.

The companion exercise Logging and Monitoring walks through wiring `ILogger<HomeController>` into a deployed ASP.NET Core MVC app, switching between the plain text and JSON console formatters, and watching the same structured fields appear in both `dotnet run` and Container Apps stdout.

44.8 Summary

Structured logging treats each log call as two outputs at once: a rendered string for humans and a set of named fields for machines. ASP.NET Core implements this through `ILogger<T>`, which the host registers in dependency injection, parameterised by the logging category. Message templates with PascalCase placeholders preserve the field names downstream, so queries can filter and aggregate on them as columns rather than substring-search a text blob. Six log levels — Trace, Debug, Information, Warning, Error, Critical — partition severity, and `appsettings.json` controls per-category verbosity without code changes. Logging scopes attach correlation fields to every inner log line, and host-level enrichment stamps machine and deployment context onto every record. The operational discipline that makes this useful in production is the unglamorous half: no PII, no secrets, sample at high volume, and keep the templates stable so dashboards survive.

PART IX · OPERATIONS AND OBSERVABILITY

CHAPTER 45

Application Insights and Telemetry

Watch the presentation

Se presentationen på svenska

A controller that calls `_logger.LogInformation("Order {OrderId} shipped", id)` produces a log line — but a log line is only useful if something downstream collects it, indexes it, and lets a developer search across many of them at once. The previous chapter on structured logging with ILogger made the call site disciplined; the question that this chapter answers is where those calls go once the application is running on a managed platform, and how that destination grows from "a place that stores log lines" into a full picture of

the system's behaviour. Application Insights is the answer that the .NET ecosystem reaches for first: a managed service that captures logs, request and dependency timing, exceptions, and custom telemetry from a deployed application without changes to most call sites. This chapter covers what the service actually is, how the SDK auto-instruments an ASP.NET Core app, the dashboards developers reach for during an incident, and the cost lever — sampling — that prevents a chatty service from generating an unmanageable bill.

45.1 Why an APM destination

The framework's `ILogger` writes to whatever logging providers are registered. In development, that is the console. In a container running on Azure Container Apps, stdout flows into the Log Analytics workspace attached to the environment, which is enough to grep through container output with KQL. What is missing from that picture is everything that is not a log line: how long each HTTP request took, which downstream calls it made, which exceptions threw with what stack trace, and how all of those signals correlate per request. A pure log stream answers "what happened" one event at a time but leaves the developer to reconstruct the rest by hand.

Application Insights is a Microsoft Application Performance Management (APM) service that collects and analyzes telemetry from deployed applications, providing dashboards for Live Metrics, Application Map (dependency graph), Failures (exception analysis), Metrics, and Logs. It integrates with Log Analytics and can run in workspace-based mode so telemetry and container logs coexist in the same queryable store. In other words, it is the destination that turns the disciplined `ILogger` calls of the previous chapter into a navigable picture of a running system, and it adds three more pillars —

request timing, dependency timing, and exception capture — that the application code does not have to author by hand.

Two operating modes exist for historical reasons. Legacy Application Insights kept telemetry in its own backing store, separate from the Log Analytics workspace where Container Apps streams stdout. Workspace-based mode points the component at an existing Log Analytics workspace, so Application Insights tables and `ContainerAppConsoleLogs` live side by side. Workspace-based mode is the default for new components and is the only mode worth using: the next chapter on KQL works against both kinds of data through one query language, and joining a request from the `requests` table with the container's stdout from `ContainerAppConsoleLogs` only works when both are in the same workspace.

45.2 How the SDK captures telemetry without explicit calls

The reason Application Insights becomes useful with a single configuration line is that the .NET SDK auto-instruments the runtime. Adding `services.AddApplicationInsightsTelemetry(...)` to `Program.cs` registers a chain of telemetry initialisers and processors that hook into ASP.NET Core's request pipeline, the HTTP client factory, the SQL client, and the Azure SDK clients. From that point on, the SDK observes:

- **Requests**: every HTTP request the app handles, with its URL, method, duration, status code, and an operation ID that ties together everything that happened during that request.
- **Dependencies**: every outbound call the app makes — HTTP calls via `HttpClient`, SQL queries via `SqlClient`,

Service Bus and Storage calls via the Azure SDK — annotated with target, duration, and success or failure.

- **Exceptions**: every unhandled exception that bubbles up to the framework, with its type, message, stack trace, and the operation ID of the request that caused it.
- **Traces**: every `ILogger` line at or above the configured level, captured into the `traces` table with the same operation ID as the request that produced it.

None of these require explicit `Track*` calls in application code. A controller method that throws an `InvalidOperationException` produces a `requests` row with a 500 status, an `exceptions` row with the stack trace, several `traces` rows for the framework's own log output, and possibly `dependencies` rows for any database queries the request executed before throwing — all sharing one operation ID. This automatic correlation is what makes the Failures blade and the Application Map possible without instrumented code.

The catch is that auto-instrumentation only captures what the SDK knows how to hook. A custom protocol over a raw socket, a worker process draining a queue without using one of the supported clients, or a mid-request side effect that does not flow through `HttpClient` will produce no dependency telemetry on its own. For those, the application has to opt in with an explicit telemetry-client call (covered later under Custom events and metrics).

45.3 Configuration via connection string

The SDK is registered once and configured by environment variable, which keeps secrets out of source control and lets the same image run against different Application Insights components in different environments.

The connection string is the configuration format that replaced the legacy instrumentation key. It includes the instrumentation key, the regional ingestion endpoint, and the Live Metrics endpoint, all in one string. The SDK reads it from the `APPLICATIONINSIGHTS_CONNECTION_STRING` environment variable by default, which means a Container App pointed at an Application Insights component needs only one secret reference to be fully wired up.

The wire-up sequence in the companion exercise looks like this. The Application Insights component is provisioned in workspace-based mode against the Log Analytics workspace that the Container Apps environment already owns. Its connection string is fetched from the resource's properties and stored as a Container Apps secret — `appinsights-cs` — rather than as a literal env var, so that `az containerapp show` does not surface the value in plaintext. The container app then declares an environment variable `APPLICATIONINSIGHTS_CONNECTION_STRING` whose value is `secretref:appinsights-cs`, which Container Apps decrypts and exposes to the running process at request time. From the SDK's perspective, the value is just an env var; from the platform's perspective, it never appears in any configuration export.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddApplicationInsightsTelemetry(options =>
{
    options.ConnectionString =
        builder.Configuration["APPLICATIONIN-
        SIGHTS_CONNECTION_STRING"];
});

builder.Services.AddControllersWithViews();

var app = builder.Build();
```

The single `AddApplicationInsightsTelemetry` call registers everything needed: the auto-instrumentation modules, the telemetry initialisers that stamp each item with the operation ID, the adaptive sampling processor, and the `TelemetryClient` that custom code can inject. Reading the connection string from configuration rather than relying on the default env-var lookup keeps the wiring explicit, which matters when the app needs to fail fast in development if the value is missing.

45.4 The dashboards developers actually use

Application Insights ships with many blades, but four are reached for repeatedly during normal operation and during incidents.

Live Metrics is a real-time Application Insights dashboard showing current request rate, latency, server health (CPU, memory), and sample requests with one-to-two-second latency; it uses a separate push-based channel (not the regular ingestion pipeline) and is free because it bypasses storage, making it ideal for real-time monitoring during deployments or incident response. The page shows the last 60 seconds of

activity, refreshing once per second. Because it bypasses ingestion, there is no 1–3 minute delay between the application emitting a signal and the dashboard rendering it, which is the property that makes it the dashboard to leave open while a deployment rolls out. If a new revision starts throwing exceptions, the failure rate spikes on Live Metrics within a couple of seconds; if the regular Failures blade is the only signal, the operator waits a minute or more before seeing anything.

The **Application Map** is an Application Insights visualization of system topology built from request and dependency telemetry; each node represents a logical service, and each edge represents a dependency (HTTP call, database query, message queue) annotated with p95 latency and failure rate. The map is generated automatically as the application makes calls to external services. For a single-service exercise, the map shows one node with its inbound request rate and any outbound dependencies — Storage, SQL, an HTTP API — drawn as edges. For a multi-service production system, the map becomes the answer to "which service is slow" or "which service is calling the database hardest", because the edges carry their own latency and failure-rate annotations.

Failure analysis, exposed by the Failures blade, refers to the part of the portal that groups exceptions by type and displays stack traces, affected operations, and sample instances. It allows developers to understand the distribution and impact of errors without querying raw logs, and links each failure back to the parent request via operation ID. The blade defaults to a time window — say the last 24 hours — and lists exception types ranked by occurrence, request URLs ranked by failure rate, and dependency targets ranked by failure rate. Clicking a row drills down to a sample failed operation, which contains the request, the exception, the stack trace, and every

log line and dependency call that ran before the failure — the full reconstruction of one bad request, in one place.

The **Performance** blade does the same triage exercise for slow operations rather than failed ones. It ranks operations by p50, p95, or p99 latency, ranks dependencies by their contribution to slow operations, and links each row to a sample slow operation with the same drill-down semantics as Failures. Together with Failures, it covers the two failure modes that user-facing systems are most often debugged for: requests that errored, and requests that completed but were slow.

45.5 Sampling as a cost lever

Telemetry is not free. Application Insights bills per gigabyte ingested into the workspace, and a chatty service that records a row per request, a row per dependency, a row per log line, and a row per exception can run into surprising bills at modest traffic. Sampling is the lever that bounds that cost without giving up statistical accuracy.

Sampling (in the telemetry sense) is the practice of sending a representative subset of telemetry items to the backend to reduce ingestion volume and cost. The Application Insights .NET SDK uses adaptive sampling, which starts at 100% (all items sent) and throttles back when the ingestion rate exceeds a threshold (e.g., 5 items per second), keeping aggregations statistically accurate through server-side reweighting. The SDK ships with adaptive sampling on by default, which means a low-traffic exercise sees every request, dependency, and exception, while a real production service automatically scales sampling down as load grows.

The thing that makes adaptive sampling tolerable rather than misleading is that it samples whole operations, not indi-

vidual telemetry items. When a request is sampled out, every telemetry item produced during that request — the dependency calls, the log lines, the exceptions — is dropped together. The result is that a sampled request either has a complete record or no record at all; the Failures blade never shows an exception orphaned from its parent request. Aggregations like request count and average latency stay accurate because the SDK records the sampling rate per item and the backend reweights the results during query.

For most services, the SDK's default behaviour is the right starting point. Sampling becomes a knob worth tuning when the bill grows uncomfortable, when a specific failure type needs to be sampled at 100% even under load (configurable by exception type), or when an exporter pipeline is moving telemetry through OpenTelemetry rather than the native SDK and needs a hand-configured sampler.

45.6 Custom events and metrics

Auto-instrumentation captures the runtime's view of the application; custom telemetry captures what the application alone knows. A "user added an item to a cart" event is not a log line in any meaningful sense — it is a business signal — and shoehorning it into `ILogger.LogInformation` is the wrong tool because the consumer is not a developer skimming logs but a dashboard counting cart-adds per hour.

A **telemetry client** is the programmatic interface (typically `TelemetryClient`) through which code sends custom telemetry (metrics, events, exceptions) to Application Insights; it is registered in the dependency injection container and provides methods like `GetMetric()`, `TrackEvent()`, and `TrackException()` for application-level instrumentation

beyond what the SDK auto-instruments. Two patterns dominate.

```
public class HomeController : Controller
{
    private readonly TelemetryClient _telemetry;

    public HomeController(TelemetryClient telemetry)
    {
        _telemetry = telemetry;
    }

    public IActionResult Index()
    {
        _telemetry.GetMetric("home-page-
        views").TrackValue(1);
        _telemetry.TrackEvent("HomePageViewed", new Dic-
        tionary<string, string>
        {
            ["UserAgent"] =
        Request.Headers.UserAgent.ToString()
        });
        return View();
    }
}
```

`GetMetric("home-page-views").TrackValue(1)` aggregates client-side: the SDK accumulates samples in memory and ships a pre-aggregated metric (count, sum, min, max, percentiles) once per minute. The cost is a single ingestion item per metric per minute regardless of traffic, which makes custom metrics cheap at any volume and is the right default for things being counted or measured many times per request. `TrackEvent("HomePageViewed", properties)` records one event per call, complete with arbitrary string properties, and is appropriate for low-frequency business signals (a checkout completed, a user signed up) where the per-occurrence detail matters.

Both kinds of custom telemetry inherit the operation ID of the surrounding request automatically, which means a custom event recorded inside a controller is correlated with the same `requests` row, the same `dependencies` rows, and the same `exceptions` row as everything else the request produced. The companion exercise Logging and Monitoring walks through the wire-up of `AddApplicationInsightsTelemetry`, the Container Apps secret reference, a deliberate `/Home/Boom` endpoint that throws to populate the Failures blade, and a `home-page-views` metric tracked from a controller — exactly the path described above.

45.7 What Application Insights does not solve

Application Insights captures telemetry from one application's runtime. It does not, on its own, give a view across multiple applications unless they all point at the same component, and it does not replace the Log Analytics workspace for non-application data — the container's stdout, the platform's audit logs, the cluster's diagnostic telemetry. For systems composed of more than one service, the conventional pattern is one Application Insights component per environment, with every service in that environment pointing at it, so that the Application Map and the Failures blade aggregate across services rather than presenting one disconnected picture per service.

The chapter on Log Analytics and KQL covers how the same workspace serves both Application Insights and Container Apps stdout via KQL, and how queries can join the two for the kind of question that neither blade answers on its own.

45.8 Summary

Application Insights is a managed APM service backed by a Log Analytics workspace; in workspace-based mode the application's telemetry and the platform's container logs coexist in one queryable store. The .NET SDK auto-instruments ASP.NET Core to capture requests, dependencies, exceptions, and traces with operation-ID correlation, and is wired up by a single `AddApplicationInsightsTelemetry` call in `Program.cs` that reads its connection string from `APPLICATIONINSIGHTS_CONNECTION_STRING`. On Container Apps, the connection string is injected as a `secretref:` rather than a literal value, keeping it out of any configuration export. The portal exposes Live Metrics for the last 60 seconds with one-second latency, the Application Map for the dependency graph with p95 latency and failure rate per edge, the Failures blade for stack-traced exceptions grouped by type with operation-ID drill-down, and the Performance blade for the same drill-down on slow operations. Adaptive sampling is the cost lever — on by default, samples whole operations rather than individual items, and reweights server-side so aggregations stay accurate. Custom events and metrics, sent through an injected `TelemetryClient`, capture business signals that auto-instrumentation cannot see; `GetMetric` aggregates client-side and is cheap at volume, while `TrackEvent` records one item per call and is reserved for low-frequency, per-occurrence signals.

CHAPTER 46

Log Analytics and KQL Basics

Watch the presentation
Se presentationen på svenska

The dashboards in Application Insights — Live Metrics, Application Map, Failures, the Performance blade — answer the questions that come up most often in incident response. They show request rate, the slowest dependencies, the most common exception types, and the topology of services that talk to each other. What they cannot answer is the long tail of questions that occur exactly once: “show every request to `/Home/Boom` from the last forty minutes whose response took longer than two seconds, grouped by the replica that served

it." For that class of question, the dashboards are the wrong shape; they were designed by someone who could not anticipate the specific incident at hand. The right shape is a query language that runs against the same data the dashboards render, and that is what this chapter develops.

46.1 Where the data lives

Application Insights and Container Apps look like separate products in the Azure portal, but they write to the same physical store. A **Log Analytics workspace** is a multi-tenant storage container in Azure where multiple services (Container Apps, Application Insights, VMs, and others) send their telemetry; data is organized into tables, each scoped to a service type. The workspace provides a single point for retention policy, role-based access control, and KQL-based querying across all ingested data.

A workspace is provisioned as a regional Azure resource and given a name; from that point on, every service that writes telemetry into it does so by referencing the workspace ID. Container Apps environments create one automatically when the environment is provisioned, and stream container stdout and stderr into it. Application Insights components in workspace-based mode (the default since 2021) persist their request, dependency, exception, and trace telemetry into the same workspace. The result is a single store where a single query can correlate `ILogger` lines from a controller with the request span the SDK auto-instrumented for it.

The "workspace" is not a physical machine; it is a logical scope over a managed time-series database (the same engine that backs Azure Data Explorer). Data is indexed on ingestion, retained for a configurable window, and queried through one query interface regardless of which service emitted it.

46.2 Tables as the unit of organization

Inside a workspace, data is partitioned into tables. A **table** (in Log Analytics) is a named collection of rows, each with the same schema, representing a single data source type (e.g., `ContainerAppConsoleLogs` for container stdout, `InsightsMetrics` for Application Insights metrics). Queries select one table as a source and apply operators to filter, project, and aggregate its rows.

A handful of tables carry most of the traffic for a containerized .NET application:

- `AppRequests` — every HTTP request the application served, with `Url`, `ResultCode`, `DurationMs`, `OperationName`, and the operation/correlation IDs that tie a request to its descendants.
- `AppExceptions` — every exception captured by the Application Insights SDK, with `Type`, `Message`, the outer stack frame, and the operation ID of the request that produced it.
- `AppDependencies` — outbound calls the application made (SQL queries, HTTP requests to other services, Azure SDK calls), each with a duration and a success flag.
- `AppTraces` — the `ILogger` lines the SDK forwarded into App Insights, with the original message template, the rendered message, the severity, and the structured fields as a JSON blob.
- `ContainerAppConsoleLogs_CL` — the raw stdout and stderr the platform captured from each replica, with `ContainerName_s`, `ContainerImage_s`, `RevisionName_s`, and the original log line in `Log_s`.

The `_CL` suffix denotes a custom log table (the legacy Container Apps schema), and the `_s` suffix on individual

columns denotes a string-typed extracted field. Newer Container Apps environments expose a managed equivalent without the suffixes (`ContainerAppConsoleLogs`); both shapes exist in real workspaces, and a query written for one will silently return zero rows against the other. Confirming the table schema before writing the first query is part of the workflow.

46.3 KQL as a pipe of transforms

The query language for these tables is **KQL** (Kusto Query Language) — a functional, case-insensitive query language used in Azure Data Explorer and Log Analytics; it uses a pipe (`|`) composition model where each operator transforms the table flowing through it, enabling users to filter, aggregate, and visualize telemetry data through expressive, readable queries that read like English.

The mental model is the Unix shell pipeline rather than SQL. A query starts with the name of a table — `AppRequests`, on its own, is already a valid query that returns every row in the table — and successive operators are chained with the pipe character, each consuming the table that flows into it and producing a new table for the next operator to consume. Reading top to bottom describes the data's journey: source, filter, narrow, aggregate, sort, limit. The engine plans the whole pipeline at once, but the pipeline is the natural unit of authoring and review.

```
AppRequests
| where TimeGenerated > ago(1h)
| where ResultCode startswith "5"
| project TimeGenerated, OperationName, ResultCode, Dura-
tionMs
| order by TimeGenerated desc
| take 50
```

Five operators, each doing one thing: pick a time window, keep only server errors, narrow to the columns of interest, sort newest-first, and stop after fifty rows. The query is its own documentation.

46.4 The workhorse operators

A small set of operators carries most queries written in practice. Investing in fluency with these pays back faster than learning the long tail.

A **query operator** (in KQL) is a function that transforms a table flowing into it and outputs a transformed table; common operators include `where` (filters rows), `project` (selects columns), `summarize` (aggregates), `order by` (sorts), `take` (limits rows), and `extend` (adds computed columns). Operators are chained via the pipe symbol to compose complex queries.

`where` filters rows by a predicate — `where ResultCode == "500"`, `where Url contains "/api/"`, `where TimeGenerated > ago(15m)`. Multiple `where` clauses are equivalent to one with `and`; placing them early in the pipeline lets the engine prune work cheaply.

`project` chooses which columns appear downstream and (optionally) renames them — `project When = TimeGenerated, OperationName, DurationMs`. Without `project`,

every column the table has flows through, which is fine while exploring but expensive once a query goes into a workbook.

`summarize` is the aggregator. `summarize count() by ResultCode` collapses the table to one row per distinct `ResultCode` with a `count_` column showing how many rows had that value; `summarize avg(DurationMs), p95 = percentile(DurationMs, 95) by OperationName` produces both an average and a 95th-percentile latency per operation. `summarize` is what turns raw events into the numbers a dashboard or alert is built on.

`extend` adds a computed column without removing existing ones — `extend Bucket = bin(TimeGenerated, 1m)` rounds each row's timestamp to the nearest minute, which is the trick behind almost every time-series chart.

`count`, `top`, and `order by` are convenience operators built on the same primitives: `count` is `summarize count()`; `top 10 by DurationMs desc` is `order by DurationMs desc | take 10`. Keep them in the toolkit even when the longer form is equally valid; they read more naturally.

46.5 Time range first, always

Every chapter on KQL eventually arrives at the same advice: filter by `TimeGenerated` before doing anything else, and keep the window as small as the question allows.

A **time range** is the window of historical data a KQL query operates on; expressed in queries as conditions like `where TimeGenerated > ago(30m)`, it controls both the scope of analysis and the query performance. Shorter time ranges are cheaper and faster; longer ranges increase cost and latency.

The mechanics matter. Log Analytics partitions table data by ingestion time, and a `where TimeGenerated > ago(1h)` clause lets the query engine read only the partitions that overlap the last hour. Without it, the engine scans the entire retention window — thirty days by default, up to two years if extended — and the query's runtime and cost scale accordingly. A query that returns in two seconds against the last hour can take minutes against a year.

The portal's time-picker control above the query editor inserts an implicit time filter at runtime, which means a query without an explicit `TimeGenerated` filter will still be scoped during interactive use. That convenience disappears the moment the query is saved into a workbook, scheduled as an alert, or run from the CLI; embedding `where TimeGenerated > ago(...)` makes the query portable.

`ago()` is the most common helper — `ago(15m)`, `ago(1h)`, `ago(7d)` — but absolute times work too: `where TimeGenerated between (datetime(2026-04-28T08:00:00Z) .. datetime(2026-04-28T09:00:00Z))` is the right form for replaying a known incident window.

46.6 A worked example: failed responses by operation

A concrete question makes the mechanics tangible: identify which operations have been returning HTTP 5xx in the last hour, and how many failures each has produced.

```
AppRequests
| where TimeGenerated > ago(1h)
| where ResultCode startswith "5"
| summarize FailureCount = count() by OperationName
| order by FailureCount desc
| take 10
```

The pipeline is short on purpose. `AppRequests` is the source. `where TimeGenerated > ago(1h)` shrinks the scan to the last hour. `where ResultCode startswith "5"` keeps server-side failures (`500`, `502`, `503`, `504`) and discards client errors and successes — the prefix match avoids the brittleness of enumerating each code. `summarize count() by OperationName` collapses the rows into one per distinct operation, with the count of failures alongside; the `FailureCount =` rename makes the resulting column self-describing. `order by FailureCount desc | take 10` produces the top-ten table.

The output is a small table with two columns and at most ten rows. From there the next move is usually to pick the worst operation and drill into it — `where OperationName == "POST Home/Boom"` followed by a `join` to `AppExceptions` on `OperationId` — to see the actual stack traces that produced the failures. The point is that each step of the investigation is a small additional pipe, not a new query written from scratch. The companion exercise Logging and Monitoring walks through this exact workflow against a Container App generating real traffic.

46.7 Retention as a cost knob

Workspace storage has a price, and the price is paid twice: once on ingestion and once on retention. A workspace

charges per gigabyte ingested (the data flowing in) and per gigabyte retained beyond the included retention window.

Retention is the length of time data is kept in a Log Analytics workspace before automatic deletion; the default is 30 days, but can be configured from 7 days to 2 years. Longer retention increases storage cost and query latency; shorter retention reduces cost but limits historical analysis and troubleshooting window.

Retention is set at the workspace level (a default for all tables) and can be overridden per table. A common pattern is to keep `AppRequests` and `AppExceptions` for ninety days because incident post-mortems often need recent history, while leaving high-volume `ContainerAppConsoleLogs_CL` at the default thirty days because the same information is also captured (in structured form) in `AppTraces`. The decision is a trade-off between the cost of storage and the cost of not being able to answer a question that arrives a month after the data was discarded.

A second knob, archive tiers, moves data older than a threshold to cheaper storage that is queryable only through a slower path; this is the right shape for compliance-driven retention where the data must exist but is rarely read. For everyday observability, sticking to the analytics tier and tuning retention per table is the simpler model.

46.8 Saving useful queries as workbooks

A query written once during an incident is a query that has to be rewritten the next time. Log Analytics offers two persistence layers for queries that are worth keeping.

The first is a saved query — a named query attached to the user's account or to the workspace itself. A workspace-saved query is RBAC-gated (writing one requires Contributor on the

workspace) and visible to anyone with read access, which makes it the right home for the queries the team agrees are canonical. A user-saved query lives in the user's account and never appears for others.

The second, and more capable, is a workbook. A workbook is a parameterized document that combines KQL queries, charts, tables, and free-form text into a single page; opening a workbook runs its embedded queries and renders the results. The "Failures Investigation" workbook that ships with Application Insights is a worked example: a date-range parameter at the top feeds the time filter for every query in the document, and the page below shows a failure-rate timechart, a top-failing-operations table, an exception-types breakdown, and a sample exceptions list, each driven by its own KQL query. A team can build domain-specific workbooks the same way — one workbook per service, or one workbook per type of incident — and reach for the right one when the situation matches.

Workbooks compose naturally with alert rules and scheduled queries, but those are the topic of the alerts and SLOs chapter rather than this one.

46.9 Summary

A Log Analytics workspace is the underlying time-series store that Container Apps and Application Insights both write into, organized into tables — `AppRequests`, `AppExceptions`, `AppDependencies`, `AppTraces`, and `ContainerAppConsoleLogs_CL` among them — each carrying a single data-source type. KQL queries those tables through a pipe-composition model in which a small set of operators (`where`, `project`, `summarize`, `extend`, `count`, `top`, `order by`, `take`) chains transforms one after another, top to bottom, each step narrowing or reshaping the table. Filtering by

`TimeGenerated` belongs at the front of every query: it bounds cost, latency, and the partitions the engine has to read. Retention controls how far back queries can reach and is the main cost knob worth tuning per table. Useful queries graduate into saved queries or, better, into workbooks that combine queries, charts, and parameters into a reusable investigation surface.

CHAPTER 47

Health Checks

Watch the presentation
Se presentationen på svenska

A container orchestrator decides every few seconds whether to keep sending traffic to a running replica, restart it, or remove it from the load balancer. That decision is only as good as the signal the replica gives back. Without an explicit signal, the orchestrator falls back to “the TCP socket on port 8080 accepted my connection” — a check that passes for a process that has crashed mid-request, deadlocked its thread pool, or lost its database connection. The application stays in rotation while every request returns 500. The chapter develops the contract that closes this gap: an HTTP endpoint the application owns, designed to answer the orchestrator’s ques-

tion honestly, and the three flavours of that question that mean different things to the platform.

47.1 Why a TCP probe is not enough

Azure Container Apps — and Kubernetes underneath — schedules many replicas of an application across a cluster, then asks each one a recurring question: are you still able to do useful work? The default answer comes from a TCP-level probe. The platform opens a connection on the application port; if the connect succeeds, the replica is considered healthy. The check is cheap and requires no cooperation from the application, which is its only virtue.

Several common failure modes pass a TCP probe while the application is no longer functioning:

- The HTTP server thread is alive and accepting connections, but every request hangs because the database connection pool is exhausted.
- The application boots, binds the port, and then a background initialiser fails — a missing Key Vault secret, a Mongo cluster the network rules forbid — and the app is permanently in a half-initialised state.
- A garbage collection pause or an unhandled exception in the request pipeline leaves the process technically alive but practically unable to answer.

Each of these is a real outage from the user's point of view, and none of them are visible to a TCP probe. The remedy is for the application to expose its own opinion of its health and for the platform to ask for that opinion instead.

47.2 What a health check is

A **health check** is an endpoint (typically `/healthz`) that reports the operational status of an application and its critical dependencies; it returns quickly with an HTTP status code and optional JSON body indicating whether the service is healthy, degraded, or unhealthy. Container orchestrators use health checks to decide whether to keep traffic routing to a replica.

The contract is small and deliberately blunt:

- A successful check returns `200 OK`. The platform reads "healthy" and keeps the replica in rotation.
- A failed check returns `503 Service Unavailable`. The platform reads "do not send me traffic right now" and acts accordingly.
- An optional JSON body lists the individual sub-checks and their results, useful for humans debugging a failure but ignored by the orchestrator's polling loop.

The endpoint's job is to be fast and cheap. A health check that takes 30 seconds to compute is itself a source of outage: the platform's probe times out, marks the replica unhealthy, and removes it from the load balancer even though it was working fine. Probes typically run every few seconds, so the budget for a single check is on the order of a few hundred milliseconds, including any dependency calls it chooses to make.

47.3 The three canonical probe types

A single endpoint cannot answer every operational question, because the platform takes different action depending on what kind of failure it sees. Container Apps and Kubernetes both expose three distinct probe types, each tied to a specific corrective response.

47.3.1 Liveness — is the process running

A **liveness probe** is a lightweight health check that verifies the process is still running and responsive; it typically checks only that the application started successfully without calling external dependencies. Container orchestrators poll liveness regularly and kill and restart the container if it reports unhealthy, preventing zombie processes from accumulating traffic.

The defining feature of a liveness probe is what it does *not* check. It must not call the database, must not reach out to Key Vault, must not depend on a downstream API. If any of those is unreachable, killing and restarting the container will not help — the new container will fail liveness for the same reason the old one did, and the platform will enter a restart loop that solves nothing and burns the replica budget. Liveness exists to recover from in-process faults: a deadlocked thread pool, a corrupted in-memory state, a managed crash that left the process up but unresponsive.

A liveness check that returns `200` from a single in-memory branch is the right design. The implementation is essentially "if my code is still executing this line, I am alive."

47.3.2 Readiness — can the process accept traffic right now

A **readiness probe** is a health check that verifies the application is ready to serve traffic, often by testing whether critical external dependencies (database, cache, downstream API) are reachable. Container orchestrators gate traffic to a replica until readiness passes, preventing requests from reaching an incompletely initialised instance.

A readiness failure is not a crash. The replica is fine; it just cannot serve traffic at this moment. Maybe it is still warming its caches, maybe the database is briefly unreachable, maybe

it has been told to drain. The platform's response is to remove the replica from the load balancer until readiness passes again — but it does not restart the container, because restarting would not change anything.

The split between liveness and readiness is the key concept of the chapter. The same JSON outcome ("unhealthy") leads to two completely different platform actions depending on which probe reported it:

PROBE	FAILURE MEANS	PLATFORM ACTION
Liveness	The process is broken	Kill and restart the container
Readiness	The process is fine but cannot serve traffic	Remove from load balancer until readiness passes

Confusing the two is one of the most common production incidents in containerised applications. A team writes a single `/healthz` endpoint that calls the database and registers it as both liveness and readiness. The database has a brief blip. Every replica fails liveness simultaneously. The platform restarts every replica simultaneously. The new replicas come up, find the database still unreachable, fail liveness again, and the cluster enters a cascading restart loop. The same wiring with the database check on readiness only would have removed the replicas from the load balancer for thirty seconds and put them back when the database recovered.

47.3.3 Dependency probe — are downstream services reachable

A **dependency probe** (or dependency health check) is a check that verifies an external service or resource the application depends on is reachable and responding; examples include

database connectivity checks and HTTP GET requests to a downstream API endpoint. Dependency probes are commonly used in readiness checks to gate traffic until external systems are available.

Dependency probes are the building blocks readiness uses. A readiness endpoint typically aggregates several dependency probes — one for the database, one for the message queue, one for the secret store — and reports the worst result. If any critical dependency is unhealthy, the application is not ready to serve traffic.

The choice of which dependencies are "critical" is a design decision. A read-only API might list its database as critical and its email provider as non-critical: it can serve reads without sending mail. A write-heavy API would invert that hierarchy. The application owns this judgement; the platform only sees the rolled-up answer.

47.4 The /healthz endpoint convention

The **/healthz endpoint** is a Kubernetes-convention HTTP endpoint (usually returning `200 OK` or `503 Service Unavailable`) that exposes health status of an application. It is called by orchestrators and monitoring systems to determine replica health and is often separated from the main application controllers to avoid coupling liveness/readiness logic with business logic.

The path is conventional, not mandatory. The trailing `z` is a Google-internal joke that escaped into the wider Kubernetes ecosystem, originally chosen to make the path unlikely to collide with a real business URL. ASP.NET Core, Spring Boot, Express, and most other web frameworks default to `/healthz` or accept it as the documented path; orchestrator

templates assume it. Using the convention removes a configuration step.

A common pattern is to expose two separate paths: `/healthz/live` for liveness and `/healthz/ready` for readiness. Each path runs only the checks appropriate to its role. The platform configuration then points the liveness probe at `/healthz/live` and the readiness probe at `/healthz/ready`, and the two responsibilities never tangle.

47.5 ASP.NET Core health checks

ASP.NET Core ships a first-class health checks subsystem. The pattern centres on the `IHealthCheck` interface — a single method that returns a `HealthCheckResult` of `Healthy`, `Degraded`, or `Unhealthy`. Custom checks implement this interface; framework-provided checks for common dependencies (SQL Server, Mongo, Redis, Azure Key Vault) implement it on the application's behalf and are registered through extension methods on the service collection.

47.5.1 Worked example: registering a Mongo dependency probe

The setup needs three pieces: registering checks in the DI container, mapping the endpoints in the request pipeline, and tagging which checks belong to which probe.

```
// Program.cs

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddHealthChecks()
    .AddCheck("self", () => HealthCheckResult.Healthy(),
        tags: new[] { "live" })
    .AddMongoDb(
        sp => sp.GetRequiredService<IMongoClient>(),
        name: "mongo",
        tags: new[] { "ready" })
    .AddAzureKeyVault(
        new Uri(builder.Configuration["KeyVault:Uri"]!),
        new DefaultAzureCredential(),
        options => { },
        name: "keyvault",
        tags: new[] { "ready" });

var app = builder.Build();

app.MapHealthChecks("/healthz/live", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("live")
});

app.MapHealthChecks("/healthz/ready", new HealthCheckOp-
        tions
{
    Predicate = check => check.Tags.Contains("ready")
});

app.Run();
```

Three things are happening. The `AddCheck("self", ...)` call registers an in-memory check tagged `live` — this is the liveness probe, returning healthy unconditionally because the only failure mode it cares about is “this code is no longer executing.” The `AddMongoDb` and `AddAzureKeyVault` calls register dependency probes tagged `ready`; each opens a short-lived connection to its target and reports whether the

round-trip succeeded. The two `MapHealthChecks` calls expose the endpoints, each one filtering by tag so that the liveness path only runs the `live`-tagged checks and the readiness path only runs the `ready`-tagged ones. A failure in Mongo will fail readiness but not liveness, and the platform will remove the replica from the load balancer without restarting it — exactly the desired behaviour.

47.6 Health checks and the smoke test gate

The deployment pipeline already has a smoke test gate (covered in Build, Test and Smoke Gates) that runs after `az containerapp update` to confirm the new revision is actually serving traffic before the pipeline reports success. The natural target for that smoke test is the `/healthz/ready` endpoint of the new revision.

The composition is clean: the readiness endpoint is the application's own answer to "am I ready to serve traffic," and the smoke test is the pipeline asking that exact question once before declaring the deployment complete. A `200` from `/healthz/ready` proves three things at once — the new image was pulled, the new container booted, and every dependency the application declared critical is reachable from the new replica's network. A failed smoke test rolls the deployment back without any user ever seeing a 503.

The same endpoint serves both the orchestrator's continuous polling and the pipeline's one-shot gate, which is the point of standardising on a single contract.

47.7 How probes integrate with autoscaling

Container Apps and Kubernetes both factor probe results into their autoscaling decisions, which is a third reason to keep

liveness and readiness distinct. A replica that fails readiness is not counted as serving capacity by the load balancer, which means an autoscaler watching "average request count per healthy replica" will see the load concentrated on the remaining replicas and may scale out. A replica that fails liveness is being restarted, which the autoscaler treats as transient — restarting replicas are not new capacity. Mixing the two leaks signal across the boundary and produces erratic scaling behaviour.

47.8 Cross-link to the exercise

The companion exercise chapter, Logging and Monitoring, runs a Container Apps deployment end-to-end with structured logging, Log Analytics, and Application Insights. Health checks slot into that same deployment as the next observability layer: the application already publishes structured logs and telemetry; adding a `/healthz` endpoint gives the platform a way to act on the application's own self-assessment, not just consume its logs after the fact.

47.9 Summary

A health check is an HTTP endpoint the application owns, used by container orchestrators to decide whether a replica should keep receiving traffic, be restarted, or be removed from the load balancer. The default TCP-connect probe misses every failure mode in which the process is alive but unable to do useful work, which is why an application-owned check is the standard contract in Container Apps and Kubernetes. The three canonical probes are liveness (is the process running, used for restart decisions), readiness (can the process accept traffic right now, used to gate the load balancer),

and dependency probes (are downstream services reachable, used as the building blocks of readiness). ASP.NET Core implements this contract through the `IHealthCheck` interface and the `AddHealthChecks()` registration pattern, with the `/healthz` endpoint convention exposing the result. The same endpoint serves the orchestrator's continuous polling and the pipeline's smoke test gate after deployment, which is the integration point with the wider DevOps story.

CHAPTER 48

Alerts and SLOs

Watch the presentation
Se presentationen på svenska

A dashboard that nobody watches is a dashboard that fails silently. Telemetry pipelines, structured logs, and Application Insights workbooks generate enormous quantities of evidence about a running system, but evidence sitting in a queryable store is not the same as a person being told that something is wrong. Alerts close that gap by turning a queryable signal into a phone vibrating at three in the morning. The harder problem — and the one this chapter develops — is deciding what is worth waking someone up for. That is what Service Level Objectives address.

48.1 What an alert rule is

An **alert rule** is an automated condition (e.g., "error count > 10 in the last 5 minutes") that, when met, triggers a notification to one or more action groups. Alert rules are configured in Application Insights, Log Analytics, or Azure Monitor and allow teams to be notified of problems without constantly checking dashboards. The rule has three moving parts: an evaluation source (a metric stream or a KQL query), a condition expressed against that source, and a destination — the action group — that receives the notification.

Two evaluation sources dominate. Metric-based alerts read from the pre-aggregated metrics pipeline that Azure Monitor maintains for every supported resource type. They evaluate cheaply because the metrics are already aggregated at one-minute or five-minute resolution and indexed for fast comparison; the cost is that only the metrics the platform exposes are available — for a Container App, that means CPU, memory, replica count, request count per revision, and a handful of others. Log-based alerts run a KQL query on a Log Analytics workspace on a schedule (typically every five minutes) and trigger when the result set crosses a threshold. The flexibility is total — anything KQL can express becomes an alertable condition — but each evaluation costs a query, and the ingestion delay of one to three minutes adds latency between event and notification.

48.1.1 Threshold-based vs dynamic baseline alerts

The simpler form of alert rule uses a static threshold. A **threshold** is the numeric boundary that, when crossed by a metric or count, triggers an alert; examples include "CPU > 80%," "response time > 500ms," or "error count >= 5." Thresholds must balance sensitivity (catching problems early) against false positives (alert fatigue); SLOs inform sens-

ible threshold choices. Static thresholds work well when the operator knows the system's normal range — a service that has consistently served below 200ms response times for six months can have a threshold set at 500ms with confidence that crossings represent real degradation.

Static thresholds fail when normal behaviour is itself variable. A retail service with five times the traffic on Friday evenings will trip a "request rate > X" alert every Friday under any threshold sensitive enough to catch a real anomaly on a Tuesday morning. Dynamic baseline alerts address this by replacing the fixed number with a model of historical behaviour. Azure Monitor learns the metric's typical pattern over the past ten days and fires only when the current value falls outside the expected band, accounting for time-of-day and day-of-week seasonality. The trade-off is that dynamic baselines need data to learn from — they perform poorly on services with under two weeks of history and on metrics that change distribution after a deployment.

48.2 Action groups as the contact-list abstraction

When the rule fires, something has to happen. An **action group** is a collection of notification channels (email, SMS, webhook, PagerDuty, Slack) that an alert rule sends to when triggered. A single action group can be reused across multiple alert rules, centralizing notification routing and enabling teams to manage escalation paths in one place.

The separation matters operationally. A team that hand-codes a destination email into every alert rule has to update fifty rules when the on-call rotation changes hands. A team that points fifty rules at the action group `oncall-payments` updates one resource and the routing change propagates

everywhere. Action groups also encapsulate escalation: the first notification can fire to a Teams channel, a follow-up sixty seconds later can SMS the on-call engineer, and a third can page a manager if the alert is still active after fifteen minutes.

Channel choice is not cosmetic. An email is appropriate for a daily-digest alert about disk usage trending upwards; an SMS is appropriate for an alert that needs human action within the hour; a webhook into PagerDuty is appropriate for a system that already has an incident-management workflow defined. Routing every alert to email guarantees that the urgent ones get lost in the noise of the routine ones — the channel itself is part of the signal.

48.3 The alert fatigue problem

A team that adds an alert rule for every metric eventually hits alert fatigue: the on-call engineer starts ignoring notifications because most of them turn out to be nothing. Once that habit forms, the genuine incidents get ignored too, and the alerting system becomes worse than no alerting at all — it generates the false confidence that someone is watching.

Alert fatigue has two causes. The first is too many rules: every metric gets a threshold, every threshold gets a notification, and the notification volume swamps the operator's attention. The second is rules that fire on transient conditions that resolve themselves before anyone can act — a thirty-second CPU spike during a deployment, a single failed health probe during a node replacement, a brief blip in error rate during a transient dependency failure. Both causes have the same fix at the rule level (require the condition to hold for a duration before firing) but the deeper question is which conditions are worth firing on at all.

That is where Service Level Objectives come in. SLOs replace the question "what can we measure" with "what does our user actually care about," and the discipline of choosing one shapes the alerting strategy that follows.

48.4 SLI, SLO, and error budget

Three terms travel together. The **SLI** (Service Level Indicator) is a measured, quantitative metric that describes a desired aspect of service performance; examples include "percentage of successful requests," "p99 response latency," and "uptime percentage." SLIs are the foundation of SLOs and error budgets — they are what you measure. The SLI is purely descriptive; it does not say what is good, only what was observed.

The **SLO** (Service Level Objective) is a target value or range for an SLI over a specified period; an example is "99.9% of requests succeed over a 30-day month." SLOs guide resource allocation, inform alerting thresholds, and establish the acceptable failure rate (error budget) for the service. The SLO is the contract — the number the team commits to deliver against.

The **error budget** is the inverse of an SLO, expressing the acceptable failure rate as a quantity; for an SLO of 99.9% availability over 30 days, the error budget is 0.1%, which amounts to roughly 43 minutes of acceptable downtime. Error budgets guide decisions about whether to ship new features or focus on reliability. The error budget reframes reliability as a finite resource that gets consumed rather than as a binary "is the system up." Over a thirty-day window, the team can spend its budget on a controlled-rollout incident, on a planned maintenance window, or on the slow drip of inter-

mittent failures — the question is whether any spending pattern will exhaust the budget before the window resets.

CONCEPT	WHAT IT IS	EXAMPLE
SLI	The measurement	99.92% of HTTP requests returned 2xx or 3xx in the last 30 days
SLO	The target	99.9% of HTTP requests must return 2xx or 3xx, measured over 30 days
Error budget	The remaining failure allowance	0.08% (out of 0.1% budget); ~35 minutes of further downtime tolerable this window

48.4.1 Alerting on the error budget

The leverage of this vocabulary is that alerts can fire on the budget rather than on raw failure events. A naive rule says "alert when the error rate exceeds 1% over five minutes." A budget-aware rule says "alert when the current burn rate would exhaust the thirty-day error budget in less than two days." The second formulation tolerates short, isolated spikes that are within the budget while still catching sustained degradation that threatens the SLO. The result is fewer alerts, each carrying more meaning.

This is also why setting realistic SLOs matters. An SLO chosen at 99.99% (52 minutes of downtime per year) on a service that has historically delivered 99.5% will trigger con-

stantly, because every normal week consumes the entire annual budget. Start by measuring the SLI for several weeks before promising any number — the baseline measurement is the prerequisite for choosing a target that is both meaningful to users and achievable by the team. An SLO that the system cannot meet under normal conditions is worse than no SLO at all; it teaches everyone that the number is a fiction.

48.5 A worked example: alerting on 5xx error rate

Consider a Container App serving HTTP traffic, instrumented with Application Insights so request telemetry flows into the workspace. The team has measured the SLI for two weeks and observed a baseline 5xx rate of around 0.05%, with brief spikes during deployments. They commit to an SLO of 99.5% successful responses over a thirty-day window — an error budget of 0.5%, or roughly three and a half hours of tolerable failures per month.

The alerting strategy has two layers. A fast burn-rate alert catches catastrophic failures within minutes; a slow burn-rate alert catches sustained degradation that would silently exhaust the budget over hours. The fast alert is the one wired in this exercise.

```
az monitor metrics alert create \
  --name "high-5xx-rate" \
  --resource-group rg-clo25-obs \
  --scopes /subscriptions/.../containerApps/web-app \
  --condition "avg requests/failed > 5 where ResultCode
        startswith '5'" \
  --window-size 5m \
  --evaluation-frequency 1m \
  --severity 2 \
  --action ag-oncall-payments
```

The rule evaluates every minute over a five-minute sliding window. The condition is “more than five percent of requests in the window returned a 5xx status code” — a single isolated 5xx will not trip it, nor will a thirty-second blip during a deployment, but a sustained failure across multiple replicas for several minutes will. The action group `ag-oncall-payments` defines what happens next: a Teams notification to the engineering channel, an SMS to the primary on-call, and a fallback email to the team distribution list after fifteen minutes if the alert is still active.

The companion exercise Logging and Monitoring provides the deployed Container App, the Log Analytics workspace, and the Application Insights resource that make this rule possible. The exercise stops short of wiring the alert; the chapter establishes why the wiring matters before the lab.

48.6 How SLOs counter alert fatigue in practice

Returning to the fatigue problem with the SLO vocabulary in hand: a team that has agreed to a 99.5% SLO has, by definition, accepted that 0.5% of requests can fail. Any alert rule that fires before the budget is meaningfully threatened is by definition a false positive — the system is operating within agreed bounds. The SLO converts the question “is something

wrong" into the more answerable "is the budget at risk," and the latter has a numeric answer.

This shifts the team's relationship to its own reliability. When the budget is healthy, the team can ship features aggressively, accept some risk, and run experiments in production. When the budget is nearly exhausted, the team focuses on reliability work and freezes risky changes until the window resets. The error budget becomes a shared instrument that aligns engineering and product priorities around an actual measurement, rather than a debate about whether the latest incident "felt bad enough" to slow down. Alerts in this regime are rare but meaningful, and on-call engineers learn to trust them.

48.7 Summary

An alert rule pairs a condition (a query or metric threshold) with an action group that delivers a notification when the condition is met. Action groups centralize the routing of those notifications across email, SMS, webhook, and incident-management integrations, decoupling rule logic from contact lists. Static thresholds suit metrics with stable baselines; dynamic baselines handle seasonal or trending behaviour at the cost of needing historical data. The SLI/SLO/error-budget triad — measurement, target, and allowance — turns reliability from a binary state into a finite resource the team can spend, and lets alert rules fire on budget burn rather than on raw failures. Setting a realistic SLO requires measuring the SLI first; an unachievable target produces constant alerts and trains the team to ignore them. The combination — meaningful SLOs, well-routed action groups, and rules that fire on sustained budget threats — produces an alerting system the team will actually trust.

PART X

Collaboration and Process

The Course Book closes where most jobs actually begin: in a team, on a board, with a Git remote that someone else also pushes to. Five chapters cover the practical mechanics — Git itself, the branching/PR/review workflow that keeps `main` healthy, the inner-loop vs outer-loop split that defines a developer's day, the agile vocabulary that organizes the work, and how all of it lands in Jira.

The companion exercise is the Code Collaboration chapter, which puts every concept into practice in a small team simulation.

PART X · COLLABORATION AND PROCESS

CHAPTER 49

Version Control with Git

Watch the presentation

Se presentationen på svenska

A folder of source code becomes unmanageable as soon as more than one person changes it. Files diverge, edits collide, and last week's working version disappears under "final_v3_really_final.zip". The problem is not bad discipline — it is that change itself has no first-class representation. Without a system that records who changed what, when, and why, every modification is a destructive overwrite. Version control solves this by treating change as the primary object and the current files as merely the latest view of an accumu-

lated history. Git is the system used to track that history in almost every professional codebase, and the rest of Part X depends on its core vocabulary.

49.1 Why version control matters

Source code evolves continuously. A feature is started, abandoned, restarted, refactored, and shipped — all on top of work other contributors are doing in parallel. Three needs follow directly from this reality. The team must be able to answer "what changed and who changed it" for any line in the codebase. Anyone must be able to return the project to any earlier state without losing later work. And multiple developers must be able to edit the same files without overwriting each other.

A version control system meets all three needs by storing a complete history of named, immutable snapshots. Files in the working folder are no longer the source of truth — the recorded history is, and the current files are reconstituted from it on demand. Once a project is under version control, "undo" stops meaning "press Ctrl-Z fast enough" and starts meaning "check out the commit from before the bug appeared".

49.1.1 Centralized vs. distributed systems

Two architectural families implement these ideas. **Centralized version control** systems such as Subversion (SVN) and CVS keep the authoritative history on a single server. Developers check out a working copy, edit it, and commit changes back to the server. The server is the only place the history lives in full; without network access, the developer cannot view past versions, compare branches, or commit work.

Distributed version control systems, of which Git is the dominant example, give every developer a complete copy of

the entire history. Commits happen locally and are later synchronized with other copies. There is no single authoritative server in the protocol itself — although teams almost always nominate one (a GitHub repository, for instance) as the convention for collaboration. The practical consequence is that operations like viewing history, branching, and committing are local and fast, and network connectivity is needed only when sharing work with others.

This distinction matters because it shapes the daily workflow. With Git, commits are cheap and frequent; branches are created for any unit of work; and synchronization with the team is a deliberate, separate step. The rest of this chapter assumes the distributed model.

49.2 What a Git repository actually is

A **repository** is a `.git` directory and its contents that stores all commits, branches, tags, and configuration for a project. Everything Git knows about a project lives inside that one hidden folder at the project root. Delete `.git/` and the surrounding files revert to ordinary, untracked text files; copy `.git/` somewhere else and the entire history travels with it.

Inside `.git/` are two structures worth understanding at a high level. The **object database** stores every snapshot Git has ever recorded as a content-addressed blob — a piece of data whose identifier is a SHA-1 hash of its contents. Identical content produces identical hashes, so duplicate files across commits are stored only once. The **refs** are short, human-readable names (such as `main` or `feature/login`) that point to specific commits in the object database. Branches and tags are nothing more than refs.

The clean implication is that Git's history is not a sequence of patches layered on top of each other. It is a graph of com-

plete snapshots, and every name a developer uses — branch names, tag names, even `HEAD` — is just a label that points to one node in that graph.

49.3 The three states of a file

Git tracks every file through three distinct states. Understanding the boundary between them eliminates most early confusion about why `git status` says what it says.

The **working tree** (or working directory) is the visible, editable folder on the filesystem where project files reside; changes you make in the working tree are not automatically tracked by Git until you stage and commit them. Edit a file in an editor and the change exists only in the working tree. Git sees the file as "modified" but has not yet recorded the change anywhere durable.

The **staging area** (also called the index) is an intermediate area between the working tree and the repository where you prepare files for the next commit; `git add` moves files to the staging area, and `git commit` transforms the staged snapshot into a permanent commit. The staging area is what makes Git commits selective. A developer who has touched ten files can stage only the three that belong to the current logical change and commit them together, leaving the rest unstaged for a later, separate commit.

The third state is **committed** — the change is now part of the repository's history, stored in the object database, and reachable through some ref. Once committed, content is immutable. The commit can be hidden, abandoned, or rewritten into a new commit, but the original object remains in the database until garbage collection removes unreachable objects.

The mental model to carry forward: edit in the working tree, prepare with `git add`, record permanently with `git commit`.

49.4 Commits as immutable snapshots

A **commit** is an immutable snapshot of a project's files at a specific point in time, identified by a unique hash; it includes a message, author, timestamp, and a pointer to its parent commit(s), forming a linked chain that represents the project's history. The hash — typically displayed as a 40-character SHA-1 string, abbreviated to the first 7 in most output — is computed from the snapshot's content plus its metadata. Change anything about the commit and the hash changes. This is what makes Git history tamper-evident: a commit cannot be silently altered without invalidating every commit that descends from it.

Each commit points back to one or more parent commits, building a directed acyclic graph (DAG). A normal commit has one parent. A merge commit has two or more, marking the point where divergent histories were brought back together. The very first commit in a repository has no parent — it is the root of the graph.

Commit messages are how the team understands intent later. A commit titled `Fix bug` is almost worthless six months on; a commit titled `Fix off-by-one in pagination when page count is zero` lets the reader scan history without opening every diff. The convention this course uses is a short imperative subject line, optionally followed by a blank line and a longer body explaining the *why*.

49.5 Branches as movable pointers

A **branch** is a named pointer to a commit that allows parallel lines of development; creating a branch duplicates the pointer from the current commit, enabling isolated work without affecting other branches until an explicit merge reunites the changes. The branch itself is just a ref — a 41-byte file inside `.git/refs/heads/` containing a commit hash. Creating a branch is therefore cheap (it writes one file); switching branches is fast (it updates `HEAD` and reshapes the working tree); and there is no architectural reason to be parsimonious about branch creation.

The default branch in a new Git repository is conventionally named `main` (older repositories use `master`). When a commit is made on a branch, Git advances that branch's pointer to the new commit. When a branch is merged into another, the receiving branch's pointer moves forward to incorporate the work — sometimes by fast-forwarding (the pointer just moves up the chain) and sometimes by creating a merge commit that ties the two histories together.

`HEAD` is a special ref that names the branch the developer is currently working on. `git switch feature/login` makes `HEAD` point to `feature/login`, and subsequent commits will advance that branch.

49.6 Remotes and synchronization

A **remote** is a reference to a version of the repository hosted on a server (typically called `origin` for the default remote on GitHub); git commands like `fetch`, `push`, and `pull` synchronize commits between your local repository and the remote. A repository can have multiple remotes — one for the team's GitHub copy, one for an internal mirror, one for a per-

sonal fork — but the overwhelmingly common case is a single remote named `origin`.

Remotes do not change Git's distributed nature. Each remote is itself a full repository; what makes one of them "the team's repository" is convention, not protocol. Three commands move commits between local and remote.

Fetch is a git command that downloads commits and branches from a remote repository to your local machine without modifying the working tree or your local branches; it updates remote-tracking branches (e.g., `origin/main`) to reflect the current state of the remote. After a fetch, the developer can inspect what teammates have done before deciding whether to integrate it.

Pull is a git command that combines `fetch` and `merge` (or `rebase`) in a single operation; it downloads commits from the remote and automatically integrates them into your current local branch, updating the working tree. Pull is convenient when the developer trusts the incoming history; fetch followed by an explicit merge or rebase gives finer control.

Push is a git command that uploads commits from your local branch to the corresponding remote branch, making your work visible to collaborators and updating the remote repository; it requires write permission on the remote. Push is the symmetric counterpart of fetch — fetch downloads what teammates have done, push uploads what the local developer has done.

The symmetry is worth pausing on: every commit has to be pushed somewhere to be visible to anyone else, and every commit produced elsewhere has to be fetched to be visible locally. Nothing is automatic, which is what allows offline work in the first place.

49.7 The daily workflow

Most days, a developer touches four commands in a tight loop.

`git status` reports the state of the working tree and staging area: which files have been modified, which are staged, which are untracked. It is the first command to run when something feels off; most “I lost my work” panic ends with `git status` showing the work is right there, just not committed.

`git add <files>` moves changes from the working tree into the staging area. Pass specific paths to stage selectively, or `git add .` to stage everything in the current directory tree.

`git commit -m "<message>"` records the staged snapshot as a new commit on the current branch.

`git push` uploads new commits on the current branch to the matching branch on the remote.

A typical exchange looks like this:

```
git status
git add src/Auth/LoginController.cs src/Auth/LoginControl-
        ler.Tests.cs
git commit -m "Reject login attempts with empty password"
git push
```

The cycle is short on purpose. Small, frequent commits with descriptive messages produce a history that is easy to read, easy to revert, and easy to review.

49.8 Worked example: from empty folder to remote repository

Consider a fresh project on a developer's machine. Nothing is under version control yet. The session below takes it from empty folder to a repository pushed to GitHub.

```
mkdir clo-demo && cd clo-demo
echo "# CLO Demo" > README.md

git init
git add .
git commit -m "Initial commit"

git remote add origin https://github.com/student/clo-
        demo.git
git branch -M main
git push -u origin main
```

Six commands move the project through every concept in this chapter. `git init` creates the `.git/` directory, turning the folder into a repository. `git add .` stages every file in the working tree. `git commit -m "Initial commit"` records the first snapshot — a commit with no parent, the root of the new history. `git remote add origin <url>` registers the GitHub repository as a remote named `origin`; nothing is sent yet, only the address is recorded. `git branch -M main` ensures the local branch is named `main` (older Git defaults emit `master`). `git push -u origin main` uploads the commit and, thanks to `-u`, sets `origin/main` as the upstream for the local `main` so that future `git push` and `git pull` know where to go without arguments.

After this sequence the same history exists in two places: the local `.git/` directory and the GitHub-hosted repository. Either could now be cloned by a teammate, who would receive the full commit graph and could begin contributing.

The companion exercise Code Collaboration walks through this initialization and a first round of remote synchronization step by step.

49.9 Where Git fits in the larger picture

Git is the foundation, not the whole story. The rest of Part X builds directly on the vocabulary established here: branches become the basis for pull requests and code review, remotes become the connection point between local development and CI/CD, and the discipline of small, descriptive commits underpins both. None of those higher-level practices make sense without a clear picture of what a commit is, what a branch is, and how local and remote histories synchronize.

The reason Git won the version control market is not that its command-line interface is friendly — it is famously not. It won because the underlying data model is simple, durable, and correct: a content-addressed graph of immutable snapshots, with cheap, movable names pointing into it. Once that model is in mind, even the rough edges of the CLI become predictable.

49.10 Summary

Version control treats change as a first-class entity instead of a side-effect of editing files. Git is a distributed version control system: every developer holds a complete copy of the repository, stored in the `.git/` directory as a graph of immutable, content-addressed snapshots. A file passes through three states — working tree, staging area, committed — and the boundaries between them are what `git add` and `git commit` cross. Commits are identified by SHA hashes and chained through parent pointers; branches are cheap,

movable refs into that chain; remotes are other copies of the repository, synchronized through the symmetric pair `fetch`/`pull` and `push`. The daily loop of `git status`, `git add`, `git commit`, and `git push` covers most working hours. With this foundation in place, the rest of Part X can address the collaborative practices — pull requests, code review, agile process — that depend on it.

PART X · COLLABORATION AND PROCESS

CHAPTER 50

Branching, Pull Requests, and Code Review

Watch the presentation

Se presentationen på svenska

A solo developer can push every commit straight to the `main` branch and get away with it most of the time. The same workflow on a team breaks within a week. Two engineers overwrite each other's work, a half-finished refactor lands at the same moment a colleague is debugging production, and nobody ever read the code that just shipped. The remedy is a piece of social infrastructure layered on top of Git: every

change moves through a short-lived branch, is proposed back to `main` through a pull request, and is reviewed by another human before it merges. The chapter develops that workflow — the branch, the pull request, the review, the protection rules that enforce the contract, and the merge strategies that decide what the history ends up looking like.

50.1 Why pushing to main is risky

Direct commits to the shared branch skip every checkpoint a team relies on. There is no chance for a colleague to spot the off-by-one error. There is no run of the test suite before the change reaches everyone else. There is no record of *why* the change was made, separate from the diff itself. And if the change is wrong, rolling it back means rewriting public history — an operation Git allows but treats as hostile, because every other clone of the repository now disagrees with the remote about which commits exist.

The deeper cost is cultural. A team that pushes directly to `main` has no shared moment in which the code is held up to scrutiny. Knowledge stays trapped inside the head of whoever wrote each line. New team members have nowhere to learn from existing code, because no one is ever asked to explain it. The pull request workflow exists as much to create that scrutiny moment as it does to prevent any specific bug.

The fix is not to forbid sharing the `main` branch — every team needs a single trunk that represents the current state of the product. The fix is to require that changes arrive on `main` through a reviewable proposal rather than a unilateral push.

50.2 The topic-branch flow

The standard rhythm for adding a feature, fixing a bug, or refactoring a module is a four-step loop: branch, push, propose, merge.

1. **Branch from `main`**. Starting from an up-to-date local `main`, the developer creates a short-lived topic branch — usually named after the work it contains (`feature/user-login`, `fix/null-pointer-on-checkout`). All commits for that piece of work land on this branch, isolating them from `main` and from anyone else's in-flight work.
2. **Push the branch to the remote**. Once there is something worth showing — even an early sketch — the branch is pushed to GitHub. The remote copy is what reviewers will look at.
3. **Open a pull request**. The developer asks GitHub to create a proposal to merge the topic branch into `main`. The PR is the conversation surface for the change.
4. **Review and merge**. One or more reviewers read the diff, ask questions, request changes if needed, and approve. When all required checks pass, the branch is merged into `main` and the topic branch is deleted.

The flow is short by design. A topic branch that lives for an afternoon merges cleanly. A topic branch that lives for three weeks accumulates conflicts with every other change that landed on `main` in the meantime, and the cost of integration grows roughly with the square of the duration. This is the same pressure that motivates trunk-based development — keep branches short, integrate often, never let two parallel histories drift far apart.

50.3 What a pull request actually is

A **pull request** (or merge request) is a proposal to merge changes from one branch into another (usually a feature branch into `main`); it opens a discussion thread where reviewers examine the code, comment on changes, request revisions, and ultimately approve or reject the merge. The name is literal: the author is requesting that the target branch *pull* in the commits from the source branch.

A PR is not a Git concept. Git itself only knows about branches, commits, and merges. The PR is a feature of the hosting platform — GitHub, GitLab, Azure DevOps, Bitbucket — and it bundles three things together that Git would otherwise leave separate:

- A diff between the source branch and the target branch, rendered file-by-file with inline review controls.
- A discussion thread attached to the proposal, where reviewers leave general comments and per-line comments that the author can resolve as they push fixes.
- A CI status panel showing the result of automated checks (continuous integration builds, test suites, linters, security scans) that ran against the proposed merge.

The PR is also a durable artifact. Long after the branch is deleted, the discussion, the diff, and the CI history remain attached to the merge commit on `main`. When someone six months later is reading a stack trace and `git blame` points them at a line, the PR is there to explain why the line looks the way it does.

50.3.1 The lifecycle of a PR

A PR moves through a small set of states. It opens in *open* state, accumulates commits as the author pushes fixes in response to review, transitions to *approved* once enough

reviewers have signed off, and ends in *merged* (or *closed without merging* if the work is abandoned). GitHub records every state change, every comment, and every commit that was added during review, so the entire decision is reconstructible.

50.4 Branch protection: turning convention into enforcement

A team can agree to use the topic-branch flow and still have someone forget on a Friday afternoon and push a hotfix straight to `main`. Conventions break under pressure. The remedy is to make the convention impossible to violate.

A **branch protection rule** is a repository setting that enforces policies on a branch (e.g., `main`), such as requiring pull request reviews, status checks, or up-to-date branches before allowing pushes or merges, preventing accidental or malicious direct edits. Once enabled on `main`, the rule replaces "please don't" with "the platform refuses." The most common policies a team enables:

- **Require a pull request before merging.** Direct pushes to `main` are rejected at the server. The only path in is a reviewed PR.
- **Require N approving reviews.** The platform refuses to merge until at least N reviewers (often one or two) have approved the diff. Stale approvals — those given before the most recent push — can be auto-dismissed so a last-minute change cannot slip through unreviewed.
- **Require status checks to pass.** The merge button stays disabled until the CI pipeline reports green. A PR that breaks the test suite cannot be merged, no matter how many humans approve it.

- **Require a linear history.** The platform forbids merge commits, pushing the team toward squash or rebase merges (covered below) so that `main` is a straight line of commits rather than a tangle of parallel ribbons.
- **Require the branch to be up to date with `main`.** Before merging, the topic branch must include the latest commits from `main`, which forces conflicts to be resolved on the topic branch rather than discovered on `main` after the fact.

These rules turn the social workflow into a technical guarantee. A new contributor who has never read the team's wiki cannot accidentally merge an unreviewed change to production, because the platform itself will not let them.

50.5 Merge strategies

Once a PR is approved and the checks are green, the platform offers a final choice: *how* should the topic branch's commits be combined into `main`? Git supports three answers, and each leaves the history looking different.

STRATEGY	RESULT ON `MAIN`	WHEN IT FITS
Merge commit	All topic commits land verbatim, plus a merge commit tying them together	Long-running feature branches where intermediate commits matter for archaeology
Squash	All topic commits collapse into one new commit on `main`	Most short-lived PRs — produces a clean, one-PR-per-commit history
Rebase	Topic commits are replayed on top of `main`, with no merge commit	Linear-history teams that want each commit on `main` to stand alone

A **merge strategy** determines how Git combines two branches: a merge commit preserves both branches' histories, a squash merge combines all commits into one, and a rebase merge replays commits on top of the target branch for a linear history.

A merge commit preserves the full topic branch as it was developed. The history shows every "wip", "fix typo", and "address review feedback" commit the author made along the way. This is honest but noisy. The branch structure is also preserved — `main` becomes a tree, not a line. That structure is occasionally useful when reasoning about long-running parallel work, and unhelpful when reading the recent history to find the commit that introduced a bug.

A squash merge flattens the topic branch into one new commit on `main`, with a message the author writes once before merging. The intermediate commits are gone. The advantage is a clean `main` history where every commit corresponds to one reviewed PR. The disadvantage is that the

granular history of how the work evolved is lost — though the original commits are still visible on the (now-deleted) branch's PR page if anyone needs them. Squash is the default for most product teams using GitHub.

A rebase merge replays each topic commit on top of the latest `main`, then fast-forwards `main` to include them. The result is a linear history with all the original commits intact, but no merge commit signaling that they were developed on a branch. This works well when the topic branch was kept tidy throughout development (each commit is meaningful and self-contained) and badly when it was not (the rebased commits include "wip" and "fix typo" forever).

The choice is a team decision, not a per-PR one. Most teams pick one strategy, configure the branch protection rule to enforce it, and stop debating it.

50.6 Conflict resolution

Most merges are uneventful — Git applies the topic branch's diffs on top of `main` and the result is unambiguous. A merge conflict happens when two branches modified the same lines of the same file in incompatible ways: a colleague renamed `getUserName()` to `fetchUser()` while the topic branch added a new caller of the old name. Git cannot decide which version is correct; it stops, marks the conflicted regions in the file with `<<<<<<<` / `=======` / `>>>>>>>` markers, and hands the decision to the developer.

Conflict resolution is the process of handling merge conflicts that occur when two branches modify the same lines of code; the developer manually edits the conflicting file to choose which changes to keep, then completes the merge. The job is one of intent, not mechanics. The tool can show

what each side did; only a human knows what the code is *supposed* to do after both changes are applied.

Conflicts are easier to resolve when topic branches are short and the team rebases or merges from `main` frequently. A branch that has been off doing its own thing for two weeks will conflict with everything; a branch that pulled `main` yesterday will conflict with almost nothing.

50.7 Draft pull requests for work in progress

Sometimes a developer wants feedback on a direction before the work is done — a sketch of an API, a partial migration, an early stab at a refactor. Opening a normal PR for this signals "ready to merge," which is misleading. The remedy is a **draft PR**: a pull request marked as work-in-progress that suppresses notifications and prevents merging; it allows developers to share early feedback on incomplete changes without formally requesting review.

A draft PR is identical to a regular PR in every way that matters for collaboration — same diff, same discussion thread, same CI runs — except that the merge button is disabled and reviewers are not paged. The author marks the PR as ready when the work is finished, at which point it transitions to a normal review request.

Draft PRs are also a useful collaboration surface for paired work. Two developers can push to the same branch, and the draft PR shows the running state to both of them and to anyone they want to consult, without anyone treating the work as done.

50.8 Code review as a craft

The mechanics of opening, approving, and merging a PR are the easy part. The skill is in the review itself. A good review catches bugs, levels up the team, and leaves the author feeling helped rather than judged. A bad review nitpicks variable names, ignores the actual logic, and corrodes trust over time.

A few habits that distinguish good review:

- **Keep PRs small.** A 50-line PR gets a careful read. A 2,000-line PR gets a rubber stamp. The author owes reviewers a reviewable diff; the reviewer owes the author a real review. Both halves break when the diff is too large to hold in one sitting.
- **Ask, do not assert.** "Why did you choose a `Dictionary` here instead of a `HashSet`?" invites a conversation. "This should be a `HashSet`" closes one. Often the author has a reason the reviewer did not see.
- **Distinguish blocking from non-blocking feedback.** A bug that will cause an outage blocks the merge. A preference about variable naming does not. Conventions like prefixing comments with `nit:` for minor stylistic suggestions help reviewers send both kinds of feedback without conflating them.
- **Be kind in the review and rigorous in the standard.** The author wrote the code at three in the afternoon under deadline pressure; the reviewer is reading it at ten in the morning with full attention. Tone matters because the asymmetry is real. Standards matter because the code will live in production for years.
- **Approve when it is good enough, not when it is perfect.** The point of review is to catch things the author missed, not to rewrite the PR in the reviewer's preferred style. Trust the author to make the small calls.

50.9 Where this fits with the exercises

The companion exercise chapter, Code Collaboration, walks through this workflow end-to-end: creating a topic branch, opening a pull request via the GitHub UI and the `gh` CLI, requesting review, resolving conflicts, and merging through each of the three strategies. The theory in this chapter exists to make those steps feel motivated rather than ceremonial — every click in the exercise corresponds to one of the protections, signals, or design decisions described above.

50.10 Summary

Pushing directly to `main` skips the social mechanism teams rely on to catch mistakes and share knowledge, which is why production codebases route every change through a topic branch and a pull request. A pull request is a platform-level proposal to merge one branch into another, bundling a diff, a discussion thread, and a CI status into a single durable artifact. Branch protection rules turn the convention into enforcement: required reviews, required green builds, and required linear history move the team's standards into the platform itself. The merge strategy — merge commit, squash, or rebase — decides what the resulting history on `main` looks like, and is a team-level choice rather than a per-PR one. Conflict resolution is the human's job, not the tool's, because only the developer knows what the merged code is supposed to mean. Draft PRs make work-in-progress collaboration explicit, and the craft of code review — small diffs, kind language, asking rather than asserting — is what turns the workflow from mechanical ceremony into a learning loop the whole team benefits from.

PART X · COLLABORATION AND PROCESS

CHAPTER 51

Inner Loop vs Outer Loop

Watch the presentation

Se presentationen på svenska

A developer's productivity is dominated by feedback latency. Every time a code change is made, some interval of time passes before the developer learns whether that change is correct, useful, or broken. That interval—measured from keystroke to verifiable answer—governs how many ideas can be tested in an hour, how willing a developer is to refactor, and how confidently a team can ship. Two distinct cycles produce that feedback at very different speeds, and writing software well requires understanding both. The inner loop runs

locally and answers narrow questions in seconds; the outer loop runs on shared infrastructure and answers integration questions in minutes. Optimising the inner loop is the highest-leverage productivity investment a developer can make, and several tools — hot reload, fast unit tests, in-memory data stores, and dev containers — exist specifically to shorten it.

51.1 The two loops of software development

Every change a developer makes travels through two feedback cycles. The first runs entirely on the developer's own machine and produces an answer in seconds. The second runs on shared infrastructure, involves the rest of the team, and produces an answer in minutes. Both cycles are necessary, but they answer different questions and operate on different timescales.

51.1.1 The inner loop

The **inner loop** is the fast, local development cycle where a developer edits code, runs it (via `dotnet run` or similar), and verifies changes instantly without waiting for CI/CD; it emphasizes rapid feedback and iteration. Concretely, this is the edit-build-run-test cycle that happens on the developer's laptop, ideally completing in seconds to single-digit minutes.

The inner loop answers narrow, immediate questions. *Does this method compile? Does this test still pass after the refactor? Does the endpoint return the JSON shape the front-end expects?* These are the questions a developer asks dozens or hundreds of times in a productive day. Each iteration is short, cheap, and private—nothing leaves the local machine until the developer is satisfied.

A typical inner-loop iteration looks like this. The developer edits a file in the editor. The build runs, either invoked manually with `dotnet build` or triggered automatically by a watcher. The application starts, or a unit test runner executes the relevant test. The developer reads the result—a passing test, a stack trace, a rendered page—and edits again. Most of this happens without the developer thinking about the cycle at all. That invisibility is the goal.

51.1.2 The outer loop

The **outer loop** is the slower, automated validation cycle triggered by pushing code (e.g., to GitHub), where CI/CD pipelines build, test, and deploy changes; it provides confidence that changes work across all environments before reaching production. This is the commit-push-CI-deploy cycle that happens on shared infrastructure, typically completing in minutes rather than seconds.

The outer loop answers broader, integrative questions. *Does the code build on a clean machine? Do the integration tests pass against a real database? Does the container image still start when deployed to a staging environment?* These questions require shared resources—a build agent identical to the production host, a real Postgres instance, the deployment target—and they require coordination with the rest of the team's work on the same branch.

The outer loop runs through a pipeline, defined as code in the repository, triggered by `git push` or by opening a pull request. It is the team's shared safety net: the place where work converges, where assumptions about "it builds on my machine" are tested against the team's reproducible environment, and where the merge to `main` is gated.

51.1.3 Why the distinction matters

The two loops are not redundant. Each catches problems the other cannot. The inner loop catches typos, broken unit tests, and wrong-shape API responses while the developer is still holding the change in working memory. The outer loop catches environment drift, missing dependencies, and integration failures that only appear when the code meets a system the developer cannot fully reproduce locally. A well-run team uses both, and uses each for the questions it is good at answering.

The mistake worth avoiding is treating the outer loop as a substitute for the inner loop. When the inner loop is broken or slow, developers compensate by pushing half-finished work to CI and using the pipeline as their build machine. The pipeline becomes a bottleneck, builds queue up, and a fifteen-second mistake takes ten minutes to discover. This pattern is recoverable but expensive—and it almost always indicates that the inner loop deserves attention.

51.2 Why optimizing the inner loop matters

A developer runs the inner loop hundreds of times a day. A team of six developers runs it tens of thousands of times a week. Every second saved per iteration compounds across that volume into hours of recovered focus, more refactors attempted, and more hypotheses tested.

The math is unforgiving in the other direction as well. A thirty-second build feels fine. A two-minute build feels like an interruption. A five-minute build is long enough that the developer will switch tabs, lose the thread of the change, and return distracted. The cost is not the five minutes themselves—it is the broken concentration that costs another ten

minutes of re-orientation. Across a week, a slow inner loop quietly erases an entire workday per developer.

Speed also changes what a developer is willing to attempt. With a sub-second test feedback cycle, a developer will write the test first, watch it fail, write the implementation, and watch it pass—the discipline of test-driven development is sustainable. With a forty-second test feedback cycle, that same developer will write the implementation first, then the test, then run both at the end of the change, hoping for the best. The slow cycle quietly degrades engineering practice without anyone deciding to lower the bar.

The inner loop is also where exploratory work happens. A developer trying three approaches to a tricky problem needs to throw away two of them. If each attempt costs a minute to verify, three attempts cost three minutes—nothing. If each attempt costs eight minutes, three attempts cost half an hour, and the developer will commit to the first approach that works rather than the best one. Inner-loop speed is, indirectly, code-quality investment.

51.3 Tools that shorten the inner loop

Several tools and patterns specifically target the inner loop. They are not interchangeable; each addresses a different source of latency.

51.3.1 Hot reload

Hot reload is a development feature that automatically rebuilds and restarts the running application when source code changes, eliminating the manual `stop → edit → run` cycle and accelerating the inner feedback loop. In .NET, this is exposed through `dotnet watch`, which monitors the source

tree and applies changes to a running application without a full restart where possible.

The mechanism collapses three steps—save, stop, run again—into one. The developer saves; the watcher detects the change; the application reloads with the new code; the next browser refresh shows the updated behavior. For UI work, where the developer is iterating on visible output, hot reload changes the felt experience of programming. The browser window becomes a live mirror of the source file.

Figure 1: Starting a .NET application in watch mode

```
dotnet watch run
```

Hot reload has limits. Some changes—altered method signatures, new dependency injection registrations, migrations—require a full restart, and the watcher will fall back to a rebuild rather than patch in place. The developer experience is not magic; it is a careful trade-off between patch-and-continue and rebuild-and-restart, with the watcher choosing the cheapest option that produces correct behavior.

51.3.2 Fast unit tests

Unit tests that run in milliseconds are a different kind of inner-loop tool. They give the developer a verifiable answer ("the function does what I claim it does") without involving the application as a whole. A test suite of a thousand fast unit tests can run in under five seconds. The developer can run the entire suite on save and treat any failure as an immediate, local signal.

Fast unit tests stay fast by avoiding I/O. They do not touch the disk, the network, or a real database. They exercise pure functions and isolated classes, with collaborators replaced by test doubles. The cost of this discipline is that unit tests

cannot validate integration—they cannot prove the code works against a real Postgres instance, only that the code works against an in-memory fake. That is a trade-off the team accepts: integration is the outer loop's job; correctness of individual units is the inner loop's job.

51.3.3 In-memory databases for tests

For tests that need data-access logic but cannot afford the latency of a real database, in-memory database providers offer a middle ground. Entity Framework Core's `InMemoryDatabase` provider, or SQLite running in `:memory:` mode, gives the test code something that responds to LINQ queries and tracks entities without a network round-trip.

The in-memory option is fast enough to run on every save, and it catches a class of bugs—wrong joins, missing `Include` calls, wrong query shape—that pure unit tests cannot. The trade-off is that the in-memory provider is not the production database. SQL features like JSON columns, full-text search, or vendor-specific functions behave differently or not at all. In-memory tests are a useful intermediate signal, not a substitute for an outer-loop test against the real database engine.

51.3.4 Dev containers

A **dev container** is a containerized development environment (e.g., via Docker and VS Code Dev Containers extension) that ensures consistent tooling, dependencies, and configuration across team members' machines, reducing "works on my machine" problems.

The problem dev containers solve is environmental drift. One developer has .NET 8 installed; another has .NET 9. One has Node 18; another has Node 20. The Postgres versions differ. The local certificates differ. Each developer's machine is a slightly different production target, and bugs that depend

on those differences cost hours to diagnose. A dev container declares the entire toolchain—runtimes, compilers, databases, formatters, linters—in a single configuration file checked into the repository.

Figure 2: Minimal `.devcontainer/devcontainer.json`

```
{
  "name": "API dev",
  "image": "mcr.microsoft.com/devcontainers/dotnet:8.0",
  "features": {
    "ghcr.io/devcontainers/features/docker-in-docker:2": {}
  },
  "postCreateCommand": "dotnet restore"
}
```

When a new developer clones the repository, the IDE detects the dev container configuration, builds (or pulls) the image, and opens the editor inside that container. The toolchain on the host machine is irrelevant. The dev container is the toolchain. Onboarding shrinks from “spend a day installing things and asking Slack” to “open the repo, click Reopen in Container, write code.”

Dev containers also bring inner-loop tooling to any IDE that supports the protocol. The container exposes a consistent runtime; the IDE—VS Code, JetBrains Rider, others—connects to it. The same dev container works on Windows, macOS, and Linux because the container itself is Linux, and the host’s role is reduced to running Docker.

The trade-off is the indirection layer. File watchers cross a virtual filesystem boundary; ports are forwarded; debugging crosses a container edge. On a fast machine with good Docker integration, this is invisible. On a slow machine, or with a heavy image, it adds noticeable latency to the inner loop and undoes some of the speed it was meant to protect. The dev

container is a tool, not a default; whether to adopt it is a judgement about consistency-versus-speed for the specific team.

51.4 The boundary conversation

A live tension runs through every test suite: deciding which checks belong in the inner loop and which earn their slot in the outer loop is a continuous engineering judgement.

The inner loop is constrained by speed. Anything that runs on every save must complete in seconds. The outer loop is constrained by infrastructure cost and by how long the team will tolerate a yellow check mark on a pull request. Anything that takes minutes belongs in the outer loop—but the outer loop is a worse place to learn about a problem, because the developer has already moved on.

The default heuristic is to push every check toward the inner loop until something forces it outward. A check earns its outer-loop slot when one of three conditions holds: it depends on infrastructure the developer cannot run locally (a real cloud database, a third-party API), it takes long enough that running it on save would break flow (a full integration suite), or it validates a property that is meaningful only in a production-like environment (deployment health, smoke tests against a staging URL).

CHECK TYPE	LOOP	WHY
Compile, type-check	Inner	Sub-second; runs on save via the IDE
Unit tests (no I/O)	Inner	Sub-second; high-volume signal
In-memory or SQLite-backed data tests	Inner	Fast enough; catches query-shape bugs
Format and lint checks	Inner (and outer as a gate)	Inner for speed, outer to enforce
Integration tests against a real database	Outer	Requires shared infra; too slow for save
End-to-end browser tests	Outer	Slow, flaky, requires a running app
Build of production container image	Outer	Reproducibility matters more than speed
Deploy and smoke-test	Outer	

CHECK TYPE	LOOP	WHY
		Requires a deployment target

The conversation is rarely settled once. As the codebase grows, slow tests creep into the inner loop and developers stop running them locally. Fast tests are sometimes promoted from outer to inner once a faster alternative (an in-memory provider, a contract test) becomes available. The team's shared judgement about where each check belongs is itself a piece of engineering practice that needs maintenance.

For hands-on practice with the local development cycle that the inner loop describes, see the exercises at /exercises/15-code-collaboration/, where the local edit-build-run cycle around `dotnet run` is the foundation of the commit and review workflow.

51.5 Summary

A developer's day is governed by feedback latency, and that latency comes in two distinct cycles. The inner loop is the fast, local edit-build-run-test cycle that runs on the developer's own machine in seconds; it answers narrow questions about correctness while the change is still in working memory. The outer loop is the slower commit-push-CI-deploy cycle that runs on shared infrastructure in minutes; it answers integration questions that require a reproducible environment and the team's converged work. Optimizing the inner loop is high-leverage because every saved second compounds across thousands of iterations a week, and because slow cycles quietly degrade engineering practice. Hot reload, fast unit tests, in-memory database providers, and dev con-

tainers each shorten the inner loop along a different axis—live patching, no-I/O testing, lightweight data-layer testing, and reproducible toolchains. The boundary between what belongs in the inner loop and what earns its slot in the outer loop is a continuous engineering judgement, not a fixed rule, and a healthy team revisits it as the codebase and the tools evolve.

CHAPTER 52

Agile, Sprints, and User Stories

Watch the presentation
Se presentationen på svenska

Software requirements rarely survive contact with real users. A team that locks down a 200-page specification, builds against it for nine months, and only then ships to production routinely discovers that priorities shifted, assumptions were wrong, or the market moved. Waterfall planning collapses under requirement change because it treats change as an exception. Agile methods invert that assumption — change is the default, and the process exists to absorb it through short, frequent delivery cycles. A working team oper-

ates that way through a small set of shared concepts: the values agile rests on, the Scrum cadence that most teams adopt, the user-story format that frames work in user-visible terms, and the acceptance criteria and definition-of-done contracts that make “done” mean something specific.

52.1 Why iterative delivery replaces big-bang planning

A waterfall project sequences phases — requirements, design, implementation, testing, release — and treats each as a gate. The plan only works if the requirements gathered at the start remain accurate when the product ships. In practice they almost never do. Stakeholders learn what they want by seeing partial systems running. Regulations change. Competitors release features that reshape the priority list. By the time a waterfall release lands, parts of it are already obsolete.

Iterative delivery accepts this and shrinks the cycle. Instead of one delivery in nine months, a team delivers something useful every one or two weeks. Each delivery is a real chance to learn — to confirm an assumption, to discover a bad one, to re-prioritize the next iteration based on what was just observed. The plan becomes a living artifact rather than a contract, and the cost of a wrong direction is bounded by the length of one iteration rather than the length of the project.

52.2 The four values of the Agile Manifesto

Agile is a family of iterative software development methodologies that emphasize flexible requirements, frequent delivery, continuous feedback, and team collaboration over rigid upfront planning, with the goal of responding quickly to change. The 2001 Agile Manifesto distilled the movement into

four value pairs. Each pair acknowledges that both sides have value, then states that the left side has more.

The first pair is individuals and interactions over processes and tools. A team of skilled people communicating well will out-deliver a more rigid team using better tools. Process exists to support people, not the other way around — when a process consistently obstructs good work, the team should reshape it.

The second pair is working software over comprehensive documentation. Working software is the only honest measure of progress. A 50-page design document does not run, cannot be exercised by a user, and does not validate any assumption. Some documentation is necessary; the value statement says it should never become a substitute for running code.

The third pair is customer collaboration over contract negotiation. A fixed-scope contract drafted before either party fully understands the problem will protect the wrong things. Continuous collaboration with the customer — letting them see partial work, adjusting based on their feedback — produces better outcomes than enforcing a contract that was wrong on the day it was signed.

The fourth pair is responding to change over following a plan. Plans are useful as snapshots of current intent, but the project must be able to incorporate new information without procedural friction. The plan is a tool, not a commitment.

These values do not constitute a method on their own. They are the criteria a team uses when choosing how to work.

52.3 Scrum as the dominant framework

Most agile teams adopt Scrum, a lightweight framework that turns the manifesto values into a concrete weekly rhythm. Scrum defines roles (product owner, Scrum master, develop-

ment team), artifacts (product backlog, sprint backlog, increment), and a small set of recurring ceremonies. The framework's core mechanism is the sprint.

A **sprint** is a fixed time-boxed iteration (usually 1–2 weeks) in Scrum where a team commits to completing a set of work items; it includes planning, daily standups, review, and retrospective ceremonies. The time-box is non-negotiable. If the team estimates poorly and cannot finish everything, the sprint still ends on the planned date and unfinished work returns to the backlog. This rule is what gives sprints their feedback value: every two weeks the team gets an honest signal about its capacity.

52.3.1 Sprint planning

Sprint planning is a ceremony where the team estimates story points for backlog items, commits to a sprint goal, and selects work to complete in the upcoming sprint based on team velocity and priorities. The product owner brings a prioritized backlog. The team reviews the top items, asks clarifying questions, and decides how many it can credibly complete given its observed velocity from previous sprints. The output is a sprint backlog — a concrete list of work the team has committed to — and a single-sentence sprint goal that explains what the sprint is for.

52.3.2 The daily sync

Once the sprint is running, the team meets briefly each day — fifteen minutes is typical — to coordinate. Each member answers three implicit questions: what was completed since the last sync, what is planned for today, and what is blocked. The daily sync is not a status report to a manager; it is the team aligning with itself. Blockers surface immediately so

someone can clear them rather than discovering at the end of the sprint that a story stalled on day three.

52.3.3 Sprint review and retrospective

At the end of the sprint the team holds two ceremonies. The sprint review demonstrates the increment to stakeholders. Working software is shown, not described. Stakeholders react, ask questions, and the product owner adjusts the backlog based on what they learn.

The **retrospective** (or retro) is a ceremony at the end of a sprint where the team reflects on what went well, what didn't, and agrees on specific improvements to implement in the next sprint, fostering continuous process improvement. The retrospective focuses on the process, not the product. It produces a small number of concrete actions — usually two or three — that the team will try in the next sprint. Without this ceremony the framework becomes mechanical and stops adapting.

52.4 The user story format

A **user story** is a brief, user-centric description of a feature in the format "As a [user type], I want [goal] so that [benefit]"; it focuses on delivering value from the user's perspective rather than technical implementation details. The format is deliberately constrained. The role names a specific user type, the capability names what they want to do, and the outcome explains why it matters to them.

The format works because each part forces a distinct conversation. Naming the role exposes whether the team understands which user is being served. Naming the capability keeps the story at the level of what the user does, not how the system implements it. Naming the outcome links the work to

user value, which is what the team uses when prioritizing or descoping under pressure.

A user story is not a specification. It is a placeholder for a conversation that the team will have when the story enters a sprint. The card reminds the team that a conversation is needed; the conversation produces the shared understanding; the acceptance criteria record the outcome of that conversation.

52.5 Acceptance criteria as the "how do we know it's done" answer

A story without acceptance criteria invites endless interpretation. **Acceptance criteria** are specific, testable conditions that define when a user story is considered complete; they provide a shared understanding between the team and stakeholders about what "done" means for that story. Each criterion is concrete enough to verify by observation — either the behavior is present or it is not.

Acceptance criteria are story-specific. They answer "what must be true about this story before we accept it" and nothing more. They do not include cross-cutting concerns like test coverage or deployment status — those belong in the definition of done.

52.6 Definition of done as the team's quality contract

The **definition of done** (DoD) is a shared checklist of criteria (e.g., code reviewed, tests passing, documentation updated) that a user story must meet before it can be marked complete; it ensures consistent quality across all work. The DoD applies to every story regardless of subject matter. It encodes

what the team has agreed counts as professional output. A typical DoD includes items like: unit tests pass, the change is reviewed and merged via pull request, the build is green on the main branch, the change is deployed to a staging environment, and any user-facing change has updated documentation.

The acceptance criteria and the DoD work together. Acceptance criteria say "this specific story does what it claims." The DoD says "this work is in the shape we ship." A story is only complete when both are satisfied.

52.7 Worked example: a sign-up form

Consider a story for a new user sign-up form on a web application. Written in user-story format, it reads:

```
As a prospective customer, I want to create an account with
email and password,
so that I can save my preferences and place orders.
```

The team agrees on three acceptance criteria during sprint planning:

- The form rejects an email address that is already registered, with a visible error message naming the conflict.
- A password shorter than 10 characters is rejected before the form submits, with a message explaining the minimum length.
- A successful sign-up sends a verification email within 60 seconds and redirects the browser to a "check your email" page.

These criteria are observable: a tester can drive a browser, attempt each path, and report pass or fail. They do not say

anything about how the email is sent, which library hashes the password, or which framework renders the form — those decisions belong to the developer.

The team's DoD adds the contract that every story must satisfy on top of its acceptance criteria. For the sign-up story, the DoD might require: unit tests cover the email-uniqueness and password-length checks; the change is merged via a reviewed pull request; an integration test exercises the full sign-up flow against a staging database; the staging deployment is healthy after the merge; and a release note mentions the new sign-up flow. A story that passes all three acceptance criteria but ships without the integration test is not done. A story with the integration test but with a broken staging deployment is not done either. The DoD is a gate, not a wish list.

52.8 Scrum compared with Kanban

Scrum is not the only agile framework. Kanban organizes work as a continuous flow rather than as time-boxed sprints. A Kanban team maintains a board with columns representing workflow stages (e.g., To Do, In Progress, In Review, Done) and applies work-in-progress (WIP) limits to each column. New work pulls into the system whenever capacity opens; work is delivered as soon as it finishes rather than at the end of a fixed iteration.

DIMENSION	SCRUM	KANBAN
Cadence	Fixed-length sprints (1–2 weeks)	Continuous flow
Commitment	Sprint backlog committed at planning	No commitment to a fixed batch
Ceremonies	Planning, daily sync, review, retrospective	Daily sync optional; flow review periodic
Capacity control	Sprint capacity from velocity	WIP limits per column
Best fit	Feature work with discoverable scope	Operational work, support, mixed-priority queues

Many real teams blend the two — running sprint planning and retrospectives while letting work flow continuously inside the sprint, or applying WIP limits within a Scrum board. The framework is a tool. Teams adopt the parts that produce useful feedback and adjust the rest.

52.9 Connecting to practice

The Scrum cadence becomes concrete only once a team practices it on a real backlog with real code. The collaboration exercises at /exercises/15-code-collaboration/ walk through writing user stories, opening pull requests against them, and integrating with DevOps automation that enforces the definition of done. Working through the exercises is where the vocabulary in this chapter stops being abstract.

52.10 Summary

Iterative delivery replaces big-bang waterfall planning because it treats requirement change as the default rather than as an exception. The Agile Manifesto's four values — individuals over process, working software over documentation, customer collaboration over contract, and responding to change over following a plan — give a team the criteria for deciding how to work. Scrum operationalizes those values through fixed-length sprints with planning, a daily sync, a review, and a retrospective. User stories frame work in the format "As a role, I want a capability, so that an outcome," keeping the team focused on user-visible value. Acceptance criteria define when a specific story is done; the definition of done is the cross-cutting quality contract that every story must satisfy. Kanban offers an alternative to time-boxed sprints when continuous flow fits the work better, and many teams blend the two.

PART X · COLLABORATION AND PROCESS

CHAPTER 53

Working with Jira

Watch the presentation
Se presentationen på svenska

A sprint board drawn on a whiteboard works for a team that sits in one room. The moment the team works across two offices, two time zones, or a mix of office and remote, that whiteboard stops being a shared instrument — half the team can no longer see it, and the other half stops trusting it because they cannot verify it from their desk. Software teams need a digital work-tracker that every member can read and update from anywhere, and that integrates with the version control and review tools the developers already use. Jira has become the standard answer to this need across professional software organisations, and the discipline of using it well —

not the tool itself — is what separates teams that ship from teams that argue about what they were supposed to be shipping. The pieces that matter for a working developer are the issue-type hierarchy, the workflow as a state machine, the sprint board, and the Git integration that hooks branches and pull requests back to the issues they implement.

53.1 What Jira is

Jira is an Atlassian project and issue tracking tool used by teams to plan work, manage backlogs, run sprints, track progress, and report on project metrics; it integrates with Git platforms like GitHub via webhooks and branch naming conventions. Strip away the marketing surface and Jira is a database of work items with a project structure on top: a project owns a backlog of issues, each issue has a type and a workflow status, and views like the sprint board and the backlog list are filters over that database.

The model has three layers. The project is the container — typically one project per product or per team, identified by a short key like `PROJ` or `WEB`. Inside the project sit individual issues, every one identified by the project key plus a number — `PROJ-123`, `WEB-47`. The issue ID is the load-bearing identifier in the rest of this chapter; it is what the agile board displays, what the developer puts in a branch name, and what the pull request description references. The third layer is the workflow that each issue moves through as work progresses, which is described later in the chapter.

53.2 The issue type hierarchy

Jira issues are organised in a deliberate hierarchy that lets a team plan at multiple levels of granularity at once. Four types

matter for the standard agile setup: epic, story, task, and subtask.

An **epic** in Jira is a large body of work that is broken down into smaller user stories; epics typically span multiple sprints and represent major features or themes (e.g., "User authentication" or "Payment integration"). Epics are the planning unit for product managers and tech leads. They sit above the sprint horizon — a single epic is rarely completed in one sprint and often runs across an entire quarter — and they serve as the answer to "what major work is in flight right now."

A **story** (in Jira) is a work item representing a user-facing feature or enhancement; it is smaller than an epic, fits within one or two sprints, and includes fields like description, story point estimate, and acceptance criteria. The story is the unit the team commits to in sprint planning. Each story belongs to one epic — its parent — and the relationship lets the product manager track the epic's progress as the percentage of its child stories completed. The user story format ("As a [user], I want [goal] so that [benefit]") fills the description field; the story-point estimate fills the estimate field; the acceptance criteria fill a checklist field.

A **task** (in Jira) is a work item representing internal work (refactoring, documentation, infrastructure) that does not directly deliver user value; tasks are often subtasks of stories and are tracked on the sprint board. Tasks fill the gap that user stories cannot — work that has to happen but does not map to a user-facing outcome. Migrating a database schema, upgrading a dependency, writing a runbook, or refactoring a module are all tasks. They appear on the sprint board alongside stories and consume the team's capacity in the same way.

Below stories and tasks sit sub-tasks, which decompose a story or task into the concrete pieces of work that individual developers pick up. A story like "User can reset their password" might decompose into sub-tasks for the database migration, the API endpoint, the email-sending integration, and the frontend form. Sub-tasks are how a story shared by the team becomes work owned by an individual.

LEVEL	TYPICAL SPAN	OWNER	EXAMPLE
Epic	Quarter	Product manager	User authentication
Story	Sprint	Team	User can reset their password via email
Task	Sprint	Team	Upgrade database driver to v8
Sub-task	A day or two	Developer	Add `PasswordResetToken` migration

The hierarchy is not pedantic — it lets a single backlog answer different questions for different roles. The product manager looks at epic progress; the team looks at sprint stories and tasks; the developer looks at the sub-task they are picking up next.

53.3 The workflow as a state machine

A **workflow** (in Jira) is a sequence of statuses and transitions that a work item passes through (e.g., TO DO → IN PROGRESS → IN REVIEW → DONE); teams customize workflows to match

their development process. The workflow is the answer to "where is this issue right now," and it is the thing that turns the issue tracker into a process tool rather than just a list of work.

The default workflow that most teams start with has four states. `TO DO` is the staging area — the issue has been planned into the sprint but no one has started it. `IN PROGRESS` means a developer has picked the issue up and is actively working on it. `IN REVIEW` means the work is complete from the developer's side and is now waiting for a code review, typically via a pull request. `DONE` means the work has been merged and meets the team's definition of done — reviewed, merged, deployed, and verified.

Teams customise this workflow when their process demands it. A team with a manual QA stage adds a `READY FOR QA` and `QA` state between `IN REVIEW` and `DONE`. A team with a deployment gate adds a `DEPLOYED TO STAGING` state. The customisation is straightforward in Jira's workflow editor, but every added state is also added friction — the more granular the workflow, the more transitions someone has to remember to make, and the more likely that the board lies about reality because someone forgot to advance an issue. Most teams settle on four to six states and resist adding more.

53.4 The sprint board

The **sprint board** (in Jira) is a visual board displaying work items in the current sprint organized by status columns (e.g., TO DO, IN PROGRESS, DONE); it provides real-time visibility into sprint progress during daily standups. The board makes work-in-progress visible, which is the single most useful thing a process tool can do — a long `IN PROGRESS` column is a signal that the team has too much in flight, a stuck `IN`

`REVIEW` column is a signal that reviews are not getting done, and an empty `TO DO` column with the sprint half over is a signal that the team underestimated.

The board is the artefact the team gathers around at the daily standup. Each developer walks through their cards, says what is moving forward, and flags anything stuck. The conversation is grounded in the board state — "this card has been in review for three days, who can pick it up" — instead of in vague status reports. When a team's board accurately reflects reality, the standup takes ten minutes; when the board lies, the standup is people remembering what they did yesterday and the manager taking notes.

The board also visualises work-in-progress limits. A team that decides "no more than three issues in `IN PROGRESS` per developer" enforces that limit on the board, and the limit makes the team finish work before starting new work. The pull-system discipline that limits encourage is the antidote to the failure mode where a team starts ten things, finishes none of them, and ends the sprint with everything still half-done.

53.5 Git integration through branch naming

The connection between the issue tracker and the source code is what makes Jira useful for developers, and it is implemented through a convention rather than a magical integration. A **branch naming convention** is a team standard for naming Git branches (e.g., `feature/JIRA-123-user-login` or `bugfix/issue-456`) that encodes the work item type and Jira ticket ID, enabling automatic linking between code and issues.

The convention has three parts. A prefix indicates the type of work — `feature/` for new functionality, `bugfix/` for fixes, `chore/` for maintenance. The Jira issue ID identifies which

specific work item the branch implements — `PROJ-123`. A short slug describes the change in human-readable form — `add-login`. The full branch name reads `feature/PROJ-123-add-login`, and from that single string a teammate can tell what kind of work it is, where to find the full context in Jira, and what the change does in plain language.

```
git checkout -b feature/PROJ-123-add-login
```

The Jira-GitHub integration watches commits and pull request titles for matching issue keys and auto-links the activity back to the issue. When the developer pushes the branch, Jira shows the branch as connected to `PROJ-123`. When a pull request is opened, Jira shows the PR. When the PR is merged, Jira can transition the issue to `DONE` automatically based on a workflow rule. The developer never opens Jira to update the status — the act of doing the work updates the issue, which is the only way status accuracy survives at scale.

53.5.1 A worked example

Consider an epic and the work that comes out of it. The product manager creates `PROJ-100: User authentication` as the epic. The team breaks it into two stories during refinement: `PROJ-101: User can sign in with email and password` and `PROJ-102: User can reset their password`. Each story decomposes into three tasks during sprint planning. `PROJ-101` becomes `PROJ-103: Add user table migration`, `PROJ-104: Implement login endpoint`, and `PROJ-105: Add login form to UI`. `PROJ-102` becomes `PROJ-106: Add password-reset-token table migration`,

`PROJ-107: Implement reset-request endpoint`, and `PROJ-108: Add reset form to UI`.

A developer picks up `PROJ-104`. They drag the card to `IN PROGRESS` on the sprint board, then create a branch:

```
git checkout -b feature/PROJ-104-login-endpoint
```

They commit progress as they go, with messages that reference the issue:

```
PROJ-104: Add LoginRequest DTO and validation
PROJ-104: Wire login endpoint to authentication service
PROJ-104: Add integration test for valid credentials
```

When the work is ready, they open a pull request titled `PROJ-104: Implement login endpoint` with a description that links to the Jira issue. The Jira-GitHub integration sees `PROJ-104` in the branch name, the commit messages, and the PR title, and attaches all of them to the issue's activity log. The developer drags the card to `IN REVIEW`. A teammate reviews and approves; the PR is merged; the workflow rule transitions `PROJ-104` to `DONE` automatically. The epic `PROJ-100` shows progress without anyone updating its fields directly — its progress bar is computed from its children.

53.6 Comment hygiene and estimating discipline

Two practices separate teams that get value from Jira from teams that find it bureaucratic. The first is comment hygiene. An issue is one conversation about one piece of work, and the comments on it are the durable record of what was discussed and decided. Putting design discussions in Slack and decisions in Jira comments — rather than burying them in a

Slack thread that the team will lose track of by next sprint — means that someone joining the team next quarter can read the issue and understand why the code looks the way it does.

The second is estimation. Jira supports estimating in story points (an abstract unit reflecting effort and complexity, calibrated against the team's historical baseline) and in hours. Story points work for stories — a five-point story is roughly twice as much work as a two-point story, regardless of who picks it up, and the team's average velocity in points per sprint is the planning instrument. Hours work for tasks where the estimator already knows roughly how long the work will take and the team needs a deadline-style number for scheduling. Mixing the two on the same backlog produces meaningless aggregates; pick one for stories and stick with it.

The honest summary is that Jira works only as well as the team's hygiene around it. A team that updates the board daily, writes comments on issues, names branches with issue IDs, and reviews the board at every standup gets a tool that tells the truth about the project. A team that does the work but neglects the tracker gets a tool that lies, and a lying tracker is worse than no tracker — it produces false confidence that erodes the moment a stakeholder asks a question the board cannot answer. The companion exercise Code Collaboration walks through the Jira-GitHub integration end to end, including the branch naming, the commit message format, and the PR linkage that makes the workflow function.

53.7 Summary

Jira is a database of work items organised under a project, displayed through views like the sprint board and the backlog list. The issue-type hierarchy — epic, story, task, sub-task — lets a single backlog serve product managers, teams, and

individual developers at the level of granularity each role needs. The workflow is a state machine, typically `TO DO → IN PROGRESS → IN REVIEW → DONE`, and the sprint board makes work-in-progress visible so the team can see when too much is in flight or when reviews are stuck. The Git integration is implemented through a branch naming convention like `feature/PROJ-123-add-login`, which lets commits, pull requests, and issue status flow through one identifier and removes the need for developers to update Jira separately. Estimating in story points works for stories; estimating in hours works for tasks; mixing them produces nonsense. The deeper truth is that Jira is only as honest as the team's discipline around it. With this chapter the Course Book closes — the path from cloud foundations through application development, data, identity, services, containers, delivery, observability, and team process is now in place, and what remains is the practice of doing the work.